

MODERN WORKPLACE AUTOMATION

M365Ops

Guida alla configurazione iniziale — Intune, Entra ID, Exchange Online e motore AI con integrazione MCP (Lokka), pensata per essere estesa nel tempo.

Tenant di riferimento in questa guida: contoso.onmicrosoft.com (profilo "contoso-test")
Versione documento: 1.34 — 19 Agosto 2026

Indice

1. Cos'è M365Ops e principio di fondo
2. Architettura
3. Prerequisiti software
4. App Registration in Entra ID
5. Certificato ed Exchange.ManageAsApp
6. Ruoli RBAC su Exchange Online
7. Registrazione del tenant nel modulo
8. Motore AI (Claude / Azure OpenAI)
9. Connettori MCP (Lokka) — configurazione per-tenant
10. Autenticazione delegata (utente + MFA) — tenant senza App Registration
11. Le 50+ cmdlet Exchange Online e come le usa l'AI
12. Avvio dell'applicazione
13. Uso quotidiano
14. Sicurezza — principi non negoziabili
15. Stato attuale del progetto
16. Roadmap ed estensioni future
17. Script personalizzati (Scripts\Custom)
18. Portabilità — spostare l'ambiente su un altro PC
19. Log delle azioni — storico e debug
20. Stato e utilizzo del motore AI
21. Stato e reset MFA — dettagli tecnici
22. Report AI-orchestrati con grafici — dettagli tecnici
23. Memoria della conversazione — dettagli tecnici
24. Appendice A — struttura dei file
25. Appendice B — tutto avviene dalla GUI, non serve PowerShell

1. Cos'è M365Ops e principio di fondo

M365Ops è un modulo PowerShell + interfaccia web di chat per amministrare un tenant Microsoft 365 Modern Workplace (Intune, Entra ID, Exchange Online) combinando:

- un **catalogo comandi locale**, deterministico e a costo zero, per le richieste note e ripetitive;
- un **motore AI con accesso reale ai dati** (tool-calling), usato quando la richiesta non rientra nel catalogo;
- un principio di conferma umana obbligatoria per ogni azione di scrittura.

Principio di fondo. L'interpretazione dei dati (spiegare perché un device non è conforme, raggruppare pattern, decidere una causa probabile) vive nel motore AI, *non* in regole scritte a mano nel codice. Le cmdlet del modulo restituiscono solo dati grezzi o eseguono azioni — mai interpretazione cablata. Vedi `Get-M365OpsCompliancePatterns.ps1` come esempio di riferimento.

Perché non un semplice bot a regole fisse

Un sistema a sole regole hardcoded (if/else su ogni possibile domanda) non scala e non generalizza: funziona solo per le domande previste in anticipo. M365Ops usa le regole solo per instradare le richieste ovvie a costo zero (senza chiamare l'AI), e delega al modello AI — con accesso reale ai dati tramite tool — tutto ciò che richiede ragionamento o non è stato previsto esplicitamente.

2. Architettura



Flusso di una richiesta in chat

- 1. Il messaggio arriva a `Handle-ChatMessage` in `Gui/Server.ps1`.
- 2. Se c'è un'azione in sospeso in attesa di conferma, la gestisce prima di tutto.
- 3. Altrimenti prova il **catalogo comandi** (regex/trigger locali, zero costo AI).
- 4. Se nessun comando corrisponde, passa la richiesta a `Invoke-M365OpsAgentTools`, che chiama Claude con gli strumenti Lokka disponibili al primo giro, e le cmdlet interne come fallback nei giri successivi.
- 5. Se l'AI propone una scrittura (`propose_graph_write`), l'azione resta in sospeso finché l'utente non conferma esplicitamente in chat.

3. Prerequisiti software

Componente	Uso	Verifica	Installazione automatica
PowerShell 7+	esecuzione del modulo e del server web	<code>pwsh -v</code>	si — <code>M365Ops.bat</code> lo controlla PRIMA di avviare qualsiasi altra cosa e lo installa da solo via winget se manca, con una finestra grafica dedicata (sezione 3.1)
Node.js + npx	avvio del server MCP Lokka (<code>npx -y @merill/lokka</code>)	<code>npx -v</code>	si — stessa finestra grafica di PowerShell 7, oppure pulsante "Installa automaticamente" nel banner delle Impostazioni (sezione 3.1)
Modulo <code>ImportExcel</code>	export report in formato .xlsx	<code>Get-Module -ListAvailable ImportExcel</code>	si — stessa finestra grafica (sezione 3.1), fallback al primo uso se saltata
Modulo <code>ExchangeOnlineManagement</code> — ⚠ VERSIONE FISSATA A 3.4.0	connessione a Exchange Online (app-only e delegata)	<code>Get-Module -ListAvailable ExchangeOnlineManagement</code>	si — stessa finestra grafica (sezione 3.1), fallback al primo uso se saltata. Fissata in COPPIA con MicrosoftTeams 6.5.0 (vedi sotto): verificata dal vivo priva del conflitto di sezione 6.6 quando Exchange si connette PRIMA di Teams — M365Ops installa/ripara sempre la 3.4.0 e rimuove attivamente qualunque versione ≥3.10.0 trovata sul disco ad ogni connessione. Non aggiornarlo mai manualmente (<code>Update-Module</code> o simili) senza prima ripetere la verifica di sezione 6.6 su una copia separata — vedi il riquadro sotto.
Microsoft Edge	generazione PDF (report ed export, incl. questo documento) via <code>--headless --print-to-pdf</code>	presente di default su Windows 11	si — stessa finestra grafica di PowerShell 7/Node.js, utile solo su PC senza Edge preinstallato (raro su Windows 11)
Modulo <code>IntuneWin32App</code>	solo per <code>New-M365OpsWin32App</code> (packaging app Win32)	<code>Get-Module -ListAvailable IntuneWin32App</code>	si — stessa finestra grafica (sezione 3.1), fallback al primo uso se saltata
Modulo <code>MicrosoftTeams</code> — ⚠ VERSIONE FISSATA A 6.5.0	connessione a Microsoft Teams	<code>Get-Module -ListAvailable MicrosoftTeams</code>	si — stessa finestra grafica (sezione 3.1), fallback al primo uso se saltata. Fissata in COPPIA con ExchangeOnlineManagement 3.4.0 (vedi sopra e sezione 6.6) — non aggiornarlo mai manualmente per lo stesso motivo.
Modulo <code>PnP.PowerShell</code>	connessione a SharePoint Online	<code>Get-Module -ListAvailable PnP.PowerShell</code>	si — stessa finestra grafica (sezione 3.1), fallback al primo uso se saltata
Modulo <code>PdfLexer</code>	estrazione testo da PDF caricati nella Knowledge Base	<code>Get-Module -ListAvailable PdfLexer</code>	si — stessa finestra grafica (sezione 3.1), fallback al primo uso se saltata

Microsoft Graph NON compare in questa tabella: questo progetto non usa mai l'SDK ufficiale `Microsoft.Graph` (pesante, molte dipendenze) - ogni chiamata Graph passa da `Invoke-M365OpsGraphRequest`, un wrapper REST puro (`Invoke-RestMethod` + bearer token), quindi non serve nessun modulo dedicato.

Tutti i moduli PowerShell mancanti vengono installati automaticamente al primo uso dalle rispettive cmdlet (`Install-Module ... -Scope CurrentUser`), non serve preinstallarli manualmente.

⚠ Procedura per cambiare la coppia di versioni fissate ExchangeOnlineManagement 3.4.0 / MicrosoftTeams 6.5.0 (leggere prima di toccare uno dei due pin): non aggiornare mai un pin per assunzione o perché "e' uscita una versione nuova" - i due moduli vanno sempre riverificati INSIEME, mai uno alla volta (sezione 6.6: nessuna coppia di versioni condivide la stessa build della libreria di autenticazione condivisa, quindi ogni combinazione va testata dal vivo, non dedotta dai numeri di versione). Prima di cambiare: (1) installare le nuove versioni IN AFFIANCAMENTO a quelle attuali su una macchina di test, mai sostituendole; (2) in un processo PowerShell isolato, importare `ExchangeOnlineManagement` (nuova versione) e chiamare `Connect-ExchangeOnline / Connect-IPPSession` con parametri anche finti (`-AppId / -CertificateThumbprint` e, separatamente, `-AccessToken` finto per il login delegato - l'eventuale conflitto di assembly si manifesta PRIMA di qualunque validazione di rete, quindi non serve un tenant vero); (3) SOLO se nessun errore "Could not load file or assembly"/"The given assembly name was invalid" compare in nessuno dei passaggi, importare `MicrosoftTeams` (nuova versione) DOPO Exchange e chiamare `Connect-MicrosoftTeams`, verificando che sia anch'esso pulito; (4) verificare anche che `-CertificateThumbprint / -Certificate`, `-AccessToken` e `-CertificateThumbprint` su `Connect-IPPSession` esistano ancora come parametri nella nuova versione Exchange (in 3.4.0, `Connect-IPPSession` non supporta ancora nessun flusso delegato device-code - verificare se questo e' cambiato); (5) solo a questo punto aggiornare i numeri di versione ovunque compaiano nel codice (`Assert-M365OpsExoSafeVersion.ps1`, `Assert-M365OpsTeamsSafeVersion.ps1`, `Install-M365OpsPrerequisites.ps1`, `Show-M365OpsPrereqInstaller.ps1` - e la soglia `3.10.0` nella logica di disinstallazione attiva di Exchange, se la nuova versione sicura non e' piu' quella immediatamente successiva). Richiesto esplicitamente dall'utente il 23/08/2026 e ribadito il 25/08/2026, dopo aver trovato lui stesso l'errore nella diagnosi due volte di seguito ("non e' che dipende da versioni diverse dei moduli?", poi "trova una soluzione con moduli piu' vecchi"): un processo scritto una volta, da ripetere identico ogni volta, invece di rifidarsi della memoria.

3.1 Controllo e installazione automatica (17/08/2026, unificata in una GUI il 22/08/2026)

Su richiesta esplicita, l'app non si limita a segnalare i prerequisiti mancanti: li installa da sola dove tecnicamente possibile, senza che l'operatore debba far nulla — un doppio click su `M365Ops.bat` è sufficiente perché l'ambiente sia pronto all'uso.

Percorso comune (tutto già presente): nessuna finestra. `M365Ops.bat` controlla PowerShell 7, winget e Node.js con un semplice `where` prima di fare qualunque altra cosa - se ci sono già tutti (il caso normale su un PC dove l'app è già stata usata), avvia il server direttamente, esattamente come prima, senza alcun ritardo o finestra aggiuntiva.

Qualcosa manca: `Show-M365OpsPrereqInstaller.ps1` . Se manca anche solo uno tra PowerShell 7, winget o Node.js, `M365Ops.bat` apre una finestra grafica dedicata (Windows Forms, eseguita in Windows PowerShell 5.1 - sempre presente, a differenza di PowerShell 7 stesso) che mostra passo per passo cosa sta verificando/installando: PowerShell 7, Node.js, Microsoft Edge, i tre moduli PowerShell (ExchangeOnlineManagement, ImportExcel, IntuneWin32App - dal 22/08/2026, vedi nota sotto), e - se manca anche winget stesso - il bootstrap di winget prima di tutto il resto (vedi sotto).

Ogni passaggio lungo (bootstrap di winget, singole installazioni winget o di modulo) mostra l'output del processo in tempo quasi reale riga per riga MENTRE gira (non solo alla fine) e un contatore di secondi nella barra di stato, così la finestra non sembra mai bloccata anche durante un passaggio silenzioso di 1-2 minuti - bug reale segnalato dal vivo il 22/08/2026 subito dopo il primo utilizzo pratico ("sembra stuck... le persone si rompono"): la prima versione leggeva l'output del processo figlio solo DOPO che finiva (bloccante), quindi durante un bootstrap lungo la finestra restava letteralmente immobile senza nessun segno di vita. La finestra si chiude da sola se tutto va a buon fine, altrimenti resta aperta con il log completo e un pulsante "Chiudi". Prima del 22/08/2026 questi tre passaggi erano divisi fra un file `.bat` di solo testo (PowerShell 7) e un secondo controllo invisibile minuti dopo (Node.js/Edge, dentro `Launch-M365Ops.ps1`) - ora sono un unico flusso visibile, richiesto esplicitamente dall'utente ("una gui che segua l'installazione dei moduli. aggrega dentro l'installazione dei prereq anche nodejs visto che cmq serve").

Bootstrap di winget stesso, se assente — raro (PC molto minimali/ datati o Windows Server), ma possibile proprio sul tipo di "PC pulito" dove serve di più. Un file a parte, `Bootstrap-Winget.ps1`, usa il modulo ufficiale Microsoft `Microsoft.WinGet.Client` (`Repair-WinGetPackageManager`) per installarlo da zero, con TLS 1.2 e il provider NuGet forzati esplicitamente prima di `Install-Module` (altrimenti, su un PC davvero vergine, il tentativo può fallire silenziosamente per un protocollo TLS troppo vecchio negoziato di default, o restare in attesa di un prompt del provider NuGet mai mostrato). Se anche questo bootstrap fallisce, la finestra mostra il link diretto per l'installazione manuale di PowerShell 7 invece di restare bloccata senza spiegazioni.

Il banner "Ambiente non completo" (tab Impostazioni) con il pulsante **"Installa automaticamente"** resta disponibile come ripetizione manuale dentro l'app stessa, per il caso in cui qualcosa cambi DOPO il primo avvio (es. Node.js disinstallato per sbaglio) - chiama `Install-M365OpsPrerequisites` (`POST /api/install-prerequisites`), che gira anche come seconda passata idempotente ad ogni avvio di `Launch-M365Ops.ps1`.

Al primo avvio, `Launch-M365Ops.ps1` crea anche un collegamento **M365Ops.lnk** sul Desktop con un'icona dedicata (`Assets\M365Ops.ico`) - un file `.bat` non può avere un'icona propria (limite di Windows: mostra sempre quella generica di `cmd.exe`), un collegamento sì. Operazione idempotente e non bloccante: se il collegamento esiste già e punta al `M365Ops.bat` corretto non viene ricreato, e un suo eventuale fallimento (es. cartella Desktop non scrivibile) è solo cosmetico, non impedisce mai l'avvio del server.

4. App Registration in Entra ID

Ogni tenant gestito da M365Ops richiede una propria App Registration, con autenticazione **app-only (client credentials)** — mai login interattivo ripetuto.

4.1 Creazione e client secret

- 1. portal.azure.com → Microsoft Entra ID → **App registrations** → New registration
- 2. Certificates & secrets → New client secret → copia subito il valore (non sarà più visibile)
- 3. Il valore del secret **non va mai salvato in un file**: si imposta una sola volta come variabile d'ambiente Windows (vedi sezione 7)

Alternativa: solo certificato, nessun secret (aggiunto il 21/08/2026) - richiesto esplicitamente dall'utente: se hai già creato il certificato per Exchange Online (sezione 6.1) puoi usare **lo stesso certificato** anche per Microsoft Graph, senza creare nessun client secret. M365Ops firma una richiesta di token con la chiave privata del certificato (JWT "client assertion", lo stesso meccanismo standard che `Connect-ExchangeOnline -CertificateThumbprint` usa per Exchange) invece di inviare un secret in chiaro - un solo materiale da gestire e rinnovare per tutta l'app, non due. Per usarlo: lascia vuoti sia "Nome variabile d'ambiente per il secret" sia "Valore client secret" nel tab Tenant, e compila solo il campo "Thumbprint certificato" (sezione 6.1). Se invece il secret È configurato e risolvibile, resta la scelta di default (nessun cambiamento per le installazioni esistenti) - il certificato viene usato solo quando il secret non è disponibile. Verificato dal vivo: token Graph ottenuto e usato con successo per una chiamata reale (`/organization`) con solo il certificato, nessun secret configurato.

4.2 Permessi Microsoft Graph (Application, non Delegated)

Permesso	Motivo
DeviceManagementManagedDevices.Read.All	lettura device Intune, stato compliance
DeviceManagementConfiguration.ReadWrite.All	creazione/lettura profili di configurazione
DeviceManagementApps.ReadWrite.All	creazione/assegnazione app Intune (Win32, Store), criteri di protezione app MAM (Android/iOS)
DeviceManagementRBAC.ReadWrite.All	Scope Tag, ruoli RBAC personalizzati, assegnazioni di ruolo - verificato dal vivo il 19/08/2026: senza questo permesso Graph risponde 403 su <code>/deviceManagement/roleScopeTags</code> e su <code>/deviceManagement/roleDefinitions</code>
DeviceManagementServiceConfig.ReadWrite.All	restrizioni/limiti di iscrizione dispositivi, Autopilot - verificato dal vivo il 19/08/2026: senza questo permesso Graph risponde 403 su <code>/deviceManagement/deviceEnrollmentConfigurations</code>
DeviceManagementScripts.ReadWrite.All	script di distribuzione Windows/macOS, Proactive Remediations - verificato dal vivo il 19/08/2026: senza questo permesso Graph risponde 403 su <code>/deviceManagement/deviceManagementScripts</code> e su <code>/deviceManagement/deviceHealthScripts</code>
Group.ReadWrite.All	creazione gruppi e gestione membri
User.ReadWrite.All	ricerca utenti per nome/UPN E creazione/modifica utenti - <code>User.Read.All</code> da solo NON basta per creare un utente (403 "Insufficient privileges", incontrato dal vivo il 17/08/2026 su un tenant che aveva solo il permesso di lettura)
Mail.Send	invio report generati (CSV/XLSX/PDF) via email
Team.ReadBasic.All	elenco Team del tenant (report Teams/canali) - non ancora aggiunto su tutti i tenant, verificare se un 403 compare su <code>/teams</code>
Channel.ReadBasic.All	elenco canali di un Team - richiesto insieme a <code>Team.ReadBasic.All</code> , uno dei due da solo non basta per un report completo
SecurityAlert.Read.All	lettura alert di sicurezza (Defender) - solo se si usa <code>graph_api_call</code> su <code>/security/alerts_v2</code> (sezione 17.7)
SecurityIncident.Read.All	lettura incidenti di sicurezza - solo per <code>/security/incidents</code> (sezione 17.7)
ThreatHunting.Read.All	query KQL avanzate (equivalente di "Explorer") - solo per lo strumento <code>security_hunting_query</code> (sezione 17.7); richiede ANCHE la licenza Microsoft Defender for Endpoint Plan 2, non solo il permesso - senza quella licenza Graph risponde comunque 403 anche a permesso concesso
Organization.Read.All	report licenze (<code>/organization</code> , <code>/subscribedSkus</code>) - verificato dal vivo il 19/08/2026: già presente e funzionante su vnsys-test, semplicemente mai documentato prima in questa tabella (era già in sezione 10.8 per la modalità Delegata, dimenticato qui)
AuditLog.Read.All	log di sign-in - verificato dal vivo il 19/08/2026: già presente e funzionante su vnsys-test, stesso motivo sopra
UserAuthenticationMethod.ReadWrite.All	stato MFA e reset (sezione 21) - scope dedicato, <i>non</i> coperto da <code>User.ReadWrite.All</code> - verificato dal vivo il 19/08/2026: già presente e funzionante su vnsys-test, stesso motivo sopra
Sites.Read.All	lettura siti SharePoint via <code>graph_api_call</code> diretto (percorso Graph nativo, DIVERSO dal permesso <code>Sites.FullControl.All</code> sotto l'API separata "SharePoint" richiesto per <code>sharepoint_query</code> /PnP - sezione 4.3) - verificato dal vivo il 19/08/2026: già presente e funzionante su vnsys-test, stesso motivo sopra
Application.Read.All	visibilità sulle App Registration del tenant (sola lettura) - verificato dal vivo il 19/08/2026: già presente e funzionante su vnsys-test, stesso motivo sopra
Policy.Read.All	lettura policy del tenant (es. Conditional Access - sola lettura) - verificato dal vivo il 19/08/2026: già presente e funzionante su vnsys-test, stesso motivo sopra
RoleManagement.Read.Directory	lettura assegnazioni di ruolo Entra ID (sola lettura) - verificato dal vivo il 19/08/2026: già presente e funzionante su vnsys-test, stesso motivo sopra
Reports.Read.All	report di utilizzo Microsoft 365 (usage report) - anche i report di utilizzo Copilot (<code>Get-M365OpsCopilotUsageReport</code> / <code>Get-M365OpsCopilotUsageSummary</code> , sezione 24) - verificato dal vivo il 19/08/2026 (e riconfermato il 22/08/2026 per Copilot): NON ancora

	concesso su vnsys-test, Graph risponde <code>{"code": "UnknownError", "message": "{\\"error\\": {\\"code\\": \"S2SUnauthorized\", \"message\": \"Invalid permission.\"}}\"}</code>
<code>IdentityRiskyUser.Read.All</code>	segnali di rischio Identity Protection (<code>/identityProtection/riskyUsers</code>) - verificato dal vivo il 19/08/2026: NON ancora concesso su vnsys-test, Graph risponde 403 "required scopes are missing in the token"; permesso corretto confermato su Microsoft Learn il 19/08/2026 - non <code>SecurityEvents.Read.All</code> (nome usato per errore in una prima stesura di questa tabella e ancora presente, da correggere, in sezione 10.8)

Questa tabella e' stata verificata e completata dal vivo il 19/08/2026 dopo che l'utente ha chiesto esplicitamente conferma che l'elenco fosse completo per TUTTO il modulo, non solo per Intune: 9 permessi risultavano gia' documentati in sezione 10.8 (modalita' Delegata) per le stesse identiche funzionalita', ma mai aggiunti qui - un vero gap di documentazione, non funzionale (le funzionalita' corrispondenti funzionavano gia' su vnsys-test, semplicemente senza che il permesso necessario fosse elencato). Due di questi risultano invece davvero NON concessi su vnsys-test (`Reports.Read.All`, `IdentityRiskyUser.Read.All`), verificato con l'errore Graph reale.

Dopo aver aggiunto i permessi, cliccare sempre **"Grant admin consent"** — senza consenso admin le chiamate app-only falliscono con errore di autorizzazione anche se il permesso risulta "aggiunto" nella lista.

Un 403 "Insufficient privileges to complete the operation" / "Authorization_RequestDenied" su una scrittura significa quasi sempre questo: manca il permesso Application specifico per QUELL'azione (es. serve `User.ReadWrite.All` per creare un utente, non basta `User.Read.All`) oppure il permesso c'è ma manca ancora il "Grant admin consent" - mai un bug del codice, sempre da controllare prima nel portale.

4.3 Permessi SharePoint (API separata da Microsoft Graph)

Aggiunto il 17/08/2026 per il supporto SharePoint/OneDrive (sezione 17.9): a differenza di **tutti** gli altri permessi di questa pagina, che sono sotto l'API **Microsoft Graph**, SharePoint online (via PnP.PowerShell, usato per l'elenco siti/permessi/OneDrive) richiede un consenso separato sotto un'API diversa, chiamata semplicemente **"SharePoint"** nel selettore "APIs my organization uses" del portale.

- 1. App registration → API permissions → Add a permission → tab "APIs my organization uses" → cerca **"SharePoint"** (non "Microsoft Graph")
- 2. Application permissions → `Sites.FullControl.All` → Add permissions
- 3. **Grant admin consent** (esattamente come per Microsoft Graph, sezione 4.2)

Verificato dal vivo il 17/08/2026: senza questo permesso, l'autenticazione col certificato riesce comunque (nessun errore di login) ma **ogni** chiamata reale (anche solo leggere un sito) fallisce con "Unexpected response from the server... status code is Unauthorized" - un errore diverso nella forma dal classico 403 di Graph, ma stesso significato: permesso mancante, non un bug.

Questo permesso riguarda SOLO i tenant App-only. Su un tenant Delegato non esiste un'App Registration a cui concedere `Sites.FullControl.All` - la lettura SharePoint dipende invece dal **ruolo Entra ID dell'utente** con cui si fa login (tipicamente serve *SharePoint Administrator* o *Global Administrator*). Verificato dal vivo il 17/08/2026 su Fabrikam-Prod (Delegato): il login a codice dispositivo può completarsi correttamente (autenticazione riuscita) e le chiamate fallire comunque con "Attempted to perform an unauthorized operation." - un problema di ruolo, distinto e successivo rispetto al login stesso. Se capita, verifica il ruolo dell'utente in Entra ID → Roles and administrators, non l'App Registration.

Il ruolo va assegnato PRIMA di fare login, non dopo. Un ruolo Entra ID concesso mentre una sessione SharePoint delegata è già attiva non si applica al token già emesso (i ruoli sono valutati al momento del login, non rivalutati in tempo reale) - se dopo aver assegnato il ruolo l'errore "Attempted to perform an unauthorized operation." persiste identico, la causa più probabile non è che il ruolo non sia stato assegnato correttamente, ma che serva un **nuovo login**: pulsante "Connetti / Test connessione SharePoint" nel tab **Tenant** (sezione "Stato connessioni", non nel tab MCP/Connettori - vedi sezione 10.7), non semplicemente ripetere la domanda in chat.

Bug reale trovato e corretto il 17/08/2026 grazie al test dal vivo su Fabrikam-Prod: il polling del login a codice dispositivo (SharePoint, Teams, Exchange e il login Graph generico - stesso schema in tutti e 4) usava `setInterval` , che pianifica il giro successivo a orario fisso *indipendentemente* da quanto impiega quello precedente a rispondere. Se lo scambio del token era più lento del solito, due poll potevano finire in volo insieme: il primo completava con successo e ripuliva lo stato lato server, il secondo (già partito) trovava lo stato appena rimosso e mostrava "nessun login in corso" *sopra* il messaggio di successo appena scritto - un login riuscito che sembrava fallito. Prova diretta nei log: la chiave di sessione veniva salvata correttamente, poi risultava assente 22 secondi dopo con due tentativi di lettura falliti nello stesso secondo. Corretto sostituendo `setInterval` con un `setTimeout` che pianifica il giro successivo SOLO dopo aver ricevuto risposta da quello corrente, rendendo la sovrapposizione impossibile per costruzione.

4.4 Permessi Teams (API separata, terza diversa da Graph e da SharePoint)

Aggiunto il 17/08/2026 per il supporto Teams (sezione 17.11): l'elenco Team/canali/membri funziona SUBITO con lo stesso certificato di Exchange, verificato dal vivo, nessun permesso aggiuntivo. I **criteri** (riunione/chiamata/messaggistica) e la **configurazione di accesso esterno/ospiti** invece richiedono una terza API separata, chiamata **"Skype and Teams Tenant Admin API"**.

- 1. App registration → API permissions → Add a permission → tab "APIs my organization uses" → cerca **"Skype and Teams Tenant Admin API"**
- 2. Application permissions → `application_access` → Add permissions
- 3. **Grant admin consent**

Non ancora verificato se basta il permesso da solo o se serve ANCHE assegnare al service principal dell'App Registration un ruolo Entra ID (es. "Teams Administrator") - la documentazione Microsoft per l'autenticazione app-only di questo modulo menziona entrambi in scenari diversi. Se dopo aver concesso il permesso le cmdlet di policy continuano a fallire con lo stesso errore, il prossimo passo da provare è assegnare quel ruolo al service principal (Entra ID → Enterprise applications → cerca l'App Registration → Roles and administrators).

Verificato dal vivo il 17/08/2026 senza questo permesso: `Get-Team` / `Get-TeamChannel` / `Get-TeamUser` funzionano comunque, ma `Get-CsTeamsMeetingPolicy` / `Get-CsTeamsCallingPolicy` / `Get-CsTeamsMessagingPolicy` e le cmdlet di configurazione ospiti falliscono con "Access Denied. Provide different credential or request access." - un terzo errore diverso da "Unauthorized" (SharePoint) e da 403 (Graph), ma stesso significato: permesso mancante.

4.5 Ruolo Purview per le retention policy (RBAC nel portale Compliance, non un permesso App Registration)

Aggiunto il 18/08/2026 per il supporto retention policy (sezione 17.12). A differenza di Graph/SharePoint/Teams sopra, qui non basta un permesso API sull'App Registration: Microsoft Purview usa un sistema di **ruoli RBAC separato**, assegnato direttamente nel portale compliance, non in Entra ID.

Correzione del 18/08/2026 rispetto alla prima stesura di questa sezione: un service principal (l'App Registration usata in modalita' AppOnly) NON puo' essere aggiunto DIRETTAMENTE come membro di un gruppo di ruoli Purview dal portale - a differenza di un utente Delegato, per cui l'assegnazione diretta funziona davvero cosi' come descritto sotto. Per un service principal serve un passaggio intermedio, verificato sulla documentazione reale (non a memoria) dopo che l'utente ha chiesto esplicitamente conferma di questa procedura.

Tenant Delegato (login utente, nessuna App Registration): assegnazione diretta, un solo passaggio.

1. Vai su compliance.microsoft.com → **Ruoli e ambiti** → **Autorizzazioni**
2. Cerca il gruppo di ruoli **"Compliance Administrator"** (o un ruolo piu' mirato tipo "Records Management"/"Retention Management" se disponibile nel tenant)
3. Aggiungi come membro l'utente delegato (lo stesso UPN gia' usato per Exchange)

Tenant AppOnly (service principal/certificato): un service principal non e' assegnabile direttamente a un gruppo di ruoli Purview - serve un gruppo di sicurezza Entra ID come tramite:

1. Crea un gruppo di sicurezza Entra ID con l'opzione **"Azure AD roles can be assigned to this group" = Yes** attiva alla creazione (va impostata alla creazione, non modificabile dopo su un gruppo gia' esistente)
2. Aggiungi il service principal dell'App Registration (AppOnly) come membro di quel gruppo
3. Da Security & Compliance PowerShell, assegna il GRUPPO (non il service principal) al ruolo: `Add-RoleGroupMember -Identity "Compliance Administrator" -Member "<nome del gruppo di sicurezza>"`

Limite aggiuntivo per AppOnly, dalla stessa fonte: anche dopo aver assegnato il ruolo, l'autenticazione app-only (certificato) copre solo i permessi Application - non sufficiente per alcuni cmdlet di ricerca compliance specifici (es. `Start-ComplianceSearch`, `New-ComplianceSearchAction`), che richiederebbero permessi Delegated anche su un tenant configurato AppOnly. Non ancora verificato se questa limitazione tocca anche `Get-RetentionCompliancePolicy` (lettura semplice, non una ricerca) - da confermare quando il ruolo sara' assegnato sul tenant di test.

Verificato dal vivo il 18/08/2026 SENZA questo ruolo: `Connect-IPSSession` autentica correttamente (nessun errore di connessione) ma espone SOLO i cmdlet coperti dai ruoli gia' assegnati - sul tenant di test, solo quelli di Quarantena (`Get-QuarantineMessage` e affini, gia' funzionanti). `Get-RetentionCompliancePolicy` risulta quindi "term not recognized", lo STESSO messaggio che comparirebbe per un nome di comando sbagliato - a differenza di SharePoint/Teams, che rispondono con un errore di permesso esplicito ("Unauthorized"/"Access Denied"). Se la lettura fallisce con questo messaggio, la causa piu' probabile e' il ruolo mancante, non un errore di battitura nel nome del cmdlet.

5. Certificato ed Exchange.ManageAsApp

Exchange Online PowerShell in modalità app-only **non supporta i client secret**: richiede autenticazione a certificato.

5.1 Generazione del certificato (una tantum, sul PC amministrativo)

```
$cert = New-SelfSignedCertificate -Subject "CN=M365Ops-Exchange" `
    -CertStoreLocation "Cert:\CurrentUser\My" -KeyExportPolicy Exportable `
    -KeySpec Signature -KeyLength 2048 -KeyAlgorithm RSA -HashAlgorithm SHA256

Export-Certificate -Cert $cert -FilePath "Config\M365Ops-Exchange.cer"
$cert.Thumbprint # salva questo valore: serve al passo 5.3
```

Il file `.cer` (chiave pubblica, nessun segreto) va caricato sul portale; la chiave privata resta nel certificate store di Windows di questo PC e non va mai copiata altrove.

5.2 Caricamento su Entra ID

1. App registration → Certificates & secrets → tab **Certificates** → Upload certificate
2. Seleziona `Config\M365Ops-Exchange.cer`

5.3 Permessi Exchange.ManageAsApp

Non è un permesso Microsoft Graph — è un'API separata, si trova in un punto diverso del portale.

1. API permissions → **Add a permission**
2. Tab **"APIs my organization uses"** (non "Microsoft APIs") → cerca **"Office 365 Exchange Online"**
3. **Application permissions** (non Delegated) → spunta **Exchange.ManageAsApp** → Add permissions
4. **Grant admin consent**

6. Ruoli RBAC su Exchange Online

Avere il permesso `Exchange.ManageAsApp` non basta: il service principal dell'app deve anche essere **registrato dentro Exchange Online** e ricevere un ruolo RBAC. Questo passo richiede una sessione interattiva come Global/Exchange Admin (non può essere automatizzato dall'app stessa, che gira app-only).

```
# una tantum, sul PC amministrativo
Install-Module ExchangeOnlineManagement -Scope CurrentUser
Connect-ExchangeOnline -UserPrincipalName tuo-admin@contoso.onmicrosoft.com
```

6.1 Registrare il service principal in Exchange Online

Serve l'**Object ID della Enterprise Application** (Entra ID → Enterprise applications → cerca il nome dell'app → Overview → Object ID) — è diverso dall'Object ID della App Registration.

```
New-ServicePrincipal -AppId "00000000-0000-0000-0000-000000000001" `
-ObjectId "<Object-ID-della-Enterprise-Application>" `
-DisplayName "M365Ops"
```

6.2 Assegnazione del ruolo — due opzioni

Opzione A — Full admin (tenant di test) usata su contoso-test

```
Add-RoleGroupMember "Organization Management" -Member "M365Ops"
```

Il ruolo più ampio disponibile. Comodo in un tenant di test dove non serve calibrare i permessi, ma va evitato su un tenant cliente reale in produzione.

Opzione B — Permessi minimi (consigliata per tenant di produzione)

```
New-RoleGroup -Name "M365Ops-ReadOnly" `
-Roles "View-Only Recipients","View-Only Configuration" `
-Members "M365Ops"
```

Sufficiente per le funzionalità attuali (lettura mailbox condivise e permessi FullAccess/SendAs/ SendOnBehalf), che sono di sola lettura.

6.3 Verifica

```
Get-ServicePrincipal | Where-Object DisplayName -eq "M365Ops"
Get-RoleGroupMember "Organization Management" # o "M365Ops-ReadOnly"
Get-ManagementRoleAssignment -RoleAssignee "M365Ops" # conferma i ruoli effettivi assegnati
Disconnect-ExchangeOnline -Confirm:$false
```

Poi, dall'app, tab Manutenzione/MCP → pulsante "Test connessione Exchange".

Attenzione ai tempi di propagazione. Anche se `Get-RoleGroupMember` mostra subito il service principal come membro, l'assegnazione RBAC può impiegare **fino a 60 minuti** (tipicamente 15-30) prima di diventare effettiva per l'autenticazione app-only/certificato. In questa finestra la connessione fallisce con:

```
Error: The user "<tenant>\<AppId>" isn't assigned to any management roles.
```

Non è un errore di configurazione: se `Get-ManagementRoleAssignment -RoleAssignee` restituisce righe, basta attendere e riprovare — non serve rifare i passi precedenti.

6.4 Alcuni cmdlet restano "term not recognized" anche con Organization Management (scoperto dal vivo il 20/08/2026)

Aggiornamento importante (21/08/2026): questa indagine si e' rivelata in parte una pista sbagliata. Il caso `Receive-/SendConnector` descritto qui sotto NON era affatto un problema di RBAC - erano semplicemente i cmdlet SBAGLIATI, esclusivi di Exchange on-premises e mai esistiti in Exchange Online (vedi il riquadro dedicato piu' sotto e la sezione 17.15). Il caso `New-M365OpsAcceptedDomain` invece resta un genuino limite RBAC/modalita' di autenticazione, ora CONFERMATO (non piu' un'ipotesi): funziona in modalita' Delegated, non in App-only. I due cmdlet producevano lo stesso identico messaggio di errore testuale ("term not recognized") per due cause completamente diverse - la lezione utile qui e' che un messaggio di errore identico non garantisce una causa comune, verificare sempre ogni caso separatamente prima di generalizzare una diagnosi.

Bug reale riscontrato testando `New-M365OpsAcceptedDomain` e le cmdlet `Receive-/SendConnector`: il service principal era già membro del role group **"Organization Management"** (sezione 6.2, opzione A) - eppure questi cmdlet restituivano comunque `"term not recognized"`, come se il ruolo non fosse affatto assegnato.

Verificato sulla documentazione ufficiale Microsoft ([app-only-auth-powershell-v2](#), non a memoria): per l'autenticazione app-only/certificato esistono **due meccanismi RBAC distinti**, non uno solo:

- **Opzione 2 (quella usata finora in questa guida, sezione 6.2):** assegnare il service principal a un role group Exchange interno (`Add-RoleGroupMember "Organization Management"`).
- **Opzione 1:** assegnare un vero **ruolo directory Microsoft Entra ID** all'app (Exchange Administrator, Global Administrator, ecc.).

Microsoft dichiara esplicitamente: *"Global Administrator e Exchange Administrator forniscono i permessi necessari per QUALSIASI task in Exchange Online PowerShell... Security Administrator NON ha i permessi necessari per gli stessi task."* I due meccanismi si **sommano** (RBAC combina i permessi da tutte le fonti) - aggiungere il ruolo directory non toglie nulla di già configurato con il role group.

Passi per assegnare il ruolo directory Entra ID

1. Vai su entra.microsoft.com (o portal.azure.com) → cerca **"Ruoli e amministratori"** ("Roles and administrators").
2. Cerca e clicca sul **nome** del ruolo **"Exchange Administrator"** (non la casella di spunta).
3. Nella pagina "Assegnazioni" → **"Aggiungi assegnazioni"**.
4. Cerca e seleziona l'App Registration di M365Ops (nome dell'app in Entra ID - può differire dal `DisplayName` con cui è registrata dentro Exchange via `New-ServicePrincipal`, sezione 6.1).
5. Conferma, poi attendi la propagazione (stessa finestra fino a 60 minuti di sopra).

ancora irrisolto Assegnato sul tenant di test (app registrata come "intune-ai-interface", ruolo directory Exchange Administrator). Nota onesta su un errore di questa stessa guida: una versione precedente di questa sezione affermava "verificato oltre 12 ore dopo l'assegnazione" - un tempo trascorso MAI verificato in modo indipendente (nessun log/timestamp consultato per calcolarlo), solo assunto in modo sbagliato leggendo il contesto della conversazione. Corretto dopo una segnalazione esplicita dell'utente ("il ruolo è stato aggiunto circa un'ora fa non 12 ore"): riprovato a ~1 ora dall'assegnazione reale (quindi entro o appena oltre la finestra di 60 minuti documentata sopra, non ben oltre come scritto prima) - `New-AcceptedDomain` e `Get-ReceiveConnector` restituiscono ancora **"term not recognized"**. A quell'orizzonte temporale la sola propagazione resta un'ipotesi plausibile (non ancora esclusa con certezza come lo sarebbe stata a 12 ore reali) - i due controlli sotto restano comunque validi ed erano già stati fatti indipendentemente dal tempo trascorso:

- Verificato via Graph (`GET /servicePrincipals/{id}/roleManagement/directory/roleAssignments`) che il ruolo directory "Exchange Administrator" risulta assegnato al service principal **giusto** (stesso `AppId` configurato in `Config\tenants.json` per questo tenant - esclusa l'ipotesi "ruolo assegnato all'app sbagliata").
- Verificato lato Exchange con `Get-ManagementRoleAssignment -RoleAssignee <NomeVisualizzatoInExchange>` (attenzione: il nome da usare qui è quello mostrato da `Get-ServicePrincipal`, es. "Exchange-for-ia" in questo test - NON il `DisplayName` dell'App Registration in Entra ID, i due possono differire) che il ruolo **"Remote and Accepted Domains"** (che dovrebbe coprire proprio `New-AcceptedDomain`) risulta effettivamente assegnato via il role group "Organization Management".

Quindi: sia il ruolo directory Entra ID sia il ruolo RBAC Exchange specifico risultano correttamente assegnati all'app giusta, eppure il cmdlet resta non esposto nella sessione app-only/certificato.

CONFERMATO dal vivo il 21/08/2026 (utente: "puoi fare test in modalita' delegata?"): login interattivo reale completato con `diego@vnsys.it` (membro di "Organization Management", verificato con `Get-ManagementRoleAssignment` - decine di ruoli, incluso "Remote and Accepted Domains") - `Get-AcceptedDomain` funziona immediatamente in modalita' **Delegated**, restituendo i 5 domini reali del tenant. L'ipotesi delle sessioni app-only/CBA limitate a un sottoinsieme curato di cmdlet, indipendente dall'RBAC assegnato, e' quindi confermata per `New-/Set-/Remove-/Get-AcceptedDomain` : questi cmdlet richiedono una sessione Delegated, non sono raggiungibili in App-only indipendentemente da quanti ruoli si assegnano al service principal. **Stato finale**: in App-only resta bloccato (limite della piattaforma, non un bug di questo progetto); in Delegated funziona. Chi ha bisogno di gestire Accepted Domain da M365Ops deve usare un profilo tenant in modalita' Delegated (sezione 10) invece di App-only.

Il caso Receive-/SendConnector era tutt'altro: un cmdlet sbagliato, non un limite RBAC. Verificato dal vivo lo stesso giorno: lo stesso "term not recognized" su `Get-ReceiveConnector` compare ANCHE in modalita' Delegated con `diego@vnsys.it` (membro completo di Organization Management) - se fosse stato un limite RBAC o di sessione app-only, un utente con quel livello di privilegio avrebbe dovuto riuscire. Causa reale, confermata su documentazione ufficiale Microsoft (non a memoria): `Get-/New-/Set-/Remove-ReceiveConnector` e `...SendConnector` sono cmdlet **esclusivi di Exchange on-premises**, non sono MAI esistiti in Exchange Online - "this cmdlet is available only in on-premises Exchange" ([Get-ReceiveConnector, Microsoft Learn](#)). Il vero equivalente Exchange Online e' `Get-/New-/Set-/Remove-InboundConnector` e `...OutboundConnector` - nomi diversi, non alias, con parametri diversi (es. `SenderDomains` invece di `Bindings / RemoteIPRanges`, che sono concetti legati a un server fisico che non esiste in cloud). Corretto sostituendo tutti gli 8 wrapper del modulo (`Get-/New-/Set-/Remove-M365OpsInboundConnector` e `...OutboundConnector`, sezione 17.15) - testato dal vivo con un ciclo completo create/verifica/modifica/verifica/rimuovi/verifica per entrambi, IN APP-ONLY (nessun limite di sessione per questi, a differenza di Accepted Domain sopra): funzionano perfettamente.

6.5 Conflitto assembly .NET passando da App-only a Delegato nello stesso processo

Scoperto dal vivo il 21/08/2026: connettendosi a Exchange Online come Delegato (login interattivo) DOPO che lo stesso processo server aveva già usato una connessione App-only (certificato), la connessione fallisce con:

```
Errore: Token ottenuto ma la connessione a Exchange e' fallita: Could not load type
'Microsoft.IdentityModel.Telemetry.TelemetryClient' from assembly 'Microsoft.IdentityModel.Tokens,
Version=8.2.0.0, Culture=neutral, PublicKeyToken=31bf3856ad364e35'.
```

Causa: conflitto di VERSIONE di una DLL condivisa, non un bug di questo progetto. I due percorsi di autenticazione (certificato vs interattivo/MFA) di [ExchangeOnlineManagement](#) caricano versioni diverse di [Microsoft.IdentityModel.Tokens](#) (una dipendenza condivisa con MSAL) - .NET non può avere due versioni della stessa DLL caricate insieme nello stesso processo. Una volta che una versione è stata caricata (dalla PRIMA connessione, di qualunque tipo), ogni tentativo successivo che ne richiederebbe una diversa fallisce con un errore di tipo/assembly poco chiaro come questo.

Unico rimedio reale: un processo NUOVO. Riavviare il server libera completamente il problema (nessuna DLL ancora caricata) - verificato dal vivo: lo stesso login Delegato che falliva ha funzionato subito dopo un riavvio pulito del server, PRIMA di qualunque altra connessione Exchange nello stesso nuovo processo.

Guardrail aggiunto lo stesso giorno (richiesto esplicitamente dall'utente: "quando c'è un'applicazione attiva e clicco su attiva la modalità delegata, abbi cura di disconnettere tutto quanto connesso prima"): [Connect-M365Ops](#) ora disconnette esplicitamente Teams ([Disconnect-M365OpsTeams](#), nuovo) e SharePoint ([Disconnect-M365OpsSharePoint](#), nuovo) ad ogni cambio di profilo tenant, oltre a Exchange e Lokka che già lo facevano.

Onestà tecnica su questo guardrail: chiudere una SESSIONE (Disconnect-ExchangeOnline ecc.) non annulla una DLL già caricata nel processo - il guardrail riduce il rischio di stato di sessione del tenant precedente rimasto attivo per sbaglio (comunque corretto di per sé), ma NON è una soluzione completa al conflitto di assembly sopra. Se il problema si ripresenta passando da un tenant all'altro con AuthMode diversa, il riavvio del server resta l'unico rimedio certo.

6.6 Conflitto DIVERSO: MicrosoftTeams e ExchangeOnlineManagement - RISOLTO DEFINITIVAMENTE con isolamento reattivo dei processi (26/08/2026, storia della diagnosi precedente lasciata sotto invariata)

Soluzione definitiva (v0.9.59, 26/08/2026), sostituisce il pin di versione sotto come rimedio primario: i pin di versione 3.4.0/6.5.0 documentati qui sotto restano installati per default (nessuna ragione per usare qualcos'altro), ma non sono più l'unico argine al conflitto - se scatta comunque (PC con versioni residue, o una futura combinazione mai testata), [Connect-M365OpsExchange](#) / [Connect-M365OpsTeams](#) / [Connect-M365OpsCompliance](#) ora tentano automaticamente un **isolamento reattivo**: la connessione che avrebbe generato il conflitto (Exchange o Teams, quale dei due arriva secondo) viene spostata in un sottoprocesso `pwh.exe` SEPARATO (`Private\Workers\M365OpsIsolatedWorker.ps1`), che importa SOLO quel modulo - i due moduli non condividono più mai lo stesso processo .NET, quindi il conflitto non può semplicemente più verificarsi. Da quel momento, ogni cmdlet del modulo isolato (Get-Mailbox, Get-DistributionGroup, Get-CASMailbox ecc. - verificato dal vivo un elenco di 835 cmdlet proxyati automaticamente sul tenant di test, non una lista scritta a mano) viene intercettato da una funzione proxy generata dinamicamente e inoltrato al sottoprocesso via JSON-RPC su stdio (stesso protocollo già usato per Lokka/CLI Microsoft 365), **senza che l'utente debba riavviare il server** - il messaggio "riavvia il server" di questa sezione resta solo per i casi non coperti (vedi limiti sotto). Attivazione sempre e solo REATTIVA, mai preventiva: un utente che usa un solo modulo per tutta la sessione non paga mai il costo di un secondo processo.

Limiti noti di questa prima versione, verificati dal vivo sul tenant "vnsys-test", non ipotizzati:

- **Solo App-only per ora.** Un tenant Delegato continua a vedere il classico messaggio "riavvia il server" - il token Exchange ottenuto via device-code non è oggi salvato in nessuna cache riutilizzabile dal sottoprocesso isolato; estenderlo richiede un login dedicato nel worker, non ancora costruito.
- **Cmdlet Purview "classici" (es. [Get-RetentionCompliancePolicy](#)) possono risultare assenti anche quando [Connect-IPPSession](#) non lancia nessun errore** - scoperto dal vivo durante la verifica di questa stessa funzionalità, e confermato con un test MINIMALE, del tutto fuori dall'architettura di isolamento (solo `Import-Module + Connect-ExchangeOnline + Connect-IPPSession` puri): su questo tenant/ questa versione del modulo, [Connect-IPPSession](#) riporta successo ma [Get-PSSession](#) subito dopo non mostra nessuna sessione remota attiva - la sessione di implicit remoting "classica" da cui derivano quei nomi di cmdlet non viene proprio stabilita. Non è un bug di questo progetto né dell'isolamento (riprodotto identico SENZA isolamento): è coerente con quanto già documentato più sotto in questa stessa sezione, che [ExchangeOnlineManagement 3.2.0+](#) ha spostato Security & Compliance sulle REST API - i vecchi nomi cmdlet legati al protocollo di remoting dismesso semplicemente non vengono più iniettati allo stesso modo. I cmdlet mailbox (Get-Mailbox, Get-CASMailbox, Get-DistributionGroup ecc., che NON dipendono da questa sessione) restano invece pienamente funzionanti - verificato con chiamate ripetute (12 invocazioni consecutive, incluse chiamate interlacciate a più cmdlet diversi) senza nessun errore.
- **Connessione al worker isolato più lenta della connessione diretta** (tipicamente 1-2 minuti la prima volta per tenant/modulo: avvio del processo + [Connect-ExchangeOnline](#) + [Connect-IPPSession](#) + attesa di stabilizzazione dell'elenco comandi) - accettabile perché succede una sola volta per tenant per la vita del processo server, non ad ogni comando.
- **CORRETTO IL 26/08/2026 (v0.9.60), bug reale trovato dal vivo durante un bug-hunt dedicato di 3 ore:** il meccanismo sopra scattava SOLO quando il conflitto si manifestava durante la CONNESSIONE stessa (dentro il blocco catch di [Connect-M365OpsExchange/Teams/ Compliance](#)) - ma Teams ed Exchange possono connettersi ENTRAMBI con successo nello stesso processo (nessun errore in quel momento) e il conflitto manifestarsi solo DOPO, su una scrittura specifica: riprodotto dal vivo con `Set-Team`, fallito con `"Method not found: 'Void Microsoft.Identity.Client.ITokenCache.Deserialize(Byte[])'"` - una TERZA variante del messaggio .NET per lo stesso conflitto (le prime due, "Could not load file or assembly" e "The given assembly name is invalid", sono fallimenti di CARICAMENTO assembly; questa è un [MissingMethodException](#) - l'assembly incompatibile è già caricato, solo quel metodo specifico non esiste più in quella versione). [Get-M365OpsModuleConflictHint](#) (Private) ora riconosce anche questo pattern ("Method not found" + "Microsoft.Identity.Client" nello stesso messaggio, non un singolo pattern fisso). Più in profondità: `Execute-PendingAction (Gui\Server.ps1)`, per i casi `ExoWrite / TeamsWrite`, ora passa dalla nuova `Invoke-M365OpsWriteWithIsolationRecovery` (Public) - se la scrittura fallisce con un conflitto riconosciuto, tenta l'isolamento reattivo ADESSO (anche se il connect era già riuscito prima) e RIPROVA la stessa scrittura una sola volta attraverso il processo isolato, invece di limitarsi a mostrare l'errore .NET grezzo. **Onestà tecnica su questo fix:** il percorso "nessun conflitto, tutto ok al primo tentativo" e la logica di riconoscimento pattern sono verificati dal vivo (test unitari mirati + una scrittura Teams reale completata con successo attraverso il wrapper); il percorso di RETRY specifico (fallimento reale seguito da isolamento-e-risuscita-al-secondo-tentativo) non è stato ri-riprodotto forzatamente in questo giro, perché il conflitto stesso è intrinsecamente non deterministico (dipende dalle versioni esatte installate in quel momento, stessa causa già documentata sopra per il conflitto originale) - il meccanismo di retry riusa però ESATTAMENTE lo stesso [Connect-M365OpsIsolatedModule](#) già verificato a fondo poco sopra in questa sezione, quindi il rischio residuo è basso, non assente. **AGGIORNAMENTO, scoperto ancora dal vivo pochi minuti dopo aver scritto il paragrafo sopra, nella STESSA sessione di test:** il conflitto di questo tipo, una volta scattato una prima volta nel processo, ha lasciato un effetto collaterale più ampio di quanto creduto inizialmente - non solo la singola scrittura che lo ha fatto emergere (recuperata con successo dal meccanismo sopra), ma ANCHE letture Exchange completamente indipendenti, eseguite diversi minuti dopo, hanno cominciato a fallire con un errore DIVERSO ma correlato (`IDX12729` , un errore di decodifica token JWT di [Microsoft.IdentityModel](#) - stessa famiglia di librerie di autenticazione condivisa, ennesima variante dello stesso problema di fondo). Confermato con un confronto diretto: la STESSA query, in un processo PowerShell completamente nuovo (fuori dal server), ha funzionato subito senza alcun errore - la degradazione è quindi specifica del processo server già "toccato" dal conflitto, non un problema di dati o di rete. Un riavvio completo del server ha risolto tutto immediatamente (stessa query, stesso processo dopo il riavvio: risultato corretto). **Conclusione onesta:** il meccanismo di recupero sopra riduce concretamente l'impatto immediato (l'azione confermata dall'utente va comunque a buon fine, verificato dal vivo), ma NON è una cura

completa per il processo nel suo complesso - un conflitto reale, una volta scattato, può lasciare un'instabilità residua su chiamate successive non correlate. La guida raccomanda quindi ancora, come indicazione prudente dopo aver visto comparire QUALUNQUE errore di questa famiglia (assembly/versione/token JWT legati a Microsoft.Identity.Client o Microsoft.IdentityModel), un riavvio completo del server come passo successivo, anche se l'azione specifica che ha innescato l'errore sembra essere andata a buon fine grazie al recupero automatico. Rimane inoltre un limite strutturale non ancora coperto: cmdlet di SOLA LETTURA che incontrano lo stesso conflitto dopo un connect riuscito (non solo le scritture) non passano ancora dal meccanismo di recupero con retry - se capita, il messaggio resta quello grezzo (o la variante IDX12729 sopra) finché non viene esteso anche a quel percorso.

Verificato dal vivo end-to-end, sul tenant reale "vnsys-test", non con credenziali finte: connesso Teams per primo (direzione nota per rompersi), poi Exchange - isolamento attivato automaticamente, confermato nel log ("Isolamento reattivo attivato per Exchange - 835 cmdlet ora eseguiti in un processo separato"); Get-Mailbox / Get-DistributionGroup / Get-CASMailbox hanno restituito dati reali attraverso il proxy; Teams è rimasto funzionante in-process per tutta la durata (mai isolato, non necessario); il worker isolato si chiude correttamente alla disconnessione/cambio tenant.

Un secondo conflitto di assembly, distinto da 6.5 (quello sopra e' fra due MODALITA' di autenticazione dello STESSO modulo Exchange; questo e' fra DUE MODULI diversi, indipendente dalla modalita' Delegata/App-only): connettersi a Microsoft Teams e poi a Exchange Online nello STESSO processo server (o viceversa) poteva fallire con un errore analogo - Microsoft.IdentityModel.Abstractions se Teams e' stato connesso per primo, Microsoft.Identity.Client se Exchange e' stato connesso per primo.

Storia completa della diagnosi, lasciata visibile perche' istruttiva - il percorso e' stato lungo e con piu' false piste corrette man mano dall'utente: (1) creduto inizialmente un conflitto Microsoft inevitabile, in ENTRAMBE le direzioni, con qualunque versione; (2) l'utente ha corretto questa diagnosi ("ieri ho connesso tutto senza errori, non dipende dalla versione dei moduli?") - confermato che dalla versione **3.10.0** di ExchangeOnlineManagement in poi il conflitto e' effettivamente SEMPRE presente, in entrambe le direzioni, qualunque versione di Teams - quella soglia resta vera; (3) un primo pin a 3.9.0 (senza pin su Teams) e' sembrato risolvere il problema su due versioni Teams testate, poi smentito da una matrice piu' ampia (Teams 7.1.0/7.3.1/7.9.0) - nessun ordine era affidabilmente sicuro con quel pin; (4) rimossa ogni guardia preventiva su richiesta dell'utente, sostituita con un messaggio reattivo; (5) l'utente ha insistito, correttamente, che "fino all'altro giorno" tutto funzionava in delegato/device-code, e ha guidato la ricerca verso versioni piu' vecchie di ENTRAMBI i moduli - analizzata la libreria di autenticazione condivisa (Microsoft.Identity.Client.dll) bundled in OGNI versione rilasciata di entrambi i moduli (16 versioni Exchange, 16 versioni Teams): nessuna coppia condivide la build esatta in nessun punto della storia di nessuno dei due moduli, ma la coppia **ExchangeOnlineManagement 3.4.0 + MicrosoftTeams 6.5.0**, con Exchange connesso PRIMA di Teams, e' risultata verificata come sicura in un test dal vivo completo di tutti i flussi del progetto (App-only, login delegato device-code via -AccessToken , Purview App-only, poi Teams App-only connesso dopo tutti e tre) - nessun conflitto in nessuno dei quattro passaggi, ripetuto due volte.

Fissata la coppia di versioni, non solo una delle due (v0.9.47): Assert-M365OpsExoSafeVersion (ExchangeOnlineManagement 3.4.0) e Assert-M365OpsTeamsSafeVersion (MicrosoftTeams 6.5.0, nuova funzione) garantiscono entrambe che la propria versione sia installata E integra (stesso controllo di integrita' dei file, non solo del manifest, introdotto dopo il bug "The given assembly name was invalid" - vedi sotto), la importano e riparano da sole un'installazione danneggiata quando e' ancora possibile farlo (PRIMA di qualunque Import-Module). Assert-M365OpsExoSafeVersion disinstalla anche attivamente qualunque versione ≥3.10.0 trovata sul disco - quella soglia resta l'unica certezza universale di questo conflitto, indipendente dalla versione di Teams. 3.4.0 e' stata scelta deliberatamente come la piu' vecchia versione ExchangeOnlineManagement che supporta TUTTO cio' che serve al progetto (-CertificateThumbprint per App-only, -AccessToken per il login delegato device-code, -CertificateThumbprint su Connect-IPPSession per Purview App-only) - non e' stata scelta a caso, e' la piu' lontana possibile (in termini di eta'/dipendenze) dalla soglia 3.10.0. 6.5.0 e' stata preferita a 5.0.0 (anch'essa verificata sicura nella stessa coppia) perche' piu' recente a parita' di Microsoft.Identity.Client bundled (condiviso da tutte le versioni Teams 4.0.0-6.5.0) - piu' bugfix, e predata anche il passaggio di Connect-MicrosoftTeams a WAM come broker di default (sezione 10.6/17.14, introdotto in 7.8.1-preview), quindi non ne soffre per costruzione. La guardia preventiva resta rimossa (decisione dell'utente: "non voglio il safe guard, nelle versioni vecchie funzionava") - Get-M365OpsModuleConflictHint (Private) resta come rete di sicurezza reattiva per l'ordine inverso o per PC con versioni residue diverse. **Verifica finale (v0.9.48) fatta con il tenant di test REALE attraverso la GUI vera**, non solo con credenziali finte in script isolati: cliccati in sequenza i pulsanti Purview, Exchange e Teams nel tab Tenant sul tenant vnsys-test - tutti e tre connessi con successo nello stesso processo server, con dati reali restituiti (mailbox condivise, Team elencati), non solo "nessun errore".

PERCHE' 3.4.0 e non 3.1.0 (che sulla carta sarebbe la piu' vecchia con tutto cio' che serve) - scoperto testando con CREDENZIALI REALI, non finte: un primo pin a 3.1.0 e' stato scelto e poi scartato lo stesso giorno dopo un test dal vivo sul tenant vnsys-test reale - Connect-IPPSession in App-only su 3.1.0 falliva con un errore WS-Management/WinRM ("Connecting to remote server ps.compliance.protection.outlook.com failed..."), non un conflitto di assembly: 3.1.0 usa ancora il vecchio protocollo Remote PowerShell (RPS) per Security & Compliance, che Microsoft ha dismesso lato server dal 15 luglio 2023 (banner ufficiale mostrato dal modulo stesso alla connessione) - un endpoint che non esiste piu', nessuna correzione lato nostro puo' farlo funzionare. I test con credenziali FINTE fatti fino a quel momento non lo avevano mai scoperto: fallivano prima, sul controllo locale del certificato, senza mai arrivare alla vera chiamata di rete - lezione diretta: un test con credenziali finte verifica l'assenza del conflitto di assembly, non che la funzionalita' sia davvero utilizzabile. ExchangeOnlineManagement 3.2.0 ha spostato Security & Compliance sulle REST API (stesso banner del modulo) - 3.4.0 (ultima versione della fascia 3.1.0-3.4.0 che condivide la stessa build di Microsoft.Identity.Client , quindi eredita la stessa garanzia di non conflitto con Teams senza dover essere riverificata da zero) e' stata poi riverificata con SUCCESSO REALE: connessione Purview autentica sul tenant di test, non solo "nessun errore".

Login delegato a Purview: implementato via popup interattivo, non device-code (v0.9.48) - Connect-IPPSession su questa versione fissata non supporta ALCUN flusso device-code/token (ne' -Device ne' -AccessToken , solo -UserPrincipalName / -Credential o i parametri App-only), a differenza di Connect-ExchangeOnline che sulla stessa versione supporta gia' -AccessToken perfettamente - limite reale del modulo su questa versione, non un bug. -UserPrincipalName da solo pero' attiva un vero prompt di autenticazione moderna (confermato dalla guida integrata del modulo: "allows you to skip entering a username in the modern authentication credentials prompt") - un popup bloccante, stesso principio gia' usato per Teams/SharePoint delegati (gated da -AllowInteractive , mai avviato da solo). Sicuro dal punto di vista del conflitto Teams perche' passa dalla STESSA libreria di autenticazione condivisa gia' verificata pulita per Purview App-only nella stessa coppia di versioni - non e' un percorso di codice diverso, solo un modo diverso di popolare le credenziali. Connect-M365OpsCompliance.ps1 lo sceglie dinamicamente (controlla se -Device esiste prima di usarlo, altrimenti usa solo -UserPrincipalName). **Novita' collaterale scoperta nello stesso giro:** Purview in modalita' Delegata non aveva NESSUN punto di ingresso in GUI prima di questo fix (Get-M365OpsRetentionCompliancePolicies chiamava Connect-M365OpsCompliance senza -AllowInteractive , quindi falliva sempre con "vai a connetterlo" senza che esistesse davvero un modo di farlo) - aggiunto un riquadro "Purview / Compliance" nel tab Tenant, stesso schema di Exchange/SharePoint/Teams, con nuovo endpoint POST /api/compliance-test .

Regola pratica: connetti sempre Exchange (o Purview) PRIMA di Teams nella stessa sessione del server - in questo ordine, con questa coppia di versioni, funziona senza conflitti in ogni modalita' testata. Se hai gia' installato questo progetto e aggiorni da una versione precedente, la prima connessione Exchange/Teams dopo l'aggiornamento allinea da sola i moduli sul disco (installa/ripara le versioni corrette, rimuove una eventuale ≥3.10.0) - **ma serve un riavvio COMPLETO del processo server, non solo un refresh del browser:** la pagina index.html (compresi questi avvisi) viene letta una sola volta all'avvio del server e tenuta in memoria per tutta la sua vita (sezione 12.2).

BUG DISTINTO scoperto durante questa indagine, gia' corretto: un errore separato, "The given assembly name was invalid.", poteva comparire anche SENZA alcun coinvolgimento di Teams - un'installazione locale di ExchangeOnlineManagement danneggiata (Get-Module -ListAvailable legge solo il manifest, non verifica che ogni DLL referenziata esista e sia integra). Entrambe le funzioni Assert-M365Ops*SafeVersion ora verificano l'integrita' dei file critici PRIMA di importare, quando e' ancora possibile ripararla - una volta che Import-Module ha caricato l'assembly nel processo, Windows blocca quei file a livello di file system e nemmeno Remove-Item puo' piu' toccarli, verificato dal vivo.

7. Registrazione del tenant nel modulo

Il file `Config\tenants.json` è la fonte di verità per ogni tenant configurato. **Non contiene mai segreti**: solo ID, thumbprint del certificato (non è un segreto) e il NOME della variabile d'ambiente che contiene il client secret. Tutta questa sezione si fa dalla GUI, tab **Tenant** → riquadro "Aggiungi / aggiorna profilo" — non serve aprire un terminale.

Messaggio di benvenuto al primo avvio (aggiunto il 21/08/2026): finché `Config\tenants.json` non contiene nessun profilo, la chat mostra un messaggio di onboarding invece del messaggio normale - indirizza al tab Tenant e riassume le due modalità di autenticazione (Delegata vs App Registration), con un cenno ai permessi da assegnare per App Registration e al fatto che Exchange Online richiede in più il certificato. Sparisce da solo non appena si salva il primo profilo, nessuna azione manuale da fare.

1. **Nome profilo** — un nome a scelta per riconoscerlo (es. `contoso-test`).
2. **Tenant ID** — dominio `....onmicrosoft.com` o GUID del tenant, indifferentemente.
3. **Modalità di autenticazione** — App-only (richiede App Registration, sezioni 4-6) o Delegata (login col tuo utente, sezione 10).
4. Se App-only: **Client ID, nome variabile d'ambiente per il secret** (es. `M365_CLIENT_SECRET`) e — nel campo separato subito sotto — **il valore del secret stesso**, copiato dal portale Entra ID: il campo lo scrive direttamente nella variabile d'ambiente del tuo utente Windows (mai in un file), lascialo vuoto per non toccare un valore già impostato in precedenza.
5. **Thumbprint certificato Exchange** (solo App-only, facoltativo se non usi Exchange).
6. **Salva profilo** → il tenant appare nell'elenco sopra: clicca **Attiva** per connetterti (equivalente a `Connect-M365Ops`).

Il mittente email (per l'invio report) e la chiave del motore AI (`ANTHROPIC_API_KEY` o le variabili Azure OpenAI) si impostano allo stesso modo, ciascuno dal proprio tab (rispettivamente "Email" e "Motore AI") — sempre un campo di testo, mai una variabile d'ambiente da impostare a mano.

Aggiungere un secondo tenant (in parallelo, senza toccare il primo)

Stesso riquadro, di nuovo: compila i campi con i dati del nuovo tenant (nome profilo diverso, propria App Registration/secret) e Salva — il profilo precedente resta intatto e selezionabile in qualsiasi momento dall'elenco.

Ogni tenant ha la propria App Registration, il proprio certificato Exchange, il proprio mittente email e i propri connettori MCP (sezione 9) — tutto annidato nel *proprio* oggetto dentro `tenants.json`, senza condivisione con altri profili.

8. Motore AI (Claude / Azure OpenAI)

Provider	Variabili richieste	Tool-calling reale (chat con dati)
Claude	ANTHROPIC_API_KEY	implementato
Azure OpenAI	AZURE_OPENAI_KEY , AZURE_OPENAI_ENDPOINT , AZURE_OPENAI_DEPLOYMENT	implementato

Il provider si sceglie dal tab "Motore AI" delle impostazioni dell'app (persistito su `Config\ai-provider.json` — sopravvive al riavvio del server) e vale per **tutto**: sia la chat libera con accesso reale ai dati (`Invoke-M365OpsAgentTools`) sia le chiamate singole senza strumenti (analisi pattern, diagnosi/correzione errori). Non è più vero, dal 15/08/2026, che la chat richieda sempre Claude a prescindere dal selettore. Claude e Azure OpenAI sono due alternative complete e alla pari: nessuno dei due è un "default" consigliato, la scelta è sempre e solo dell'utente (vedi anche v0.9.26 in changelog).

8.1 Due API diverse, stesso set di strumenti

Claude (Anthropic Messages API) e Azure OpenAI (Chat Completions API, parametro `tools / tool_choice`) hanno una forma di richiesta e risposta strutturalmente diversa — verificato sulla documentazione Microsoft Learn reale, non assunto per analogia. `Invoke-M365OpsAgentTools` gestisce questa differenza ramificando **solo** la parte che parla con l'API (costruzione della richiesta, lettura della risposta); le **definizioni degli strumenti** (quali dati leggono/scrivono) e la logica che li esegue restano un unico blocco condiviso tra i due provider — ogni chiamata a uno strumento, da qualunque provider arrivi, viene prima ricondotta alla stessa forma interna prima di essere eseguita, così aggiungere uno strumento nuovo richiede una sola modifica, non due.

8.2 Un limite reale di Azure OpenAI: 1024 caratteri per descrizione strumento

Azure OpenAI rifiuta (o tronca, a seconda del client) una descrizione di funzione oltre i 1024 caratteri — Claude non ha questo limite. Due strumenti di questo modulo (`exo_query` e `propose_exo_write` , che elencano l'intero catalogo delle 50+ cmdlet Exchange nella propria descrizione) superano abbondantemente quella soglia. Invece di troncarli a metà elenco perdendo cmdlet in modo silenzioso, per il ramo Azure la descrizione lunga viene accorciata nella definizione dello strumento e il contenuto integrale spostato nel messaggio di sistema della conversazione — il modello vede comunque l'elenco completo, solo in una posizione diversa della richiesta.

Bug reale corretto lo stesso giorno. La prima versione del supporto Azure OpenAI aveva un difetto di scoping identico a un altro già risolto in precedenza: `Invoke-M365OpsAgentTools` vive nel modulo, quindi il suo `$script` : è lo scope del *modulo*, non quello di `Server.ps1` — un valore di default basato su `$script:ActiveIPProvider` dentro la funzione vedeva sempre `$null` e cadeva silenziosamente su Claude, a prescindere dalla scelta reale in GUI. Corretto obbligando `Server.ps1` a passare `-Provider` in modo esplicito ad ogni chiamata, come già avviene per ogni altra funzione AI del modulo.

8.3 Configurare un deployment Azure OpenAI in Azure AI Foundry

Serve un **deployment** (non basta avere una sottoscrizione Azure) — un modello scelto e attivato dentro una risorsa Azure OpenAI/progetto Azure AI Foundry, con un nome che poi va inserito nell'app. Percorso completo, testato dal vivo il 15/08/2026:

1. Vai su ai.azure.com (Azure AI Foundry) → apri o crea un progetto.
2. **Deployments** → **Deploy a base model**.
3. Scegli un modello — vedi 8.3.1 per quale.
4. Dai un nome al deployment (può essere uguale al nome del modello, es. `gpt-5-mini`) → conferma.
5. A deployment completato ("Succeeded"), nel pannello di destra trovi i tre valori che servono all'app: **Project endpoint**, **API Key** (icona occhio per rivelarla), e il nome del deployment appena scelto.
6. Nell'app, tab **Motore AI** → provider Azure OpenAI → incolla i tre valori (endpoint, chiave, nome deployment esatto — case-sensitive) → Salva → **Testa connessione**.

8.3.1 Quale modello scegliere

Nel catalogo modelli di Azure AI Foundry, evita:

- **varianti "codex"** — specializzate per generazione di codice, non per ragionamento/uso di strumenti generico: non è il compito di questa app;
- **tier "nano"** — il più economico, ma anche il meno affidabile su orchestrazione multi-passo di strumenti (esattamente il compito di `Invoke-M365OpsAgentTools`);
- **tier "pro"** — capacità più alta ma latenza/costo maggiori, non necessari qui: la chat fa già fino a 8 giri di chiamate per una singola domanda.

Il **tier "mini"** del modello più recente disponibile nel catalogo è il compromesso giusto tra costo, velocità e affidabilità sul tool-calling — usato e verificato dal vivo con `gpt-5-mini` .

8.3.2 Due incidenti reali, entrambi corretti nel codice (non serve più pensarci)

1. Due formati di endpoint diversi. Il portale Azure mostra l'endpoint in due forme diverse a seconda di dove lo copi: quella "classica" della risorsa Azure OpenAI (`https://risorsa.openai.azure.com/`) e quella più recente "Project endpoint" di Azure AI Foundry (`https://risorsa.services.ai.azure.com/openai/v1`). Il modulo costruisce sempre la stessa chiamata REST classica (`/openai/deployments/{deployment}/chat/completions?api-version=...`), che va applicata alla *radice* della risorsa — incollare la seconda forma senza normalizzare produceva un URL con `/openai/` duplicato e un 404. Corretto: l'endpoint viene normalizzato automaticamente (rimosso un eventuale `/openai/v1` o `/openai` finale) prima di costruire la chiamata, qualunque delle due forme incolli.

2. max_tokens rifiutato dai modelli "reasoning". I modelli più recenti (serie o1/o3/o4 e famiglia gpt-5.x, incluso `gpt-5-mini`) rispondono **400 Bad Request** al parametro `max_tokens` , e vogliono `max_completion_tokens` al suo posto — non deducibile in anticipo dal solo nome del deployment. Corretto: il modulo prova prima con `max_tokens` (compatibile con i modelli più vecchi) e, solo se Azure risponde con quell'errore specifico, riprova automaticamente con `max_completion_tokens` — nessuna azione manuale richiesta, la scelta giusta viene anche ricordata tra un giro di tool-calling e il successivo nella stessa conversazione.

Testato dal vivo. Dopo questi due fix, una domanda reale ("elenco licenze del tenant") ha attraversato l'intero ciclo nuovo — costruzione della richiesta, chiamata a `graph_api_call/subscribedSkus`, e risposta finale — con Azure OpenAI (`gpt-5-mini`) come provider attivo, in circa 15 secondi.

9. Connettori MCP (Lokka) — configurazione per-tenant

Lokka è un server MCP open-source che esegue chiamate Graph/Azure REST generiche. È lo strumento **primario** usato dall'AI per leggere e proporre scritture — le cmdlet interne del modulo restano come fallback, usate solo se Lokka non copre il caso.

Sui tenant **Delegati**, Lokka (client credentials) non è mai disponibile — ma gli strumenti `graph_api_call` / `propose_graph_write` restano comunque offerti all'AI: a runtime scelgono da soli se passare da Lokka o da una chiamata diretta con `Invoke-M365OpsGraphRequest` usando lo stesso token delegato già in uso per le altre funzioni. Bug reale corretto: in una prima versione questi strumenti venivano esclusi del tutto sui tenant Delegati, impedendo di leggere dati Graph generici (es. licenze utente via `/users/{id}/licenseDetails`) nonostante il token valido fosse già disponibile.

La configurazione (comando, argomenti, mapping variabili) vive dentro la sotto-sezione `McpServers` del profilo tenant attivo in `tenants.json`, modificabile anche da GUI (tab MCP-Connettori):

```
{
  "contoso-test": {
    ...
    "McpServers": {
      "lokka": { "command": "npx", "args": "-y @merill/lokka", "envMapping": "none" }
    }
  }
}
```

Perché tenant-scoped: due tenant diversi possono avere connettori, credenziali o versioni del server MCP diverse in parallelo, senza che la configurazione dell'uno interferisca con quella dell'altro.

Anche la gestione a runtime è interamente da GUI, tab MCP-Connettori: il pulsante "🔄 **Riconnetti con la nuova configurazione**" chiama `Connect-M365OpsLokka -Force` dopo ogni modifica ai campi comando/argomenti. Ogni server elencato sotto "Altri server MCP" ha un proprio pulsante "Rimuovi" (con conferma). Lokka non è rimovibile da lì (è builtin): "rimuoverlo" significherebbe solo azzerare una personalizzazione, quindi si gestisce modificando i campi comando/argomenti invece che con una rimozione vera e propria.

10. Autenticazione delegata (utente + MFA) — tenant senza App Registration

Tutto il modulo, finora, presume che tu possa creare un'App Registration sul tenant target (sezione 4). Nella pratica capita spesso di gestire tenant dove hai ruoli admin delegati (Exchange Administrator, User Administrator, Teams Administrator...) ma **non** i permessi per registrare un'app (serve Global Administrator, Application Administrator o Cloud Application Administrator). Per questi casi esiste una seconda modalità, per-tenant, che non richiede alcuna App Registration.

10.1 Come funziona

Il profilo tenant si imposta con `AuthMode: Delegated` invece di `AppOnly`. In questa modalità il modulo non usa client credentials: usa un **login interattivo con il tuo utente** (username + password + MFA), tramite il client pubblico multi-tenant di Microsoft "Microsoft Graph Command Line Tools" (id `14d82eec-204b-4c2f-b7e8-296a70dab67e`) — un'app di Microsoft stessa, già presente in ogni tenant, che non richiede nessuna registrazione da parte tua. Il token risultante riflette esattamente i ruoli RBAC già assegnati al tuo utente su quel tenant: se hai Exchange Administrator, puoi leggere/scrivere ciò che quel ruolo permette; se non hai un ruolo su una risorsa, quella chiamata fallirà — esattamente come nel portale.

10.2 Isolamento tra tenant — non negoziabile

Un tenant AppOnly e un tenant Delegated (o due tenant Delegated diversi) non condividono MAI stato. Ad ogni cambio tenant (`Connect-M365Ops -TenantProfile ...`), il modulo:

- chiude esplicitamente la sessione Exchange Online precedente, se attiva;
- ferma il sottoprocesso Lokka precedente, se attivo.

Senza questo, Lokka (un unico sottoprocesso globale) e la sessione Exchange Online (remoting implicito, anch'esso globale) potrebbero restare agganciati al tenant precedente e far eseguire un comando pensato per il tenant appena attivato contro quello sbagliato. Il cambio tenant è sempre un'azione esplicita tua — non avviene mai automaticamente in mezzo a una conversazione.

10.3 Configurazione

Dalla GUI, tab Tenant → riquadro "Aggiungi / aggiorna profilo": "Modalità di autenticazione" → Delegata → inserisci il tuo UPN su quel tenant (es. `mario.rossi@clientey.onmicrosoft.com`) → Salva profilo → Attiva. Compare un riquadro "Login delegato" con un pulsante **"Accedi con il mio utente"**.

10.4 Primo login (device code flow)

1. Clic su "Accedi con il mio utente": l'app mostra un URL (`microsoft.com/devicelogin`) e un codice a 9 caratteri.
2. Apri l'URL in un browser (anche su un altro dispositivo), inserisci il codice, accedi con il tuo utente e completa l'MFA.
3. La GUI verifica in background (polling automatico) e conferma appena il login è completato — nessuna azione ulteriore da parte tua.

Il token viene tenuto in cache **solo in memoria** per la durata del processo (mai scritto su disco, stesso principio di zero-segreti-in-chiaro delle altre credenziali) e si auto-rinnova finché resta usato regolarmente; se il server si riavvia, va rifatto il login.

10.5 Limiti noti di questa modalità

Aspetto	AppOnly	Delegated
Serve App Registration	Sì	No
Esecuzione unattended	Sì, sempre	No — richiede un login umano ogni volta che la sessione scade
Permessi effettivi	Quelli concessi all'app (admin consent)	Quelli del ruolo RBAC del tuo utente su quel tenant
Lokka (MCP)	Disponibile	Non disponibile — richiede client credentials. L'AI usa solo <code>exo_query</code> /le cmdlet dirette
Connessione Exchange	Silenziosa (certificato)	Apri una finestra di login interattiva — blocca il server finché non completi

In pratica: usa AppOnly ovunque puoi (è il modello con cui è stato pensato tutto il resto del sistema); usa Delegated solo sui tenant dove non hai alternativa, sapendo che quel tenant specifico richiederà un tuo login occasionale invece di girare 24/7 senza interazione.

10.6 Rischio di blocco — perché la connessione Exchange delegata non è mai automatica

Il server di questo modulo gestisce le richieste HTTP **una alla volta** (single-threaded): se una richiesta si blocca in attesa di qualcosa, **l'intera app resta ferma per chiunque**, non solo per chi ha fatto quella richiesta specifica.

Incidente reale. Su un tenant Delegated appena attivato (login Graph già completato, ma sessione Exchange non ancora stabilita), una domanda in chat ("elenco licenze") ha fatto sì che l'AI provasse una cmdlet Exchange (`exo_query`). Ogni cmdlet Exchange chiama `Connect-M365OpsExchange` come primo passo, che in modalità Delegated avrebbe aperto un login interattivo — bloccando il server *per tutti*, per minuti, senza alcun feedback, finché il processo non è stato terminato manualmente da riga di comando.

Prima correzione (fail-fast): `Connect-M365OpsExchange` su un tenant Delegato senza sessione già attiva fallisce **immediatamente** con un messaggio chiaro invece di tentare un login bloccante, quando la connessione scatterebbe come effetto collaterale implicito di un tool AI (es. `exo_query`).

Seconda correzione (device code non bloccante) — la connessione esplicita stessa non blocca più: il primo tentativo di connessione esplicita (pulsante "Connetti Exchange") usava `Connect-ExchangeOnline -Device`, che stampa il codice solo nella console del processo — invisibile per un server lanciato in background — e blocca comunque l'intera app fino al completamento. Ora il pulsante usa lo stesso principio già collaudato per il login Graph (sezione 10.4): richiesta del codice device separata e non bloccante (client id `fb78d390-0c51-40cd-8e17-fdbfab77341b`, estratto e verificato empiricamente dal .dll del modulo `ExchangeOnlineManagement` — il client "legacy" `a0c73c16-...` talvolta citato online risulta disabilitato sui

tenant moderni), polling non bloccante per il completamento, e solo alla fine `Connect-ExchangeOnline -AccessToken` (rapido, nessuna interazione umana necessaria a quel punto) per stabilire la sessione vera e propria. Il codice compare direttamente nella GUI, non serve più che qualcuno lo legga dalla console per te.

Testato dal vivo, incidente reale: prima il server è rimasto bloccato senza feedback finché non è stato terminato a mano da riga di comando; il codice generato era visibile solo nella console del processo (`To sign in, use a web browser to open the page https://login.microsoft.com/device and enter the code ...`). Con la correzione, lo stesso flusso ora restituisce il codice immediatamente nella risposta HTTP, senza mai bloccare il server.

10.7 GUI riorganizzata — stato connessioni annidato nella riga del tenant attivo

Il tab **Tenant** mostra un indicatore per ciascun servizio (pallino verde/giallo): modalità (App-only/Delegata), Microsoft Graph, Exchange Online, e Lokka (solo App-only, non pertinente in modalità Delegata). La sezione **Exchange Online** (pulsante di connessione/test) vive qui, non in MCP/Connettori, dato che è dal tab Tenant che si gestisce tutto il resto del tenant attivo — il tab MCP/Connettori contiene solo Lokka e altri server MCP. Dal 18/08/2026 questo pannello non è più un blocco fisso sotto l'elenco profili: ogni riga tenant ha un proprio chevron espandi/collassa, e solo il tenant **attivo** mostra il pannello reale (espanso di default) — le righe degli altri profili, quando espande, mostrano solo un avviso che non c'è nulla di reale da verificare finché non li attivi. Attivare un tenant diverso sposta il pannello reale sulla sua riga automaticamente.

Il provider AI selezionato nel tab "Motore AI" è persistito su `Config\ai-provider.json` (solo il nome, non è un segreto) — un riavvio del server lo ricarica automaticamente, non torna più a Claude come default se stavi usando Azure OpenAI.

10.8 Permessi Microsoft Graph richiesti in modalità Delegata

A differenza di AppOnly (dove i permessi si scelgono e si concedono manualmente in fase di App Registration, sezione 4.2), in modalità Delegata i permessi sono uno **scope OAuth richiesto al login**: il token contiene solo ciò che è stato esplicitamente domandato in `Private\Invoke-M365OpsDeviceCodeFlow.ps1 ( $script:M365OpsDeviceCodeScopes )`, non ciò che l'utente avrebbe "in teoria" tramite i propri ruoli RBAC. Uno scope non richiesto qui produce un 403 che sembra un problema di consenso admin per un'app estranea, ma è quasi sempre solo che il token non l'ha mai chiesto — incontrato due volte dal vivo (`Organization.Read.All` per il report licenze, `UserAuthenticationMethod.ReadWrite.All` per lo stato/reset MFA) prima di arrivare all'elenco completo sotto.

Permesso	Motivo
<code>DeviceManagementManagedDevices.Read.All</code>	lettura device Intune, stato compliance
<code>DeviceManagementConfiguration.ReadWrite.All</code>	profili di configurazione
<code>DeviceManagementApps.ReadWrite.All</code>	app Intune (Win32, Store), assegnazioni, criteri di protezione app MAM
<code>DeviceManagementRBAC.ReadWrite.All</code>	Scope Tag, ruoli RBAC personalizzati, assegnazioni di ruolo
<code>DeviceManagementServiceConfig.ReadWrite.All</code>	restrizioni/limiti di iscrizione dispositivi, Autopilot
<code>DeviceManagementScripts.ReadWrite.All</code>	script di distribuzione Windows/macOS, Proactive Remediations
<code>Group.ReadWrite.All</code>	creazione gruppi, gestione membri
<code>User.ReadWrite.All</code>	lettura e scrittura su utenti (es. disabilitare un account)
<code>UserAuthenticationMethod.ReadWrite.All</code>	stato MFA e reset (sezione 21) — scope dedicato, <i>non</i> coperto da <code>User.ReadWrite.All</code>
<code>Directory.Read.All</code>	lettura oggetti directory generici oltre a utenti/gruppi
<code>Organization.Read.All</code>	informazioni tenant, licenze (<code>/subscribedSkus</code> , <code>/organization</code>)
<code>AuditLog.Read.All</code>	log di sign-in
<code>RoleManagement.Read.Directory</code>	lettura assegnazioni di ruolo (sola lettura — vedi nota sotto)
<code>Reports.Read.All</code>	report di utilizzo (usage report Microsoft 365)
<code>Sites.Read.All</code>	lettura siti SharePoint
<code>Application.Read.All</code>	visibilità sulle App Registration del tenant (sola lettura)
<code>IdentityRiskyUser.Read.All</code>	segnali di rischio Identity Protection (<code>/identityProtection/riskyUsers</code>) - corretto il 19/08/2026: questa riga elencava per errore <code>SecurityEvents.Read.All</code> (permesso per un'API diversa, gli alert Defender), mai verificato dal vivo fino a quel momento - nome corretto confermato su Microsoft Learn
<code>Policy.Read.All</code>	lettura policy del tenant (es. Conditional Access — sola lettura)
<code>Team.ReadBasic.All</code> / <code>Channel.ReadBasic.All</code>	dati Teams di base
<code>Mail.Send</code>	invio report generati via email
<code>offline_access</code>	refresh token — senza, la sessione non si rinnova da sola

Scritture ad alto raggio d'azione escluse di proposito. `RoleManagement.ReadWrite.Directory` (assegna/revoca ruoli admin), `Policy.ReadWrite.ConditionalAccess` (un Conditional Access sbagliato può bloccare fuori l'intero tenant) e `Application.ReadWrite.All` (può creare credenziali su un'app) non sono nello scope richiesto — vanno aggiunti solo su richiesta esplicita per una funzionalità reale già costruita, mai "perché potrebbe servire".

Ogni volta che si aggiunge un nuovo scope, serve un **nuovo login** (tab Tenant → "Accedi con il mio utente") — un token già in cache non guadagna retroattivamente un permesso non richiesto al momento in cui è stato emesso. La prima volta che un nuovo scope viene effettivamente usato può comparire una schermata di consenso durante il login stesso, normale per un permesso mai richiesto prima su quel tenant.

10.9 Connessione Intune in modalità Delegata — un limite del modulo di terze parti, non nostro

La pacchettizzazione app Win32 (`New-M365OpsWin32App`) passa dal modulo di terze parti `IntuneWin32App` , che gestisce la propria autenticazione in modo indipendente dal resto di questo progetto — non condivide il token Graph già in uso per tutto il resto.

Incidente reale. Il flusso a codice dispositivo (`Connect-MSIntuneGraph -DeviceCode`) di `IntuneWin32App` ha un bug reale su PowerShell 7: il gestore d'errore del ciclo di polling chiama `$ErrorResponse.GetResponseStream()` , un metodo che esiste solo su `System.Net.WebResponse` (.NET Framework), non su `System.Net.Http.HttpResponseMessage` (le eccezioni di PS7) — crolla sul primissimo "in attesa che l'utente completi il login", cioè sempre, a prescindere da quanto in fretta si inserisca il codice. Il flusso a codice dispositivo per Intune quindi **non può funzionare** su questo sistema, indipendentemente da qualunque cosa faccia questo modulo.

Corretto passando a `-Interactive` (`Connect-MSIntuneGraph -Interactive`), un flusso PKCE con redirect su `localhost` scritto da zero dal modulo stesso, che non condivide quel bug — apre il browser di sistema con **selezione account esplicita** (`prompt=select_account`), utile anche per essere certi di quale utenza si stia usando.

Connessione esplicita, mai implicita

Come per Exchange (sezione 10.6), la connessione Intune su un tenant Delegato non parte mai da sola dentro un'azione automatica: `Connect-M365OpsIntune` fallisce subito con un messaggio chiaro se non c'è già una sessione valida, a meno di essere chiamata esplicitamente con `-AllowInteractive` — riservato al pulsante dedicato **"Connetti Intune (browser)"** nel tab Tenant. Il pulsante avvisa esplicitamente che il server resterà bloccato (a thread singolo, sezione 10.6) finché non completi il login nel browser appena aperto — va quindi cliccato PRIMA di chiedere in chat di pacchettizzare un'app, mai durante una richiesta composta dove bloccherebbe l'app senza preavviso.

Verifica dal vivo, non solo un flag

La connessione non si considera valida solo perché `Connect-MSIntuneGraph` non ha sollevato un errore: `Test-M365OpsIntuneAuthValid` chiama Graph con il token appena ottenuto (`/organization`) e confronta il tenant risolto con quello attivo — se non combaciano (token/cache riusato da un'altra sessione) o la chiamata fallisce, la connessione viene rifiutata e va rifatta, invece di scoprire il problema più tardi dentro `New-M365OpsWin32App` con un errore criptico del modulo. La GUI mostra l'utenza e il tenant effettivamente verificati, non solo un'icona verde presunta.

10.10 Prompt di admin consent al login Graph delegato — e perché CLI Microsoft 365 può risolverlo (27/08/2026)

Domanda reale dell'utente: "mi loggo nel tenant e mi chiede Graph come admin consent anche se mi loggo con il mio account? questo CLI 365 risolve?" — risposta completa, non ipotetica.

Il login delegato descritto in questa sezione (10.4) usa il client pubblico Microsoft **"Microsoft Graph Command Line Tools"** (`14d82eec-...`). Il login delegato di **CLI Microsoft 365** (`Connect-M365OpsCLIMicrosoft365 -AllowInteractive` , tab Tenant → riquadro "CLI Microsoft 365") usa invece un client pubblico Microsoft **diverso**, di proprietà del progetto PnP/CLI Microsoft 365. Sono due app Microsoft first-party SEPARATE, ciascuna con il proprio stato di consenso INDIPENDENTE su un dato tenant — è quindi del tutto normale che una richieda ancora un consenso (o lo richieda "admin", vedi sotto) mentre l'altra sia già pronta all'uso, anche stesso account, stesso tenant.

Perché normalmente NON serve un admin che clicchi "Grant admin consent": un login DELEGATO (con la tua utenza reale, non un'App Registration Application) chiede consenso solo per gli scope DELEGATI necessari — per la stragrande maggioranza di questi scope, in un tenant con le impostazioni di default, il consenso è **auto-consenso dell'utente** al primo utilizzo (un semplice popup "Questa app vuole accedere a... Accetta"), non un passaggio riservato a un amministratore. Il permesso EFFETTIVO che ottieni resta comunque limitato al tuo ruolo RBAC reale su quel tenant (sezione 10.1) — l'auto-consenso concede solo la possibilità tecnica di chiedere quello scope, non un privilegio aggiuntivo.

L'eccezione che spiega un prompt "admin consent" anche loggandoti con il tuo utente: se il tenant ha la policy Entra ID *"Users can consent to apps"* disabilitata (impostazione di sicurezza comune in ambienti aziendali/hardened), OGNI app — inclusi entrambi i client pubblici Microsoft sopra — richiede un consenso amministratore una tantum, a prescindere da quale dei due stai usando. In quel caso passare a CLI Microsoft 365 NON risolve da solo: risolve solo se quel client specifico ha già ricevuto il consenso admin in passato (es. da un utente Global Admin che lo ha usato prima), mentre l'altro client (Graph CLI Tools) non lo ha mai ricevuto — è quindi possibile che "risolva" per puro caso di quale dei due è già stato consentito su quel tenant, non per un motivo strutturale che lo garantisca sempre.

Come verificare qual è il caso reale, invece di indovinare: se il prompt che compare menziona esplicitamente "chiedi al tuo amministratore di concedere il consenso" (`AADSTS65001` con indicazione admin-only, o la UI mostra "Serve l'approvazione di un amministratore" invece del normale pulsante "Accetta"), è la policy tenant-wide — un amministratore deve concedere il consenso una tantum per QUEL client specifico (Entra ID → App aziendali → cerca il nome del client → Autorizzazioni → "Concedi consenso amministratore"), dopo di che qualunque utente può accedere normalmente. Se invece il prompt è il normale popup "Accetta" che chiunque può cliccare da solo, non è affatto una policy di blocco — è solo il primo utilizzo di quel client su quel tenant, e basta cliccare Accetta una volta.

Conclusione pratica: sì, tenere CLI Microsoft 365 come connettore alternativo ha senso proprio per questo scenario — se il client Graph CLI Tools risulta bloccato/ non ancora consentito su un tenant specifico, il login delegato di CLI Microsoft 365 (client diverso) vale la pena provarlo come seconda strada, prima di dover coinvolgere un amministratore del tenant per un consenso esplicito. Non è garantito che eviti SEMPRE il problema (la policy tenant-wide, se presente, blocca comunque entrambi finché un admin non consente il client specifico in uso), ma nella pratica risolve la maggioranza dei casi in cui il blocco non è una policy ma solo "questo client non è ancora stato usato/consentito qui".

11. Le 50+ cmdlet Exchange Online e come le usa l'AI

Oltre alle due cmdlet iniziali (mailbox condivise e relativi permessi), il modulo espone oggi oltre 50 cmdlet Exchange Online, organizzate per area. Tutte richiedono `Connect-M365OpsExchange` (chiamata automaticamente al loro interno) e rispettano lo stesso principio di conferma umana per le scritture.

Area	Esempi di cmdlet
Mailbox condivise	<code>New-M365OpsSharedMailbox</code> , <code>Grant-/Revoke-M365OpsMailboxPermission</code> , <code>Add-/Remove-M365OpsSendOnBehalf</code> , <code>Get-M365OpsSharedMailboxReport</code>
Gruppi Exchange	Distribution list, mail-security group, dynamic distribution group — <code>Get/New/Remove</code> , gestione membri, <code>Get-M365OpsGroupsOverviewReport</code>
Mail flow	<code>Get-/New-/Set-/Remove-M365OpsTransportRule</code> , <code>Get-M365OpsMailFlowReport</code>
Tracking	<code>Get-M365OpsMessageTrace</code> , <code>Get-M365OpsMessageTraceDetail</code>
Domini	<code>Get-M365OpsAcceptedDomains</code>
Contatti	<code>Get-/New-/Remove-M365OpsMailContact</code> , <code>Get-M365OpsMailUsers</code>
Mailbox risorsa	<code>Get-M365OpsResourceMailboxes</code> , <code>New-M365OpsRoomMailbox / EquipmentMailbox</code> , policy di prenotazione
Migrazioni	<code>Get-M365OpsMigrationBatches / MigrationUserStatus / MigrationEndpoints</code> (lettura); <code>New-M365OpsMigrationBatch</code> — SOLO su un endpoint già configurato (mai credenziali nuove da chat), creato sempre non avviato; <code>Start-M365OpsMigrationBatch</code> per avviarlo, passo separato
Report mailbox	Utilizzo/quota, mailbox inattive, inoltri automatici, risposte automatiche, delegati aggregati, regole sospette, litigation hold
Sicurezza	<code>Get-M365OpsInboxRulesReport</code> (regole sospette), <code>Get-M365OpsSharedMailboxSignInStatus</code> (mailbox condivise con login abilitato — rischio)
Calendario / Cartelle pubbliche	<code>Get-/Set-M365OpsCalendarPermission</code> , <code>Get-M365OpsPublicFolders</code>

11.1 Come l'AI le usa

L'AI non ha 50 strumenti separati da scegliere (costerebbe troppo token e confonderebbe il modello): ha due strumenti generici, sullo stesso principio di `graph_api_call / propose_graph_write` usati per Lokka:

- **exo_query** — sola lettura, il modello specifica quale cmdlet (dalla lista consentita) e con quali parametri, viene eseguita subito;
- **propose_exo_write** — come `propose_graph_write` : il modello propone cmdlet + parametri + motivazione, ma non esegue mai nulla. La proposta torna in chat con "Cmdlet: ... Parametri: ... Motivo: ...", in attesa di conferma esplicita.

Entrambi gli strumenti validano il nome della cmdlet contro una lista consentita (non è possibile far eseguire all'AI una cmdlet PowerShell arbitraria).

11.2 Report via catalogo (senza costo AI)

I report più richiesti sono anche comandi diretti del catalogo, zero costo AI: "esporta report utilizzo mailbox in excel", "esporta report mailbox condivise in pdf", "esporta report mailbox totali", "esporta report mail flow", "esporta report inoltri", "esporta report regole sospette".

Nota sul routing: un trigger generico come "permessi mailbox" senza specificare "condivise" passa volutamente all'AI (non al comando dedicato alle sole mailbox condivise) — così può usare `Get-M365OpsMailboxDelegatesReport` , che copre *tutte* le mailbox, non solo quelle condivise, ed essere esplicita sui limiti invece di rispondere con uno scope ristretto senza dirlo.

11.3 Migrazioni — solo su endpoint già esistenti

La creazione di un **nuovo** endpoint di migrazione (che richiede le credenziali del sistema sorgente esterno — server IMAP, Exchange on-premises, ecc.) resta volutamente fuori da questo modulo: va fatta manualmente dal centro amministrazione Exchange o con `New-MigrationEndpoint` diretto, con supervisione — inserire credenziali di un sistema esterno tramite la chat non è un caso d'uso che vogliamo abilitare.

Quello che l'AI *può* fare è creare un **batch** su un endpoint *già configurato*: prima cerca gli endpoint disponibili con `exo_query` su `Get-M365OpsMigrationEndpoints` , poi propone `New-M365OpsMigrationBatch` con quell'identity esatta — `New-M365OpsMigrationBatch` rifiuta qualunque `SourceEndpoint` che non corrisponda esattamente a uno di quelli configurati, invece di lasciar passare un nome indovinato. Il batch viene sempre creato **non avviato**: avviarlo è un passo separato ed esplicito (`Start-M365OpsMigrationBatch`), con la sua stessa conferma umana di ogni altra scrittura.

Opzioni reali supportate

Opzione	Cmdlet	Effetto
<code>-Users</code>	<code>New-M365OpsMigrationBatch</code>	Elenco preciso di indirizzi da migrare (batch mirato) — via CSV caricato o elencati in chat
<code>-Cutover</code>	<code>New-M365OpsMigrationBatch</code>	Batch sull'intera organizzazione, nessun elenco
<code>-AutoComplete</code>	<code>New-M365OpsMigrationBatch</code>	Ogni mailbox si finalizza da sola non appena sincronizzata; di default assente — resta in sospeso, serve conferma manuale
<code>-CompleteAfter <data/ora></code>	<code>New-M365OpsMigrationBatch</code>	Programma la finalizzazione a un momento preciso nel futuro (es. fuori orario lavorativo) invece che subito
<code>-StartAfter <data/ora></code>	<code>Start-M365OpsMigrationBatch</code>	Programma l'inizio della sincronizzazione invece di farla partire immediatamente all'avvio

Caricare un elenco di indirizzi da CSV

Sopra la casella di chat c'è un pulsante **"Carica CSV migrazione..."** (prima colonna del file: `EmailAddress`, un indirizzo per riga). Una volta caricato, il contenuto viene fornito automaticamente all'AI quando chiedi di creare un batch — non serve incollare gli indirizzi a mano in chat.

Basta un solo prompt in chat — esempio verificato dal vivo

Si: dopo aver caricato il CSV, un singolo messaggio in linguaggio naturale è sufficiente. L'AI cerca da sola l'endpoint disponibile via `exo_query`, legge gli indirizzi dal CSV caricato, e costruisce la proposta completa:

```
Utente: crea un batch di migrazione chiamato TestBatch1 con gli indirizzi
del csv caricato, usando l'endpoint disponibile, senza completamento automatico

Risposta (proposta, non ancora eseguita):
Cmdlet: New-M365OpsMigrationBatch
Parametri: {"SourceEndpoint":"Hybrid Migration Endpoint - EWS (Default Web Site)",
            "Name":"TestBatch1",
            "Users":["mario.rossi@contoso.com","anna.bianchi@contoso.com"]},
            "AutoComplete":false}
Motivo: [spiegazione in italiano di cosa fa, mostrata prima della conferma]

(rispondi 'si' per eseguirla davvero, 'no' per annullare)
```

Per programmare una finalizzazione futura basta aggiungerlo alla stessa frase, es. "...e completala automaticamente il 1 settembre alle 22" — l'AI traduce la data in `CompleteAfter` senza bisogno di comandi PowerShell separati.

11.4 Copertura completa dei parametri — mai a memoria, sempre da Microsoft Learn

Le cmdlet di scrittura di questo modulo non limitano a priori quali parametri nativi puoi usare: ognuna espone anche `ExtraParams` (un oggetto libero) per qualunque parametro reale della cmdlet Exchange/Graph sottostante non già previsto esplicitamente — es. `Alias`, `RetentionPolicy`, `ModeratedBy` su `New-Mailbox`, e così per ogni altra cmdlet.

Per evitare che l'AI inventi un nome di parametro plausibile-ma-sbagliato (un rischio reale con la sola conoscenza pregressa del modello, che può essere incompleta o superata), ha accesso a uno strumento aggiuntivo:

- **lookup_ms_docs** — consulta la pagina reale e aggiornata di Microsoft Learn per una cmdlet Exchange (URL diretto alla pagina di riferimento, con sintassi completa e tipo di ogni parametro) o, in fallback, la ricerca Microsoft Learn per argomenti Graph. Il modello è istruito a usarlo *prima* di includere in `ExtraParams` un parametro non già verificato in conversazione, e quando l'utente chiede esplicitamente "quali altre opzioni ci sono per X".

Testato dal vivo. Alla domanda "quali altre opzioni ha New-Mailbox oltre a quelle che usi di solito per creare una mailbox condivisa? dammene 5", l'AI ha consultato `lookup_ms_docs` "New-Mailbox" e risposto con 5 parametri reali (`RetentionPolicy`, `AddressBookPolicy`, `ModeratedBy` / `ModerationEnabled`, `Archive`, `ThrottlingPolicy`), spiegando anche come usarli via `ExtraParams` in `propose_exo_write`.

Questo principio è scritto anche nel `README.md` del modulo e in `Scripts\Custom\README.md` come convenzione permanente per chi (umano o AI) scrive o corregge codice in questo progetto: mai una chiamata a una cmdlet nativa costruita a memoria, sempre verificata contro la documentazione reale prima di proporla.

12. Avvio dell'applicazione

12.1 Avvio con un click — M365Ops.bat

Nella cartella radice del progetto, `M365Ops.bat` (che richiama `Launch-M365Ops.ps1`) è il modo consigliato per l'uso quotidiano: un doppio click avvia il server e apre il browser sulla chat, senza passare dal terminale.

Non duplica mai la sessione. Prima di avviare qualunque cosa, controlla con una vera chiamata HTTP a `/api/status` se un'istanza è già attiva sulla porta: se sì, si limita ad aprire il browser sulla sessione già in corso (stesso tenant, stessa cronologia chat) invece di avviarne una seconda in conflitto sulla stessa porta o lasciarne una orfana in background. Legge inoltre `Config\last-active-tenant.txt` (scritto da `Connect-M365Ops` ad ogni cambio tenant) per far ripartire il server esattamente sul tenant su cui stavi lavorando l'ultima volta, non su un default fisso.

12.2 Avvio manuale da terminale (facoltativo — non serve per l'uso normale)

Il doppio click su `M365Ops.bat` (12.1) è sufficiente in ogni caso normale: questa sezione esiste solo per chi preferisce vedere l'output in un terminale invece che in una finestra nascosta, o per diagnosticare un avvio che fallisce silenziosamente in modo "con un click".

```
cd <percorso-della-cartella-M365Ops>
.\Launch-M365Ops.ps1
```

È lo stesso identico script richiamato da `M365Ops.bat` (resume-session, rilevamento porta, controllo prerequisiti inclusi) — la sola differenza è che l'output resta visibile nella finestra invece che nascosto. Il server web parte sulla porta configurata (8743 di default, sezione "Aggiornamenti" nel tab Manutenzione per cambiarla), carica automaticamente l'ultimo tenant attivo e avvia la connessione a Lokka. Dal tab Manutenzione della GUI è comunque disponibile un pulsante di riavvio che non richiede di tornare al terminale.

Il riavvio (pulsante GUI o `POST /api/restart`) è l'unico modo sicuro di far ripartire il server: essendo a thread singolo, la richiesta di riavvio viene "eseguita" solo subito dopo aver chiuso la risposta HTTP corrente, quindi non può mai interrompere una scrittura confermata a metà. Terminare il processo dall'esterno (es. Task Manager, `Stop-Process -Force`) non ha questa garanzia: se c'è una richiesta bloccante in corso (es. una scrittura Graph appena confermata con "sì"), un kill esterno può interromperla a metà - un incidente reale del 15/08/2026, poi verificato senza conseguenze solo controllando a mano lo stato del tenant dopo un nuovo login.

12.3 Se il server non risponde affatto — "Termina e riavvia" (22/08/2026)

Il pulsante di riavvio del tab Manutenzione (12.2) funziona solo se il server può ancora rispondere a una richiesta HTTP - se è davvero bloccato (es. un login interattivo Teams/ SharePoint rimasto in attesa, sezione 10.6), nemmeno quel pulsante risponde, perché il server è a thread singolo: una richiesta bloccata blocca anche tutte le altre, incluso il tab Log stesso. Per questo caso specifico, un collegamento **"M365Ops - Termina e riavvia"** viene creato in automatico sul Desktop (stessa icona di M365Ops, accanto al collegamento normale) - termina forzatamente il processo del server (anche se bloccato) e lo riavvia da `M365Ops.bat`, tutto da un doppio click, senza dover aprire Task Manager a mano.

Usalo SOLO come ultima risorsa, quando il pulsante di riavvio normale (12.2) non risponde affatto - non ha la stessa garanzia di "mai a metà di una scrittura" descritta sopra (un kill esterno, con qualunque strumento, può in teoria interrompere una scrittura Graph appena confermata). Se il server risponde ancora anche solo lentamente, preferisci sempre il pulsante di riavvio normale.

Tecnicamente: `Kill-And-Restart-M365Ops.ps1 / .bat` nella cartella dell'app, Windows PowerShell 5.1 (non richiede che `pwsh.exe` funzioni - deve poter terminare `pwsh.exe` anche quando è lui stesso ad essere bloccato). Termina SOLO i processi `pwsh.exe` che eseguono `Gui\Server.ps1` di QUESTA installazione specifica (confronto esplicito del percorso completo) - mai un generico "tutti i pwsh.exe di sistema", che potrebbero appartenere a tutt'altro.

13. Uso quotidiano

Il modulo non copre solo Intune: gestisce Intune/Entra ID (via Graph, direttamente o tramite Lokka) ed Exchange Online (tramite le 50+ cmdlet della sezione 11) nello stesso strumento di chat. Ecco esempi reali per ciascuna area.

13.1 Intune (device e compliance)

- "elenca i dispositivi" / "dispositivi non conformi"
- "analizza i pattern di non conformità" (usa l'AI)
- "esporta dispositivi in excel"

13.2 Entra ID (utenti e gruppi)

- "panoramica utente mario.rossi@contoso.com" — gruppi, dispositivi, app e policy assegnate
- "panoramica gruppo Marketing" — membri, app e policy assegnate al gruppo
- "crea gruppo Vendite-Test con membro mario.rossi@contoso.com" (richiede conferma)
- "trova l'utente Mario Rossi" — risoluzione nome → UPN
- qualunque altra domanda su utenti/gruppi/licenze non coperta sopra passa all'AI, che interroga Microsoft Graph direttamente tramite Lokka (sezione 9)

13.3 Exchange Online

- "permessi delle mailbox condivise" — solo condivise, comando diretto da catalogo
- "tira fuori tutti i permessi delle mailbox" — generico, passa all'AI che usa `Get-M365OpsMailboxDelegatesReport` e copre *tutte* le mailbox
- "esporta report mailbox condivise in pdf" / "esporta report utilizzo mailbox in excel"
- "quali mailbox hanno l'inoltro automatico configurato?" (report sicurezza)
- "crea una mailbox condivisa fatture@contoso.com" (richiede conferma)
- "dai FullAccess su fatture@contoso.com a mario.rossi@contoso.com" (richiede conferma)
- "elenca le regole di trasporto" / "crea una regola che blocca gli allegati .exe" (creata sempre disabilitata, richiede conferma per l'attivazione)
- "quali domini sono accettati dal tenant?"

13.4 Report ed export (trasversali)

Qualunque area sopra supporta l'export diretto in CSV/Excel/PDF e l'invio per email dell'ultimo report generato ("invio per email a nome@dominio.it").

Per richieste generiche ("fammi un report licenze", "report caselle di posta con grafico per dimensione") l'AI usa lo strumento `generate_report`: raccoglie prima i dati REALI del tenant (mai inventati/riassunti a memoria) con `graph_api_call / exo_query`, poi genera un file Excel (dati completi) e un PDF con grafici a barre/torta calcolati dal codice via `Group-Object` - mai chiedendo il conteggio all'AI, per evitare che un conteggio sbagliato su tanti record diventi un errore silenzioso in un report. I grafici sono SVG generati internamente, nessuna libreria esterna o dipendenza da internet in fase di stampa. Entrambi i file compaiono in chat come pulsanti di download diretti - non serve più andare a cercarli sul disco, e il testo di risposta non mostra mai il percorso assoluto del file.

13.5 Stato e reset MFA di un utente

- "stato mfa di mario.rossi@contoso.com" / "che metodi di autenticazione ha registrato mario.rossi?" — sola lettura, elenca i metodi registrati (Authenticator, telefono, FIDO2, Windows Hello, app OATH, Temporary Access Pass), mai la password
- "resetta l'mfa di mario.rossi@contoso.com" / "fallo reregistrare l'mfa" — **richiede sempre conferma esplicita** prima di rimuovere qualunque metodo: e' un'azione con impatto immediato (l'utente perde l'accesso ai metodi rimossi finché non li riconfigura). La password non viene mai toccata.

Non esiste in Microsoft Graph v1.0 un singolo flag "MFA abilitata" per utente (dipende da Conditional Access, non da questa API) - lo stato si deduce da quali metodi risultano registrati oltre alla password (verificato su Microsoft Learn, sezione 21). Serve lo scope `UserAuthenticationMethod.ReadWrite.All` **più** un ruolo Entra (Global Reader per la sola lettura; Authentication Administrator o Privileged Authentication Administrator per il reset) - un requisito di ruolo separato dallo scope, un 403 con lo scope corretto ma senza il ruolo è normale, non un bug.

13.6 Memoria della conversazione

Dal 15/08/2026 l'AI ricorda gli scambi precedenti con lo stesso tenant: gli ultimi turni della conversazione sono salvati in locale (`Config\ChatHistory-<tenant>.json`), con le eventuali password redatte prima del salvataggio) e rimandati come contesto ad ogni nuova domanda - una domanda di follow-up ("e per quell'utente creato prima?") funziona senza dover ripetere il contesto da zero. Un refresh della pagina ricarica lo stesso storico (e un'eventuale proposta di scrittura ancora in sospeso), invece di mostrare una chat vuota. Il pulsante **"Nuova conversazione"** (o scrivere "nuova conversazione"/"dimentica tutto" in chat) azzerà lo storico per il tenant attivo, per ripartire puliti su un argomento non correlato.

13.7 Conferma umana obbligatoria — vale sempre, senza eccezioni

Ogni azione di scrittura (creazione, modifica, eliminazione) segue sempre il flusso *proponi* → *conferma* → *esegui*, indipendentemente da:

- **dove avviene la scrittura** — Microsoft Graph (`propose_graph_write`, via Lokka) o Exchange Online (`propose_exo_write`, sezione 11.1): stesso principio, stesso meccanismo di proposta;
- **la modalità di autenticazione del tenant** — AppOnly o Delegated (sezione 10): il gate di conferma vive nel livello chat/AI, non nel livello di autenticazione, quindi si applica identicamente in entrambi i casi.

Le due liste di cmdlet consentite all'AI (lettura ed effettivamente eseguibili subito, vs. scrittura e solo proponibili) sono statiche e senza sovrapposizioni: una cmdlet di scrittura non può mai finire nel percorso di sola lettura per errore. Verificato anche con un test dal vivo: una richiesta di scrittura (creazione di un contatto) si è fermata alla proposta, un messaggio successivo inviato con la proposta ancora in sospeso non l'ha bypassata, e il rifiuto esplicito ("no") l'ha annullata senza mai chiamare la cmdlet reale.

Una sola proposta di scrittura per risposta. Bug reale osservato il 15/08/2026: l'AI aveva proposto "crea utente" e, nella stessa risposta, subito dopo "assegna licenza" - la seconda proposta sovrascriveva silenziosamente la prima, l'utente confermava pensando di approvare entrambe ma solo l'ultima era davvero in sospeso. Ora una seconda proposta nella stessa risposta viene rifiutata dallo strumento stesso: per un piano a più passaggi (es. crea utente POI assegna licenza), l'AI propone solo il primo passo e si ferma, mostrando un indicatore **"Passo 1 di 2"** nel testo della conferma - il passo successivo viene proposto solo in un messaggio separato, dopo che il primo è stato confermato ed eseguito.

13.8 Quando manca uno strumento: l'AI può proporre di crearne uno

Se chiedi qualcosa di ricorrente che nessuno strumento esistente copre (un report specifico di questo tenant, un'estrazione ad hoc), l'AI può proporre di scrivere un nuovo script permanente invece di limitarsi a spiegarti come farlo a mano — vedi sezione 17.6 per i dettagli completi (conferma sempre col codice per intero, validazioni automatiche, riavvio per caricarlo).

14. Sicurezza — principi non negoziabili

1. **Zero segreti su disco.** `tenants.json` contiene solo ID, thumbprint (pubblico) e nomi di variabili d'ambiente — mai client secret o API key in chiaro.
2. **Propose → confirm → execute.** Nessuna scrittura contro Graph o Exchange avviene senza conferma umana esplicita in chat.
3. **Le cmdlet di scrittura creano senza assegnare/attivare per default** — l'assegnazione è sempre un passo separato ed esplicito.
4. **Le cmdlet di lettura non interpretano mai i dati** — l'interpretazione passa sempre dal motore AI, mai da regole cablate nel codice.
5. **Isolamento multi-tenant** — credenziali, certificati e connettori MCP di un tenant non sono mai visibili o raggiungibili da un altro profilo.

15. Stato attuale del progetto

Area	Stato	Note
Graph app-only (Intune/Entra ID)	funzionante	Device, gruppi, app, profili di configurazione
Lokka MCP	funzionante	Tool primario per l'AI, tenant-scoped
Export report CSV/XLSX/PDF	funzionante	PDF via Microsoft Edge headless
Invio report via email	funzionante	Graph Mail.Send , testato end-to-end
Exchange Online — connessione app-only (certificato)	funzionante	Verificato dal vivo dopo completamento RBAC (sezione 6)
Exchange Online — 50+ cmdlet (sezione 11)	funzionante	Mailbox, gruppi, mail flow, tracking, contatti, risorse, report — testate dal vivo sul tenant
Autenticazione delegata (utente + MFA)	funzionante	Device code flow verificato dal vivo; per tenant senza App Registration (sezione 10)
Tool-calling AI — Claude	funzionante	Loop tool-calling completo, con <code>exo_query / propose_exo_write</code>
Tool-calling AI — Azure OpenAI	funzionante	Stesso loop/stessi strumenti di Claude su Chat Completions API (sezione 8.1/8.2), scelta in GUI persisti
Persistenza scelta provider AI	funzionante	<code>Config\ai-provider.json</code> , sopravvive al riavvio (sezione 10.8)
Server MCP CLI Microsoft 365	collegato	Copre Entra/Outlook/Planner/OneDrive/Purview (audit/compliance search)/Teams lato Graph - vedi se Teams, ne' (per ora) SharePoint su tenant a client secret. Prerequisito (<code>npm install -g @pnp/cli-m</code> box al primo avvio (.bat), non serve piu' installarlo a mano. Supportato sia App-only (client secret) sia GUI) - pulsante "Connetti/Test" nel tab Tenant, sezione Stato connessioni (non nel tab MCP/Connettor per ogni tenant, resta vivo in background al cambio profilo
Altri server MCP oltre Lokka e CLI Microsoft 365	salvabili ma non collegati	La GUI permette di registrarli/rimuoverli; l'infrastruttura di connessione (Connect-M365OpsMcpServer server diversi da questi due - andrebbero aggiunti caso per caso come per CLI Microsoft 365
Rimozione server MCP da GUI	funzionante	Pulsante "Rimuovi" per riga, testato dal vivo (aggiunto e rimosso)
Creazione batch di migrazione	funzionante	Solo su endpoint già configurati (sezione 11.3), creato sempre non avviato — testato dal vivo, endpoir
Script personalizzati (Scripts\Custom)	funzionante	Scoperta automatica, testato dal vivo con uno script reale (sezione 17)
Controllo stato ambiente / portabilità	funzionante	<code>Get-M365OpsSetupStatus</code> + banner GUI, testato dal vivo (sezione 18)
Log persistente delle azioni	funzionante	Un file al giorno in <code>Logs\</code> + visualizzatore nel tab Manutenzione (sezione 19)
Blocco server su login Exchange delegato implicito	corretto	Bug reale riscontrato in uso e risolto stesso giorno — vedi sezione 10.6
Copertura parametri Exchange/Graph (ExtraParams + lookup_ms_docs)	funzionante	Ogni cmdlet di scrittura accetta parametri nativi aggiuntivi verificati dal vivo su Microsoft Learn (sezior
Login Exchange delegato non bloccante (device code)	corretto	Sostituito il flusso bloccante con lo stesso schema del login Graph — codice mostrato in GUI, mai più
graph_api_call su tenant delegati	corretto	Bug reale: era escluso del tutto, ora chiama Graph direttamente quando Lokka non è disponibile (sezic
Pannello stato connessioni + Exchange/Intune sotto Tenant	funzionante	Indicatori chiari Graph/Exchange/Intune/Lokka per entrambe le modalità (sezione 10.7)
Stato/contatore chiamate motore AI	funzionante	Contatore per provider + test di connessione dal vivo (sezione 20)
Connessione Intune su tenant Delegati	funzionante	<code>-Interactive</code> (non <code>-DeviceCode</code> , bug reale nel modulo terzo), verifica dal vivo del token (sezion
Stato/reset MFA utente	funzionante	Reset sempre con conferma esplicita, testato dal vivo (sezione 13.5, 21)
Report AI-orchestrati con grafici (Excel + PDF)	funzionante	<code>generate_report</code> , grafici SVG generati internamente, download diretto da GUI (sezione 13.4, 22)
Avvio con un click (M365Ops.bat)	funzionante	Resume-session su porta già attiva, tenant ripreso da <code>last-active-tenant.txt</code> (sezione 12.1)
Indicatore di avanzamento onesto in GUI	corretto	Bug reale: messaggio statico "sto elaborando" indistinguibile da un blocco reale — ora un contatore d
Memoria della conversazione	funzionante	Storico persistito per tenant, ricaricato al refresh pagina, azzerabile a comando (sezione 13.6, 23)
Una sola proposta di scrittura per risposta + indicatore "passo X di N"	corretto	Bug reale: due proposte nella stessa risposta si sovrascrivevano in silenzio (sezione 13.7)
Message trace su Get-MessageTraceV2	corretto	Le cmdlet classiche sono in dismissione dal 1/09/2025, verificato su Microsoft Learn (sezione 11)
Ricerca registro di controllo unificato (Purview <code>Search-UnifiedAuditLog</code>)	non costruito	Tecnicamente fattibile (cmdlet reale, richiede licenza/ruolo Audit Log) ma non ancora implementato —
Creazione di nuovi script da parte dell'AI	funzionante	<code>propose_new_custom_script</code> , conferma con codice completo sempre obbligatoria, validazione sin automatico per caricarlo (sezione 17.6)
Bug: risposta AI vuota su richieste complesse	corretto	Budget token 2000→4000 + rilevamento e messaggio esplicito invece di un messaggio bianco silenzic
Bug: crash su array vuoto serializzato in JSON	corretto	<code>ConvertTo-Json</code> pipelato su un array vuoto restituisce <code>\$null</code> vero, non <code>[]</code> - causava un errore su risolto in tutti i punti a rischio del progetto (sezione 23.1)
Sicurezza email (antispam, threat policy, allow/block list, quarantena+rilascio)	funzionante	Nuove cmdlet lette/scritte via <code>exo_query / propose_exo_write</code> , stessa connessione Exchange gia

Message trace oltre i 10 giorni	funzionante	Incatenamento automatico delle finestre da 10 giorni (sezione 17.7) - bug reale scoperto e corretto lo contesto del modello, ora troncato a 1000 righe con avviso
Export diretto senza passare dati grezzi nella conversazione	funzionante	<code>generate_raw_export</code> : query+scrittura file in un solo passaggio lato server, per volumi che altrime verificate dal vivo in un solo export, ~10mila token invece di 437mila) (sezione 17.7)
Controllo prerequisiti con installazione automatica	funzionante	PowerShell 7 controllato/installato da <code>M365Ops.ps.bat</code> via winget prima dell'avvio; Node.js/Edge instal
Invio report via email dalla conversazione libera	funzionante	<code>propose_send_report_email</code> , conferma sempre richiesta, testo email scritto dall'IA con overview r
SharePoint/OneDrive (siti, condivisione esterna, storage, permessi, account orfani)	funzionante	Verificato dal vivo il 17/08/2026 dopo la concessione del permesso SharePoint (sezione 4.3) - 44 siti, 1 correttamente (sezione 17.9)
Bug: URL centro amministrazione SharePoint non risolvibile su TenantId a GUID	corretto	Derivazione assumeva sempre il formato dominio - ora risolve il dominio iniziale verificato via Graph
Microsoft Teams (elenco/canali/membri, criteri, accesso esterno)	funzionante	Elenco/canali/membri verificati dal vivo (14 Team reali); criteri e accesso esterno costruiti, in attesa del 17.11)
Bug: richiesta AI rifiutata per un messaggio con 'role' nullo in un punto imprecisato	mitigato	Filtro difensivo sull'array messages appena prima di ogni chiamata (sia Azure sia Claude) - una voce m l'intera risposta (sezione 20.5)
Bug: login a codice dispositivo (Exchange/Graph) mostrava "nessun login in corso" dopo un successo reale	corretto	Race condition da <code>setInterval</code> - due poll potevano sovrapporsi, il secondo trovava lo stato appen auto-rischedulato, sovrapposizione impossibile per costruzione (sezione 20.5)
Bug: login Delegato SharePoint/Teams falliva sempre (AADSTS700016, client_id indovinato)	corretto	Il flusso a codice dispositivo custom per SharePoint/Teams richiedeva un <code>client_id</code> OAuth verificato cor - sostituito con <code>-Interactive</code> / <code>-UseDeviceAuthentication</code> nativi (bloccano il server su un click (sezione 17.10)
Bug: pulsante "Connetti SharePoint/Teams" continuava a rifiutare il login anche dopo il fix precedente	corretto	<code>\$script:M365OpsContext</code> letto direttamente in Server.ps1 (fuori dal modulo) è sempre vuoto - ste progetto. <code>isDelegated</code> risultava sempre <code>false</code> , bloccando il login interattivo anche su tenant Del <code>M365OpsActiveTenantInfo</code> (sezione 17.10) - trovato anche lo stesso bug, mai innescato, nel codice
Testo: riferimento al tab sbagliato ("MCP/Connettori" invece di "Tenant") in più messaggi di errore	corretto	La sezione "Stato connessioni" (Exchange/SharePoint/Teams/Intune) è nel tab Tenant dalla riorganizza Exchange/SharePoint/Teams e il system prompt dell'IA citavano ancora la vecchia posizione
App Registration dedicata per il login SharePoint interattivo (discovery automatico)	funzionante	<code>Sync-M365OpsSharePointAppRegistration</code> : scopre via Graph se l'app esiste già nel tenant e ne s per crearla - verificato dal vivo sia il percorso "trovata" (Fabrikam-Prod) sia "non trovata" (contoso-tes entrambi i casi (sezione 17.10)
Bug: catalogo comandi deterministico intercettava richieste piene ("dispositiv"/"non conform" come sottostringa)	corretto	Trigger troppo generici (<code>ListDevices</code> , <code>ListNonCompliant</code> in <code>CommandCatalog.ps1</code>) rispondev completamente diverse che nominavano "dispositivo"/"non conformi" di sfuggita (troubleshooting Te arrivando all'AI - trovato con un batch di 55 prompt reali estratti da mail. Trigger ristretti a un'intenzio
Scrittura SharePoint (creazione sito, membri, ereditarietà permessi, sharing, quota)	funzionante	5 cmdlet nuove via PnP.PowerShell, stesso meccanismo di proposta/conferma del resto del modulo - \ ritrovandolo in una query separata (sezione 17.9.1)
Scrittura Teams (crea/modifica/rimuovi Team, assegna policy)	funzionante	4 cmdlet nuove - creazione Team verificata dal vivo (via Graph); assegnazione policy verificata fino al li stesso permesso "Skype and Teams Tenant Admin API" già mancante per la lettura (sezione 17.11.1)
Bug: catalogo intercettava anche richieste composte di export ("scarica...dispositivi...e invialo a...")	corretto	Stesso schema del bug sopra su <code>ExportDevices</code> : un report con colonne personalizzate + invio ema colonne richieste sia "invialo a" (nessuna parola "email"/"mail", solo un indirizzo) - aggiunto <code>DeferWo</code>
Bug: report generato dal catalogo non trovato da un invio email richiesto in un turno successivo	corretto	<code>\$script:LastReportPath</code> impostato da Server.ps1 (catalogo) non è visibile al modulo (dove vive scope vista più volte in questo progetto, stavolta nella direzione opposta. Aggiunta <code>Set-M365OpsLas</code>
Bug: trigger catalogo troppo generici trovati in un secondo batch di test (MfaStatus, assegnazione app)	corretto	<code>MfaStatus</code> ('metodi di autenticazione' da solo) e il rilevamento 'assegna...gruppo' in Server.ps1 (per CA/MFA e di processo/ticketing completamente estranee - stesso schema del batch precedente
Bug: due trigger latenti del catalogo trovati per ispezione proattiva (mai innescati dal vivo)	corretto	<code>CompliancePatterns</code> ('pattern' da solo) ed <code>ExportCompliancePatterns</code> ('esporta.*analisi' senza problema reale, non dopo averlo osservato
Packaging app custom da script (.ps1/.bat/.cmd), oltre a .exe/.msi	funzionante	<code>New-M365OpsWin32App</code> era già agnostico al tipo di file (nessuna modifica al packaging .intunewin ir (<code>powershell.exe -ExecutionPolicy Bypass -File</code> per .ps1, invocazione diretta per .bat/.cmd) (<code>FileExists</code> / <code>RegistryExists</code>) accanto a quella su versione file. A differenza di un exe, uno scrip non viene MAI dedotta, va sempre indicata esplicitamente nel messaggio - se manca, il tool la chiede end: script .ps1 caricato, pacchettizzato con detection a file, app Intune reale creata e riletta da Graph Verificato anche con .bat
BUG SERIO: <code>Get-M365OpsManagedDevices -NonCompliantOnly</code> restituiva i dispositivi SBAGLIATI	corretto	Il filtro server-side OData <code>\$filter=complianceState ne 'compliant'</code> su <code>/deviceManagement/</code> richiesto - su un tenant con 5 dispositivi noncompliant e 2 compliant, il filtro ha restituito i 2 complian questo campo). Trovato durante un giro di bug-hunting proattivo del 18/08/2026 confrontando l'elen SEMPRE tutti i dispositivi e filtrando lato client con <code>Where-Object</code> , mai fidandosi del <code>\$filter</code> sen restituisce esattamente i 5 dispositivi non conformi reali
Bug: 5 ulteriori trigger del catalogo troppo generici, trovati con un giro di stress-test mirato	corretto	Stesso schema ricorrente (18/08/2026): <code>ExportMailFlowReport</code> / <code>ExportForwardingReport</code> / <code>ExportMailboxUsageReport</code> / <code>ExportSh</code> intercettavano "report su X ... e inoltralo/mandalo a persona Y" (un indirizzo email presente basta a se report sui forwarding"); <code>ListNonCompliant</code> troncava una richiesta che chiedeva ANCHE un piano di una domanda di formazione/spiegazione; <code>Help</code> (^aiuto) intercettava un vero problema dell'utente DeferWords mirati, verificati dal vivo uno per uno

Bug: <code>Set-PnWeb -BreakRoleInheritance / -ResetRoleInheritance</code> non esistono in PnP.PowerShell 3.4.0	corretto	Rimossi rispetto a versioni precedenti della libreria. Sostituiti con la REST API diretta (<code>Invoke-PnPSPF..._api/web/breakroleinheritance / resetroleinheritance</code>), che richiede un URL ASSOLUTO (u del vero errore) e, per "break", un body esplicito anche vuoto ("{}") - altrimenti "Cannot handle the data Scoperto nello stesso giro: SharePoint rifiuta SEMPRE questa operazione sulla WEB ROOT di un sito (" della piattaforma (la web root non ha un genitore da cui ereditare), non un bug M365Ops - tradotto in critico. L'operazione ha senso solo su sotto-siti o liste/raccolte, non ancora supportati da questo cmd
Nota: il bug di serializzazione MicrosoftTeams (sezione 17.11.1) colpisce anche <code>Set-Team</code> , non solo l'assegnazione policy	limite noto	Trovato durante il bug-hunting del 18/08/2026: lo stesso errore "Dictionary...Keys must be strings" di una modifica Team (rinomina) tramite <code>Set-Team</code> , non solo sul percorso legacy Cs* legato al permess ampio/intermittente del modulo <code>MicrosoftTeams</code> (versione 1.7.3.0), non limitato alle sole cmdlet di API". <code>New-Team</code> invece ha funzionato correttamente in un test precedente nella stessa sessione - il p trattato come limite ambientale noto da monitorare, non (ancora) un bug risolvibile in M365Ops
Nuovo: <code>Invoke-M365OpsProvisioningRecipientDiagnostic</code> - diagnosi provisioning mailbox	funzionante	Diagnostica in un colpo solo i problemi dietro NDR 550 5.1.10/mailbox introvabile/mailbox che non si ExchangeGuid con una mailbox inattiva/soft-deleted, indirizzi proxy duplicati su altri oggetti, stato Ent Bug trovato PRIMA di essere mai usato dal vivo: <code>Get-EXOMailbox</code> (a differenza del classico <code>Get-Ma senza -Properties ExchangeGuid</code> esplicito, il confronto ExchangeGuid (il cuore della diagnosi) av rilevare un conflitto reale ma senza dare nessun errore. Verificato dal vivo tramite chat sia su una mail inesistente (correttamente segnalato come oggetto mancante, non provisioning bloccato)
Nuovo: <code>Get-M365OpsMoveRequestDiagnostic</code> - diagnosi dettagliata migrazione mailbox	funzionante	Stesso livello di dettaglio di <code>Get-MoveRequestStatistics -IncludeReport -DiagnosticInfo</code> " il Move Request classico, poi ripiega su <code>Get-MigrationUserStatistics</code> (stessi parametri) per gli u <code>M365OpsMigrationBatch</code> , che non usano un Move Request diretto. Supporta anche <code>-ExportClick completo (non solo i campi riassunti), utile per un ticket Microsoft Support</code>
Nuovo: workaround OneDrive "accesso negato nonostante lo sharing sia corretto"	funzionante	3 cmdlet che coprono i passi automatizzabili del workaround noto Microsoft (concedi/revoca accesso rimuovi il destinatario dalla condivisione): <code>Grant-M365OpsOneDriveDelegateAccess</code> , <code>Remove-M365OpsOneDriveDelegateAccess</code> . Il passo di verifica sulla pagina "membri" (<code>people.aspx?membri un browser interattivo</code>) - il primo cmdlet restituisce l'URL diretto pronto da aprire
BUG SERIO trovato dal vivo nel primo test end-to-end del workaround sopra	corretto	<code>Remove-M365OpsOneDriveSharingRecipient</code> non distingueva un permesso "owner" (che rappres amministratore della raccolta siti, MAI una condivisione fatta a un collega - quelle sono sempre "read" rimuovere coincideva con l'amministratore appena delegato, la scansione trovava e tentava di cancell collection admin - la cancellazione non aveva pero' alcun effetto reale (e' un permesso calcolato, non danno reale, ma il comando riportava falsamente "39 permessi rimossi" senza aver rimosso nulla di ve scansione/rimozione - non sono mai il bersaglio corretto per questo comando. Stato del tenant di tes
Nuovo: retention policy Purview (<code>Get-M365OpsRetentionCompliancePolicies</code>)	in attesa di permesso	<code>Connect-M365OpsCompliance</code> (Connect-IPPSession, stesso certificato di Exchange) verificato dal v ruolo RBAC Purview dedicato (sezione 4.5) non ancora assegnato sul tenant di test - <code>Connect-IPPS assegnati (oggi solo Quarantena), quindi <code>Get-RetentionCompliancePolicy</code> risulta "term not reco spiega gia' correttamente la causa e rimanda alla sezione 4.5 - verificato dal vivo tramite chat</code>
Nuovo: Knowledge Base per tenant (tab "Documentazione")	funzionante	Upload .txt/.md/.docx/.xlsx/.xls/.pdf (Pdflexer per i PDF, installato automaticamente al primo uso) dal sola volta all'upload (titolo/argomenti/riassunto), poi il catalogo (testo leggero) resta sempre nel prorr completo si recupera con lo strumento <code>kb_query</code> solo quando davvero serve. Isolamento per tenan test con un "codice segreto" di prova, l'AI lo ha recuperato e citato correttamente SOLO su quel tenan l'AI ha rifiutato esplicitamente di inventare una risposta invece di confondere i due tenant. <code>kb_query sempre e solo <code>\$script:M365OpsContext.Name</code> (il tenant realmente attivo in quel momento), garar convenzione</code>
Bug reale trovato dal vivo testando la cancellazione di un documento dalla Knowledge Base	corretto	Stessa classe di bug <code>ConvertTo-Json</code> gia' vista altrove nel progetto (array a un elemento appiattito sbdola: rimuovendo l'ULTIMO documento, l'array diventava vuoto e " <code>\$catalogVuoto ConvertTo pipeline - <code>Set-Content</code> a valle non veniva mai eseguito, il file restava quello vecchio, e la funzione r usando <code>ConvertTo-Json -InputObject</code> (mai in pipeline) in <code>Add-/Remove-M365OpsKnowledgeDo documenti fino a un catalogo vuoto</code></code>
Nuovo: pubblicazione su GitHub e aggiornamento in-app (canale Stable/Testing)	funzionante	Repository pubblicato (sorgente sanificato, nessun dato del tenant reale), tab Manutenzione mostra la aggiornamenti"/"Aggiorna ora" - vedi sezione 17.14. Verificato dal vivo: lettura versione da <code>M365Ops. api.github.com</code> (nessuna Release ancora pubblicata al momento del test, gestito correttamente cc
BUG reale: repository pubblicato ma lasciato Privato per errore	corretto	Con un repository Privato, GitHub risponde 404 (non un errore di permesso esplicito) a QUALUNQUE design non usa mai un token (deve funzionare per chiunque scarichi l'app, non solo per il proprietario pubblicata" anche con una Release gia' presente. Trovato dal vivo verificando <code>api.github.com/repc cui lo interroga l'app</code>) - risolto rendendo il repository Pubblico da Settings → Danger Zone, verificato s
Nuovo: prima Release pubblicata (<code>v0.1.0</code>)	funzionante	Baseline pubblica, non pre-release (canale Stable). Verificato dal vivo che l'app la riconosca come versi restituiscono lo stesso risultato quando esiste una sola Release)
Nuovo: <code>Get-M365OpsLogHistory</code> - ricerca log su piu' giorni dalla GUI	funzionante	Il tab Manutenzione mostrava solo il log DI OGGI (ultime 200 righe) nonostante lo storico su disco no modulo elimina i file <code>Logs\m365ops-*.log</code> - restano indefinitamente, gia' oggi ben oltre 6 mesi). A (oggi/7/30/183/365 giorni/tutto), testo libero e livello (Info/Warn/Error) - verificato dal vivo: filtro per 17.11.1), ricerca testo "kb_query" trova le 4 righe corrette
Nuovo: timestamp sui messaggi in chat	funzionante	Ogni messaggio mostra ora data/ora (formato it-IT) - salvato in <code>Add-M365OpsChatHistoryTurn</code> , m (non solo sui messaggi live)
Nuovo: <code>Remove-M365OpsTenant</code> - rimuovi un profilo tenant dal tab Tenant	funzionante	Rimuove SOLO la registrazione in <code>tenants.json</code> (Tenant ID, Client ID, riferimento al secret) - non to accumulati per quel tenant, che restano sul disco. Blocca esplicitamente la rimozione del tenant ATTUA

		ripetuto lato server) per non lasciare l'app senza un tenant attivo. Verificato dal vivo con un profilo usa scrittura), rimozione di un profilo non attivo confermata, integrita' degli altri profili reali verificata byte
Giro di test dal vivo su tutto il lavoro della giornata (18/08/2026)	completato	<code>Install-M365OpsUpdate</code> eseguito per la PRIMA volta per davvero (mai testato prima): copia isolata di v0.2.1 da GitHub, sovrascrittura, verificato byte per byte che i file nuovi siano corretti e che nessuna stata toccata - funziona correttamente al primo utilizzo reale. Rimozione profilo tenant testata con un pulita, integrita' dei profili reali confermata). Ricerca log testata su tutti i filtri incrociati (periodo/testo/
BUG reale trovato dal vivo: <code>Get-M365OpsLogHistory</code> , periodo "Oggi" includeva anche ieri	corretto	Off-by-one nel calcolo della soglia: <code>oggi.AddDays(-Days)</code> con <code>Days=1</code> produceva la mezzanotte soglia" - "Oggi" mostrava quindi anche le righe del giorno precedente (verificato dal vivo: la ricerca co non solo del 18/08). <code>-Days</code> conta i giorni TOTALI da includere (oggi compreso), la soglia corretta arr dal vivo su tutti i periodi (oggi/7/30 giorni)
BUG reale trovato dal vivo: timestamp chat, lo storico precedente alla funzione mostrava un orario falso	corretto	Segnalato dall'utente con uno screenshot: messaggi con orario 15:53 comparivano PRIMA di un mess: nell'array era in realta' sempre corretto - il problema era che le voci di storico salvate PRIMA dell'intro "adesso" come ripiego, un valore che cambia ad ogni reload della pagina e puo' superare l'orario reale <code>undefined</code> (messaggio live, "adesso" e' corretto) da <code>null</code> esplicito (voce di storico senza timestan inventato) - verificato dal vivo su entrambi i casi
Giro di stress test "hard" su tutti i servizi (18/08/2026): 45 prompt reali su Exchange/Entra ID/Intune/Teams/SharePoint/Purview	completato	Batch di richieste realistiche (dirette, composte, con typo, ambigue) eseguito dal vivo sul tenant di test analizzati riga per riga. Ha portato a 3 correzioni di routing (sotto) + 1 bug serio nel livello AI (sotto) + distribuzione, creati e ripuliti per davvero) + un limite ambientale diagnosticato (vedi sotto)
BUG SERIO trovato dal vivo: <code>EmailLastReport</code> inviava il report SBAGLIATO se il messaggio nominava un argomento diverso dall'ultimo generato	corretto	"mandami via mail un report dei dispositivi non conformi a X" dopo aver appena generato un report C sbagliato - nessun controllo di coerenza tra l'argomento richiesto e <code>\$script:LastReportPath</code> . Cor report giusto) ogni messaggio che descrive esplicitamente il contenuto voluto ("report dei/delle/di...") generici ("invalio", "mandalo"). Riverificato dal vivo: ora genera e propone il report corretto
BUG SERIO trovato dal vivo, stessa sessione di test: <code>ListNonCompliant</code> ignorava silenziosamente "mandalo via mail" in una richiesta composta	corretto	Corretto il bug sopra, la stessa richiesta cadeva SUBITO dopo su questa voce del catalogo, che mostra mail" - nessuna email mai proposta. Aggiunti <code>mail</code> / <code>email</code> ai DeferWords: una richiesta che nomin richiesta composta (genera + propone invio) in un solo ragionamento
BUG trovato dal vivo: trigger "Help" non deviava un problema reale con verbi coniugati diversamente dai DeferWords letterali	corretto	"aiuto, un dipendente non riesce piu' ad ACCEDERE alla mail" (terza persona, verbo "accedere") non ve ("accesso" un sostantivo, 'non riesco' solo prima persona) - l'utente riceveva l'elenco comandi invece di piu' ampi ('access\w*jacced\w*', 'non riesc\w*') che coprono sostantivo e coniugazioni verbali comuni
BUG SERIO trovato dal vivo: <code>teams_query</code> non aveva MAI avuto un case nello switch di dispatch	corretto	Lo strumento e' dichiarato nello schema mostrato all'AI (che lo chiama per davvero, log "Tool AI chiam nel <code>default</code> dello switch, che restituisce "Tool sconosciuto" - l'AI riportava all'utente un fuorviante " <code>M365OpsTeamsPolicies</code> / <code>Get-M365OpsTeamsExternalAccessConfig</code> , le uniche due cmdlet che p (list/canali/membri restano invisibili al problema perche' l'AI le recupera quasi sempre via <code>graph_api</code> 18/08/2026), mai innescato prima perche' mai chiamato per davvero in un test dal vivo. Aggiunto il ca <code>sharepoint_query/compliance_query</code>) - verificato dal vivo su entrambe le cmdlet, ora mostrano corrett messaggio fuorviante
BUG trovato in combinazione col precedente: <code>Get-M365OpsTeamsExternalAccessConfig</code> senza <code>-ErrorAction Stop</code>	corretto	Stessa classe di bug gia' corretta il 17/08/2026 in <code>Get-M365OpsTeamsPolicies</code> per lo stesso identic <code>-ErrorAction Stop</code> , "Access Denied" e' un errore NON terminante che produce un array vuoto (<code>\$ cmdlet Cs</code> della funzione
Limite ambientale reale diagnosticato dal vivo: conflitto di assembly .NET dopo aver usato molti servizi diversi nello stesso processo server	limite noto	Una scrittura Exchange reale (creazione mailbox condivisa) e' fallita con <code>Could not load file or Version=8.6.0.0' ... 0x80131040</code> sul processo server che aveva gia' usato Graph, PnP (SharePoin lunga. Isolato il problema testando la STESSA identica scrittura in un processo PowerShell fresco (appo colpo. Confermato anche sul server reale: fallita prima del riavvio, riuscita subito dopo un riavvio pulit - e' un conflitto di versioni tra le dipendenze .NET condivise di piu' moduli PowerShell pesanti (Graph/ stesso processo a lungo termine. Workaround affidabile: riavviare il server (tab Manutenzione) p passati da molti servizi diversi nella stessa sessione.
Nuovo: <code>Remove-M365OpsSharedMailbox</code> - colma il gap trovato durante lo stress test	funzionante	<code>New-M365OpsSharedMailbox</code> / <code>New-M365OpsRoomMailbox</code> / <code>New-M365OpsEquipmentMailbox</code> es corrispondente - la pulizia di una mailbox di risorsa richiedeva il cmdlet nativo fuori da M365Ops. Re nativo (funziona anche su sale/attrezzature nonostante il nome) - verificato dal vivo end-to-end: mailb verificata assente dopo
Nuovo: rilevamento automatico del conflitto di assembly .NET, con riavvio proponibile	funzionante	L'errore <code>0x80131040</code> (conflitto tra i moduli PowerShell caricati nella sessione, vedi voce sopra) viene scrittura fallita passa comunque) PRIMA della diagnosi AI generica, che non conosce questa causa spe immediata e deterministica che spiega la causa reale e propone il riavvio con un semplice 'si' (stessa ir di azione <code>RestartServer</code>). Verificato: il pattern di rilevamento corrisponde esattamente al messagg di riavvio riusa quello gia' collaudato del pulsante Manutenzione - non riprodotto una seconda volta c comando (dipende dall'ordine di risoluzione assembly di .NET), verificato quindi sul pattern esatto inv
Nuovo: porta del server personalizzabile, con rilevamento automatico di conflitti	funzionante	Richiesto dall'utente dopo aver notato la porta 8743 fissa. Preferenza persistita in <code>Config\server-pi avvio completo</code> - "Riavvia server" resta volutamente sulla porta gia' in uso, per non lasciare la scheda c la porta di partenza (preferenza o default 8743) e' gia' occupata da un altro programma, <code>Server.ps1 \$listener.Start()</code> come test (non un proxy separato, per evitare falsi positivi/negativi con http.sy che <code>Launch-M365Ops.ps1</code> legge per sapere sempre su quale porta aprire il browser o riconoscere u reale (porta occupata dal server stesso, in una copia isolata): rilevato, passato automaticamente alla pc
Nuovo: copertura estesa Exchange Online - 36 cmdlet su 4 aree (gruppi, Tenant Allow/Block spoof, mail flow/connettori, quarantena policy)	funzionante	Richiesto dall'utente dopo un censimento dei cmdlet EXO ufficiali (1.438, scaricati dal repository sorge copertura reale - vedi sezione 17.15 per l'elenco completo e cosa resta comunque fuori scope
BUG SERIO trovato dal vivo durante il test della nuova area spoof: stessa classe "errore non terminante ignorato in silenzio" gia' vista per Teams,	corretto	<code>New-M365OpsTenantAllowBlockListSpoofItem</code> riportava "Fatto" anche quando <code>New-TenantAll server Microsoft ("A server side error has occurred..."),</code> perche' mancava <code>-ErrorAction Stop</code> - lo st

applicata preventivamente a tutti i 27 nuovi cmdlet di scrittura		M365OpsTeamsExternalAccessConfig trovato poche ore prima nello stesso giorno. Corretto su TU' quello dove si e' manifestato), non solo su quello dove il bug e' stato osservato per la prima volta - ve per questa specifica funzione, non un bug M365Ops) ora viene mostrato correttamente invece del fals
BUG reale trovato dal vivo dalla diagnosi automatica dell'app stessa: Remove-QuarantinePolicy non supporta -Confirm	corretto	A differenza di quasi ogni altro Remove-* di Exchange (verificato: tutti gli altri Remove- di questa es: QuarantinePolicy non lo espone nella sua sintassi ufficiale - passarlo produceva un errore di bindir autodiagnosi dell'app (Invoke-M365OpsErrorTriage) durante un test dal vivo di pulizia, non da revisior esatta (rifiutata l'applicazione automatica per prudenza, applicata a mano). Verificato dal vivo end-to-e verificata assente
BUG SERIO trovato dal vivo (non collegato alle estensioni sopra): un client disconnesso a meta' risposta poteva far crashare l'INTERO processo server, non solo quella richiesta	corretto	Il server e' realmente crashato durante il test odierno (processo terminato, porta non piu' in ascolto) d mentre il server stava ancora scrivendo la risposta ("The specified network name is no longer available richiesta provava a sua volta a scrivere una risposta d'errore sulla stessa connessione ormai morta, sen ("This operation cannot be performed after the response has been submitted") risaliva non gestita finc corso. Corretto avvolgendo sia la scrittura della risposta d'errore sia la chiusura dello stream nel fina ora fa fallire silenziosamente solo quella singola richiesta, mai l'app per tutti gli altri. Verificato dal vivo con timeout forzato, anche se la stessa identica race condition di rete non e' riproducibile a comando
BUG di documentazione trovato dal vivo dall'utente: la guida si contraddiceva su Node.js/PowerShell 7 (sezione 3 diceva "si installano da soli", sezione 18.4 diceva "no, manuale")	corretto	La tabella di 18.4 era rimasta ferma a prima dell'introduzione dell'auto-installer (sezione 3.1, 17/08/20: stato vero, e resa coerente riscrivendo anche 18.5 (checklist PC nuovo, che indicava ancora .\Start-
Nuovo: installazione di Node.js e Microsoft Edge resa completamente silenziosa all'avvio, nessun click richiesto	funzionante	Richiesto esplicitamente dall'utente ("deve esserci l'autoinstallazione di tutto cio' che manca per esser pulsante "Installa automaticamente" nel banner Impostazioni. Launch-M365Ops.ps1 ora chiama Ir silenziosamente, PRIMA di avviare il server - stesso principio gia' in uso per PowerShell 7 in M365Ops. caso raro in cui il controllo silenzioso fallisca (winget assente/bloccato)
Guida riscritta per non presentare piu' comandi PowerShell come modo d'uso dell'app (richiesto esplicitamente due volte dall'utente)	completato	Le sezioni 7 (registrazione tenant), 9 (riconnessione Lokka), 10.3 (setup delegato) e 12.2 (avvio manual M365OpsTenant , setx , Connect-M365OpsLokka ...) per operazioni che hanno gia' tutte un equiva B (intero "reference rapido" di comandi, incluso un esempio con lo script di avvio ormai superato) e' st ogni operazione. Restano documentati come comandi PowerShell SOLO i passaggi realmente esterni i ruolo RBAC Exchange, trasferimento certificato su PC nuovo) - mai piu' presentati come "modo d'uso"
Nuovo: tab Tenant riorganizzato ad accordion - stato connessioni annidato dentro la riga del tenant attivo	funzionante	Richiesto dall'utente: prima il blocco "Stato connessioni" (Exchange/SharePoint/Teams/Intune) restava mostrando dati non pertinenti se non coincideva col tenant attivo. Ora ogni riga tenant ha un proprio pannello reale (spostato li' via appendChild , mai clonato - gli ID interni come mcp-exchange-sta default; le righe inattive mostrano solo un avviso "attiva questo tenant per vedere lo stato delle conne test, AlePiras): espandi/collassa, hint sulle righe inattive, nessun errore in console
BUG SERIO trovato dal vivo testando l'accordion sopra: il pannello reale spariva del tutto al SECONDO cambio di tenant attivo (non al primo)	corretto	#active-tenant-detail vive nell'HTML statico come fratello di #profile-list solo al primissir spostato DENTRO una riga (annidato in #profile-list). loadProfiles() svuotava incondizion: distruggendo per sempre il pannello reale (con tutti i suoi listener) se gia' annidato li' da un render pr da #profile-list) ma riproducibile al secondo cambio tenant. Corretto staccando #active-teni dopo. Verificato dal vivo ripetendo esattamente lo scenario che aveva rotto: due cambi di tenant cons presente e funzionante a ogni passaggio
Secondo giro di stress test "hard" (18/08/2026), mirato ai 36 cmdlet EXO estesi mai esercitati dal vivo e alla coerenza dei flussi di collegamento dopo il rework dell'accordion	completato	28 richieste reali su vnsys-test (letture + scritture con conferma reale, oggetti "StressTest2-"/"StressTes quarantena policy, Tenant Allow/Block List spoof, mailbox quarantine, mail flow/connettori; piu' verific list/attiva, test Exchange/SharePoint/Teams/Intune, install-prerequisites, update-status, server-port) - r reali (sotto) + 2 limiti ambientali confermati (sotto)
BUG SERIO trovato dal vivo: trigger catalogo MfaStatus ignorava silenziosamente la seconda meta' di una richiesta composta	corretto	Stesso schema gia' visto 3 volte in questo progetto (EmailLastReport/ListNonCompliant/Help): "contro veniva gestito interamente dal catalogo a costo zero (trigger 'stato mfa)', che si limita a eseguire Get "e poi..." spariva senza che nessuno (ne' il catalogo ne' l'AI) la eseguisse mai. Corretto aggiungendo p DeferredWords di MfaStatus, che ora fanno passare l'intero messaggio all'AI quando rilevate. Riverificato correttamente entrambe le parti
BUG reale trovato dal vivo dalla diagnosi automatica dell'app: Update-M365OpsDistributionGroupMember non poteva svuotare un gruppo	corretto	"sostituisci i membri con nessuno, svuotato" e' un uso esplicitamente documentato della funzione, ma [string[]] senza AllowNull() / AllowEmptyCollection() - PowerShell rifiutava il binding di con un errore di binding parametro invece di eseguire la sostituzione con lista vuota. Trovato dalla ste dal vivo, che ha anche proposto la correzione esatta. Corretto e riverificato dal vivo end-to-end: grupp verificato vuoto, eliminato
BUG SERIO trovato dal vivo: Enable-/Disable-M365OpsMailboxQuarantine non potevano MAI funzionare su Exchange Online	rimossi	I cmdlet nativi Enable-MailboxQuarantine / Disable-MailboxQuarantine su cui erano costruit disponibili solo su Exchange Server on-premises (2013/2016/2019/SE) - "term not recognized" su Ex verifica commesso durante l'estensione del 18/08/2026 (controllata la sintassi dei parametri ma non l'i: supporta). Rimossi entrambi i wrapper e i relativi riferimenti nello strato AI invece di lasciare una funzio propone correttamente un'alternativa reale via Graph (disabilitazione del sign-in dell'account), verifica
Limite ambientale confermato (non nuovo, ri-osservato 2 volte in piu' con parametri diversi): New-M365OpsTenantAllowBlockListSpoofItem con SpoofType: External fallisce sempre con "A server side error has occurred" su vnsys-test	limite noto	Riprodotta altre due volte con valori diversi (indirizzo email completo e poi solo dominio come Spoof l'esempio ufficiale Microsoft) - stesso identico errore lato server sempre. Sommato al primo caso gia' c completato con successo su questo tenant nonostante sintassi verificata corretta 4 volte. Non e' un dil documentazione ufficiale fallisce), probabile gate di licenza/servizio lato Microsoft (es. piano Defender spoof intelligence) non ancora confermato - -ErrorAction Stop continua a impedire correttamen
Nota informativa (non un bug): il tenant vnsys-test risulta rinominato in passato da "vnsysit" a "vnsys"	solo informativo	Trovato per caso verificando un'apparente discrepanza: Get-M365OpsAcceptedDomains (Exchange) vnsys.it / vnsys.onmicrosoft.com / vnsys.mail.onmicrosoft.com , ma vnsysit.onmicroso (usato con successo in questo stesso giro di test) perche' e' il dominio ORIGINALE del tenant in Entra I

file:///C:/Users/diego_o6s3y1g/Documents/claude/M365Ops/docs/Guida-Configurazione.html 33/82

		DeviceManagementRBAC.ReadWrite.All / DeviceManagementServiceConfig.ReadWrite.All sezione 17.16 per lo stato dettagliato di ogni area
BUG trovato durante il syntax-check del nuovo codice: "\$Identity:" dentro una stringa fra doppi apici viene interpretato da PowerShell come variabile con scope/drive (\$Identity:), non come variabile seguita da due punti letterali	corretto	Errore di parsing ("': was not followed by a valid variable name character") su due dei nuovi cmdlet (Set-M365OpsAdminTemplateSetting) che scrivevano "Configurazione \$Identity: priorit� \${{Identity}}: (delimitatore esplicito) invece di \$Identity: . Cercato lo stesso schema su tutti gli trovata) prima di considerare il fix completo
Stress test massiccio pre-commit (19/08/2026) sulla seconda ondata Intune - prompt composti, casi limite, cicli scrittura reali con conferma, richiesta esplicita dell'utente	completato	~15 richieste reali su vnsys-test: letture composte multi-area, un piano a 2 scritture con fallimento par notifica e anello di aggiornamento, tentativo di doppia proposta nella stessa risposta (correttamente r - vedi le 3 voci sotto
BUG SERIO trovato dal vivo: ListNonCompliant (catalogo deterministico) troncava silenziosamente la seconda meta' di una richiesta composta nonostante contenesse "e poi"/"e anche"	corretto	Stesso schema gia' corretto 8 volte in questo progetto (EmailLastReport/ListNonCompliant/Help/MfaStatus/ListDevices/CompliancePatterns/UserOverview/G DeferWords erano gia' state estese con parole SPECIFICHE (piano/remediation/mail) in un fix preceder gia' a posto, senza mai ricevere le parole di continuazione GENERICHE ('e poi'/'e anche'/'quindi') aggi dispositivi non conformi e poi dimmi se ci sono anelli di aggiornamento Windows configurati e anche dispositivi. Corretto aggiungendo le 3 parole generiche; riverificato dal vivo con la stessa identica richi
BUG reale trovato dal vivo: New-M365OpsNotificationTemplate falliva sempre con 400 "An error has occurred" (nessun dettaglio dal servizio Intune) alla creazione	corretto	La diagnosi automatica dell'app (Invoke-M365OpsErrorTriage) ha proposto DUE correzioni sbagliate d brandingOptions in array) - entrambe respinte prima di applicarle, verificando lo schema Graph gi� ipotesi). Isolata la causa reale per esclusione, provando ogni proprieta' del corpo singolarmente in una scrivibile in creazione a QUALUNQUE valore/formato (provato "en-us"/"en-US"/"it-IT", falliti identicam creando la prima localizedNotificationMessage con isDefault=true (gia' il comportament non andava anche passata sul corpo del template). Rimossa dal corpo di creazione; riverificato dal vivc popolato correttamente, eliminazione) - lezione: non fidarsi della diagnosi AI di un errore Graph opacc puo' sbagliare con sicurezza tanto quanto un umano
BUG GRAVE trovato dal vivo: l'AI ha dichiarato di aver proposto ed eseguito con successo l'eliminazione di un oggetto Intune reale, inventando un JSON di risultato plausibile, SENZA MAI chiamare alcuno strumento - l'oggetto non e' mai stato toccato	mitigato	Al messaggio "elimina il modello di notifica X" l'AI ha risposto "Ho proposto l'eliminazione... in attesa c propose_intune_write nei log; al successivo "si" ha risposto "Fatto." con un JSON di risultato inve senza alcuna chiamata. Il codice server (\$script:PendingAction / Execute-PendingAction) ha azione era davvero in sospeso, quindi nessuna esecuzione e' avvenuta) - il bug e' stato puramente nel conversazionale visto poco prima in una lunga sequenza di propose/conferma reali senza davvero rich l'oggetto da Graph subito dopo: esisteva ancora, nonostante "Removed: true" mostrato all'utente. Mit prompt di sistema (generica, vale per ogni propose_*, non solo Intune): vietato scrivere "ho propos strumento in quel turno; se l'utente conferma ("si") ma nel contesto non risulta una vera chiamata pr riproporre per davvero o chiedere chiarimento, mai fingere un completamento. Riprodotto lo scenario registrato una proposta REALE (confermata da /api/status), eseguito con role: system dopo successiva. Trattandosi di un comportamento probabilistico del modello (non un bug deterministico n non lo azzer� per costruzione - resta la raccomandazione permanente di verificare un'azione distruttiv dubbio � lecito
Audit completo delle tabelle permessi (19/08/2026), richiesto esplicitamente dall'utente dopo aver notato che la verifica precedente copriva solo Intune ("parlo di tutto non solo di intune")	corretto	La tabella 4.2 (permessi AppOnly, quella seguita da chi configura il modulo) mancava 9 righe gia' pres stesse identiche funzionalita' - report licenze, log di sign-in, stato/reset MFA, lettura siti SharePoint via Access, ruoli Entra ID. Verificato dal vivo ognuna delle 9 su vnsys-test: 7 funzionavano gia' (permesso c (Reports.Read.All per i report di utilizzo Microsoft 365, IdentityRiskUser.Read.All per i s mostrato per entrambe. Trovato anche un secondo bug nello stesso giro: la tabella 10.8 elencava per c (SecurityEvents.Read.All , che serve per un'API diversa, gli alert Defender) - mai verificato dal viv Microsoft Learn. Aggiunte le 9 righe mancanti a 4.2 e corretto il nome in 10.8
BUG reale trovato dal vivo dall'utente: una domanda di audit sicurezza ("ci sono stati accessi sospetti nelle ultime settimane?") ha ricevuto un'analisi basata SOLO sui primi 100 sign-in, con @odata.nextLink presente ma mai seguito - risposta presentata come valida invece che come campione parziale	corretto	Nessuno strumento/istruzione diceva mai all'AI di seguire la paginazione Graph quando la domanda r presente per ridurre il volume con \$select / \$top , che invece c'era). Verificato dal vivo prima di scr valore di @odata.nextLink COSI' COM'E' come path a graph_api_call restituisce correttame \$skiptoken , nessuna modifica necessaria). Aggiunta una regola esplicita alla descrizione dello strun "quanti in totale", non un rapido controllo), seguire @odata.nextLink fino a un tetto di sicurezza d troncamento invece di presentare un campione come quadro completo - stesso principio gia' in uso p Riverificato dal vivo con la stessa identica domanda dell'utente: 2 chiamate graph_api_call nei log finale che dichiara esplicitamente "controllato completamente... 2 pagine, senza ulteriori nextLink" inv
Nuovo (v0.9.3, 19/08/2026): avvio (Launch-M3650ps.ps1) reso robusto - i fallimenti ora sono visibili invece di silenziosi	funzionante	Lo script gira sempre -WindowStyle Hidden (lanciato da M3650ps.bat): fino ad ora un falliment� risponde entro il timeout di 10s) restava del tutto invisibile, sia per chi fa doppio click sia per un'autor osservato dal vivo mentre un agente AI tentava di riavviare il server dopo una reinstallazione del PC: il cmd /c M3650ps.bat) non ha mai avviato davvero il server (nessun errore visibile, nessuna traccia r risolto solo rilanciando lo script in foreground fino al suo completamento. Aggiunto Config\last-s avvio: istanza riusata, avviato con successo, o fallito con il motivo) e un MessageBox visibile anche a server non pronto entro il timeout) - il controllo prerequisiti (Node/Edge) resta volutamente non blocc come WARNING nello stesso file invece di sparire in una finestra nascosta. Verificato dal vivo: percorsi correttamente, status file scritto (OK Istanza gia' attiva, porta 8743), server rimasto raggiun
Bug-hunt mirato (v0.9.4-0.9.5, 19/08/2026): 21 cmdlet di scrittura senza - ErrorAction Stop su Exchange/SharePoint/Teams/Intune, stesso schema gia' visto altrove nel progetto ma mai propagato	corretto	Un errore reale e non terminante del cmdlet nativo sottostante veniva ignorato in silenzio: la funzione Verificato dal vivo su Exchange (gruppo di distribuzione di test creato, tentata aggiunta di un membro reali, dopo il fix eccezione reale sollevata) e su Teams end-to-end tramite chat reale (proposta -> conf Access Denied catturato e diagnosticato invece di un falso successo). Su SharePoint il caso testato (gia' un'eccezione anche senza il fix - li' la correzione resta difensiva/di consistenza, dichiarato onestam New-M3650psWin32App (modulo IntuneWin32App) anch'esso difensivo, non testato dal vivo (packag
Bug-hunt mirato sul catalogo comandi (v0.9.6-0.9.7, 19/08/2026): risposte locali sbagliate su richieste scritte in modo diverso dal previsto, mai arrivate	corretto	Richiesto esplicitamente dall'utente dopo aver chiesto "siamo sicuri che l'AI intervenga sempre, o il cal - risposta onesta: no, nessuna garanzia c'era. Costruito un harness che verifica sistematicamente, per u

all'AI		ATTESA, con l'AI (sicuro), o con una voce SBAGLIATA (il caso pericoloso). Trovati 5 casi reali: tipo "nn" rispondere <code>ListDevices</code> con TUTTI i dispositivi invece dei soli non conformi; "pattern di conformita richiesta; un verbo tra la negazione e "conforme" (forma grammaticale normale, non un refuso) non ri ignoravano "invia/manda/spedisci via mail" silenziosamente. Trovato anche un bug piu' profondo nel voci): il confine di parola \b finale faceva fallire SEMPRE, in silenzio, ogni DeferWord scritta come radic empiricamente (<code>'consiglio' -match '\bconsigli\b'</code> e <code>'\$false'</code>) e corretto in un solo punto in anche <code>Get-M365OpsNormalizedChatText</code> (apostrofi doppi, abbreviazioni SMS come "nn"/"sn"/"cm (es. "quta" per "questa") restano deliberatamente fuori scope, verificato dal vivo che il motore AI li ges
Causa radice trovata (v0.9.8, 19/08/2026) del bug "role mancante" - prima solo mitigato, sezione 20.5	corretto	Ripreso su richiesta esplicita dell'utente dopo averlo rivisto ricomparire durante il bug-hunt sul catalogo conversazione che parte da uno storico vuoto lo innescava. Causa in due parti verificate empiricamente un array vuoto che crolla a <code>\$null</code> quando il chiamante (<code>Server.ps1</code>) lo assegna senza <code>@(...)</code> zero iterazioni come una collezione vuota, ne esegue una con <code>\$_ = \$null</code> , producendo la voce fantasma nel punto mancante) sia con una rete di sicurezza simmetrica sull'input in <code>Invoke-M365OpsAgentToolbox</code> che prima innescava sempre il bug ora non produce piu' alcun "Filtrate...role mancante" nei log
Diagnosi/auto-correzione di una scrittura fallita estesa con ricerca documentale reale (v0.9.9-0.9.11, 19/08/2026)	corretto	Richiesto dall'utente dopo un caso reale (regola di un Assignment Filter Intune rifiutata da Graph, "per <code>M365OpsErrorTriage</code> (diagnosi post-fallimento) sia <code>Invoke-M365OpsWriteRecovery</code> (propone il sul testo dell'errore, senza alcun modo di consultare documentazione quando l'errore non si auto-spiega capire cosa cercare, poi consultano Microsoft Learn reale (<code>Invoke-M365OpsLookupMsDocs</code> , incluso il payload di hijacking/prompt injection sul nuovo meccanismo: nessuna scrittura pericolosa mai proposta l'altro riconosciuto e ignorato dal modello stesso. Aggiunta anche (v0.9.10) la sintassi verificata delle richieste al sistema (<code>operatingSystemVersion</code> vs <code>osVersion</code> , quale supporta quali operatori) - verificato da produce la regola corretta AL PRIMO tentativo, non solo dopo un fallimento
BUG SERIO: array a un elemento crollato a scalare in <code>ConvertTo-M365OpsHashtable</code> (v0.9.12, 19/08/2026)	corretto	Trovato durante uno stress test multi-round richiesto dall'utente: <code>"groupTypes": ["DynamicMembership"</code> diventava silenziosamente <code>"groupTypes": "DynamicMembership"</code> attraversando questa funzione - di quasi ogni scrittura del modulo, quindi capace di colpire qualunque campo array a un solo elemento "role mancante" sopra (un array restituito da una funzione PowerShell viene enumerato sul flusso di output a scalare), qui pero' annidato in una funzione ricorsiva generica. Corretto con la virgola unaria (<code>return</code> Trovato in parallelo un secondo bug in <code>Invoke-M365OpsWriteRecovery</code> : rifiutava correzioni gia' identificate campo diverso - corretto chiarendo che va risolto solo cio' che l'errore CORRENTE segnala, i round successivi manifestati
Nuovo: <code>generate_raw_graph_export</code> - i report su dati Graph sono sempre righe reali, mai aggregate (v0.9.13, 19/08/2026)	corretto	"fai un report dei sign-in log degli ultimi 7 giorni di diego@vnsys.it" ha prodotto un file Excel con UNA riga (verificato dal vivo). <code>generate_raw_export</code> esisteva gia' per questo problema ma SOLO per le cmdlet equivalenti, quindi con un volume anche solo moderato il modello rischiava di sintetizzare un riepilogo token, nonostante il divieto esplicito nella descrizione dello strumento. Aggiunto lo strumento gemello automatica (@odata.nextLink, tetto 50000 righe) - deliberatamente generale (non specifico ai sign-in) aree diverse: sign-in log (114 righe reali) e utenti+licenze del tenant (55 righe reali). Trovato e corretto "aiuto": la Description di <code>SharedMailboxPermissions</code> conteneva gergo da sviluppatore invece di un
BUG STRUTTURALE: <code>-ErrorAction Stop</code> non protegge da un cmdlet nativo IRRISOLVIBILE, solo da un cmdlet che restituisce un errore morbido (v0.9.14, 20/08/2026)	corretto	Trovato testando <code>New-M365OpsAcceptedDomain</code> (area "implementata ma mai testata dal vivo", sezione <code>AcceptedDomain</code> nativo non fosse mai stato eseguito (stesso limite RBAC di Receive/SendConnector Causa verificata empiricamente in isolamento: quando il NOME del cmdlet non e' risolvibile ("term non viene ignorato (PowerShell lo lega solo DOPO aver risolto il cmdlet) - l'esecuzione prosegue secondo 'Continue'). Questo significa che TUTTE le correzioni <code>-ErrorAction Stop</code> cmdlet-per-cmdlet del bug in questo scenario (cmdlet indisponibile per ruolo RBAC mancante o altra causa), una classe di errore divisa in cmdlet: <code>\$ErrorActionPreference = 'Stop'</code> a livello di SCOPE DEL MODULO in <code>M365Ops.ps1</code> ereditano anche per un comando irrisolvibile, e che <code>Gui\Server.ps1</code> non chiama mai cmdlet native a livello di modulo copre l'intera superficie nativa dell'app. Verificato dal vivo con una regressione ampia di un gruppo, fallimento intenzionale ancora onesto) - nessuna rottura
Nuovo: Knowledge Base GLOBALE + diagnosi errori che consulta anche la KB (v0.9.15, 20/08/2026)	corretto	Richiesto esplicitamente dall'utente: la guida di configurazione ora e' caricabile come documento in un file sempre, a prescindere dal tenant attivo - cosi' si puo' chiedere "come configuro l'app per Exchange Online" e <code>M365OpsErrorTriage</code> e <code>Invoke-M365OpsWriteRecovery</code> ora consultano anche questa KB (oltre a setup/configurazione di questa app. Bug reale trovato caricando la guida (52 pagine): <code>PdfLexer</code> su un file (pagina), non una stringa - il codice lo passava cosi' com'era, quindi <code>.Length</code> contava le pagine (52) errore la catalogazione AI (array passato a un parametro tipizzato stringa). Corretto unendo esplicitamente <code>M365OpsExtractedFileText.ps1</code> - stessa famiglia "array trattato come scalare" gia' vista piu' volte in configurazione via chat risponde con dettagli reali della guida (log: chiamata effettiva a <code>kb_query</code>); <code>Invoke-M365OpsErrorTriage</code> ora cita la documentazione interna invece di rispondere alla cieca, con match per parola chiave (bypassando l'AI)
Nuovo: la guida nella Knowledge Base globale si autoaggiorna, messaggio di benvenuto accorciato (v0.9.16, 21/08/2026)	corretto	Richiesto esplicitamente dall'utente dopo aver notato che il documento appena caricato nella KB globale guida (come quelle della riga sopra, v0.9.15) richiedeva un ricaricamento manuale facile da dimenticare SHA256 di <code>docs\Guida-Configurazione.pdf</code> con l'ultimo caricato e ricarica automaticamente solo la guida e' invariata) - vedi sezione 17.13. Verificato dal vivo con due riavvii consecutivi: il primo (guida mancante) no. Aggiornato anche il messaggio di benvenuto in chat all'avvio (<code>Gui\index.html</code>): prima elencava le domande alla guida ("fammi domande specifiche su cio' che vuoi sapere"), coerente con la nuova possibilita' di chiedere
Nuovo: autenticazione Graph via certificato (senza secret) + messaggio di onboarding al primo avvio (v0.9.17, 21/08/2026)	corretto	Richiesto esplicitamente dall'utente. Prima domanda: "possiamo usare il certificato anziche' il secret?" Entra ID supporta client credentials con un JWT "client assertion" firmato dal certificato al posto del client secret. <code>Connect-ExchangeOnline -CertificateThumbprint</code> per Exchange. Nuova funzione <code>New-M365OpsCert</code> (Cert:\CurrentUser\My poi Cert:\LocalMachine\My, come fa gia' <code>Connect-ExchangeOnline</code>) e <code>Get-M365OpsCert</code> (default, nessun cambiamento per installazioni esistenti), altrimenti il certificato se configurato - mai utilizzato. Verificato dal vivo dentro lo scope del modulo (secret sostituito con una variabile inesistente, cache to

		per una chiamata reale a <code>/organization</code> . Rinominato di conseguenza il campo "Thumbprint certificato App / Exchange, facoltativo)" - non e' piu' specifico di Exchange - e aggiunta una nota che i due campi messaggio di onboarding al primo avvio (nessun profilo tenant salvato) che indirizza al tab Tenant e la panoramica dei permessi per App Registration - nuovo campo <code>firstRun</code> in <code>GET /api/status</code> (il primo posto del messaggio di benvenuto normale finche' non si salva il primo profilo. Verificato dal vivo con <code>Config\tenants.json</code> (mai perso il contenuto reale): <code>firstRun</code> passa correttamente da true a false
Nuovo: check permessi app in tempo reale, pulsante "🔥 Stato permessi" (v0.9.18, 21/08/2026)	corretto	Richiesto esplicitamente dall'utente: "vorrei fare una sezione check permessi app dove... l'interfaccia sia etc, in lettura o anche in scrittura". Discusso con l'utente dove farlo vivere (sotto-sezione statica nel tab coerente con come ogni altro insight di questa app viene presentato. Nuova funzione <code>Get-M365OpsAppPermissions</code> del service principal (<code>GET /servicePrincipals/{id}/appRoleAssignments</code>), le risolve in nomi di permessi radicata nella sezione 4.2/4.3/4.4/6 della guida (mai a memoria) per dare uno stato lettura/Teams (due percorsi distinti: base via certificato, policy via API separata), SharePoint (due percorsi distinti scritta in codice DURANTE lo sviluppo, prima di consegnare: il nome risorsa Graph per SharePoint e' "Graph" come si potrebbe supporre - un primo tentativo con quel nome produceva sempre "nessun accesso" e l'output con le assegnazioni grezze reali di vnsys-test. Nuova voce nel catalogo comandi locali (<code>Comm</code>) precisi, un riassunto Al aggiungerebbe solo rischio) e nuovo pulsante "🔥 Stato permessi" nell'header del pulsante nel browser: risultato consistente con le assegnazioni grezze verificate a mano (3 permessi quanto gia' documentato in sezione 17.16)
Tema chiaro/scuro, pannello Upload a scomparsa, pulsanti header unificati (v0.9.19, 21/08/2026)	corretto	Richiesto esplicitamente dall'utente, in due parti. Tema scuro: pulsante nell'header, scelta persistita in localStorage prima del rendering per evitare un lampo del tema sbagliato - fattibile con una sola nuova regola CSS nessuna riscrittura dei singoli stili. Pannello Upload: i 3 pulsanti di caricamento, prima sempre visibili si è scomparsa (pulsante "📁 Upload" nell'header), stesso pattern toggle del pannello Impostazioni. Osservato "Impostazioni tenant" e "Stato permessi" avevano due grafiche diverse (il secondo, aggiunto in v0.9.18, era una sola classe condivisa (<code>.header-btn</code>), applicata anche ai due nuovi pulsanti. Bug di processo trovato viene tenuto in memoria dal server all'avvio (mai riletto ad ogni richiesta, a differenza di quanto assunto dopo l'edit mostrava ancora la pagina vecchia, risolto con un riavvio del server, poi verificato dal vivo dopo reload, nessun colore rimasto sbagliato in nessun pannello)
Testato dal vivo: Set-M365OpsRemoteDomain e ciclo completo gruppi dinamici; corretta un'informazione sbagliata sulla sezione 6.4 (21/08/2026)	corretto	Ripreso il compito rimasto sospeso dall'inizio della sessione ("copiamo cio' che e' implementato ma non un toggle reale (<code>AutoForwardEnabled</code> sulla voce "Default"), letto indipendentemente, ripristinato a <code>M365OpsDynamicDistributionGroup</code> verificato con un ciclo completo: gruppo dinamico di test creato passaggio confermato con una lettura indipendente, ambiente di test tornato pulito. <code>New-/Set-/Remove-M365OpsDynamicDistributionGroup</code> non provato (azione ad alto impatto, bloccata comunque dallo stesso limite RBAC di Receive/SendConnector in questa stessa guida poche ore prima: la sezione 6.4 affermava "verificato oltre 12 ore dopo l'assegnazione trascorso MAI verificato in modo indipendente, solo assunto in modo sbagliato dal contesto della guida con stato aggiunto circa un'ora fa non 12 ore"). Retestato a ~1 ora reale dall'assegnazione: New-Accepted quell'orizzonte la propagazione resta un'ipotesi plausibile non ancora esclusa con la stessa certezza - questa incertezza invece di un tempo trascorso inventato
BUG: check permessi non riconosceva un permesso Graph piu' ampio come sufficiente per uno piu' stretto (v0.9.21, 21/08/2026)	corretto	Trovato dal vivo dall'utente, non durante lo sviluppo: "l'app ha Sites.FullControl.All che dovrebbe essere il confronto in <code>Get-M365OpsAppPermissionsCheck.ps1</code> era per stringa esatta, quindi <code>Sites.FullControl.All</code> e <code>Sites.Read.All</code> . Corretto riconoscendo la gerarchia standard Microsoft Graph (<code>Read.All < Read.FullControl.All</code> stesso prefisso): un permesso piu' ampio ora soddisfa automaticamente un requisito piu' stretto, sia in console che in browser. Verificato dal vivo via chat dopo il fix: l'area SharePoint via Graph e' passata da "nessun accesso" a "let Intune, che l'utente aveva segnalato come apparentemente simile ma non lo era: i 3 permessi (Device, App, Delegated) risultavano ancora genuinamente assenti dalle assegnazioni reali anche dopo averli "aggiunti" in Entra aggiungere un permesso nella lista "API permissions" non crea un'assegnazione finche' non si preme e le assegnazioni grezze live prima di concludere
BUG STRUTTURALE: Receive-/SendConnector erano cmdlet on-premises-only, mai esistiti in Exchange Online - risolto con InboundConnector/OutboundConnector (v0.9.22, 21/08/2026)	corretto	Richiesto esplicitamente dall'utente: "puoi fare test in modalita' delegata?" sul limite RBAC apparente un utente membro completo di "Organization Management" (verificato con Get-ManagementRoleAssignment domini reali) - CONFERMANDO che Accepted Domain e' un genuino limite App-only/Delegated, ora <code>ReceiveConnector</code> ha dato lo STESSO "term not recognized" anche con quell'utente pienamente provato verificando la documentazione ufficiale Microsoft (non a memoria): <code>Get-/New-/Set-/Remove-M365OpsReceiveConnector</code> cmdlet ESCLUSIVI di Exchange on-premises, mai esistiti in Exchange Online - "this cmdlet is available only in on-premises environments" (sezione 6.4) era in parte una pista sbagliata per questo caso specifico: stesso messaggio di errore testato con 8 wrapper (<code>Get-/New-/Set-/Remove-M365OpsReceiveConnector/SendConnector</code> rimossi, sostituiti da veri verificati su Microsoft Learn (<code>SenderDomains/ConnectorType</code> per Inbound, <code>RecipientDomains/SmtpBindings/RemoteIPRanges/AddressSpaces</code> dei vecchi wrapper, concetti on-premises che non esistono prompt Al e nelle allowlist <code>exo_query/propose_exo_write</code> (Invoke-M365OpsAgentTools.ps1). Verificato create/verifica/modifica/verifica/rimuovi/verifica per ENTRAMBI i connettori, in App-only: funzionano non era mai stato un limite RBAC per questi due cmdlet
Anti-spam/anti-phishing/malware in scrittura, check permessi Delegated con ruoli directory, guardrail disconnect, fix login Teams (v0.9.23, 21/08/2026)	corretto	Round esteso, richiesto esplicitamente dall'utente. (1) Implementate le 3 aree anti-spam/anti-phishing (17.19): 24 nuovi wrapper (<code>Get-/New-/Set-/Remove-</code> per Policy+Rule x3 aree), nessun nuovo ruolo RBAC verificato dal vivo con un ciclo completo per ognuna). Bug reale trovato durante il test: nessuno dei tre Command, non dalla documentazione che suggeriva il contrario) - aggiunti 6 cmdlet Enable-/Disable-per i tenant Delegati (prima si limitava a un errore: nuova <code>Get-M365OpsDelegatedPermissionsCheck</code> nel ManagementRoleAssignment). Due bug segnalati dall'utente subito dopo e corretti lo stesso giorno: i tenant Delegato, e mancanza dei ruoli DIRECTORY Entra ID (Global Admin ecc, sistema separato da RBAC) /me/transitiveMemberOf/directoryRole riusando il token Graph delegato gia' ottenuto. (3) Guardrail: C Teams/SharePoint (prima solo Exchange/Lokka) ad ogni cambio tenant - richiesto esplicitamente dopo (Microsoft.IdentityModel.Tokens) passando da un tenant App-only a uno Delegato nello stesso processo solo un riavvio del processo risolve davvero il conflitto. (4) BUG STRUTTURALE trovato dal vivo (successo UseDeviceAuthentication) bloccava il server per sempre senza mostrare alcun codice da nessuna parte

		corretto rimuovendo il parametro, così il modulo usa il proprio flusso interattivo di default (popup br fix, login completato senza bloccare nulla
Stress test feroce post-v0.9.23: tutto verificato via AI/chat reale, 1 bug reale trovato e corretto (v0.9.24, 21/08/2026)	corretto	Richiesto esplicitamente dall'utente: "esegui un feroce stress test... con prove reali di utilizzo... non tras cerca regressioni... non dare nulla per scontato". Verificato dal vivo, via chat/AI reale (non chiamate dir creato/verificato/disabilitato/rimosso interamente in linguaggio naturale con conferme separate a ogr (non Set- con -Enabled, il bug di v0.9.23); lettura anti-phishing e filtro malware con dati reali del tenan (regressione negativa, nessun impatto dagli ultimi 5 round di modifiche); creazione/verifica indipende consolidata, nessuna regressione); comando locale "aiuto" aggiornato con la nuova voce; guardrail dis (riattivazione tenant senza nulla da disconnettere, nessun errore); 2 tentativi di hijacking (uno bloccato respinto dal ragionamento del modello nonostante rivendicasse un'approvazione già data in una chat "come funziona il check permessi per un tenant delegato?" veniva intercettata dal trigger locale <code>check</code> ESEGUIVA il controllo vero invece di rispondere con l'AI/la guida - corretto aggiungendo <code>DeferWords</code> "perche") alla voce <code>AppPermissionsCheck</code> del catalogo, verificato dal vivo che la stessa domanda c
Audit del codice preesistente su richiesta esplicita ("mai dare per scontato"): 3 bug reali trovati e corretti + 5 critica' da un test su PC pulito (v0.9.25, 21/08/2026)	corretto	Audit sistematico del codice preesistente (mai toccato in questa sessione fino ad ora), richiesto esp si era concentrato solo sulle novità del giorno: "hai dato un occhio agli script preesistenti? mai assum Grep sistematico di tutti i cmdlet nativi usati nel codice (210 nomi distinti) verificati uno per uno contr a memoria - 18 "non risolvibili" trovati, la maggior parte falsi positivi (funzioni locali con nomi coincide in elenchi di pattern pericolosi, riferimenti in commenti a funzioni interne di ALTRI moduli). 3 bug real <code>M365OpsCompliancePatterns.ps1</code> - stesso meccanismo di collasso array già visto più volte (sezior "\$enriched = foreach(...)" collassa a scalare con esattamente 1 dispositivo non conforme, E anche un v ConvertTo-Json collassa comunque un array a 1 elemento - riprodotto empiricamente con un disposit <code>Enable-/Disable-DistributionGroup</code> - TERZA istanza della stessa classe di bug già vista per Re ESCLUSIVI di Exchange on-premises, mai esistiti in Exchange Online, confermato su documentazione u sessione EXO reale) - ma qui, a differenza dei connettori, NON esiste alcun equivalente cloud da sostit premise): corretto facendo fallire IMMEDIATAMENTE con un errore chiaro invece di un criptico "term n disponibile.
Stesso commit: 5 critica' reali segnalate dall'utente durante un test end-to-end su un PC pulito (nessuna delle 5 era mai stata provata dal vivo su un ambiente davvero vergine prima d'ora): 1. Bug di regressione critico: il salvataggio di un profilo tenant richiedeva ANCORA -SecretEnvVar obbligatorio (sia lato JS sia lato <code>Set-M365OpsTenant.ps1</code>) anche dopo che il certificato era diver monte esattamente la configurazione "solo certificato" che quella funzionalità era pensata per abilitare. Corretto in entrambi i punti: serve ClientId sempre, e SecretEnvVar O ExchangeCertThumbpr 2. winget bootstrap automatico: su un PC molto minimale winget stesso può mancare, bloccando l'intera catena di auto-installazione. Aggiunto un tentativo di bootstrap automatico (modulo uffici <code>Repair-WinGetPackageManager</code> - gestisce da solo le dipendenze VCLibs/UI.Xaml, niente URL di pacchetti fissati a mano) prima di arrendersi con istruzioni manuali. 3. Bug reale nell'installazione Node.js: <code>winget install</code> senza <code>--source winget</code> esplicito interrogava ANCHE la sorgente msstore - un errore del backend Microsoft Store ("0x8a15003b Rest A quando il pacchetto era già trovabile nella sorgente giusta. Corretto specificando sempre <code>--source winget</code> . 4. Messaggio fuorviante sulla chiave AI mancante: citava solo la variabile ANTHROPIC_API_KEY, dando l'impressione sbagliata che fosse l'unico provider supportato. Prima correzione (questa riga, v correzione definitiva in v0.9.26 subito sotto. 5. Problema architetturale reale, non solo di messaggistica: senza nessuna chiave AI configurata, ANCHE la guida stessa era irraggiungibile (leggibile solo tramite <code>kb_query</code> dentro il motore AI) per sapere come configurare l'AI ma la guida richiedeva l'AI per essere letta. Aggiunta una route statica <code>GET /guida</code> che serve <code>docs\Guida-Configurazione.pdf</code> direttamente, senza passare zero chiavi, zero tenant salvati. Messaggi di errore delle chiavi AI mancanti e testo di onboarding aggiornati per indicarla esplicitamente. Tutti e 5 verificati dal vivo dopo la correzione: profilo solo-certificato salvato con successo via API reale, <code>Install-M365OpsPrerequisites</code> testato senza regressioni con winget già presente, <code>GET /guida</code> correttamente).		
BUG: la v0.9.25 aveva corretto solo il TESTO del messaggio "chiave AI mancante", non la logica - nessun provider e' un default (v0.9.26, 22/08/2026)	corretto	Correzione richiesta esplicitamente dall'utente dopo aver riletto la riga del changelog v0.9.25: "manca di default, non l'unico -- correggi, non ci deve essere un provider di default". Aveva ragione: <code>Get-M365OpsPrerequisites.ps1</code> SOLA variabile scelta in base a <code>\$script:ActiveAIProvider</code> . Correggendo la logica e' emerso un b del comportamento originale: <code>\$script:ActiveAIProvider</code> e' una variabile di <code>Gui\Server.ps1</code> , ma <code>\$script:</code> e' un altro scope, quindi la lettura risultava SEMPRE <code>\$null</code> , e il controllo finiva SEMPRE quale provider fosse davvero selezionato lato server (verificato dal vivo: anche con un server reale app vuota). Non un problema di testo quindi, ma la causa strutturale del "perche' solo Anthropic?" segnala <code>ANTHROPIC_API_KEY</code> e le tre variabili <code>AZURE_OPENAI *</code> in modo indipendente e alla pari, senza pi Azure OpenAI sono presentati come due alternative equivalenti, mai una "predefinita, non l'unica". Ve server reale riavviato, con entrambi i provider configurati: stato OK, messaggio corretto senza alcun rif
BUG STRUTTURALE: "ps7 assente e winget non disponibile" ricomparso identico su un secondo PC pulito con la v0.9.26 già installata - GUI di installazione prerequisiti unificata, icona e collegamento Desktop (v0.9.27, 22/08/2026)	corretto	Segnalato dal vivo dall'utente dopo aver riprovato l'installazione: "provata la 0.9.26 avviata installazion disponibile" - lo stesso blocco già segnalato prima della v0.9.25. Causa reale: il bootstrap winget aggi <code>M365OpsPrerequisites.ps1</code> , dentro il MODULO PowerShell - inutile per il caso vero, che scatta in solo PowerShell 7 (il modulo richiede un ambiente PowerShell già funzionante per essere importato). eseguito da Windows PowerShell 5.1 (sempre presente su Windows 10/11 a differenza di <code>pwsh.exe</code>) (<code>Microsoft.WinGet.Client / Repair-WinGetPackageManager</code>) ma dal punto giusto della cater (causa più comune di fallimenti silenziosi di <code>Install-Module</code> su un PC vergine, applicato anche al t <code>M365OpsPrerequisites.ps1</code> per coerenza). <code>M365Ops.bat</code> riscritto usando "call" a sottoroutine im parsing di cmd.exe: l'intero blocco tra parentesi viene letto come un'unica unità prima di essere esegui già presente, bootstrap fallito, bootstrap riuscito via fallback WindowsApps prima di toccare il file rea Richiesto in aggiunta dallo stesso utente prima del commit: "disegna una icona bellina per il bat e dentro l'installazione dei prereq anche nodejs visto che cmq serve". Realizzati: (1) icona originale <code>Ass</code> <code>blu/turchese</code> , generata via System.Drawing, nessun logo Microsoft riprodotto), assegnata a un nuovo c Desktop al primo avvio - un <code>.bat</code> non può avere un'icona propria in Windows, un collegamento sì; Windows Forms che mostra dal vivo il controllo/installazione di PowerShell 7, Node.js e Microsoft Edg solo se manca davvero qualcosa - il percorso comune con tutto già presente resta istantaneo e senza un secondo momento invisibile dentro <code>Launch-M365Ops.ps1</code> , e' ora aggregato nello stesso flusso v

BUG UX: la GUI di installazione (v0.9.27) sembrava bloccata durante il bootstrap di winget - output del processo figlio mostrato solo alla fine (v0.9.28, 22/08/2026)	corretto	Segnalato dal vivo dall'utente al primo utilizzo pratico della GUI v0.9.27, con screenshot: la finestra res nessuna nuova riga per oltre un minuto - "sembra stuck... le persone si rompono". Causa reale: la funz singola installazione winget) leggeva il suo output SOLO dopo che il processo usciva (<code>StandardOutp</code> <code>Install-Module / Repair-WinGetPackageManager</code> , che puo' durare 1-2 minuti su un PC vergine, mostrare progresso nel frattempo, anche se il processo stava lavorando normalmente. Corretto reindir e "tail-andolo" ad ogni giro del polling gia' esistente (verificato empiricamente: un ritardo di circa 3-4 file, dovuto al buffering interno dell'host PowerShell su stdout rediretto - accettabile, molto meglio di secondi nella barra di stato aggiornato ad ogni giro di polling (ogni 200ms), indipendente dal bufferin tratti senza nuove righe di log. Verificato dal vivo (per un incidente di test: un tentativo di simulare l'as pwsh/winget dal processo di test, facendo scattare per errore una VERA installazione/aggiornamento correttamente e senza danni, <code>node --version / npx --version</code> confermati funzionanti dopo) che ticchettato in modo continuo ogni ~0.2s e le righe di output reali di winget ("Downloading https://no era ancora in corso, non solo alla fine.
BUG (diagnosi da codice, in attesa di conferma dal vivo): winget appena installato risultava "non trovato" a un riavvio successivo, PATH figlio ereditato dal genitore prima del bootstrap (v0.9.29, 22/08/2026)	corretto	Segnalato dal vivo dall'utente su Windows Sandbox (utente <code>WDAGUtilityAccount</code> nello screenshot, v0.9.27/v0.9.28 aveva installato tutto con successo (lentamente), un errore "il server non ha risposto er winget sembrava di nuovo non disponibile. Diagnosi dalla rilettura del codice (non ancora confermata pulita... fammi sapere"): un processo figlio Windows eredita il <code>%PATH%</code> del genitore AL MOMENTO in <code>M3650ps.bat</code> , dopo aver richiamato la GUI di bootstrap, non rileggeva mai il proprio <code>%PATH%</code> prir quindi erediterebbe ancora il PATH precedente al bootstrap. Dentro quel processo, <code>Install-M3650p</code> (controllava solo <code>Get-Command</code> , nessun fallback sul percorso diretto WindowsApps a differenza degl ridondante sotto PowerShell 7 (percorso moduli diverso da Windows PowerShell 5.1, quindi nessuna c sia della lentezza percepita sia del timeout di avvio del server. Tre correzioni indipendenti, ciascuna sul rilegge il PATH vero dal registro (Machine+User) subito dopo la GUI, prima di procedere - verificato cc PowerShell embedded non rompono il parsing del blocco <code>if()</code> che lo contiene, stesso tipo di rischi <code>M3650psPrerequisites.ps1</code> controlla ora anche il percorso WindowsApps diretto prima di conclud nome nudo "winget") per la chiamata di installazione vera e propria - un secondo bug latente scopert controllo falliva sempre allo stesso modo; (3) il timeout di attesa del server in <code>Launch-M3650ps.ps1</code> freddo su hardware lento (import di 350+ file del modulo) senza mostrare un errore fuorviante mentre dal vivo su questa macchina di sviluppo: percorso comune (tutto gia' presente, resume-session) ancor aggiunta in v0.9.30: questa diagnosi NON era la causa reale dell'errore segnalato - le tre correzic era altrove, vedi sotto.
BUG CONFERMATO (dal log reale): il server non partiva affatto su un'installazione DAVVERO vergine, zero profili tenant salvati - crash immediato, non lentezza (v0.9.30, 22/08/2026)	corretto	L'utente ha condiviso il contenuto reale di <code>Logs\server-console-error.log</code> su richiesta esplicita allegato) - la vera causa era completamente diversa dalla diagnosi (poi rivelatasi sbagliata) della v0.9.2 salvato. Usa prima <code>Set-M3650psTenant.</code> <code>Gui\Server.ps1</code> chiamava <code>Connect-M3650ps -Tenantf</code> script, PRIMA che il listener HTTP venga mai creato molto piu' sotto - su un'installazione con <code>Config</code> significato di "PC pulito": non solo senza PowerShell 7/winget/Nodejs, ma anche senza NESSUNA con moriva all'istante senza mai aprire la porta. Da fuori il sintomo osservato da <code>Launch-M3650ps.ps1</code> € messaggio generico "non ha risposto entro N secondi", ma la causa vera era un crash immediato, nor sulla rilettura del codice, mai confermata dal log reale) aveva puntato nella direzione sbagliata. Il mecc <code>/api/status</code>) era gia' pensato esattamente per questo caso e gia' funzionante altrove (<code>Get-M3650p</code> <code>tenants.json</code> assente) - ma non poteva mai essere raggiunto, perche' il server non arrivava mai ad stato esercitato prima d'ora in questa sessione: ogni test precedente partiva gia' da un PC con almeno il default hardcoded, creato durante lo sviluppo iniziale del progetto) - la vera prima installazione a ze questo test su Windows Sandbox. Corretto avvolgendo la chiamata in try/catch: un fallimento ora si lir nessun tenant attivo (stato gia' previsto e gestito correttamente altrove nel codice) e l'utente vede la s pagina, come da design originale. Verificato dal vivo con una simulazione fedele: copia isolata comple avviato con il profilo di default inesistente su una porta separata - partito correttamente, <code>GET /api/</code> <code>/api/setup-status</code> e la pagina principale serviti correttamente senza nessun tenant attivo.
BUG: login Exchange delegato riuscito ma poi fallito per un modulo mancante mai auto-installato in quel percorso; progress bar Marquee invisibile su Windows Sandbox (v0.9.31, 22/08/2026)	corretto	Segnalato dal vivo dall'utente su Windows Sandbox: login a Microsoft Graph delegato riuscito, ma Exc connessione a Exchange e' fallita: The specified module 'ExchangeOnlineManagement' was not loaded directory." Causa reale: <code>Complete-M3650psExchangeDelegatedLogin.ps1</code> chiamava <code>Import-Mod</code> fallback di auto-installazione, a differenza di <code>Connect-M3650psExchange.ps1</code> (percorso app-only) € percorsi di codice diversi, uno solo dei due gestiva l'assenza. Stesso buco trovato anche in <code>New-M3650</code> commento diceva "gia' installati oggi", un'assunzione mai vera su un PC nuovo). L'utente ha richiesto € rincorrere un fallback alla volta: "ti avevo detto di implementare tutti i moduli come prerequisito" / "in <code>M3650psPrerequisites.ps1</code> ora installa in anticipo anche i tre moduli PowerShell (<code>ExchangeOnli</code> <code>Install-Module -Scope CurrentUser</code> , con lo stesso irrobustimento TLS 1.2/provider NuGet gia' cmdlet restano comunque presenti per difesa in profondita', non piu' come unico punto di installazion dentro <code>Show-M3650psPrereqInstaller.ps1</code> (visibile, non solo nel secondo passaggio invisibile di moduli vengono installati dentro un processo <code>pwsh.exe (PowerShell 7) figlio</code>, non nello scope di Wir <code>Install-Module -Scope CurrentUser</code> sotto le due versioni di PowerShell scrive in due cartelle di <code>Documents\PowerShell\Modules</code>) - installarli nello scope sbagliato li avrebbe lasciati comunque in Progress bar: segnalata "mancante" dal vivo - causa reale: mai chiamato <code>[System.Windows.Forms.</code> WinForms richiede per disegnare correttamente uno stile <code>Marquee</code> (senza, puo' renderizzare come s moderno - un problema noto, non un'ipotesi) - aggiunta la chiamata mancante, e sostituito comunqu avanzamento manuale a passi (0-4, un incremento per sezione completata: PowerShell 7, Nodejs/Edg su sessioni remote/RDP come Windows Sandbox e capace di mostrare un progresso reale invece del s questa macchina di sviluppo: <code>Install-M3650psPrerequisites</code> e la GUI aggiornata eseguite senza comune, nessun rallentamento aggiunto).
Completato l'elenco moduli prerequisito: mancavano MicrosoftTeams, PnP.PowerShell, PdfLerx (v0.9.32, 22/08/2026)	corretto	L'utente ha notato subito dopo la v0.9.31 che l'elenco copriva solo Exchange/Excel/Intune: "ma i moduli legittima, la v0.9.31 aveva corretto solo i moduli coinvolti nel bug appena segnalato (Exchange/Intune sistematico su tutto il codebase (<code>Install-Module</code> <code>lw</code>) usato per gli altri: trovati altri due moduli co

		(<code>MicrosoftTeams</code> in <code>Connect-M365OpsTeams.ps1</code> , <code>PnP.PowerShell</code> in <code>Connect-M365OpsSharePoint.ps1</code>) e <code>Get-M365OpsExchangeDelegatedLogin.ps1</code> (estrazione testo dai PDF caricati nella Knowledge Base, <code>Get-M365OpsExchangeDelegatedLogin.ps1</code> locale, quindi non erano "bug zero-fallback" come Exchange/Intune, ma restavano comunque fuori da <code>M365OpsPrerequisites.ps1</code> sia a <code>Show-M365OpsPrereqInstaller.ps1</code> (stesso meccanismo già lo scope corretto). Microsoft Graph chiarito esplicitamente in guida come intenzionalmente escluso con <code>Microsoft.Graph</code>, ogni chiamata passa da <code>Invoke-M365OpsGraphRequest</code> (REST puro via <code>Invoke-WebRequest</code>), nessun modulo dedicato per Graph. Verificato dal vivo su questa macchina di sviluppo: tutti e 6 i moduli rilevati correttamente come già presenti, nessuna regressione.
BUG: tutti e 6 i moduli PowerShell fallivano l'installazione nella GUI con "PackageManagement... currently in use" (v0.9.33, 22/08/2026)	corretto	Segnalato dal vivo dall'utente subito dopo la v0.9.32, log incollato in chat: ogni singolo modulo falliva con l'errore: <code>'X' of module 'PackageManagement'/'PowerShellGet' is currently in use. Retry the command (exit code)"</code> vuoto invece di un numero. Causa reale: <code>Show-M365OpsPrereqInstaller.ps1</code> lanciava i 6 moduli, e ciascuno ripeteva da zero il bootstrap TLS1.2/provider NuGet - su un PC dove il provider per correre a ridosso l'uno dell'altro (il secondo processo poteva partire prima che il primo avesse rilasciato <code>PackageManagement/PowerShellGet</code>) e collidere sullo stesso file, facendo fallire l'installazione per TUTTI i moduli. Sintomo dello stesso problema: il processo falliva in un modo che lasciava <code>\$?LASTEXITCODE</code> non 0 ma <code>M365OpsModuleInstall</code> per installare TUTTI i moduli mancanti in UN SOLO processo <code>pwsh.exe</code> (bootstrapping) - installa ogni modulo in sequenza nello stesso processo) - elimina la possibilità stessa della collisione, file insieme. Stessa ottimizzazione applicata anche al controllo di presenza (prima 6 processi separati si avviavano, ora <code>-SkipPublisherCheck</code> a tutte le chiamate <code>Install-Module</code> del progetto (GUI e <code>Install-Module</code> causa nota di blocchi silenziosi su un repository non ancora segnato come attendibile. Verificato dal vivo su questa macchina di sviluppo: <code>PSScriptAnalyzer</code>, mai usato da questo progetto, rimosso subito dopo il test) installa insieme a un nome di modulo volutamente inesistente, che ha riportato il proprio errore specifico invece di "currently in use" nel nuovo percorso a processo singolo.
Diagnostica aggiunta (non ancora un fix confermato): login Teams delegato segnalato dal vivo come bloccato dopo il popup su Windows Sandbox (v0.9.34, 22/08/2026)	diagnostica, non ancora risolto	Segnalato dal vivo dall'utente: "exch sharepoint si sono connessi, teams mi ha chiesto login l'ho fatta e non funziona". Questa macchina di sviluppo (il tenant di test qui è App-only, non Delegato, il ramo interessato di <code>Connect-M365OpsTeams.ps1</code> era <code>Connect-M365OpsTeams.ps1</code>), invece di tentare un fix alla cieca è stata aggiunta diagnostica prima/dopo attorno alla chiamata <code>Connect-M365OpsTeams.ps1</code> (ramo delegato interattivo), scritta su <code>Logs\m365ops-YYYYMMDD.log</code> (persiste per differenza di <code>Write-Host</code>, che finisce solo sulla console nascosta del server e non è mai stato utile a debug del device-code già risolto in sezione 17.14) con i secondi trascorsi sia in caso di successo sia di errore. La chiamata non è MAI arrivata a completarsi (bloccata per sempre, es. componente mancante nell'ambiente di test loopback locale che non torna mai indietro) oppure se è solo lenta (es. MFA/conditional access concausa da un'osservazione soggettiva ("sembra bloccato"). Verificato dal vivo su questa macchina di sviluppo: <code>Connect-M365OpsTeams.ps1</code> (non toccato da questa modifica) continua a funzionare correttamente via <code>POST /api/teams-test</code> log come atteso.
BUG STRUTTURALE trovato: login Teams delegato bloccato per sempre da WAM (Web Account Manager), il broker diventato default nel modulo MicrosoftTeams (v0.9.35, 22/08/2026)	corretto	Causa vera del blocco Teams delegato, trovata dopo che l'utente ha confermato dal vivo un blocco CC di un vero blocco a thread singolo) e ha suggerito il confronto decisivo: "non puoi implementarlo con <code>Connect-M365OpsIntune.ps1</code> (funzionante) con <code>Connect-M365OpsTeams.ps1</code> (bloccato): dal modulo <code>MicrosoftTeams</code> usa di DEFAULT Web Account Manager (WAM) come broker di autenticazione su Windows. WAM non a memoria. WAM richiede una sessione Windows interattiva con una UI reale a cui agganciare il server (avviato con <code>-WindowStyle Hidden</code>), WAM non ha dove mostrare la propria interfaccia e la chiamata non arriva mai. Stessa CLASSE di bug del device-code già risolto (sezione 17.14: un flusso di lavoro interattivo che qui non esiste), ma causa DIVERSA e più recente: WAM è diventato il default DOPO che è stato aggiornato sotto silenzio nel frattempo (installato per la prima volta su un PC pulito prende sempre l'ultima versione ufficiale <code>-DisableWAM</code>, introdotto proprio in v7.8.1-preview "per scenari come contesti RunAs dove Windows non corrisponde esattamente a questo caso. Applicato in modo CONDIZIONALE, non incondizionato, il modulo installato è ancora 7.3.1, dove <code>-DisableWAM</code> non esiste affatto come parametro - passarlo senza il parametro binding, un bug diverso ma reale introdotto per evitarne un altro. La versione del modulo v7.8.1 di <code>MicrosoftTeams</code> Version -ge [version]'7.8.1') prima di aggiungere lo switch. Aggiunta anche <code>Connect-M365OpsTeams.ps1</code> stesso (oltre a quella già aggiunta in v0.9.34 attorno a <code>Connect-M365OpsTeams.ps1</code>) del modulo invece che nella connessione. Verificato dal vivo su questa macchina di sviluppo: il ramo A di <code>Connect-M365OpsTeams.ps1</code> (non toccato da questa modifica) continua a funzionare correttamente via <code>POST /api/teams-test</code> reale. Il ramo B di <code>Connect-M365OpsTeams.ps1</code> di test App-only - in attesa di conferma dal vivo dall'utente sul prossimo test.
-DisableWAM confermato funzionante (il popup c'era, era solo nascosto dietro il browser) + irrobustimento sistematico di tutti i fallback Install-Module + nuovo strumento "Termina e riavvia" (v0.9.36, 22/08/2026)	corretto	L'utente ha riprovato il login Teams delegato sulla v0.9.35 e ha segnalato di nuovo un blocco, stavolta nemmeno "Import-Module MicrosoftTeams avviato" (la primissima diagnostica aggiunta, PRIMA del popup di blocco (se reale) sarebbe scattato ancora più in alto, nel fallback <code>Install-Module MicrosoftTeams</code> TLS1.2/provider NuGet/ -SkipPublisherCheck già applicato altrove nel progetto (Bootstrap-Winget-Install-M365OpsPrereqInstaller.ps1) ma mai a QUESTO fallback specifico, ne' agli altri identici sparsi nel codice dello stesso tipo di blocco silenzioso per una causa diversa. Ma subito dopo l'utente ha trovato la via <code>Connect-M365OpsTeams.ps1</code> - <code>-DisableWAM</code> aveva funzionato correttamente fin dalla v0.9.35 semplicemente dietro la finestra già in primo piano, mai notato. Nessun vero blocco in questo episodio restava un rischio reale indipendente, quindi applicato comunque: stesso pattern TLS1.2/provider NuGet/ -SkipPublisherCheck rimasti scoperti (<code>Connect-M365OpsTeams.ps1</code> , <code>Connect-M365OpsSharePoint.ps1</code> , <code>Complete-M365OpsExchangeDelegatedLogin.ps1</code> , <code>New-M365OpsWAM.ps1</code> per ImportExcel e PdfLexer) - stesso identico grep sistematico già usato per trovare i moduli mancanti dell'installazione invece che al suo elenco. Nuovo strumento richiesto esplicitamente dall'utente ("come che killi l'istanza powershell e riavvii il .bat?"): un vero pulsante DENTRO la pagina web non aveva server e davvero bloccato, non può nemmeno ricevere/processare il click sul proprio pulsante (stesso metodo di rispondere). Aggiunti <code>Kill-And-Restart-M365Ops.ps1</code> / <code>.bat</code> (Windows PowerShell 5.1 <code>pwsh.exe</code> anche se è lui stesso ad essere bloccato): termina SOLO i processi <code>pwsh.exe</code> che eseguono (confronto esplicito del percorso completo, mai "tutti i pwsh.exe di sistema"), poi rilancia via <code>Connect-M365OpsTeams.ps1</code> ("M365Ops - Termina e riavvia", stessa icona) creato automaticamente da <code>Tools\New-M365OpsShortcuts.ps1</code>

		idempotente per ciascuno indipendentemente). Verificato dal vivo su questa macchina di sviluppo con successo dallo strumento, riavviato automaticamente via <code>M365Ops.bat</code> , tornato in linea con il te in circa 20 secondi; entrambi i collegamenti Desktop creati con percorso/icona corretti.
BUG DI TERZE PARTI trovato e riprodotto: MicrosoftTeams e ExchangeOnlineManagement non possono coesistere nello stesso processo server - conflitto noto di Microsoft, guardia aggiunta con messaggio chiaro invece di un crash apparente (v0.9.36, 22/08/2026)	mitigato (nessun fix possibile lato Microsoft)	Segnalato dal vivo dall'utente SUL SUO PC REALE (non sul Sandbox - dettaglio confermato esplicitame aver connesso Teams poi Graph, la connessione a Exchange falliva con "Could not load file or assembl '..\ExchangeOnlineManagement\3.10.1\netCore\Microsoft.IdentityModel.Abstractions.dll'. The located assembly reference. (0x80131040)" - e subito dopo il tab Log/Aggiornamenti smetteva di rispondere c non solo un errore isolato. Riprodotto IDENTICO su questa macchina di sviluppo, in App-only (quindi modalita' Delegata): connettere Teams poi Exchange nello stesso processo fallisce SEMPRE su Micro: l'ordine inverso (Exchange poi Teams): fallisce ANCH'ESSO, stavolta su <code>Microsoft.Identity.Clien</code> BIDIREZIONALE. Confermato con una ricerca reale (non a memoria) che si tratta di un problema pubb progetto: i moduli <code>MicrosoftTeams</code> e <code>ExchangeOnlineManagement</code> portano con se' versioni incc (<code>Microsoft.Identity.Client</code> / <code>Microsoft.IdentityModel.*</code>) - una volta che un modulo le cè lato Microsoft (vedi "PowerShell Module Clash for Teams and Exchange Online", office365itpros.com, 2 processo/job separato) sarebbe un refactoring architetturale sostanzioso, non fatto qui. Mitigato inv M365OpsExchange / Connect-M365OpsCompliance / Complete-M365OpsExchangeDelegatedLogi se l'ALTRO modulo (Teams vs Exchange) e' gia' stato caricato in questo processo - se si', restituiscono di riavviare il server, invece di lasciar propagare il criptico errore .NET nativo che aveva causato il blocc sviluppo in entrambe le direzioni: il messaggio chiaro compare immediatamente, PRIMA di qualunque blocco, solo un errore comprensibile.
Nuovo: report di utilizzo Microsoft 365 Copilot - primo passo del supporto Copilot, ambito volutamente ristretto ai soli report (v0.9.37, 22/08/2026)	implementato	Richiesto esplicitamente dall'utente: "dobbiamo aggiungere tutto quello che riguarda il supporto copi fatta contro Microsoft Learn reale (non a memoria) prima di implementare qualunque cosa - risultato: App-only/Delegato oggi. Impostazioni Copilot (<code>/copilot/admin/policySettings</code>) risultano letter documentazione ufficiale, solo Delegato - l'utente ha confermato esplicitamente di NON voler proced mi stai dicendo ci sono i limiti allora non procediamo oltre se non con i report"). Catalogo agenti/app permessi corretti - troppo instabile. Connettori federati (<code>Set-FederatedConnectorToggle</code>) in dism qualcosa gia' morto in pochi giorni. Cronologia interazioni reali (<code>AIEnterpriseInteraction.Read</code> , dato di livello compliance/eDiscovery - non aggiunto senza una richiesta esplicita separata. Aggiunte <code>Get-M365OpsCopilotUsageSummary</code> (aggregato), endpoint Graph v1.0 stabile, permesso <code>Reports</code> mancante su vnsys-test da un giro precedente (19/08/2026) per altri report, riconfermato mancante ar permission."). La risposta v1.0 e' sempre CSV (comportamento documentato, non un bug) - entrambe parsing dei dati reali non e' ancora verificato dal vivo (solo il percorso di errore 403, in attesa che il pe esplicitamente all'utente, su sua richiesta di chiarimento: M365Ops non chiama mai Copilot come mot Base globale (sezione 17.13, gia' esistente) puo' gia' rispondere a domande concettuali su Copilot sen documentazione dedicata con <code>Add-M365OpsKnowledgeDocument -TenantName '_global'</code> , nesso sezione 24 per il dettaglio completo.
BUG GRAVISSIMO trovato: l'auto-update non aveva MAI applicato nessuna modifica di codice da settimane, nonostante ogni giro si dichiarasse riuscito e il numero di versione salisse correttamente (v0.9.38, 22/08/2026)	corretto	Spiega retroattivamente buona parte della difficolta' di questa intera sessione di debug: dopo che l'ut anche dopo piu' fix e riavvii, incollando il contenuto REALE di <code>Connect-M365OpsTeams.ps1</code> dalla su settimane prima (niente <code>-DisableWAM</code> , niente guardia, niente diagnostica) - nonostante il log mostr <code>0.9.33</code> a <code>0.9.34</code> , poi a <code>0.9.35</code> , <code>0.9.36</code> , <code>0.9.37</code> , ognuno senza errori. Causa reale, riprodotta empiric sorgente\Public -Destination dest\Public -Recurse -Force , quando <code>dest\Public</code> ESIST un'installazione nuova - motivo per cui questo bug non era mai stato notato prima), non sovrascrive il DENTRO quella di destinazione, producendo <code>dest\Public\Public\...</code> invece di sovrascrivere de <code>M365Ops.psd1</code> (un FILE, non una cartella, non soggetto a questo problema) si aggiornava correttam l'aggiornamento fosse riuscito, mentre il codice VERO sotto <code>Public\ / Private\ / Gui\ / Tools\</code> da nessun aggiornamento successivo. Corretto con una copia ricorsiva file-per-file (<code>Copy-M365OpsUp</code> verso una destinazione che potrebbe gia' esistere, solo singoli file copiati con <code>Copy-Item -Force</code> d impossibile annidare, indipendentemente dalla profondita' della struttura. Aggiunta anche <code>Scripts</code> mai toccate dall'update, protezione mancante prima. Verificato empiricamente sia il bug sia la corr file "nuovo" che finiva in <code>dest\Public\Public\NuovoFile.ps1</code> con la vecchia logica; con la nuova cartella annidata, file estraneo) produce il file sovrascritto al posto giusto, il file annidato al livello corre estraneo intatto.
Azione necessaria per chi ha gia' installato una versione precedente: questo fix corregge solo gli aggiornamenti FUTURI - non ripulisce l'annidamento gia' eventualmente presente su disco da aggiorna installato M365Ops e aggiornato piu' volte prima di questa versione, la cartella <code>Public\</code> (e probabilmente <code>Private\</code> , <code>Gui\</code> , <code>Tools\</code>) potrebbe contenere sottocartelle annidate con lo stesso nome (mai applicate. L'unico modo sicuro di ripartire e' una reinstallazione pulita: scarica lo zip dell'ultima Release da GitHub ed estrailo in una cartella NUOVA (mai sovrascrivere quella vecchia) - <code>Config\ /</code> vecchia cartella, quindi se servono i dati di un tenant gia' configurato vanno copiati a mano nella cartella nuova prima del primo avvio.		
Avviso diretto in GUI vicino a Exchange Online e Teams sul conflitto 6.6 - confermato empiricamente che Disconnect/Remove-Module NON liberano le assembly caricate (v0.9.39, 22/08/2026)	implementato	Richiesto esplicitamente dall'utente: prima "c'e' modo da GUI di smontare il modulo... un disconnect r vicino a exchange online e teams". Prima di rispondere, verificato dal vivo (non per ipotesi) se <code>Discor ExchangeOnlineManagement -Force</code> permettesse poi di caricare MicrosoftTeams senza conflitto: N conferma che una volta che una DLL e' caricata in un processo .NET, ne resta parte per tutta la vita del vengano disconnessi o rimossi da quella sessione PowerShell (rimuovono solo i comandi dalla session Aggiunto un avviso permanente in <code>Gui\index.html</code> , dentro i riquadri "Exchange Online" e "Microso "NB: i moduli Exchange e Teams non convivono nello stesso processo server (conflitto noto di Microso connessioni per volta e riavvia il server prima di passare all'altra", con rimando alla sezione 6.6. Verifica che il testo compare correttamente in entrambi i riquadri dopo il riavvio del server necessario per ricar 12.2).

Due bug reali segnalati dal vivo su PC di test: tab Manutenzione mostrava "Cannot bind argument to parameter 'Name' because it is an empty string" nel controllo stato; auto-sync guida-> Knowledge Base falliva con "Could not find a part of the path ...Config\KnowledgeBase-global.json" (v0.9.40, 23/08/2026)	corretto	Primo bug: <code>Get-M365OpsSetupStatus.ps1</code> chiamava <code>Get-M365OpsSecret -Name \$ctx.Secret</code> fosse impostato - legittimamente vuoto per un tenant AppOnly configurato SOLO con certificato (<code>Se</code> <code>ExchangeCertThumbprint</code> , non richiede entrambi). <code>Get-M365OpsSecret</code> ha <code>-Name Mandatory</code>: parametro Mandatory come "valore mancante", e in un host non interattivo come questo processo se quell'eccezione invece di un prompt - stesso motivo per cui <code>Get-M365OpsToken.ps1</code> e <code>Connect-M</code> chiamata con un controllo a monte, protezione che qui mancava. Corretto aggiungendo lo stesso con Secondo bug: <code>Add-M365OpsKnowledgeDocument.ps1</code> creava la cartella <code>Uploads\kb\<tenant>\</code> <code>Config\KnowledgeBase-<tenant>.json</code> - su un'installazione dove <code>Config\</code> non era ancora mai di aggiornamento mai cambiati - le uniche altre funzioni che la creano), l'auto-sync della guida nella K avvio del server, quindi puo' essere la primissima cosa scritta su disco) falliva silenziosamente (loggato <code>Directory -Force su Config\</code> prima della scrittura, stesso pattern gia' usato per <code>Uploads\kb\</code> (tokenizer PowerShell); il secondo bug era visibile nei log della macchina di test con un percorso <code>...</code> annidato - probabile estrazione manuale dello zip GitHub con "Estrai tutto" di Windows, che duplica l un problema del codice ma va segnalato all'utente (reinstallazione pulita in una cartella vuota, non inn
Il conflitto MicrosoftTeams/ExchangeOnlineManagement (sezione 6.6) NON e' universale: risolto fissando ExchangeOnlineManagement alla versione 3.9.0, l'unica verificata dal vivo come conflict-free (v0.9.41, 23/08/2026)	corretto	L'utente ha correttamente messo in dubbio la diagnosi precedente ("mitigato, nessun fix possibile") cc servizi uno dopo l'altro senza errori, non e' che dipende da versioni diverse dei moduli?" Verificato dal REALE e credenziali finte - l'assembly conflict scatta PRIMA di qualunque validazione di rete, quindi e' NON dipende dalla versione di <code>MicrosoftTeams</code> installata (riprodotto identico con 7.3.1 e con 7.9.0 <code>ExchangeOnlineManagement - 3.9.0 non va MAI in conflitto</code> (testato con entrambe le versioni Tez (riprodotto anche l'errore esatto originariamente segnalato dall'utente, "Microsoft.Identity.Client.dll... r Exchange 3.10.1 - probabilmente 3.10.0 ha aggiornato la libreria di autenticazione condivisa, coerente nella stessa versione). Confermato inoltre che 3.9.0 supporta comunque tutto cio' che serve al progett only) e <code>-AccessToken</code> (login delegato) sono entrambi presenti come parametri di <code>Connect-Excha '3.9.0'</code> , mai piu' "l'ultima disponibile") in tutti i punti che installano o importano il modulo: <code>Connect-M365OpsCompliance.ps1</code> , <code>Complete-M365OpsExchangeDelegatedLogin.ps1</code> , <code>Install-M365OpsM365OpsPrereqInstaller.ps1</code> - sia l'installazione sia il controllo di presenza ora verificano la versio <code>Import-Module</code> senza <code>-RequiredVersion</code> avrebbe comunque caricato la versione piu' recente di <code>Install-Module -RequiredVersion</code> non tocca ne' rimuove altre versioni gia' installate (es. una 3. propria cartella di versione, PowerShell le tiene separate. La guardia anti-crash aggiunta in v0.9.36 (sez profondita', ma con questo fix non dovrebbe piu' scattare nell'uso normale. Un aggiornamento futuro recente andrebbe riverificato allo stesso modo (test dal vivo con <code>Connect-ExchangeOnline</code> reale, n
Rinforzo del fix precedente: disinstallazione ATTIVA di ExchangeOnlineManagement >= 3.10.0 se trovata sul disco (non solo pin passivo), avviso reso molto piu' visibile in GUI e guida, procedura scritta per testare le prossime versioni (v0.9.42, 23/08/2026)	implementato	Richiesto esplicitamente dall'utente subito dopo il fix precedente: "aggiungi un check che se e' installa questa macchina, metti bene in evidenza in GUI e guida questa dipendenza cosi non la aggiornano pe facciamo test specifici". Il fix v0.9.41 fissava <code>-RequiredVersion</code> su <code>Import-Module / Install-Mo</code> presente sul disco - sufficiente per QUESTO progetto (che chiama sempre <code>-RequiredVersion</code> esplici qualunque altro uso non protetto dallo stesso pin. Aggiunta <code>Assert-M365OpsExoSafeVersion</code> (nu <code>M365OpsExoSafeVersion.ps1</code>): disinstalla attivamente ogni versione <code>ExchangeOnlineManagement RequiredVersion</code> , best-effort - un fallimento e' solo loggato, il pin nei chiamanti resta comunque ef da tutti e tre i punti di connessione (<code>Connect-M365OpsExchange.ps1</code> , <code>Connect-M365OpsComplia</code> <code>M365OpsExchangeDelegatedLogin.ps1</code> , che ora condividono questa unica implementazione invece <code>M365OpsPrerequisites.ps1</code> . Stessa logica duplicata (necessariamente, come testo generato) dentro processo pwsh.exe figlio PRIMA che il modulo M365Ops sia importato - anche il controllo di presenza da rimediare) il caso "3.9.0 presente MA anche una 3.10.x affiancata", non solo "3.9.0 assente". Verifica 3.10.1 su questa macchina di sviluppo appositamente per il test, poi eseguita <code>Assert-M365OpsExoSa</code> <code>RemovedVersions: {3.10.1}</code>), 3.9.0 confermata presente; ripetuto lo stesso test end-to-end anch <code>M365OpsPrereqInstaller.ps1</code> (non solo tokenizzato: eseguito per davvero via <code>Invoke-Expressi</code> alla versione corretta (solo 3.9.0 rimasta per ExchangeOnlineManagement, oltre a una 2.0.5 residua ch innocua, non in conflitto, lasciata li'). Visibilita' aggiunta: avvisi in GUI (riquadri Exchange Online e Mi esplicitamente la versione fissata e avvertire di non aggiornarla mai a mano; riga della tabella prerequ evidenziata con l'avviso di versione fissata; nuovo riquadro in sezione 3 con la procedura scritta da se versione del modulo (installare in affiancamento su una macchina di test, mai sostituire; test con <code>Coni</code> a far scattare l'eventuale conflitto senza serve un tenant vero; solo se pulito aggiornare il numero ovur
BUG REALE trovato dopo il fix precedente: la guardia anti-crash del 22/08/2026 era rimasta attiva anche nella direzione ormai sicura (Teams poi Exchange), bloccando esattamente quello che il pin 3.9.0 aveva appena risolto; scoperto anche che l'ordine INVERSO (Exchange poi Teams) resta rotto anche con la 3.9.0 (v0.9.43, 24/08/2026)	corretto	L'utente ha segnalato dal vivo su v0.9.42: "sono alla 0.9.42 ma non vedo nella GUI l'avviso che dici e i c rispondere, non per ipotesi: (1) testato per la prima volta l'ordine INVERSO rispetto a tutti i test precec <code>Module MicrosoftTeams</code> - risultato: fallisce comunque ("Could not load file or assembly Microsoft riprodotto anche con la 2.0.5 (la piu' vecchia disponibile) e con Teams 7.3.1/7.9.0 - il fix v0.9.41/42 cop nell'altro verso. (2) Rileggendo <code>Connect-M365OpsExchange.ps1 / Connect-M365OpsCompliance.} <code>M365OpsExchangeDelegatedLogin.ps1</code> con questo in mente: la guardia del 22/08/2026 (blocca SEI direzione) era rimasta li' intatta anche dopo aver fissato la versione - bloccava quindi ANCHE la direzi messaggio "nessuna soluzione lato modulo" che pero' non era piu' vero per quella direzione. Root cau versione moduli ne' di GUI non aggiornata, ma una guardia dimenticata che bloccava un caso ormai ri importano ExchangeOnlineManagement (verificato dal vivo, non solo letto: chiamate reali a <code>Connect</code> attraverso il modulo vero, con un tenant finto - nessun blocco, fallisce pulito solo sul certificato finto). piu' azionabile ("riavvia e riparti da Teams"), nell'UNICA direzione dove serve davvero: <code>Connect-M365</code> prima (verificato che continua a bloccare correttamente in quel caso). Avvisi in GUI e sezione 6.6 riscrit sicuro senza riavvii, ordine inverso richiede ancora un riavvio. Sulla domanda "non vedo l'avviso in t server e tenuto in memoria per tutta la sua vita (sezione 12.2, gia' documentato) - un aggiornamento processo server (tab Manutenzione o "M365Ops - Termina e riavvia") perche' la pagina rilegga il file aq ora esplicitata anche in guida.</code>
CORREZIONE della correzione precedente, stesso giorno: nessun ordine di connessione si e' rivelato affidabilmente sicuro - rimossa OGNI guardia	corretto	L'utente ha messo in dubbio la v0.9.43 subito dopo averla vista: "non c'e' magari una versione un po' p mia macchina funzionava in qualunque direzione, tra l'altro il problema prima era inverso (Exchange-p

preventiva, sostituita con un messaggio chiaro SOLO se il conflitto scatta davvero (v0.9.44, 24/08/2026)		rispondere, eseguita una matrice di test molto piu' ampia della precedente: 2 versioni Exchange (2.0.5, 12 combinazioni, ciascuna in un processo pwsh.exe isolato separato, con chiamate REALI a <code>Connect-</code> (credenziali finte). Risultato inatteso: con Teams 7.1.0 + Exchange 3.9.0, ANCHE l'ordine Teams-poi-Exc conflitto - la regola "Teams prima e' sempre sicuro col pin 3.9.0" non regge, smentita da una terza vers causa di disturbo concreta sulla macchina di sviluppo: un'installazione AllUsers di ExchangeOnlineMan <code>Files\WindowsPowerShell\Modules\</code> , non rimovibile per permessi insufficienti, mescolata con le ir ambientale che rende ancora meno affidabile fidarsi di un singolo ordine come regola universale. L'utb bidirezionale ripristinata: "non voglio il safe guard, nelle versioni vecchie funzionava" - aveva ragione s si e' dimostrato ne' sempre sicuro ne' sempre rotto, bloccare A PRIORI un tentativo che su un PC spec rischio incerto. Cambio di approccio, non solo di soglia: rimossa ogni guardia preventiva ("l'altro mc tutti e quattro i punti di connessione (<code>Connect-M3650psExchange.ps1</code> , <code>Connect-M3650psCompli M3650psExchangeDelegatedLogin.ps1</code> , <code>Connect-M3650psTeams.ps1</code>) - si tenta sempre la conni condivisa <code>Get-M3650psModuleConflictHint</code> (<code>Private/Get-M3650psModuleConflictHint.ps1</code> nel messaggio di un'eccezione REALE (non prevista in anticipo) e lo riveste con un messaggio leggibile conflitto scatta davvero - qualunque altro errore (incluso il successo) passa inalterato. La versione FISS 3.10.0 in poi il conflitto e' sempre presente, senza eccezioni in nessun test) insieme alla disinstallazione chiamate reali a <code>Connect-M3650psTeams</code> poi <code>Connect-M3650psExchange</code> e viceversa attraverso i blocco preventivo, fallisce pulito solo sulle credenziali finte; <code>Get-M3650psModuleConflictHint</code> tes esattamente il pattern del conflitto reale, riconosciuto correttamente, e con un messaggio di errore no sezione 6.6 riscritti un'ultima volta per riflettere questo: nessun ordine "giusto" da consigliare, solo "se
BUG DISTINTO trovato lo stesso giorno: "The given assembly name was invalid." su Exchange dopo un riavvio pulito, SENZA Teams coinvolto - non era il conflitto Teams/Exchange ma un'installazione locale di ExchangeOnlineManagement danneggiata, ora rilevata e riparata PRIMA che diventi irreparabile (v0.9.46, 25/08/2026)	corretto	L'utente ha segnalato dal vivo: "connesso teams, vado su exch e ho questo errore: The given assembly SENZA mai toccare Teams, lo stesso identico errore su Exchange da solo, dimostrando che NON era (s questa macchina di sviluppo cancellando deliberatamente <code>Microsoft.Identity.Client.dll</code> dalla 3.9.0: <code>Get-Module -ListAvailable</code> continuava a segnalare la versione come "presente" (legge sol esista), e perfino <code>Import-Module</code> riusciva SENZA errori (non carica ogni DLL referenziata, solo l'asse tentativo di <code>Connect-ExchangeOnline</code> , quando il modulo tenta di caricare pigramente quella diper riparazione "in-process" (rilevare l'errore dentro il blocco catch della connessione, cancellare e reinstal SCARTATO lo stesso giorno: una volta che <code>Import-Module</code> ha caricato l'assembly principale nel proc nemmeno <code>Remove-Item</code> riesce piu' a toccarli (stesso identico vincolo gia' visto per il conflitto Teams ma qui il blocco e' ancora piu' basso, a livello di file, non solo di AppDomain.NET). Fix vero, spostato dopo: <code>Assert-M3650psExoSafeVersion</code> (chiamata SEMPRE da tutti i punti di connessione PRIMA (ancora bloccati da nessuno) ora verifica l'INTEGRITA' dei file critici (il modulo principale e <code>Microsoft manifest</code>) - se mancano o sono vuoti, tratta l'installazione come "da reinstallare" esattamente come fa installandola di nuovo PRIMA che qualunque <code>Import-Module</code> possa bloccarla. Nuova funzione <code>Pri</code> aggiunta come rete di sicurezza SECONDARIA (avvolge le chiamate reali <code>Connect-ExchangeOnline</code> avenga DOPO che il modulo e' gia' stato importato in quella sessione - li' una riparazione vera non e chiaramente il problema e la necessita' di un riavvio invece del criptico errore .NET nativo. Bug collater questa rete di sicurezza veniva a sua volta intercettato e sovrascritto da <code>Get-M3650psModuleConfli</code> come sottostringa) - aggiunta una frase-sentinella per evitare il doppio rivestimento. Verificato dal vi completo: cancellato il file critico, chiamata la funzione vera <code>Connect-M3650psExchange</code> attraverso rilevato e riparato PRIMA dell'import, connessione proseguita fino al normale errore di certificato finto
RISOLTO DAVVERO il conflitto MicrosoftTeams/ExchangeOnlineManagement: fissata la coppia ExchangeOnlineManagement 3.1.0 + MicrosoftTeams 6.5.0 (Exchange connesso prima), verificata dal vivo priva del conflitto in ogni flusso del progetto incluso il login delegato device-code (v0.9.47, 25/08/2026 - versione Exchange poi corretta a 3.4.0 in v0.9.48, vedi riga sotto)	corretto	L'utente ha rifiutato la conclusione precedente ("nessuna coppia e' affidabile, si gestisce solo l'errore q usato fino all'altro giorno! qualche versione supporta il device code? hai eseguito test reali con modul ragione a insistere - non erano ancora stati testati Exchange "serie 3" con Teams di eta' comparabile, s Prima di implementare, una domanda intermedia dell'utente ("puo' essere un problema di ORDINE DI suggerimento online) e' stata verificata dal vivo e scartata: import Exchange poi Teams, ma CONNECT confermando che conta l'ordine con cui si CONNETTE (si tocca per la prima volta la libreria di autentic modulo. Trovato poi che ExchangeOnlineManagement 3.1.0 e' la piu' vecchia versione con TUTTO cio' aggiunto fra 3.0.0 e 3.1.0: <code>-CertificateThumbprint</code> su <code>Connect-IPPSession</code> per Purview, pre: e 6.5.0 (stesso <code>Microsoft.Identity.Client</code> 4.29.0.0 bundled in tutta la fascia Teams 4.0.0-6.5.0): c del progetto (App-only certificato, delegato device-code via <code>-AccessToken</code> , Purview App-only via Teams App-only connesso dopo tutti e tre) risultavano puliti, senza alcun conflitto - ripetuto due volte versioni, non solo una: nuova funzione <code>Private/Assert-M3650psTeamsSafeVersion.ps1</code> (stess incluso il controllo di integrita' dei file introdotto nel fix precedente) garantisce che MicrosoftTeams 6. connessione - richiamata da <code>Connect-M3650psTeams.ps1</code> e, in modalita' solo-installazione, da In: reale in quel momento (poi ridimensionato in v0.9.48, vedi riga sotto): <code>Connect-IPPSession</code> code/token (solo <code>-UserPrincipalName</code> / <code>-Credential</code> o i parametri App-only) - <code>Connect-M3650</code> messaggio chiaro invece del criptico errore di parametro mancante. Bug collaterale corretto nello stes <code>IPPSession</code> in 3.1.0 (esiste invece su <code>Connect-ExchangeOnline</code> , da sempre) - controllato dinam <code>Connect-M3650psTeams.ps1</code> . Verificato dal vivo end-to-end attraverso le funzioni reali del mo giro successivo, v0.9.48, ha scoperto perche' questo non bastava): <code>Connect-M3650psExchange</code> , <code>Co M3650psExchangeDelegatedLogin</code> , poi <code>Connect-M3650psTeams</code> - nessun blocco, nessun conflittt usate per il test.
CORREZIONE il giorno stesso: 3.1.0 sembrava a posto con credenziali finte ma era in realta' INUTILIZZABILE per Purview con credenziali reali (protocollo dismesso da Microsoft) - corretto il pin a 3.4.0, implementato il login delegato interattivo a Purview, aggiunto il relativo pulsante in GUI (mai esistito prima) (v0.9.48, 25/08/2026)	corretto	Dopo aver implementato v0.9.47, l'utente ha chiesto "esiste un'altra via per Purview [delegato]?" - indi: correlato al conflitto Teams: testando <code>Connect-M3650psCompliance</code> dal vivo nella GUI vera, sul ten nei test precedenti), Purview App-only falliva con "Connecting to remote server ps.compliance.protect errore di rete/protocollo, non un conflitto di assembly. Causa: ExchangeOnlineManagement 3.1.0 usa (RPS/WinRM) per Security & Compliance, che Microsoft ha dismesso LATO SERVER dal 15 luglio 2023 connessione) - un endpoint che semplicemente non esiste piu', nessuna correzione lato progetto pote non lo avevano mai scoperto perche' fallivano PRIMA, sul controllo locale del certificato, senza mai arr un test con credenziali finte verifica l'assenza di un conflitto di assembly, non che una funzionalita' sia ExchangeOnlineManagement 3.2.0 ha spostato Security & Compliance sulle REST API (stesso banner c condividono TUTTE la stessa build di <code>Microsoft.Identity.Client</code> (4.44.0.0), la garanzia di non-cc

		automaticamente a 3.4.0 senza dover essere riverificata da zero (stessa libreria condivisa, stesso comp vivo per sicurezza, stavolta con Purview REALE: connessione autentica riuscita sul tenant di test, non sc 3.1.0 a 3.4.0. Login delegato a Purview implementato per davvero (non solo segnalato come assen Device ne' -AccessToken , ma -UserPrincipalName da solo attiva un vero prompt di autentica modulo) - un popup bloccante, stesso principio gia' usato per Teams/SharePoint delegati, sicuro dal p libreria condivisa gia' verificata pulita. Connect-M365OpsCompliance.ps1 ora sceglie dinamicamen futura) e il solo -UserPrincipalName . Scoperto nello stesso giro che Purview Delegata non ave M365OpsRetentionCompliancePolicies chiamava Connect-M365OpsCompliance senza -Allow connetterlo" senza che esistesse davvero un modo di farlo) - aggiunto un riquadro "Purview / Compli Exchange/SharePoint/Teams) con nuovo endpoint POST /api/compliance-test e stato esposto ir END-TO-END attraverso la GUI vera, sul tenant REALE, non piu' con credenziali finte: cliccati in s tutti e tre connessi con successo nello stesso processo server, con dati reali restituiti (mailbox condivis Macchina di sviluppo riallineata alla coppia corretta 3.4.0/6.5.0 (rimossa ogni altra versione installata d
BUG REALE trovato su una macchina di test nuova: Intune App-only rifiutava un profilo configurato SOLO con certificato ("non ha un SecretEnvVar configurato"), anche se lo stesso certificato funzionava gia' per Exchange sullo stesso profilo (v0.9.49, 25/08/2026)	corretto	Segnalato dal vivo: "da macchina nuova di test, provo connessione ad intune con app e ottengo Error Connect-M365OpsIntune.ps1 era l'UNICA cmdlet di connessione del progetto a richiedere sempre certificato come alternativa - a differenza di Exchange/Teams/SharePoint, che accettano da tempo -S Set-M365OpsTenant). Verificato dal vivo sul modulo IntuneWin32App installato che Connect-M365OpsIntune.ps1 non solo il thumbprint) - il parametro esisteva da sempre, semplicemente non certificato-o-secret gia' usato ovunque nel progetto: se SecretEnvVar non e' configurato ma Exch Cert:\CurrentUser\My poi Cert:\LocalMachine\My (stessi due store gia' provati da Connect passa a -ClientCert . Se nessuno dei due e' configurato, il messaggio d'errore ora lo dice esplicitar con credenziali reali, non solo per lettura del codice: simulato un profilo "solo certificato" (stesso cert nessun secret) e chiamata Connect-M365OpsIntune per davvero - connessione riuscita attraverso il
TRE bug reali segnalati dal vivo dopo un test pulito (nessun riavvio server nel mezzo) del login delegato Exchange: (1) poll "fantasma" mostrava "Nessun login Exchange in corso" a connessione gia' riuscita; (2) voce "Sessione delegata" in Ambiente non completo confondeva la sessione Graph generica con quella Exchange; (3) salvare il profilo tenant GIA' ATTIVO (es. Intune restava "connesso" dopo aver modificato/riattivato il profilo) non ne resettava mai lo stato di connessione (v0.9.50, 26/08/2026)	corretto	L'utente ha incollato il codice dispositivo, atteso il completamento e ricevuto "Errore: Nessun login Exc Start-M365OpsExchangeDelegatedLogin" - ma cambiando tab la connessione risultava GIA' attiva, pr Gui\Server.ps1 serve le richieste HTTP in un unico ciclo sincro (\$listener.GetContext()) , n impiega qualche secondo in piu' (dentro Complete-M365OpsExchangeDelegatedLogin si chiama e questa funzione puo' essere piu' lenta del solito, gia' documentato nel suo stesso .SYNOPSIS) e il f (rete/proxy/antivirus locali, fuori dal nostro controllo), il browser pianifica un poll successivo - che perc riuscito e con la voce "pending" gia' rimossa: il poll "in piu'" trova il dizionario vuoto e restituisce l'erro livelli: (a) Complete-M365OpsExchangeDelegatedLogin.ps1 , prima di dichiarare errore per "nessu (\$script:M365OpsExchangeConnected) - se gia' vero, risponde Status = 'Completed' invece Intune in Test-M365OpsIntuneAuthValid (mai fidarsi ciecamente dell'assenza di un flag quando lo Gui\index.html , che prima lasciava il pannello di stato bloccato sul messaggio vecchio finche' l'ute LoadConnectionStatus() / LoadSetupStatus() , cosi' un eventuale errore residuo si autocorregg report, la voce "Sessione delegata" di Get-M365OpsSetupStatus.ps1 (segnalata dall'utente come ' sessione attiva" a login Exchange gia' riuscito) si e' rivelata corretta ma ambigua: controlla la sessione utente", usata per utenti/gruppi/licenze), del tutto separata dalla sessione Exchange PowerShell apper delegata (generale)" con una nota esplicita che restare "Missing" li' mentre Exchange e' gia' connesso Investigando infine la segnalazione separata "torno sul tenant delegato e Intune risulta ancora connes il reset di tutti i flag di connessione (incluso Intune) avviene SOLO dentro Connect-M365Ops , chiam: "Salva profilo" (POST /api/tenants , usato anche per modificare le impostazioni del tenant GIA' att richiamava mai dopo Set-M365OpsTenant , a differenza di POST /api/email-settings che lo fa \$script:M365OpsContext e ogni flag di connessione (Intune compreso) fermi ai valori di prima de TenantProfile \$body.name subito dopo il salvataggio, ma solo quando il profilo salvato e' quello : eq \$body.name) - un salvataggio di un profilo NON attivo resta innocuo come prima. Tutti e tre i file PowerShell/parser.NET); il fix (a) e (c) non riproducibili in isolamento senza un tenant Delegato reale e end-to-end sul codice sorgente reale (non un frammento), non con un nuovo test dal vivo dedicato; u
GUI Tenant: sezione "Aggiungi/aggiorna profilo" ora collassata di default con un pulsante "Modifica" per riga che la apre precompilata; corretto anche il glitch grafico gemello Intune (messaggio "Connesso" rimasto accanto al pulsante dopo un vero disconnect) (v0.9.51, 26/08/2026)	implementato	Richiesto esplicitamente dall'utente: "la sezione aggiungi/aggiorna profilo deve essere collassata e no da nome profilo richiamare i profili attivi per modificarli, cosi' com'e' non si capisce se per aggiornare (tab Tenant) e' ora dentro #add-profile-section , chiusa di default, apribile con un pulsante "+ Nu #profile-list ha un nuovo pulsante "Modifica" che apre la sezione GIA' precompilata con i dati s autenticazione, Client ID, nome variabile secret, UPN delegato, thumbprint certificato) - MAI il valore c nome variabile lo e', gia' spiegato nell'hint sopra il form). Get-M365OpsTenantList (Gui\Server . esponeva solo nome/tenantId/authMode, insufficiente per precompilare). Un pulsante "Annulla" chiuc sezione si richiude automaticamente invece di restare aperta col form "consumato". Salvare con lo ste duplicazione - comportamento gia' esistente lato Set-M365OpsTenant , solo ora reso scopribile dall avviato apposta sul profilo reale vnsys-test di questa macchina di sviluppo): cliccato "Modifica" su con tutti i campi attesi e secret vuoto; "Annulla" - sezione chiusa, form svuotato; creato un profilo di p conferma corretto), poi rimosso lo stesso profilo di prova per non lasciare residui. Bug gemello corre mentre testava il fix del 21/08/2026 sul reset dei flag di connessione al salvataggio del profilo attivo: p stato Intune tornava correttamente rosso ("Non connesso") ma il testo accanto al pulsante "Connetti l sessione precedente - stesso identico bug gia' corretto il 17/08/2026 per Exchange/SharePoint/Teams, un refresh di stato successivo), ma mai applicato al riquadro Intune, aggiunto dopo quel fix. Corretta c quattro riquadri in LoadConnectionStatus() .
ROOT CAUSE REALE del "Nessun login Exchange in corso" trovata riverificando dal vivo il fix v0.9.50: un pipeline leak trasformava la risposta del login riuscito in un array malformato (Status sempre undefined lato client), non un poll duplicato/ritardato come ipotizzato - scoperto un BUG DI SICUREZZA nello stesso punto, AccessToken/RefreshToken veri arrivavano fino al browser via JSON (v0.9.52, 26/08/2026)	corretto	Su richiesta esplicita dell'utente ("fai questo prima" - riverificare il fix v0.9.50 con un riavvio pulito prim login delegato Exchange dal vivo (device code + MFA reale sul tenant "vnsys delegata"), non solo per risultato inatteso: un ARRAY di due oggetti invece del singolo {Status; Message} atteso. Causa re v0.9.50: dentro Complete-M365OpsExchangeDelegatedLogin.ps1 , la chiamata Assert-M365Ops con RemovedVersions/Installed) non era mai stata soppressa ne' catturata - il suo valore di ritorno fini al risultato vero, producendo un array. Lato client (Gui\index.html) result.Status su un array quello "Error" scattavano mai su un login riuscito, il poll continuava a ripetersi mostrando "in attesa" -

		pending ormai gia' rimossa (consumata dal primo poll, quello riuscito ma malformato): QUESTO e' il n Exchange in corso", con la connessione gia' stabilita da tempo dietro le quinte. Il fix v0.9.50 (controllar dichiarare errore) restava corretto e utile come rete di sicurezza, ma curava il sintomo di QUESTO bug, Assert-M365OpsExoSafeVersion (Out-Null) e riscrivendo la funzione per costruire sempre e invece di restituire variabili intermedie cosi' come sono. Scoperto nello stesso momento un problem vecchia funzione restituiva \$result (l'oggetto di Receive-M365OpsDeviceCodeToken) COSI' COI VERI (servono per passarli a Connect-ExchangeOnline -AccessToken subito dopo) - Gui\Serve la risposta HTTP: il refresh token (il materiale piu' sensibile, permette di ottenere nuovi access token se che servisse mai fuori dal processo server. Corretto nello stesso giro: la funzione ora restituisce sempre mai un oggetto intermedio grezzo, in ogni ramo (Completed/Pending/Error). Fix gemello applicato f M365OpsWithExoRepairRetry (usata anche da Connect-M365OpsExchange.ps1 e Connect-M36 sopprimerne l'output - nessuno dei quattro punti di chiamata usa quel valore, quindi un'eventuale em ExchangeOnline / Connect-IPPSession avrebbe potuto propagarsi con lo stesso identico mecca condivisa invece che in ogni chiamante. Verificato dal vivo end-to-end due volte, non per lettura de malformato con token veri nella risposta) su un login delegato reale con MFA sul tenant di test; poi, di delegato reale da zero - risposta del poll un singolo oggetto pulito {Status:"Completed", Messag connessione Exchange confermata vera via /api/mcp-status (exchangeConnected:true).
Nuovo connettore AI: server MCP CLI Microsoft 365 (@pnp/cli-microsoft365-mcp-server) collegato al motore di ragionamento in chat, con lo stesso livello di sicurezza (proposta+conferma) gia' in uso per Lokka; livello di connessione MCP reso generico (Lokka non e' piu' l'unico server tracciabile); parecchi dettagli reali del pacchetto risultati diversi dalla sua documentazione pubblica, corretti dopo test dal vivo (v0.9.53, 26/08/2026)	implementato	Richiedo esplicitamente dall'utente: valutare se CLI Microsoft 365 (pnp.github.io/cli-microsoft365) pot Graph/Teams/SharePoint/Exchange per gli ambiti che copre. Valutazione, prima di implementare: g ufficiale - spo, entra, outlook, planner, onedrive, onenote, todo, teams (solo lato Graph: app/meeting/ Power Platform. Il gruppo 'exo' esiste ma e' stretto (solo application role assignment, non gestione ma comando di policy Teams (meeting/voice/calling policy, federation - API "Teams Admin Center" mai es moduli PS nelle due aree che generano davvero il conflitto Teams/Exchange di sezione 6.6 (policy Tear complemento per SharePoint/Entra/Outlook/Planner/Purview-audit, mai un fallback generale. Archite connessione MCP, prima scritto solo per Lokka (variabili singolari \$script:M365OpsLokkaProcess / M365OpsMcpServer / Invoke-M365OpsMcpServerTool / Disconnect-M365OpsMcpServer / Disc due dizionari per nome (\$script:M365OpsMcpProcesses / ...McpTools) cosi' piu' server MCP re Connect-/Invoke-/Disconnect-M365OpsLokka restano come alias sottili, nessun altro codice esis M365OpsCliMicrosoft365.ps1 (nuovo): sincronizza l'autenticazione CLI Microsoft 365 (comando # disco condiviso per l'utente Windows, non per sessione) col tenant M365Ops attivo usando connessio 1:1 sul nome del profilo tenant - stesso principio gia' applicato due volte in questa sessione per evitar cambio profilo (Intune, sezione 6.4). Supporta solo login App-only a client secret per ora (riusa lo stess profilo solo a certificato o un tenant Delegato vengono rifiutati con un messaggio chiaro invece di un .NOTES del file). (3) Quattro nuovi tool AI (cli_m365_run_command / cli_m365_search_commands / cli_m365_get_command_docs / prop configurato per il tenant attivo - le scritture passano sempre da propose_cli_m365_command (stess \$pendingWrite / \$script:PendingAction / Execute-PendingAction) di ogni altra scrittura del sottostante non distingue lettura da scrittura al suo interno (un solo tool generico esegue qualunque (Private) classifica per VERBO FINALE del comando invece di mantenere una lista di ~400 comandi a r altro verbo (incluso uno mai visto) = scrittura per difetto, mai il contrario - verificata con una batteria c diversi dalla documentazione pubblica, trovati SOLO chiamando davvero il server MCP installat (m365_run_command , non m365RunCommand come mostrato sui siti di terze parti che lo elencano); " (rifiutato altrimenti con "Only m365 commands are allowed"); --output json NON va mai impo automaticamente, un suggerimento embedded nella risposta di m365_search_commands lo dice esj commandName sia docs (un percorso file), entrambi presi dal risultato di m365_search_commands reale piu' importante, mai documentato da nessuna parte: con login a client secret, ogni comando not support authentication using client ID and secret" - verificato con un comando spo site list perfettamente su entra / outlook / planner (dati reali recuperati con successo). SharePoint via q login a certificato, non ancora implementato. Prerequisito nuovo, non gia' presente nel progetto: MCP lo richiede installato globalmente, non lo scarica da solo) - un'installazione globale npm appena esecuzione (il PATH aggiornato a livello di registro non si propaga a processi gia' avviati), quindi il solc chiudere tutto e riavviare da capo (collegamento sul Desktop). Errore intercettato con un messaggio a Verificato dal vivo end-to-end, non per lettura del codice: sul tenant reale "vnsys-test" (App-only) confermato reale), riconnessione con connessione gia' esistente (connection use) verificata separa restituiti, pulizia dello stato confermata passando a un altro tenant (Disconnect-M365OpsAllMcpSei "non connesso"), e il messaggio d'errore azionabile riprodotto e verificato attraverso il server GUI vero PATH gia' corretto). Riquadro GUI "Altri server MCP" (tab MCP/Connettori) esteso con un pulsante "Co bug di UI trovato e corretto nello stesso giro: il refresh della lista subito dopo aver mostrato l'esito car breve.
Ogni risposta dell'AI in chat (e ogni riga di log per tool chiamato) mostra ora la SORGENTE reale usata - Lokka (MCP), CLI Microsoft 365 (MCP), Graph diretto (Delegato) o moduli PowerShell interni; aggiunto avviso "server a thread singolo" per CLI Microsoft 365, analizzato un errore ricorrente nei log all'avvio (v0.9.54, 26/08/2026)	implementato	Richiedo esplicitamente dall'utente dopo l'introduzione di CLI Microsoft 365: sapere sempre quale ba traccia era il nome tecnico del tool (es. "graph_api_call"), che da solo non dice se e' passato da Lokka c M365OpsToolSourceLabel (Private) mappa ogni tool AI alla sua sorgente reale (attenta al caso grap NON passa mai da Lokka ma da Graph diretto con token delegato - etichetta diversa in quel caso). Og ...] ; ogni risposta finale dell'AI (sia Claude sia Azure OpenAI) aggiunge in fondo una riga Fonte dati tool e' stato davvero chiamato - una risposta puramente conversazionale non la mostra. Verificato dal utenti ci sono nel tenant?"): risposta "Nel tenant ci sono 55 utenti... Fonte dati: Lokka (MCP)." e riga di davvero, non per lettura del codice. Nello stesso giro, analizzato un errore visto ripetersi nei log dell' calling Write... Bytes to be written to the stream exceed the Content-Length bytes size specified"): ripr connessione TCP speculativa/interrotta del browser subito dopo l'apertura automatica della pagina (Lz server all'avvio) - GIA' contenuto in modo sicuro dalla protezione del 18/08/2026 (il server non si inter basta). Verificato con l'elenco delle richieste di rete del browser che NESSUNA richiesta reale fallisce p solo rumore nei log, non e' stato modificato altro codice per questo. Aggiunto infine un avviso explicit

		thread singolo, "Connetti/Test" puo' richiedere parecchi secondi al primo utilizzo (scaricamento npx) e stesso principio gia' scritto per Teams/SharePoint/Intune su tenant Delegati, mancava per questo nuov
BUG REALE trovato dal vivo dallo screenshot dell'utente: tab MCP/Connettori mostrava "Errore nel caricare i server MCP" quando nessun tenant e' attivo, invece di degradare come le altre sezioni della GUI (v0.9.55, 26/08/2026)	corretto	<code>Get-M365OpsMcpServers</code> lanciava un'eccezione quando <code>\$script:M365OpsContext</code> e' vuoto (ne: in <code>Gui\Server.ps1</code> non aveva un try/catch proprio, quindi l'eccezione arrivava al gestore d'errore g che il JS della GUI (<code>servers.find / .filter</code>) non sa interpretare - da qui il messaggio "Errore nel c profilo dal tab Tenant" gia' mostrato altrove. Nessun altro chiamante dipendeva dal throw (verificato l @() , coerente con <code>Get-M365OpsTenantList</code> , che gia' fa lo stesso quando non c'e' nessun profilo s (mai ancora manifestato, trovato in revisione mentre si sistemava quello sopra) nel campo <code>mcpServe</code> attivo, <code>Get-M365OpsActiveTenantInfo</code> restituisce <code>\$null</code> , e " <code>\$null ForEach-Object {...}</code> vuota - ne produce UNA con una voce fantasma (<code>{name:null,...}</code>) - stesso identico bug di scope progetto per lo storico chat vuoto, qui prevenuto con <code>Where-Object { \$_ }</code> prima del <code>ForEach-</code> modulo importato SENZA mai chiamare <code>Connect-M365Ops</code> , <code>Get-M365OpsMcpServers</code> restituisce
CLI Microsoft 365 esteso ai tenant Delegati (login a browser bloccante) e collegato anche nel tab Tenant, sezione "Stato connessioni" - prima disponibile solo App-only e solo dal tab MCP/Connettori (v0.9.56, 26/08/2026)	implementato	Due richieste esplicite dell'utente sullo stesso giro. (1) "la CLI supporta anche il login a browser, perche i tenant Delegati per intero, ragionando che 'm365 login --authType deviceCode' e' bloccante e quindi corretto qui: il login bloccante NON e' un motivo per escludere un tenant, e' esattamente lo stesso lim Teams/SharePoint/Intune su tenant Delegati (parametro <code>-AllowInteractive</code> esplicito, un click con bloccato per tutti", mai un login automatico dentro un flusso composto). <code>Connect-M365OpsCliMicr</code> un tenant Delegato senza connessione gia' salvata restituisce lo stesso messaggio "vai in GUI e clicca" <code>authType browser --tenant ... --connectionName ...</code> (scelto browser invece di deviceCode: Teams/SharePoint/Intune, non per un vincolo tecnico reale - deviceCode e' anch'esso bloccante sulla C <code>Connect-M365OpsMcpServer</code> e l'endpoint <code>POST /api/mcp-servers/connect</code> propagano il para il nuovo pulsante nel tab Tenant mostrano lo stesso avviso di conferma gia' usato per Intune prima di nel tab Tenant, non solo nel tab MCP": nuovo riquadro "CLI Microsoft 365" nella sezione "Stato conn: Exchange/SharePoint/Teams/Purview/Intune, stesso schema identico), con pulsante "Connetti CLI Micr <code>data.mcpServers</code> - stesso backend del pulsante gia' esistente nel tab MCP/Connettori, solo un sec quotidiano sul tenant, invece di dover cambiare tab. Verificato dal vivo: sul tenant App-only "vnsys-ti (stesso comportamento del tab MCP/Connettori, stesso messaggio d'errore azionabile quando il prere Delegato "vnsys delegata", confermato che il pulsante richiede la conferma esplicita PRIMA di procedo davvero il ramo di login a browser (non il messaggio "serve -AllowInteractive") - il completamento RE/ in una finestra visibile) va confermato dall'utente stesso al primo uso vero.
CLI Microsoft 365: installazione automatica del prerequisito mancante (npm install -g @pnp/cli-microsoft365), invece di chiedere solo all'utente di farlo a mano (v0.9.57, 26/08/2026)	implementato	Segnalato dal vivo dall'utente: dopo un riavvio completo, stesso errore "comando 'm365' non trovato" per Lokka no?". Aspettativa legittima ma basata su un'analogia imperfetta, chiarita e poi colmata: Lokk esegue il pacchetto al volo, nessuna installazione globale necessaria), ma <code>@pnp/cli-microsoft365-</code> internamente il comando <code>m365</code> come processo ESTERNO gia' installato - npx scarica il wrapper, mai Nuova <code>Assert-M365OpsCliMicrosoft365Installed</code> (Private, stesso principio di <code>Assert-M365Op</code> se <code>m365 / m365.cmd</code> e' risolvibile PRIMA di avviare il server MCP (fallisce/installa subito invece di far sola <code>npm install -g @pnp/cli-microsoft365</code> (richiede solo Node.js/npm, stesso prerequisito gi spiegato chiaramente invece di lasciarlo scoprire per errore: anche con l'installazione automatica r appena installato a un processo GIA' in esecuzione - il processo server ATTUALE non vedra' comunque tutto e riaprire dal collegamento sul Desktop, non basta "Riavvia server" se anche quello discende dall funzione lo dice esplicitamente nel messaggio finale invece di un generico "installato, riprova". Testo c aggiornato per riflettere l'installazione automatica invece di dire solo "esegui questo comando a manc tenta piu' nessuna reinstallazione inutile (nessun messaggio "lo installo ora" quando il comando e' gia <code>M365OpsMcpServer</code> .
Processi MCP isolati per TENANT (non piu' terminati ad ogni cambio profilo); CLI Microsoft 365 spostata dal tab MCP/Connettori al tab Tenant; installazione automatica out-of-the-box del prerequisito npm; rimossi i pin fissi di versione ExchangeOnlineManagement/MicrosoftTeams (ora sempre l'ultima disponibile) come primo passo verso l'isolamento reattivo dei processi Teams/Exchange (v0.9.58, 26/08/2026)	implementato	Tre richieste esplicite dell'utente sullo stesso giro. (1) "rendi ogni processo MCP isolato da altri tenant, <code>\$script:M365OpsMcpProcesses / ...McpTools</code> erano dizionari a un livello (per nome server, con terminato e riavviato ad ogni cambio profilo (<code>Disconnect-M365OpsAllMcpServers</code> chiamata da C nome server - un processo OS distinto per ogni coppia, mai condiviso; il cambio tenant non termina p riattivano istantaneamente se si torna su un tenant gia' visitato in questa sessione. Verificato dal vivo ("attiva" di CLI Microsoft 365 vive in un file di configurazione CONDIVISO per l'intero utente Windows processi isolati di due tenant diversi potrebbero quindi in teoria interferire tramite quel file condiviso. documentazione se i comandi rileggano quell'identita' dal file ad ogni chiamata o la mettano in cache un'operazione che tocca dati reali di un tenant: <code>Invoke-M365OpsMcpServerTool</code> ora riafferma espl immediatamente PRIMA di ogni comando reale (mai sulle sole chiamate di ricerca/documentazione, c minimo che garantisce correttezza per costruzione invece di fidarsi di un comportamento interno non per una pulizia totale (riavvio vero del server, rimozione di un profilo tenant) ma non e' piu' automatic almeno per il 365 cli": il pulsante "Connetti/Test" di CLI Microsoft 365 e' stato tolto dal tab MCP/Conn una nota che rimanda al tab Tenant) - vive solo nel tab Tenant, sezione Stato connessioni, insieme a Ex fatto che tutti sono specifici del tenant attivo. (3) "la CLI 365 non arriva preconfigurata su una macchin <code>install -g @pnp/cli-microsoft365</code> aggiunto sia a <code>Show-M365OpsPrereqInstaller.ps1</code> (l'in: primissima volta - cosi' il primo processo mai avviato ha gia' 'm365' sul PATH, niente piu' "serve un ria <code>M365OpsPrerequisites.ps1</code> (seconda passata idempotente) - <code>Assert-M365OpsCliMicrosoft365</code> per chi salta entrambi questi percorsi. Nello stesso giro , primo passo verso l'isolamento reattivo dei p lavoro grosso resta da fare, questo ne e' solo la base): <code>Assert-M365OpsExoSafeVersion / Assert-</code> versione fissa (3.4.0/6.5.0) - installano e mantengono sempre l'ULTIMA versione disponibile, rimuovenu contattare PSGallery ad ogni connessione (solo quando serve davvero installare) per non aggiungere l ragion d'essere di un pin - evitare il conflitto scegliendo con cura una coppia di versioni - non regge p connessione al bisogno invece di evitarlo a monte (la stessa coppia 3.4.0/6.5.0 presentata come "verifi e v0.9.52). Aggiornati in coerenza <code>Invoke-M365OpsWithExoRepairRetry.ps1</code> (rimossa una version <code>Show-M365OpsPrereqInstaller.ps1 / Install-M365OpsPrerequisites.ps1</code> (nessun pin, nessu controllo sintassi su tutti i 320 file .ps1 del progetto; isolamento MCP per tenant verificato dal vivo (co senza terminare il primo, switch di ritorno con riuso confermato dai log, comando reale eseguito corre

Isolamento reattivo dei processi Teams/Exchange: il conflitto .NET di sezione 6.6 viene ora risolto DAVVERO nello stesso turno (sottoprocesso separato per il modulo in conflitto), senza più richiedere un riavvio del server - due bug reali trovati e corretti durante la verifica dal vivo (v0.9.59, 26/08/2026)	implementato	Completamento del lavoro iniziato in v0.9.58 (rimozione dei pin di versione come base per questo). N Private\Workers\M3650psIsolatedWorker.ps1 : script standalone eseguito come processo pws ExchangeOnlineManagement OPPURE SOLO MicrosoftTeams (mai entrambi), comunica col proc già usato per Lokka/CLI Microsoft 365). Connect-M3650psExchange.ps1 / Connect-M3650psTeam l'isolamento (nuova Private\Connect-M3650psIsolatedModule.ps1) SOLO dentro il blocco catcl riconosce davvero il pattern del conflitto in un'eccezione reale già avvenuta - mai preventivo, un utent mai il costo di un secondo processo. Il worker, alla connessione, riporta al processo padre l'elenco CO Connect-M3650psIsolatedModule.ps1 usa per generare dinamicamente una funzione PROXY glo sotto ~800 nomi diversi via Set-Item function:global:<nome> , con un blocco DynamicParam interrogandoli dal worker - verificato dal vivo che \$MyInvocation.MyCommand.Name riporta corrett anche condividendo lo stesso scriptblock) - da quel momento, QUALUNQUE chiamata bare a uno di q viene automaticamente intercettata e inoltrata al processo isolato, senza dover modificare nessuno de vivo durante la prima verifica end-to-end (non teorico): un primo tentativo filtrava i comandi resi di ExchangeOnlineManagement dopo il connect, trovando solo 32 cmdlet invece degli oltre 130 attesi mancanti) - causa reale, non un problema di tempistica (un primo tentativo di correzione con un'attes ExchangeOnline inietta la stragrande maggioranza dei comandi tramite implicit remoting in un moc letteralmente "ExchangeOnlineManagement" - un filtro per nome di modulo li manca quasi tutti per c TUTTI i comandi disponibili nella sessione (Get-Command -CommandType Function,Cmdlet , nesses comando il connect abbia reso disponibile, ovunque viva; verificato dal vivo che porta l'elenco a 835 c CASMailbox tutti presenti e funzionanti attraverso il proxy. SECONDO bug reale trovato nello stesso IPPSession (Purview, tentato dal worker come parte del connect Exchange, di proposito non blocc nessun chiamante legge mai quando il "connect" nel suo complesso riesce - il fallimento spariva quinc cmdlet Purview risultassero assenti. Corretto restituendo esplicitamente purviewConnected / purvi Connect-M3650psIsolatedModule.ps1 con Write-M3650psLog quando il collegamento Purview RetentionCompliancePolicy è risultato assente sul tenant di test PUR SENZA nessun errore di Con test minimale fuori da questa architettura (vedi sezione 6.6 per il dettaglio completo): limite pre-esiste qui, non risolvibile allungando l'attesa di stabilizzazione (verificato dal vivo, nessun effetto - comunqu che si ferma solo quando il conteggio totale dei comandi smette di crescere, fino a un tetto di 15s, più caso specifico). Un errore isolato di conflitto assembly su Get-CASMailbox visto UNA SOLA volta di chiamate consecutive successive (incluse chiamate interlacciate a più cmdlet) - trattato come anomalie grazie al logging aggiunto in questo stesso giro. Verificato dal vivo end-to-end sul tenant reale "vr primo, poi Exchange - isolamento attivato automaticamente e confermato nei log; Get-Mailbox/Get-D reali ripetutamente attraverso il proxy; Teams rimasto funzionante in-process per tutta la durata; disco sessione/cambio tenant.
Bug-hunt dedicato di 3 ore (mandato esplicito dell'utente): trovati e corretti un pipeline leak sistemico in tre funzioni di connessione condivise, 15 cmdlet di scrittura Exchange senza protezione da errori non terminanti, un limite CLI Microsoft 365 evitabile, e un vero buco nell'isolamento reattivo di sezione 6.6 (il conflitto può manifestarsi anche DOPO un connect riuscito, non solo durante) - tutti riprodotti e verificati dal vivo sul tenant reale "vnsys-test", non ipotizzati (v0.9.60, 26/08/2026)	corretto	Sessione di test strutturata come richiесто esplicitamente dall'utente: (1) verifica del fresh-install (revis M3650psPrerequisites.ps1/Show-M3650psPrereqInstaller.ps1, corretti due test obsoleti - un messag reale di 30s alzato in una sessione precedente, e un commento che descriveva un pin di versione orma (tutti i tab, form, pulsanti, dark mode, edge case come submit con campi vuoti, click ripetuti, testo dig tenant vnsys-test in modalità App-only (script dedicato, non ipotetico) - 96 riusciti, i 6 "falliti" tutt già noti (Copilot usage report), 1 limite Purview già documentato in sezione 6.6, 3 errori del mio stess apposta per zero-parametri, non bug del prodotto); (4) audit reale (non teorico) di quando l'IA usa da con conversazioni chat reali end-to-end (dettaglio in fondo a questa voce). BUG PIÙ SIGNIFICATIVO: pipeline leak in Connect-M3650psExchange.ps1 / Connect-M3650psT classe di bug già trovata e corretta UNA VOLTA in Complete-M3650psExchangeDelegatedLogin.ps soppressa, il suo valore di ritorno finiva sul pipeline insieme al risultato vero), ma quel fix copriva solo Assert-M3650psExoSafeVersion / Assert-M3650psTeamsSafeVersion dentro le tre funzioni di OGNI cmdlet del progetto che tocca Exchange/Teams/Purview come primo passo di un processo serv Riprodotta dal vivo creando una distribution list tramite la chat su un processo server fresco: la rispost l'oggetto di diagnostica versione ANCHE insieme al risultato vero della scrittura, invece del solo risulta chiamate mancanti; verificato dal vivo PRIMA (bug riprodotto) e DOPO (risposta pulita, un solo oggett distribution list attraverso la GUI vera. 15 cmdlet di scrittura senza -ErrorAction Stop - stessa classe di bug già corretta UNA VOLTA p hunt 19/08/2026: i cmdlet nativi Exchange/Teams generano di default errori NON terminanti, che senz funzione dichiara "riuscito" anche quando l'operazione vera è fallita), ma mai estesa sistematicamente mirata ai verbi New/Set/Remove/Enable/Disable ha trovato e corretto 12 file (New-M3650psDistrib M3650psDynamicDistributionGroup , New-M3650psEquipmentMailbox , New-M3650psMailCont M3650psRoomMailbox , New-M3650psSharedMailbox , New-M3650psTeam , New-M3650psTransp M3650psTransportRule , Set-M3650psTransportRuleState); una seconda scansione più ampia (s solo i verbi comuni) ha trovato altri 3 file sfuggiti alla prima perché usano verbi nativi meno comuni nc M3650psQuarantineMessage / Remove-M3650psQuarantineMessage , verbi "Release"/"Delete"; S filtro ma il file era stato inizialmente saltato) - lezione diretta: una scansione automatica è utile ma no sono uniformi. Verificato dal vivo, non solo per lettura del codice: creato un contatto di posta reale creazione di un SECONDO contatto con lo stesso nome (duplicato) - PRIMA del fix questo genere di f DOPO il fix, l'eccezione reale di Exchange ("There are multiple recipients matching identity...") viene co Graph (Invoke-M3650psGraphRequest , già con -ErrorAction Stop al suo interno), SharePoint/ già sicuri per costruzione (wrapper condivisi che già gestiscono l'errore correttamente) - il problema e Exchange/Teams sparse nei singoli file. CLI Microsoft 365: eliminato un riavvio non più necessario - Assert-M3650psCliMicrosoft365 automatica riuscita (npm install -g @pnp/cli-microsoft365) lanciava sempre un errore "riavvio propaga un PATH di registro appena aggiornato a un processo già in esecuzione) - ma QUESTA funzio sottoprocesso npx del server MCP CLI-Microsoft365 (Connect-M3650psMcpServer.ps1 , chiamat

		QUESTO stesso processo (stesso trucco già usato con successo altrove nel progetto per lo stesso scen <code>M365OpsPrereqInstaller.ps1</code>), il sottoprocesso spawnato subito dopo eredita il PATH aggiornato end-to-end attraverso il flusso reale: una domanda in chat sui piani Planner ha attivato l'installazion semplice restart del processo server, non un riavvio completo del PC) ha connesso CLI Microsoft 365 c (correttamente riportando poi che CLI M365 non espone un comando per l'elenco Planner a livello di M365Ops, riportato onestamente invece di essere nascosto o inventato). Buco reale nell'isolamento reattivo di sezione 6.6: vedi quella sezione per il dettaglio completo (nu <code>M365OpsWriteWithIsolationRecovery.ps1</code> , pattern <code>MSAL_MissingMethodException</code> aggiunto durante un test dal vivo apparentemente innocuo (rinominare la descrizione di un Team di test), non u sessione: il conflitto ha lasciato un'instabilità residua nel processo oltre alla singola operazione che lo dopo, hanno cominciato a fallire con un errore correlato ma diverso (<code>IDX12729</code> , decodifica token JA un processo nuovo: nessun errore) che un riavvio completo resta l'unico rimedio COMPLETO, il recupe cura totale - dettaglio e raccomandazione onesta in sezione 6.6. Altri fix minori trovati durante il giro GUI: (a) i pulsanti Si/No di una proposta di scrittura risolta scr restavano abilitati nella cronologia della chat - un click tardivo su di essi non riesegue nulla (il server ic ma restava confuso vedere più paia di pulsanti cliccabili insieme; corretto disabilitando ogni Si/No pre qualunque tipo (<code>Gui\index.html</code>); (b) il riquadro "Stato connessioni" del tab Tenant, sezioni Exchan FISSATO alla versione X, verificata come priva del conflitto" senza menzionare l'isolamento reattivo int reale attuale. Audit reale CLI Microsoft 365 vs Lokka vs script PS1 interni (richiesto esplicitamente, verificato cor codice): interrogato "quanti team ci sono e quanti membri ha ciascuno" → Lokka (Graph diretto, dati d Scope Tag Intune → moduli PowerShell interni (<code>New-/Remove-M365OpsScopeTag</code> , l'area Intune resta nel prompt di sistema); creazione/rimozione di una distribution list e di un gruppo di sicurezza Entra - non su Graph) e Lokka (Graph diretto, <code>POST /groups</code>); query sui siti SharePoint con più storage → n confermato ancora una volta il limite già noto: SharePoint via CLI M365 rifiuta sempre l'auth a client se permesso mancante), poi CLI Microsoft 365 come fallback dichiarato (limite reale dello strumento CLI, priorità già scritto nel prompt di sistema per l'IA, confermato nella pratica non solo sulla carta.
Log scritture separato + trasparenza completa su fonte/comando eseguito per ogni scrittura confermata (anche nel turno di esecuzione); isolamento reattivo Teams/Exchange/Purview esteso ai tenant Delegati; corretto un glitch reale del modello AI che dichiarava una scrittura già' eseguita (con un ID inventato) nella stessa risposta in cui la proponeva soltanto; guida aggiornata su CLI Microsoft 365 (v0.9.61, 27/08/2026)	implementato	Quattro richieste/scoperte sullo stesso giro, in ordine di priorit� dichiarata dall'utente. (1) "una finestre eseguono write su settings": nuovo <code>Write-M365OpsWriteLog</code> (Public) scrive in <code>Logs\writes-YYYYmm365ops-*.log</code> gi� usato dal log generale (altrimenti i due si sarebbero mescolati nelle ricerche esi <code>LogFilePrefix</code> (default invariato <code>'m365ops'</code>) per riusare la stessa logica di scansione/filtro su ent <code>scope=writes</code> su <code>GET /api/logs/search</code> ; GUI (tab Manutenzione, Cronologia log) ha ora un se stessi filtri di ricerca/livello/periodo gi� esistenti riusati identici. (2) "quando nella finestra principale ch lanciare, sia che sia Lokka, CLI 365 o qualunque cosa": la proposta PRIMA di confermare mostrava gi� turno di ESECUZIONE (dopo aver risposto "s�") non lo mostrava mai, ne' la sorgente - <code>Execute-Pend</code> costruire il trailer "Fonte dati: ..." oggi), quindi quel meccanismo non scattava mai su quel turno. Nuov in <code>Gui\Server.ps1</code>) applicato a TUTTI i case di scrittura (<code>ExoWrite/TeamsWrite/SharePointWrite/IntuneWrite/CustomWrite/LokkaWrite/CliM365Write/CreateG</code> loggia nel nuovo log scritture E aggiunge un trailer <code>"_Fonte: ... Comando eseguito: ..._"</code> al r <code>M365OpsCommandLine</code> (Public) costruisce una riga leggibile stile PowerShell da <code>cmdlet+hashtable</code> di <code>Get-M365OpsWriteActionSourceLabel</code> (Public) mappa il <code>Type</code> di un'azione confermata alla stes Microsoft 365/moduli interni/Graph diretto su Delegato). Anche i FALLIMENTI vengono loggati nel nu catch generico di <code>Execute-PendingAction</code>) invece di ripetere la stessa chiamata in ogni case, con l come dettaglio. Guida: nuova sezione 10.10, risposta a una domanda reale dell'utente ("mi chiede adr risolve?") - spiega che il login delegato Graph e quello di CLI Microsoft 365 usano DUE client pubblici perch� normalmente un login delegato NON richiede admin consent (auto-consenso utente), l'eccezio disabilitata, che blocca comunque entrambi i client finch� un admin non consente quello in uso), e cor Verificato dal vivo: controllo sintassi su tutti i file toccati e reimport pulito del modulo (funzioni nuov la stesura della guida stessa - un <code><div class="warn"></code> della nuova sezione 10.10 non richiuse corr trovato con un controllo di bilanciamento tag prima di procedere, non lasciato per caso. Isolamento reattivo esteso ai tenant Delegati (priorita' massima, richiesta urgente dal vivo): sub conflitto .NET reale di sezione 6.6 su un tenant Delegato (Teams+Exchange connessi, poi Purview fallit manifest definition does not match") - l'isolamento reattivo (v0.9.59/60) fino a qui copriva SOLO App- un errore esplicito su Delegato, quindi il tentativo di isolamento falliva a sua volta e l'utente vedeva so <code>Get-M365OpsIsolatedConnectParams</code> ora costruisce parametri Delegati (UserPrincipalName per E lanciare un errore; il worker isolato (<code>M365OpsIsolatedWorker.ps1</code>) ha un nuovo ramo Exchange Dc <code>UserPrincipalName</code> (popup interattivo) poi tenta <code>Connect-IPPSession</code> nello stesso processo is un processo senza Teams caricato); deliberatamente NIENTE device-code (-Device) in questo wo stdout per il protocollo JSON-RPC stesso, un codice dispositivo stampato li' non sarebbe mai visto da timeout (stessa classe di bug gi� vista con Teams device-code, sezione 17.14); il popup browser/WAM richiesta "connect" alzata da 60s a 240s quando il tenant e' Delegato (un popup richiede un'azione un non attraversa mai il confine tra processi, quindi il worker isolato rifa' un login interattivo TUTTO SUO per quel tenant in quel processo server - costo pagato una tantum per la vita del worker, non ripetuto test funzionale diretto di <code>Get-M365OpsIsolatedConnectParams</code> (parametri corretti per Exchange/I nessuna regressione su App-only) - il flusso di popup interattivo vero e proprio richiede un login uma autonomia in questo giro; da riverificare dal vivo alla prima occasione reale. Glitch reale trovato e corretto durante il test dal vivo del log scritture: creando un modello di not correttamente <code>propose_intune_write</code> (la proposta era reale) ma nella stessa risposta ha ANCHE sc eseguita correttamente... e' stato creato con ID <GUID inventato>" - l'esecuzione vera, avvenuta solo c diverso. La "REGOLA CRITICA ASSOLUTA anti-fabbricazione" gi� nel system prompt (aggiunta il 19/08

		di un modello che viola un'istruzione testuale. Doppia difesa aggiunta: (1) system prompt rinforzato con l'avvertimento esplicito di non citare MAI un ID che nessuno strumento ha ancora restituito; (2) rete di quando una scrittura risulta genuinamente proposta (<code>PendingWrite</code> valorizzato, quindi nulla e' stat con un pattern di frasi tipiche di completamento gia' avvenuto ("risulta eseguita", "e' stato creato", "hc ben visibile invece di editare la prosa libera del modello (troppo fragile da riscrivere in modo affidabile vivo: riprodotto il bug originale (ID inventato confermato diverso da quello reale), poi ripetuto lo stes: gia' impedito la ricorrenza al primo tentativo successivo ("Non e' ancora stato creato: la proposta e' in resta comunque attiva per i casi futuri in cui il solo prompt non bastasse.
Maratona di bug-hunt (16 ore, richiesta esplicita) - 11 bug reali trovati e corretti su Intune/Exchange/GUI/report/console, logo animato in header, corretto un errore criptico ricorrente nei log (v0.9.62, 23/08/2026)	implementato	Prima meta' della maratona di stress-test/bug-hunt di almeno 16 ore richiesta esplicitamente dall'uter rispettivamente l'area Exchange/Teams/Compliance/isolamento, l'area Intune, l'area SharePoint/Entra/allowlist/descrizione strumento AI in <code>Invoke-M365OpsAgentTools.ps1</code> con le vere firme <code>param()</code> linguaggio naturale (incluso lo stile "utente confuso" richiesto). Trovati e corretti: Intune - 7 disallineamenti reali tra la descrizione mostrata all'AI e i parametri veri (<code>Invoke-M365OpsConfigurationPolicy</code> documentava <code>DisplayName</code> invece del vero <code>Name</code>, e <code>Technology</code> parametro mandatory mancante in un processo server senza sessione interattiva rischia di restare bloccare compilare (stessa classe di bug gia' vista per WAM/device-code); <code>New-M365OpsDeviceScript</code> documentava quattro cmdlet (<code>New-M365OpsAutopilotDeploymentProfile</code>, <code>New-M365OpsAppProtectionPolicy</code>, <code>New-M365OpsAdminTemplate</code>) documentavano un parametro <code>ExtraParams</code> che semplicemente non funzionava in Exchange); <code>Get-M365OpsConfigurationSettingDefinitions</code> documentava <code>SearchText</code> / <code>Top</code> prima del fix, avrebbe fatto fallire silenziosamente l'esecuzione DOPO che l'utente aveva gia' confermato <code>\$cmdlet @params</code>, senza traduzione parametri). Aggiunta anche una nota esplicita nella descrizione l'azione Graph "assign" SOSTITUISCE sempre l'intera collezione di assegnazioni dell'app invece di aggi (che infatti accettano <code>TargetGroupIds</code> al plurale) - una seconda chiamata su un gruppo diverso car Exchange - fallimento totale riportato per un successo parziale: <code>New-M365OpsDistributionGroup</code> gruppo con successo ma poi, se UN SOLO membro iniziale non era valido, l'eccezione di <code>Add-DistributionGroupMember</code> (che era corretta in un giro precedente) si propagava non gestita fino al chiamante, che riportava "la scrittura non riuscita" sul tenant. Ora ogni membro viene aggiunto con il proprio try/catch: il gruppo conta sempre come creato, ma esplicitamente nel risultato invece di far sparire l'intera operazione. Fuga di dati tra tenant: <code>\$script:LastReportPath</code> (l'ultimo report generato, usato da <code>Propose-ExchangeConnect-M365Ops</code> ad un cambio tenant, a differenza di ogni altro stato per-tenant (Intune/Exchange) allegato a un'email inviata dopo essere passati al Tenant B nello stesso processo server, senza rigenerare). GUI - sicurezza sulla conferma "si" (il piu' serio): il controllo era <code>'^(si ok conferma procedi va</code> normalissimo pronome impersonale italiano ("si puo' fare anche per l'account di backup?"), quindi una domanda sospesa (es. un reset MFA proposto) veniva interpretata come CONFERMA ed eseguita alla cieca. Corretto (da punteggiatura finale) sia una parola di conferma nota, non solo la prima - riprodotto dal vivo il bug ora NON esegue piu' nulla e richiede una conferma esplicita separata. GUI - altri 3 bug reali: (1) il catalogo comandi locale intercettava "reseta l'mfa di X" (contiene "mfa di X" e "reset richiesto, ignorando in silenzio il verbo - corretto aggiungendo verbi di reset/rimozione alle Definizioni "panoramica gruppo X, elenca anche i membri" falliva con "nessun gruppo trovato" perche' la cattura si fermava al primo segnale di frase successiva (virgola o "e" + verbo di continuazione); (3) una domanda di reset passato) sia "gruppo di test" veniva interpretata come una richiesta composta di ripacchettizzazione + dal trigger di pacchettizzazione e aggiungendo una guardia anti-domanda-di-supporto condivisa. Console/log - errore criptico ricorrente: "Errore interno: Exception calling 'Write' with '3' argument(s). Length bytes size specified." comparsa nei log segnalata dal vivo dall'utente - causa reale: un client di durante l'invio del corpo della risposta HTTP, per cui la scrittura principale non aveva una protezione per un fix dell'18/08/2026) - ora isolata in un try/catch dedicato, classificata correttamente come nota info allarmante. Logo animato (richiesto dall'utente durante la maratona): SVG inline di una rete neurale (2-3-2 nodi) dashoffset sulle sinapsi, pulsazione scaglionata sui nodi), a sinistra del titolo nell'header - nessun asset <code>accent-strong</code> quindi coerenti automaticamente col tema chiaro/scuro, <code>prefers-reduced-motion</code> Verificato dal vivo: sintassi su tutti i 382 file + reimport pulito del modulo dopo ogni gruppo di modi GUI/CommandCatalog tramite un'istanza reale del server (creazione/eliminazione di due modelli di nc seguita dalla verifica che il fix lo blocchi, riproduzione dal vivo del bug MFA-status-invece-di-reset seg fuga di report tra tenant sono stati verificati per lettura diretta dei <code>param()</code> reali (non solo dal report disciplina "mai fidarsi ciecamente di un report, verificare sempre" gia' in uso in questo progetto. Non e' un popup interattivo del worker isolato Delegato (v0.9.61) e la semantica "added vs. updated" di <code>Set-M365Ops</code> ma non confermata (nota aggiunta alla descrizione strumento invece di una correzione non verificata)
Maratona di bug-hunt (16 ore, seconda parte) - 9 bug reali in piu' nel catalogo comandi locale di chat (Gui\CommandCatalog.ps1), 1 sistemico su 6 voci (v0.9.63, 23/08/2026)	implementato	Un secondo agente di audit, dedicato stavolta esclusivamente a <code>Gui\CommandCatalog.ps1</code> (dopo il primo), ha trovato altri 9 problemi reali con lo stesso schema: un trigger locale (a costo zero, bypassa l'AI) da quello che quella voce sa gestire, rispondendo con un dato plausibile ma sbagliato invece di deviare. ListNonCompliant/UserOverview/MfaStatus/GroupOverview/CompliancePatterns/ListDevices): LETTERALI "e poi"/"e anche"/"quindi" - "stato mfa di X, poi disattivagli l'account" (virgola invece di "e" e silenzio. Aggiunte 'poi'/'dopo' isolate a tutte e sei. Altri 8 fix mirati: ExportCompliancePatterns non aveva (phishing/attacchi/naming) - aggiunte DeferWords dedicate (impossibile richiedere "conform" nel trigger contiene mai). SharedMailboxPermissions (sola lettura) non aveva alcun controllo sul verbo - "aggiungi risposta con l'elenco permessi ESISTENTI invece di proporre la scrittura richiesta (stesso schema del bug

		ExportMailboxUsageReport/ExportForwardingReport avevano un'alternativa di trigger senza alcun ver cosa faccio?" veniva risposto con un export invece che con la risposta alla domanda). ExportAllMailbox sono state migrate, quali mancano?" faceva scattare comunque l'export completo). ExportDevices usa ExportCompliancePatterns era già stata corretta a '{0,30}' il 18/08/2026, mai propagato qui) - "un repc dispositivi compromessi" faceva scattare un export dispositivi grezzo. Help usava la stringa letterale 'b "Teams si blocca" (presente) non veniva riconosciuto come un problema reale da passare all'AI. Verifica mini-harness PowerShell che riproduce esattamente la logica DeferWords/Triggers di Handle-ChatM esempio REALI riportati dall'agente, confermando che ognuno ora devia correttamente all'AI; il caso "s anche end-to-end su un'istanza reale del server (prima del fix sarebbe stato gestito SOLO dal catalogo richiesta passa correttamente all'AI e risponde a entrambe le parti).
Maratona di bug-hunt (16 ore, terza parte) - fuga di dati tra tenant su nomi profilo simili, crash che perdeva un Excel già generato, log scritture incompleto, packaging .msi rotto, glitch AI su risultato vuoto (v0.9.64, 23/08/2026)	implementato	Ripresa della maratona dopo una pausa forzata dal limite di utilizzo (4 ore, come previsto dall'utente s audit paralleli su aree non ancora coperte (packaging Win32/Autopilot/Installer Insight; generazione r trovato e diagnosticato dal vivo durante il test manuale. Fuga di dati tra tenant (il più serio, riprodotto dal vivo): storico chat e Knowledge Base sono salva replace '[^w\.-]','_-' , ripetuto identico in 7 funzioni diverse - questa regex NON e' iniettiva ("N Test", si riducono allo STESSO nome file, riprodotto dal vivo con pwsh). Due profili tenant diversi con n condividere lo stesso file di storico/KB - conversazioni e documenti di un tenant visibili nell'altro. Set profilo il cui nome collide con uno già esistente, con un messaggio che spiega perché. Crash che perdeva un Excel già generato (riprodotto dal vivo): Export-M365OpsDataReport cc Object) PRIMA del try/catch che protegge solo la scrittura PDF vera e propria - un campo con un og comune) fa lanciare a Group-Object "Cannot compare... because the object does not implement IC funzione perdendo anche l'xlsx già scritto con successo poco sopra. Corretto alla radice in ConvertT nidificato singolo in "chiave=valore; ...", non solo gli array come prima - questo elimina anche la diver cella vuota, e PDF, che stampava il dump grezzo "@(X=1)", per lo stesso identico dato) E con un try/ca PDF, così' un problema impreveduto futuro degrada sempre a un avviso, mai a un'eccezione che si porta confermato che con il fix il PDF si genera senza avvisi. Log scritture incompleto: il blocco catch generico di Execute-PendingAction era l'UNICO punto ma diversi case (LokkaWrite quando Graph rifiuta con un errore restituito come risultato normale inve schema, PackageApp/AssignApp con una dipendenza/gruppo non risolvibile) uscivano con un return log nonostante fossero fallimenti di scrittura reali e quotidiani. Nuovo Write-M365OpsWriteFailure Execute-PendingAction , così' resta corretto anche per un case futuro che dimenticasse di loggare Packaging .msi rotto: un file .msi caricato produceva un InstallCommandLine tipo "App.msi /S" - Aggiunto un ramo dedicato (in entrambi i punti, flusso singolo e a coda) che costruisce msiexec /i ProductCode, non deducibile in sicurezza dai soli metadati) resta esplicitamente da completare a man Guardia mancante su AssignApp standalone: il ramo singolo di assegnazione app usava lo stesso tr senza la guardia anti-domanda-di-supporto già applicata lì - "Ho assegnato l'app come available a tu proponeva una nuova assegnazione invece di rispondere alla domanda. Ora esclusa allo stesso modo. Glitch AI su risultato vuoto (trovato e diagnosticato dal vivo): alla domanda "mostrami i modelli a CORRETTAMENTE Get-M365OpsAdminTemplates (0 risultati, tenant vuoto) ma la risposta finale e' s testo di uno scambio precedente completamente diverso, confermato dai log del server (tool chiamat nuova regola al prompt che impone di dichiarare esplicitamente un risultato vuoto per la domanda Af precedente. Riprodotto lo scenario esatto dopo il fix: risposta corretta ("Non risultano modelli ammini
Maratona di bug-hunt (16 ore, quarta parte) - retry via isolamento reattivo anche sui dati Teams (non solo sulla connessione), colonna Teams sempre vuota nelle retention policy (v0.9.65, 23/08/2026)	implementato	Un agente di audit dedicato a Purview/retention e alle policy Teams (aree toccate solo di striscio dagli test manuale della sessione live del server (non riproducibile con un semplice script pwsh isolato - sen coerente con la natura non deterministica già documentata di questo conflitto in sezione 6.6). Isolamento reattivo incompleto sui dati Teams: Connect-M365OpsTeams ha già una logica di rec proprio tentativo di connessione - se il conflitto .NET scatta invece sulle chiamate dirette ai cmdlet Cs* M365OpsTeamsExternalAccessConfig / Get-M365OpsTeamsPolicies (possibile anche con la con assembly sono già caricati nel processo in quel momento, non solo dal momento della connessione), locale. Riprodotto dal vivo: "quali policy di sicurezza per l'accesso esterno ha configurato teams" ha m mostrasse che l'isolamento reattivo si era GIÀ attivato con successo pochi istanti prima (per la connes modo di ritentare dopo che l'isolamento era pronto. Corretto in entrambe le funzioni: il corpo vero e p (primo tentativo + un singolo retry esplicito dopo aver riattivato l'isolamento, se l'eccezione corrispon riverifica dal vivo di questo stesso fix, e' emerso un SECONDO sintomo diverso della stessa classe di cc Microsoft Teams", invece del solito "Could not load file or assembly") - non riproducibile a comando c un processo server già attivo da ore), quindi non ancora coperto dai pattern riconosciuti da Get-M36 nota invece di un fix alla cieca sul solo sospetto. Colonna Teams sempre vuota nelle retention policy: Get-M365OpsRetentionCompliancePolic che NON esiste sull'oggetto restituito da Get-RetentionCompliancePolicy - Teams espone due p TeamsChatLocation , messaggi canale e chat vivono in store diversi). Riferirsi a una proprietà inesis PowerShell, restituisce silenziosamente \$null - la colonna era quindi SEMPRE vuota per ogni policy, affermato con sicurezza che nessuna policy copre Teams. Corretto separando le due proprietà reali.
Maratona di bug-hunt (16 ore, quinta parte) - autoreview: 4 regressioni introdotte dai fix precedenti di questa stessa maratona, trovate da un	implementato	Diverso dagli altri giri di questa maratona: qui non si cercano bug nuovi, si rilegge git diff dell'int SOLO a trovare errori introdotti DAI fix stessi - la disciplina "misura due volte" applicata alla propria str regressioni reali, tutte confermate dal vivo prima di correggerle:

secondo agente prima che diventassero un problema reale (v0.9.66, 23/08/2026)		1) 'poi'/'dopo' matchava dentro "poiché": il fix del giro precedente (CommandCatalog.ps1) aggiunge parola INIZIALE (stesso stile già in uso per le radici come 'perch'/'consigl', dove serve apposta) - ma 'p senza un confine anche alla fine matchavano anche dentro "poiché"/"dopodomani", deviando all'AI m 'poi\b' / 'dopo\b' - verificato dal vivo che "poiché" ora non matcha più mentre "X, poi Y" contin 2) "si, esegui pure" non veniva più riconosciuta come conferma: il fix di sicurezza sulla conferma " SOLO alla fine dell'intera frase, non parola per parola - una virgola interna dopo "si" (frase di conferma non risultava mai uguale a "si"/"si" nell'elenco delle parole di conferma, bloccando una conferma reale singola parola dopo lo split - verificato dal vivo end-to-end: creato un gruppo di test rispondendo esa ripulito. 3) Fallimenti con eccezione loggati DUE VOLTE nel log scritture: Write-M365OpsWriteFailureI precedente, per coprire i fallimenti senza eccezione) veniva chiamato ANCHE per i fallimenti CON ecce PendingAction registrava già da solo, indipendentemente. Rimossa la chiamata di log ridondante d punti che invocano Execute-PendingAction , e' ora l'UNICO punto che scrive nel log scritture, per 4) Il fix per gli Hashtable non veniva mai raggiunto: il ramo dedicato aggiunto in ConvertTo-M36 Hashtable/dizionario nidificato non veniva MAI eseguito, perché un Hashtable implementa anche IEnumerable generico per gli array (che lo appiattiva in modo sbagliato) - codice scritto ma di fatto irraggiungibile. generico - verificato dal vivo che ora produce "chiave=valore" invece del testo grezzo del tipo .NET.
CLI Microsoft 365 sempre pronto su ogni tenant + login delegato realmente funzionante (era rotto) + pulsanti di login coerenti (v0.9.67, 23/08/2026)	corretto	Tre problemi reali segnalati dal vivo dall'utente durante un test su un nuovo tenant Delegato ("AlePira 1) CLI-Microsoft365 richiedeva configurazione manuale per ogni singolo tenant: a differenza di L andava aggiunto a mano da "Altri server MCP" per ciascun profilo - un nuovo tenant falliva con "Serve tenant". L'utente si aspettava lo stesso comportamento "pronto all'uso" già vero per Lokka. Corretto i ora un secondo default incorporato, sempre presente per ogni tenant (nuovo o già esistente), con lo tenant ne personalizza comando/argomenti. 2) Login delegato di CLI Microsoft 365 completamente rotto (URGENTE, riprodotto dal vivo): cli Delegato non si apriva alcun browser - il comando restava bloccato in un prompt interattivo invisibile criptico contenente testo binario/emoji corrotto. Causa reale, verificata con m365 login --help su appId e' ora "cruciale" per QUALUNQUE tipo di login, incluso --authType browser - non più ver riverificato su questa versione) che il flusso browser usasse un client pubblico integrato senza bisogno un prompt di testo in attesa di input da tastiera - impossibile da soddisfare dato che stidio e' il canale (stessa classe di bug già vista e corretta il 21/08/2026 su Teams con -UseDeviceAuthentication). passando esplicitamente lo stesso client pubblico multi-tenant Microsoft "Microsoft Graph Command stesso progetto - un'app di primissima parte presente di default in ogni tenant, nessuna App Registr di "Delegato = nessuna App Registration". Verificato dal vivo con un probe diretto (m365 login --a comando procede subito a "To sign in, use the web browser that just has been opened", nessun prom 3) Pulsante di login "sparisce" per ~15 minuti se il popup viene chiuso senza completare il login Exchange, chiudere il popup di login senza cliccare ne' "Accetta" ne' "Annulla" dentro la pagina Micros "authorization_pending" agli occhi del server Microsoft, quindi il poll lato client continuava a rischedu lasciando il pulsante disabilitato fino alla scadenza naturale del codice. Corretto aggiungendo un seco uso per cancel-edit-profile-btn , non un testo scambiato sul pulsante principale - coerenza di s polling lato client, il device code lato server scade comunque da solo e Start-M365OpsDelegatedLc ancora pendente al riavvio. Per Teams/SharePoint/Purview/Intune/CLI-Microsoft365 delegato, che inve thread singolo resta davvero occupato finché non finisce, non solo il pulsante un "Annulla" lato client pronto mentre il server resta comunque inchiodato per tutti. Aggiunto invece un avviso onesto che co server e' probabilmente bloccato per davvero e l'unico modo per sbloccarlo e' riavviarlo dal tab Manu
L'AI chiedeva un secondo login inutile quando CLI Microsoft 365 poteva già rispondere (v0.9.68, 23/08/2026)	corretto	Bug reale segnalato dal vivo su un tenant Delegato ("AlePiras") con SOLO CLI Microsoft 365 connesso domanda "quanti utenti ha il tenant?", graph_api_call e' fallito con "Nessuna sessione delegata al Graph generica, distinta e indipendente dal login di CLI Microsoft 365), non di dato non trovato - e la ignorando che CLI Microsoft 365 era già connesso e avrebbe potuto rispondere subito con un coman system prompt (Invoke-M365OpsAgentTools.ps1) diceva già di usare cli_m365.* come fallback q quella regola copriva solo il caso "endpoint senza quel dato", non il caso "connessione stessa non disp i tenant Delegati: se graph_api_call fallisce per sessione/connessione mancante (non un 403/404 sul di: 365 (Entra/Outlook/Planner/OneDrive/Purview-ricerca/Teams-app-chat), il modello deve provarlo PRII indipendenti, uno non implica l'altro.
CLI Microsoft 365 trattato come sostituto proattivo di Graph, esteso a scritture e alla scoperta app SharePoint (v0.9.69, 23/08/2026)	corretto	Estensione richiesta esplicitamente dal vivo dopo v0.9.68 ("senza graph attivo bisogna usare il cli per c tool idempotente a graph laddove ha possibilità"), su due fronti. 1) Preferenza proattiva, non solo reattiva, e anche sulle scritture: v0.9.68 correggeva solo graph_ fallito. Riprodotto dal vivo un caso analogo su propose_graph_write (scrittura, es. "crea un accour nessun fallback su CLI365. Corretto in Invoke-M365OpsAgentTools.ps1 calcolando PRIMA di costr generica risulta attiva - se NO e CLI Microsoft 365 e' configurato, il modello viene istruito esplicitamen cli_m365_run_command / propose_cli_m365_command come PRIMO tentativo (non solo dopo ur di CLI365 (Entra/Outlook/Planner/OneDrive/Purview-ricerca/Teams-app-chat) - evita un tentativo Gra sia in scrittura. 2) SharePoint bloccato dallo stesso problema per un motivo nascosto: segnalato dal vivo, "perche - un tenant Delegato con SOLO CLI Microsoft 365 connesso non riusciva a connettere SharePoint, con reale: Sync-M365OpsSharePointAppRegistration (che cerca l'App Registration dedicata "M365O

		Graph per quella ricerca - un dettaglio implementativo interno, non un vero bisogno di SharePoint ste disponibile. Corretto aggiungendo lo stesso principio: se la sessione Graph generica non e' attiva e CL <code>app get --name</code> (sintassi verificata dal vivo con <code>--help</code> , non indovinata, incluso il comportament arriva l'errore originale che chiede il login Graph. Una volta trovato il Client ID si salva come sempre n tenant.
Messaggi onesti durante l'attesa dei login interattivi (v0.9.70, 23/08/2026, maratona di debug)	corretto	Segnalato dal vivo: "da che clicco connetti a che apre il browser passa un botto, a volte si apre a volte delle versioni moduli installate) su tutti e 5 i pulsanti bloccanti (SharePoint/Teams/Purview/Intune/CLIE (le versioni effettivamente pinnate su questo progetto, ExchangeOnlineManagement 3.4.0 e Microsoft Microsoft ha reso WAM il broker predefinito - verificato dal vivo con <code>Get-Module -ListAvailable</code> testo di stato ("apertura browser in corso...") appariva identico sia che il server stesse davvero aprendo un modulo (PnP.PowerShell/ExchangeOnlineManagement, fino a decine di secondi la prima volta, <code>Co M365Ops*SafeVersion</code>) - lo stesso identico testo statico per due fasi molto diverse rendeva imposs lento. Corretto: messaggio iniziale che avvisa esplicitamente della possibile attesa per l'installazione m "server bloccato" alzata da 25s a 45s (25s scattava spesso durante un'installazione ancora legittimame le cause plausibili (modulo in installazione, oppure finestra chiusa/aperta dietro le altre) invece di pres
4 bug reali trovati e corretti dall'agente di audit sulle cmdlet Exchange (maratona di debug, v0.9.71, 23/08/2026)	corretto	Primo lotto dell'agente dedicato al test dal vivo (non solo lettura di codice) delle cmdlet Get-M365Op invocazioni reali, 4 bug trovati e corretti, tutti riverificati dal vivo dopo il fix. 1-3) Colonna data creazione sempre vuota (<code>Get-M365OpsAllMailboxes</code> , <code>Get-M365OpsSharedM Get-EXOMailbox</code> restituisce di default un set RIDOTTO di proprieta' - <code>WhenMailboxCreated</code> non <code>Object</code> produceva sempre <code>\$null</code> anche se il dato esiste davvero, senza nessun errore che lo segna <code>WhenMailboxCreated</code> esplicitamente a ciascuna chiamata - verificato dal vivo: tutte le 25+7+7 mailb 4) Il fallback del report mail flow falliva sempre, proprio quando serviva (<code>Get-M365OpsMailFlo</code> non e' disponibile, il codice ripiegava su <code>Get-MessageTrace -PageSize</code> (V1) - cmdlet in dismission errore TERMINANTE invece di un semplice avviso, quindi il fallback (pensato per essere piu' affidabile passando a <code>Get-MessageTraceV2 -ResultSize</code> (stesso fix gia' applicato a <code>Get-M365OpsMessage</code> dal vivo: il fallback ora restituisce conteggi reali per stato (Failed/Expanded/Delivered). Confermati NON bug (dati mancanti legittimi sul tenant di test, non fallimenti silenziosi): retention pol AntiPhishRules/AutopilotDevices/UpdateRings/AppProtectionPolicies/ResourceMailboxes/AdminTemp piccolo, non per un bug di lettura.
Ciclo completo scrittura Intune verificato dal vivo con pulizia garantita + documentazione Settings corretta (maratona di debug, v0.9.72, 23/08/2026)	corretto	Secondo agente della maratona: ciclo crea→verifica→elimina→verifica su cmdlet di scrittura Intune rea Catalog reali, assegnazione policy) su "vnsys-test", oggetti nominati <code>ZZTEST-marathon-*</code> , pulizia fir test residui). Nessun bug funzionale nei cmdlet di scrittura stessi. Bug di documentazione trovato e corretto: <code>New-M365OpsConfigurationPolicy</code> dichiarava che i un contenitore vuoto, utile soprattutto con <code>-TemplateId</code>, dove il template stesso fornisce spesso dei de caso provato: sia senza <code>-TemplateId</code> sia con un template Endpoint Security reale ("Defender Updat "dCV2Policy.Settings : The Settings field is required." . Corretta la documentazione (r l'informazione era sbagliata) per riflettere che va sempre passata almeno una voce Settings reale. Lecture Entra aggiuntive verificate senza problemi: conditional access policies, riepilogo stato conform
Autoreview della maratona: 1 regressione reale trovata e corretta nel fix CLI365/SharePoint di poco prima (v0.9.72, 23/08/2026)	corretto	Terzo agente della maratona, con lo stesso compito gia' collaudato nelle maratone precedenti: non ce commit v0.9.67-v0.9.71 di questa stessa maratona) cercando regressioni introdotte dai fix stessi. Trova <code>M365OpsSharePointAppRegistration.ps1</code> (il fix CLI365/SharePoint di poco prima, stessa release p risponde correttamente "l'app non esiste" (testo semplice, non JSON valido - <code>"Error: App with na codice lo trattava allo stesso modo di un errore CLI365 generico, cadendo nel tentativo Graph success specifico (nessuna sessione Graph, e' proprio per questo che si e' provato CLI365 per primo), risultand invece del corretto "NotFound" con il comando di registrazione gia' pronto - esattamente il caso PIU' d'essere stessa di questa funzione. Corretto riconoscendo il testo "not found" nella risposta CLI365 e r contro l'output reale della CLI installata su questa macchina.</code>
La GUI era illeggibile su mobile - nessun tag viewport (maratona di debug, v0.9.73, 23/08/2026)	corretto	Trovato testando dal vivo la GUI a larghezza mobile (375px, mai fatto prima in nessuna maratona prec <code><meta name="viewport"></code> - senza, un browser mobile reale ignora la larghezza fisica dello schermo (tipicamente 980px), poi rimpicciolisce tutto per farcelo stare: risultato, testo illeggibile senza zoom m (<code>window.visualViewport.width</code> restituiva 980 invece di 375). Aggiunto <code><meta name="viewport scale=1"></code> - verificato dal vivo che risolve, <code>visualViewport.width</code> ora corretto. Trovato un secon corretta, il gruppo di pulsanti dell'header (Upload/Stato permessi/Impostazioni tenant) non andava m piccolo overflow orizzontale (435px di contenuto su 375px di schermo, misurato via <code>getBoundingCL</code> dei pulsanti - verificato dal vivo che i pulsanti ora vanno a capo correttamente, zero overflow orizzont nessun impatto sul layout desktop (verificato ridimensionando avanti e indietro).
2 bug reali su Teams trovati dall'agente di audit lettura Teams/SharePoint (maratona di debug, v0.9.74, 23/08/2026)	corretto	Quarto agente della maratona: tutte le Get-M365Ops* di Teams e SharePoint/OneDrive invocate per d corretti, entrambi riverificati dal vivo. 1) Riga fantasma nei risultati Teams alla prima connessione (<code>Connect-M365OpsTeams.ps1</code>): <code>Co</code> (Account/Environment/Tenant/TenantId) quando riesce - non soppresso, quell'oggetto finiva sulla pipi chiamante (es. <code>Get-M365OpsTeamsList</code> restituiva 16 righe invece di 15, con una riga vuota in testa) (le successive fanno return anticipato), motivo per cui non era mai stato notato prima. Stesso identico propagati qui con <code> Out-Null</code> su entrambi i rami (Delegato e AppOnly). 2) Dati reali spariti silenziosamente dagli export CSV/tabella (<code>Get-M365OpsTeamsExternalAcce</code>

		ospiti, riunioni ospiti, chiamate ospiti) avevano ciascuna il proprio set di colonne invece di uno schema <code>M3650psTeamsPolicies</code> (che unisce le colonne apposta per questo). <code>ConvertTo-Csv / Export-Cs</code> oggetto per l'intera tabella - risultato verificato dal vivo: valori reali come <code>AllowUserChat=True</code> spa leggibili sull'oggetto stesso. Corretto unendo tutte le colonne su ogni riga, stesso schema del gemello Nota di documentazione aggiornata nello stesso giro: <code>Get-M3650psTeamsPolicies</code> dichiarava ancora permesso risulta invece concesso su questo tenant, verificato con 19 righe reali restituite. Aggiornata l
Bug reale su aggiornamento messaggi notifica localizzati + gap scoperto su rimozione siti SharePoint (maratona di debug, v0.9.75, 23/08/2026)	corretto	Quinto agente della maratona: ciclo completo scrittura su Exchange (gruppi distribuzione, mailbox cor SharePoint (crea sito, quota, condivisione, membri) su "vnsys-test". Bug reale trovato e corretto: <code>Set-M3650psNotificationTemplateMessage</code> - aggiornare un mes: refuso) falliva SEMPRE con 400 "Cannot Patch Locale Property": il body del PATCH includeva ancora <code>1</code> invece serve) - Graph lo rifiuta sempre perche' il locale fa gia' parte dell'ID della risorsa, immutabile via correttamente, solo l'aggiornamento di uno esistente era rotto. Corretto separando i due body - verifi confermando che l'aggiornamento riesce dopo il fix. Gap reale scoperto (non ancora colmato): il modulo non ha NESSUN wrapper <code>Remove-M3650psSh</code> usare <code>Remove-PnPtenantSite</code> nativo direttamente, che ha fallito ripetutamente (4 tentativi dell'age Graph DELETE, tutti con "The requested operation is not supported for site") - un vincolo della piattafo modulo. ⚠ Residuo reale lasciato sul tenant "vnsys-test": il sito di test <code>https://vnsysit.sharepoint.</code> rimosso, resta <code>Active</code> sul tenant. Va rimosso manualmente piu' avanti (probabile restrizione temp <code>Remove-PnPtenantSite -Url 'https://vnsysit.sharepoint.com/sites/ZZTEST-marathon</code> <code>SharePoint</code>.
Un "crash" segnalato dal vivo su Teams era in realta' un errore di rete client dopo un successo lato server + rete di sicurezza sul loop principale (v0.9.76, 23/08/2026)	corretto	Segnalazione dal vivo: "provando Teams delegato, dopo il login ho ricevuto Errore di comunicazione: Indagando i log reali: il server NON era crashato - il conflitto .NET noto (sezione 6.6) era scattato, l'iso SECONDO popup di login (nel processo worker separato), la connessione Teams era riuscita davvero. I (spesso lunga, specialmente con un secondo popup mai segnalato prima), la connessione TCP del bro - <code>fetch()</code> restituisce un generico "NetworkError", indistinguibile per l'utente da un fallimento vero, Corretto su due fronti: (1) tutti e 5 i pulsanti di connessione bloccanti (SharePoint/Teams/Purview/Intu ricontrollano subito lo stato REALE della connessione invece di lasciare l'utente con solo il messaggio (come in questo caso), lo si vede subito; (2) il popup di conferma e il messaggio di stato di Teams avvi SECONDO popup separato se scatta il conflitto noto, invece di lasciare l'utente a chiedersi perche' un Aggiunta anche una rete di sicurezza indipendente, non legata a questo episodio specifico: il ciclo prir <code>{while(...)} finally {...}</code> SENZA MAI un <code>catch</code> - qualunque eccezione sfuggita al gestore p lasciare traccia nei log. Aggiunto un <code>catch</code> esterno che logga l'eccezione fatale (tipo, messaggio, sta impedisce un crash genuino (es. un fault nativo non catturabile da .NET), ma garantisce che se succede indovinare da log parziali.
Domande in prima persona su un tenant Delegato ricevevano un rifiuto invece di una risposta (maratona di debug, ambiente Delegato, v0.9.77, 23/08/2026)	corretto	Primo test dal vivo sul tenant Delegato "AlePiras" dopo che l'utente ha completato tutti i login: "mostr determinarlo... serve interrogare Teams/Graph per l'utente autenticato" - un rifiuto onesto (nessuna in nessuna chiamata reale, quando la risposta era in realta' ottenibile subito. Causa: nessuna guida nel sy risolve automaticamente all'identita' di chi ha fatto il login delegato - il modello non sapeva di poterlo miei dispositivi" senza dover prima conoscere lo UPN dell'utente. Corretto aggiungendo una regola es una domanda in prima persona usa <code>graph_api_call</code> su <code>/me/joinedTeams /me/mailboxSetti</code> chiedere un nome utente; su un tenant AppOnly <code>/me</code> non ha senso (l'app non e' una persona), in qu domanda.
Conteggio MFA tenant-wide: il modello a volte rispondeva senza tentare nessuna chiamata (maratona di debug, ambiente Delegato, v0.9.78, 23/08/2026)	corretto	Secondo test dal vivo sul tenant Delegato "AlePiras": "quanti utenti hanno mfa non configurata?" ha ri posso stabilire... serve interrogare i metodi di autenticazione" - verificato nel log della Cronologia (Ma "completato" sono passati solo 3 secondi, **nessuna riga "Tool AI chiamato" nel mezzo** - il modello altre domande nella STESSA conversazione (es. "ci sono piani planner attivi?" poco dopo, che mostra c <code>/groups/{id}/planner/plans</code>) - quindi gli strumenti funzionano, il modello ha scelto specificamente di dedicato esiste per un CONTEGGIO aggregato su tutto il tenant (solo <code>get_user_mfa_status</code>, che r improvvisare l'endpoint Graph corretto (<code>/reports/authenticationMethods/userRegistrationD</code> intrinsecamente incoerente (aveva funzionato su un altro tenant in precedenza nella stessa maratona, (52 utenti, 35 senza MFA registrata, stesso conteggio esatto ottenuto in precedenza) - aggiunta la gui <code>get_user_mfa_status</code>, cosi' il modello non deve piu' indovinare l'endpoint ogni volta.
"Stato permessi" mostrava note di sviluppo interne all'utente finale (v0.9.79, 23/08/2026)	corretto	Segnalato dal vivo: il pulsante "Stato permessi" mostrava righe come "nota: Verificato dal vivo il 21/08 subito in Delegated" - testo di lavoro interno (date, metodologia di verifica, riferimenti a questo stess all'utente finale, giustamente giudicato fuorviante e senza senso in quel contesto. Corretto in <code>Get-M3 M3650psAppPermissionsCheck.ps1</code>: le date e la dicitura "verificato dal vivo" restano SOLO nei com sorgente), il campo <code>Note</code> mostrato all'utente ora dice solo cio' che gli serve sapere ("dedotti dal no verificate) - nessuna data, nessun riferimento allo sviluppo di questo progetto.
2 bug reali su assegnazione ruoli RBAC Intune trovati dall'agente di audit scritture (maratona di debug, v0.9.80, 23/08/2026)	corretto	Sesto agente della maratona, ciclo crea→verifica→elimina→verifica su ruoli custom Intune e relative as faceva POST su <code>/deviceManagement/roleDefinitions/{id}/roleAssignments</code>, la rotta docum backend reale rifiuta QUALSIASI POST li' con 400 "No OData route exists". Il tipo concreto usato davve (<code>deviceAndAppManagementRoleAssignment</code>) si crea invece sulla collezione di primo livello <code>/devi</code>

		esplicito e il ruolo collegato via <code>roleDefinition@odata.bind</code> - corretto e verificato dal vivo, propa <code>M365OpsRoleAssignment</code> (mai verificata prima perche' nessuna assegnazione era mai stata creata cc ruolo": non <code>scopeMembers</code> (nome fuorviante) ma <code>members</code> . Bug 2: con un solo gruppo in <code>-ScopeGroupIds</code> , il body inviato aveva <code>resourceScopes</code> come strir 'StartArray' node was expected". Causa: <code>if (\$ScopeGroupIds) { \$ScopeGroupIds } else { @()</code> attraverso l'output di uno statement if/else (comportamento noto della pipeline PowerShell) - corrett verificato confrontando l'output JSON prima/dopo.
Login Teams Delegato non piu' bloccante per il server + 3 bug reali trovati durante la verifica dal vivo del refactor (v0.9.81, 23/08/2026)	implementato	Richiesta esplicita dal vivo dopo l'episodio v0.9.76 ("rendi la questione multi thread cosi' che queste o velocizzare sti login"). Riscrivere l'intero server per essere davvero multi-thread e' stato scartato di pro condiviso critico per la sicurezza delle scritture - <code>\$pendingWrite</code> , <code>\$script:M365OpsContext</code> , i di: risolverebbe). Implementata invece l'alternativa mirata concordata: il login Teams Delegato ora passa ! separato (finora usato solo REATTIVAMENTE dopo un conflitto .NET reale) tramite due nuove funzioni <code>M365OpsIsolatedModuleConnectAsync*</code> - il processo server principale manda la richiesta di login e gia' in uso per il codice dispositivo Graph/Exchange, pulsante "Annulla" dedicato incluso) invece di res Costo accettato: un secondo popup di login sempre per Teams Delegato, anche quando nessun confli un server che non si blocca mai durante l'attesa. Durante la verifica dal vivo del refactor (necessaria perche' la logica di completamento connessione, g <code>Complete-M365OpsIsolatedModuleConnect</code> per essere riusata identica dal nuovo percorso asincr riverificati dal vivo: 1) <code>ConvertFrom-Json</code> su un oggetto JSON vuoto (<code>{ }</code>) restituisce un <code>PSCUs</code> <code>PJsonObject.Properties.Name</code> contiene UN elemento <code>\$null</code> invece di zero elementi - un quirk Po Quando il worker isolato non trovava nessun comando nuovo da proxare, quel <code>\$null</code> finiva come cl the array index evaluated to null." Corretto filtrando i nomi vuoti/nulli prima del ciclo. 2) Causa di que proxare con un diff prima/dopo su <code>Get-Command -CommandType Function, Cmdlet</code> (SENZA <code>-Mo</code> dinamicamente via implicit remoting dentro <code>Connect-ExchangeOnline</code> , mai catalogabili in anticipo <code>Team</code> ' -in (<code>Get-Command -CommandType Function, Cmdlet</code>). <code>Name</code> risulta gia' vero in un process qualunque <code>Import-Module</code> - PowerShell cataloga i nomi esportati da un modulo staticamente dichi importare davvero. Risultato pratico senza il fix: l'isolamento Teams "riusciva" (nessuna eccezione, log Teams successivo sarebbe fallito silenziosamente con "term not recognized", un fallimento molto piu' includendo anche i comandi il cui <code>ModuleName</code> corrisponde davvero al modulo appena connesso, in cmdlet dinamici non hanno mai quel <code>ModuleName</code>). Verificato dal vivo dopo il fix: isolamento Teams correttamente, <code>Get-Team</code> via proxy isolato restituisce le 15 righe reali del tenant. 3) Un terzo bug reale, trovato SOLO testando il nuovo percorso asincrono end-to-end (mai emerso su asincrono riuscito, una chiamata successiva a <code>Get-M365OpsTeamsList</code> falliva con "Errore MCP: The [...String...]]' is not supported for serialization... Keys must be strings." Causa: <code>Connect-M365OpsTeams</code> <code>Connect-MicrosoftTeams</code>) solo se <code>\$script:M365OpsTeamsConnected</code> e' gia' vero - un flag imp nuovo percorso asincrono (che puo' rendere l'isolamento attivo senza mai passarci). Risultato: <code>Get-M</code> <code>M365OpsTeams</code> , che provava a "connettersi" di nuovo chiamando <code>Connect-MicrosoftTeams</code> per n globale (installata per ogni comando del modulo appena connesso, <code>Connect-MicrosoftTeams</code> con un'esecuzione qualsiasi invece che ignorata: il worker eseguiva davvero un secondo <code>Connect-Micro</code>: (oggetto <code>Account/Environment/Tenant/TenantId</code>) via JSON - fallendo sulla serializzazione di un Dictior flag <code>\$script:M365OpsTeamsConnected</code> / <code>\$script:M365OpsExchangeConnected</code> direttamente in unico raggiunto da entrambi i percorsi, sincrono e asincrono, dopo un connect riuscito) invece che ne <code>M365OpsExchange</code> . Verificato dal vivo end-to-end: login asincrono completo su "vnsys-test", poi <code>Get</code> correttamente le 15 righe reali senza errori.
CLI Microsoft 365 verificato per la prima volta con comandi reali - trovato e corretto un falso "installazione fallita" su Windows PowerShell 5.1 (agente 9 della maratona, v0.9.82, 23/08/2026)	corretto	Nono agente della maratona: primo test dal vivo di CLI Microsoft 365 con comandi REALI (<code>entra</code> / <code>s</code> stesso percorso usato davvero dall'AI in chat, non la CLI <code>m365</code> grezza - area segnalata "mai toccata" nessun comando reale era mai stato eseguito ed osservato. Bug reale trovato e corretto: <code>Assert-M365OpsCliMicrosoft365Installed.ps1</code> cattura l'output <code>2>&1</code> - npm scrive normalissimi avvisi non fatali su stderr (es. "npm warn allow-scripts..." per una dip pacchetto). Sotto <code>\$ErrorActionPreference = 'Stop'</code> (impostato deliberatamente a livello di mo quegli avvisi a eccezione terminante non appena attraversa la pipeline, ben prima del controllo esplici riuscita per davvero (exit 0, pacchetto installato, verificato dal vivo) veniva segnalata come fallita, con l che PowerShell 7 (il runtime reale della GUI) non soffre di questo problema - resta comunque un bug PowerShell 5.1, stessa classe di gap "solo 5.1" gia' nota per la lettura file senza <code>-Encoding</code> . Corretto locale solo per la durata della chiamata npm (ripristinato in un blocco <code>finally</code>) - il controllo su <code>\$?L</code> successo/fallimento, come previsto in origine. Riprodotto e riverificato dal vivo due volte sotto <code>power</code> dopo. Risultato del test end-to-end: CLI Microsoft 365 e' genuinamente utilizzabile in chat oggi per Entra/f reale, log di audit sostanziali) e fallisce CORRETTAMENTE e onestamente su SharePoint (limite docume Planner/Outlook (permessi Graph realmente mancanti su questa App Registration - <code>Calendar</code> . <code>Reac</code> ripetizione del blocco di ~3 minuti segnalato in precedenza durante questo giro (ogni comando testat costo reale di ~2 minuti per l'installazione npm alla primissima connessione resta confermato, pagato
Script personalizzati (Scripts\Custom), testati dal vivo per la prima volta - 2 bug reali trovati e corretti (agente 10 della maratona, v0.9.83, 23/08/2026)	corretto	Decimo agente della maratona: area segnalata "mai stress-testata" - verificato dal vivo che il caricame test", e l'esposizione all'AI tramite <code>Invoke-M365OpsAgentTools.ps1</code> funzionano davvero end-to-er <code>M365OpsOneDriveSharingReport.ps1</code> , elenca link di condivisione OneDrive attivi) - 0 righe restituit silenzioso: il tenant di test semplicemente non ha link di condivisione attivi in questo momento, verific

		Bug 1: <code>Get-M365OpsCustomScriptCatalog.ps1</code> - la lista di esclusione dei parametri comuni non è aggiunto da PowerShell 7.4 in poi) - su questo ambiente (PS 7.6.5) il catalogo esposto all'AI mostrava del solo <code>Upn</code> reale per ogni script personalizzato, che sarebbe finito nello schema dei tool <code>custom_</code> Corretto aggiungendo <code>ProgressAction</code> all'esclusione - verificato dal vivo che il catalogo ora mostr Bug 2: <code>M365Ops.psm1</code>, il loader degli script personalizzati - il blocco <code>catch</code> usava <code>\$_Name</code> per <code>\$_</code> e l'ErrorRecord dell'eccezione (senza proprietà <code>Name</code>), non più l'oggetto file della pipeline - l'a personalizzato " non caricato...", inutile con più di uno script presente. Riprodotto dal vivo con un file catturando l'oggetto file in una variabile dedicata PRIMA del <code>try</code> - verificato dal vivo che l'avviso ora percorso di validazione del tag Mode: uno script di test senza <code>.NOTES Mode:</code> si carica come funzione nel catalogo, non raggiunge mai il livello tool dell'AI.
Chiuso l'item prioritario sul fallback CLI365 per le scritture Entra - guard deterministico, non traduzione automatica (agente 11 della maratona, v0.9.84, 23/08/2026)	implementato	Undicesimo agente della maratona: risolto l'item lasciato deliberatamente aperto dal v0.9.68/69 ("una lettura... va indagato con calma, non patchato al volo"). Confronto dal vivo su "vnsys-test": creazione+ diretto (<code>New-M365OpsGroup</code>) sia via <code>m365 entra group add</code> equivalente. Evidenza reale raccolta, non ipotetica: <code>m365 entra group add</code> RIFIUTA il comando senza un <code>--</code> obbligatorio senza equivalente nel body Graph (che usa <code>mailEnabled / securityEnabled</code> booleani) ha <code>mailNickname</code> generato automaticamente (non derivato dal nome, a differenza del percorso Graph normalizzazione e default REALMENTE diversi tra i due percorsi. Confermato quindi il rischio già sospeso in un comando CLI365 e' stata implementata, ne' lo sarà senza una revisione dedicata per ogni singolo Implementato invece un guard deterministico in <code>Invoke-M365OpsAgentTools.ps1</code>: quando la sessione CLI Microsoft 365 e' configurato, una <code>propose_graph_write</code> su un percorso Entra ID (<code>/users</code> , <code>/organization</code> , <code>/domains</code>) - destinata a fallire SEMPRE in esecuzione con "Nessuna sessione delegata anche solo una richiesta di conferma, rimandando l'AI a costruire da sola un <code>propose_cli_m365_command</code> di ogni altra scrittura, nessun bypass). Ambito deliberatamente ristretto a Entra ID - Exchange/Intune/<code>propose_graph_write</code> . Verificato dal vivo attraverso il percorso reale del guard (stato Delegato+sessione-mancante simulato) ha davvero chiamato <code>propose_cli_m365_command</code> invece di <code>propose_graph_write</code> - guard con comando per analogia col body Graph invece di verificarne la sintassi reale, lo stesso rischio di normalizzazione un errore chiaro, nessun oggetto creato. Confermato che il guard resta sicuro per costruzione (mai un sintassi CLI365 - lo stesso rischio pre-esistente di qualunque <code>propose_cli_m365_command</code> scritto negli oggetti <code>ZZTEST-marathon-fallback*</code> creati durante l'indagine confermata con query indipendenti
Autoreview del blocco v0.9.81-v0.9.84: gemello del fix installazione CLI365 mai propagato (agente 12 della maratona, v0.9.85, 24/08/2026)	corretto	Dodicesimo agente della maratona, con lo stesso compito già collaudato: non cercare bug nuovi, rilegga asincrono, fix installazione CLI365 su PS 5.1, 2 bug Scripts(Custom, guard scritture Entra→CLI365) cercato tentativo era stato interrotto dal limite di utilizzo della sessione senza produrre modifiche; questo e' il Trovato e corretto: non una vera regressione, ma un gemello del bug v0.9.82 mai propagato. <code>Public</code> prerequisiti "in anticipo", separata dalla connessione "al primo uso" già corretta in v0.9.82) faceva la <code>Script(microsoft365 2>&1</code> senza il fix - sotto Windows PowerShell 5.1, un'installazione riuscita per davvero packages", exit 0) veniva comunque segnalata come fallita per lo stesso motivo già diagnosticato (un terminante da <code>\$ErrorActionPreference = 'Stop'</code> , prima del controllo su <code>\$LASTEXITCODE</code>). Corretto <code>'Continue'</code> , ripristinato in <code>finally</code>) - riverificato dal vivo con la stessa installazione reale, ora senza Altre aree controllate, nessuna regressione trovata: il guard scritture Entra non può mai scattare su <code>\$graphDelegatedSessionActive</code> e' sempre vero lì, per costruzione) ne' confligge con la preferenza Teams asincrono riverificato end-to-end dal vivo (start→poll→Connected, flag corretto, 561 cmdlet per Scripts(Custom) riprodotto dal vivo con un nuovo file dalla sintassi rotta, entrambi confermati.
La nota finale in chat ora dice sempre quale motore IA ha risposto, non solo quali dati sono stati consultati (v0.9.86, 24/08/2026)	implementato	Richiesto esplicitamente dall'utente durante un'analisi sui token inviati alle API IA: la nota "Fonte dati: 26/08/2026) compariva SOLO quando l'AI aveva davvero interrogato dati reali del tenant tramite uno puramente conversazionale (es. una spiegazione generica non legata al tenant) non aveva nessuna nota mai dirlo esplicitamente. Aggiunta un'etichetta del motore IA usato (<code>Claude Sonnet 4.5 (Anthropic)</code> sempre presente in fondo a OGNI risposta di <code>Invoke-M365OpsAgentTools</code> (i tre punti di uscita dell conversazionale, risposta interrotta per limite di passaggi) - quando sono stati consultati dati reali la nota "Risposta generata da IA (Y) senza consultare dati del tenant." Verificato dal vivo entrambe le varianti : <code>graph_api_call</code> reale e una puramente esplicativa).
La nota sorgente estesa a OGNI interazione in chat, non solo al loop AI con tool-calling (v0.9.87, 24/08/2026)	implementato	Seguito diretto del v0.9.86, segnalato dal vivo dall'utente: "dammi i fattori mfa di X" non mostrava nessuna del catalogo locale (<code>Gui/CommandCatalog.ps1</code> , voce <code>MfaStatus</code> , <code>RequiresAI=\$false</code>) , un per <code>M365OpsAgentTools</code> (dove vive la nota dal v0.9.86) - i comandi del catalogo (circa 20 voci: stato MFA utente/gruppo, ecc.) non ci sono mai passati vicino. Estratta la mappa nomi motore IA in una funzione esportata) invece di duplicarla, poi applicata la stessa nota a TRE punti in <code>Gui/Server.ps1</code> finora solo comando locale "NomeVoce", nessuna IA usata." per le ~19 voci <code>RequiresAI=\$false</code> , "Fonte: come per le voci <code>RequiresAI=\$true</code> (es. <code>CompliancePatterns</code> , che chiama davvero <code>Get-M365OpsCompliancePatterns</code> ora riportano comunque "nessuna IA usata (fallito prima di completare)"; (2) il percorso di diagnosi/co <code>M365OpsErrorTriage / Invoke-M365OpsActionRecovery</code>), che interPELLA davvero l'IA ma non lo fa <code>M365OpsAgentTools</code> stessa lancia un'eccezione e si ripiega su <code>Invoke-M365OpsAgent</code> (IA senza stessa identica domanda ("stato mfa di X") che prima non mostrava nulla ora chiude con "_Fonte: comando l voce <code>CompliancePatterns</code> (<code>RequiresAI=\$true</code>) con la sua analisi IA reale.
Autoreview del v0.9.87: la nota d'errore diceva sempre "nessuna IA usata", anche per le voci che usano davvero l'IA (agente 14 della maratona, v0.9.88,	corretto	Quattordicesimo agente della maratona, su richiesta esplicita dell'utente subito dopo il v0.9.87 ("doipio modifiche del commit appena spedito cercando regressioni introdotte dai fix stessi.

24/08/2026)		Trovato e corretto: il ramo di successo del dispatch catalogo distingue correttamente "nessuna IA usata" dal blocco <code>catch</code> aggiunto dallo stesso commit no, diceva SEMPRE "nessuna IA usata (fallito prima di (<code>CompliancePatterns</code> / <code>ExportCompliancePatterns</code>) dove l'IA e' esattamente cio' che puo' aver contraddicendo lo scopo stesso della trasparenza appena introdotta. Riprodotto dal vivo forzando un <code>M365OpsCompliancePatterns</code> : il messaggio d'errore reale diceva "nessuna IA usata" nonostante la <code>catch</code> sullo stesso <code>\$entry.RequiresAI</code> del ramo di successo - riverificato dal vivo sia il caso AI (c di controllo non-AI (invariato, ancora corretto). Altre aree controllate, nessun problema trovato: tutti i ~19 Formatter del catalogo restituiscono se al punto di concatenazione, quindi <code>+=</code> fa sempre una concatenazione di stringhe pulita; nessuna intersezione nessuna collisione con parser lato GUI (la chat renderizza il testo come nodo di testo puro, <code>body.text</code> <code>Get-M365OpsAiProviderLabel</code> confermata raggiungibile da una sessione reale di <code>Gui/Server.ps1</code> testate (<code>ListDevices</code> , <code>ExportDevices</code> con allegato, <code>GroupOverview</code> , piu' i due casi sintetici di
Cache-hit sulle chiamate IA da 12,9% a ~99% - elenco strumenti reso identico su ogni round (v0.9.89, 24/08/2026)	implementato	Seguito diretto di un'indagine sui token spediti alle API IA, richiesta esplicitamente dall'utente dopo <code>M365OpsAgentTools.ps1</code> escludeva 6 strumenti (<code>list_devices</code> , <code>list_noncompliant_devices</code> , <code>get_user_overview</code> , <code>get_group_overview</code> , <code>get_user_mfa_status</code>) SOLO dal primo round di primo tentativo (fix storico contro un comportamento reale osservato in passato). Questo rendeva per i round successivi - e sia il prompt caching di Anthropic (<code>cache_control</code> , mai attivato in questo progetto attivo, nessuna configurazione richiesta) funzionano solo se il prefisso della richiesta e' IDENTICO byte strumenti diverso rompe il confronto proprio nel punto piu' pesante del payload (~17-25.000 token secondi con un test dal vivo PRIMA del fix: solo 12,9% di cache-hit sul secondo round di una conversazione Az quasi identico a pochi secondi di distanza. Corretto: l'elenco strumenti e' ora IDENTICO su ogni round (nessuna esclusione per round). La preferenza comportamento che l'esclusione doveva garantire - e' stata spostata in una frase esplicita nel prompt mai il prefisso condiviso), che elenca per nome i 6 strumenti da evitare al primo tentativo invece della Verificato dal vivo su "vnsys-test": cache-hit sul round 1 passato da 12,9% (prima) a ~99% (dopo, con preferenza per <code>graph_api_call</code>, verificata con 6 scenari dal vivo diversi pensati apposta per tentare il round precedenza ("panoramica utente", "dispositivi non conformi", "dettaglio MFA", "panoramica gruppo", i deliberatamente le parole-trigger, un riepilogo ambiguo tra utente/gruppo) - in tutti e sei i casi il modo scorciatoie, cache-hit stabile nel range 90-99% su tutta la conversazione. Verifica indipendente aggiunta nessun problema trovato, nessun riferimento residuo al vecchio meccanismo, comportamento confermato resta pero' solo per Azure, che ha il caching automatico - Claude non ne beneficia finche' non viene attivato in questo giro).
Nuova funzionalita': analisi intestazioni email/NDR con spiegazione IA, tab "Intestazioni email" (v0.9.90, 25/08/2026)	nuovo	Richiesta esplicitamente dall'utente durante un caso reale di troubleshooting su un NDR ("550 5.4.1 Recipient's email address is blocked by FortiAnalyzer): replica in locale l'analisi che offre https://mha.azurewebsites.net (Message Header Analyzer) servizio esterno per il parsing. Nuova tab "Intestazioni email" nel pannello impostazioni: si incolla il testo del messaggio in Outlook, OPPURE un blocco di diagnostica NDR/message-trace di Exchange come quello automaticamente) e si ottiene subito: riepilogo (oggetto/mittente/destinatario/data), tabella degli hop, risultato SPF/DKIM/DMARC, decodifica del rapporto antispam (X-Forefront-Antispam-Report, con etichette codice SMTP/esteso scomposto con una spiegazione mirata (es. il 5.4.1 viene segnalato esplicitamente Edge Blocking sul lato destinatario, non un blocco di sicurezza/relay come il nome farebbe pensare - v memoria). Un secondo pulsante separato, "Chiedi all'IA una spiegazione in chiaro", manda il testo in chiaro (stesso principio di trasparenza del v0.9.86/87 - la nota "Elaborata da IA" compare sempre) per una diagnosi di troubleshooting - mai eseguito automaticamente, un click esplicito e separato dal solo parsing locale Nuovo file <code>Private/Get-M365OpsMessageHeaderAnalysis.ps1</code> (parser, nessuna dipendenza esterna, righe per RFC 5322 sezione 2.2.3), due nuove route in <code>Gui/Server.ps1</code> (<code>/api/analyze-headers</code> spiegazione IA). Verificato dal vivo end-to-end, incluso un bug reale trovato e corretto durante il dall'utente (non un caso costruito a mano) restituiva i codici SMTP/esteso vuoti nonostante il testo del messaggio di valore arrivava spezzato su piu' righe (l'ID di tracciamento tra parentesi quadre va spesso a capo da solo l'opzione Singleline; corretto normalizzando gli spazi/a-capo prima di estrarre i codici. Test finale riuscito in pochi secondi, verificato identico all'output di riferimento di Microsoft Message Header Analyzer) sia sul fronte conversazione, diagnosi IA corretta e coerente con l'analisi locale), sia tramite chiamata diretta alle nuci
Analizzatore intestazioni: pulsante combinato copia+apri MHA, riferimento ai dati grezzi, 3 bug reali corretti da 2 agenti dedicati (v0.9.91, 25/08/2026)	corretto	Seguito diretto del v0.9.90, su richiesta esplicita dell'utente ("fai partire gli agenti per controllare tutto" i dati grezzi analizzati, e un unico pulsante che copia i dati e apre mha.azurewebsites.net senza un tast Verificato PRIMA di costruire: se lo strumento Microsoft supportasse un modo per pre-compilarsi via documentazione ne' evidenza nel repository ufficiale lo conferma. Costruire un finto "auto-fill" sarebbe implementato invece un pulsante che copia negli appunti E apre il tool in un click solo - un solo Ctrl+V separato. Due agenti dedicati, in parallelo: uno di regression-review sul commit v0.9.90, uno di stress-test del (Postfix/Sendmail, Cisco IronPort/ESA, Gmail/Google Workspace). 2 bug reali nel parser (<code>Private/Get-M365OpsMessageHeaderAnalysis.ps1</code>), entrambi su variant versione: (1) date con un commento di fuso orario in coda tipo Gmail (<code>Mon, 24 Aug 2026 01:23:31</code>) calcolo del ritardo per quell'hop - corretto rimuovendo il commento prima di interpretare la data; (2) i (facoltativo per RFC 5322, omissso da gateway come Cisco IronPort/ESA, es. <code>24 Aug 2026 10:00:00</code>) buttando via anche mittente/destinatario gia' corretti - corretto rendendo il giorno della settimana fac il riconoscimento del mittente per includere hop di consegna locale Sendmail/Postfix (<code>from user@lo</code>

		esplicito. 1 bug reale nella GUI (<code>Gui/index.html</code>), trovato dall'agente di regression-review: la nuova sezione consultabile) veniva calcolata ma accodata SOLO nel ramo di analisi NDR - il ramo piu' comune (intest senza nessun errore in console. Corretto accodandola in entrambi i rami. Tutti e tre i fix riverificati dal dopo ogni correzione.
Conteggio token spediti/ricevuti nella nota "Elaborata da IA" della spiegazione intestazioni (v0.9.92, 25/08/2026)	implementato	Richiesta esplicitamente dall'utente per l'interazione IA dell'analizzatore intestazioni ("in questo caso d richiesta esplicita di verificare PRIMA che non introducesse rallentamenti percepibili. Verifica fatta: il cc gia' presente in ogni risposta HTTP di Claude/Azure OpenAI - nessuna chiamata aggiuntiva, nessuna li risposta prima di implementare, non assunto. Aggiunto un parametro opzionale <code>-ReturnUsage</code> a <code>Invoke-M365OpsAgent</code> (Public): se presente r <code>CachedTokens</code> } invece della sola stringa, con il default a <code>\$false</code> per non toccare il contratto della una stringa semplice (<code>Invoke-M365OpsErrorTriage</code> , <code>Get-M365OpsCompliancePatterns</code> , ecc. - ' identico). Usato per ora solo nella route <code>/api/analyze-headers-ai</code> : la nota finale diventa "Elabora cache)" - il dettaglio sulla cache compare solo se effettivamente maggiore di zero. Verificato dal vivo: 631 ricevuti", nessuna regressione sugli altri chiamanti (testato <code>CompliancePatterns</code> del catalogo, continua a restituire una stringa semplice come prima).
Risposte lunghe in chat non piu' tagliate a meta' frase dopo un ricaricamento pagina + toggle "Mostra tutto" (v0.9.93, 25/08/2026)	corretto	Bug reale segnalato dal vivo dall'utente con uno screenshot: un'analisi dettagliata (pattern di non conf a meta' frase con "[...troncato]" nella chat - non nella risposta live appena ricevuta, ma dopo un ricaric Causa reale: <code>Add-M365OpsChatHistoryTurn.ps1</code> troncava il testo a 3000 caratteri PRIMA di salvar opposti: (1) ridisegnare la chat per l'utente al caricamento pagina (<code>GET /api/chat/history</code>), dove conversazionale all'IA nei turni successivi, dove un limite di lunghezza e' invece una scelta corretta per report lungo per intero ad ogni chiamata. Applicare lo stesso taglio a entrambi gli scopi cancellava pe umana, l'unico caso dove non ha senso. Corretto separando i due scopi: <code>Add-M365OpsChatHistoryTurn.ps1</code> ora salva SEMPRE il testo cc caratteri si applica invece SOLO nel punto in cui lo storico diventa contesto per l'IA (<code>Invoke-M365Ops \$maxHistoryCharsPerTurn</code>), mai piu' al salvataggio. Verificato dal vivo: una risposta di 5091 caratteri turno successivo che usa quello storico come contesto continua a funzionare correttamente (riassunto Aggiunto anche (richiesto esplicitamente, "aggiungere un click per mostrare tutto su richiesta"): un se perdita di contenuto) - un messaggio oltre 1500 caratteri appare collassato con un'ellissi e un pulsante espandere/ricomprimere il testo completo, sempre disponibile per intero, mai scartato. Verificato dal v lunghi, espansione e ricomprensione funzionanti in entrambe le direzioni.
Autoreview del v0.9.93: un taglio a 3000/1500 caratteri poteva spezzare un'emoji, corrompendo silenziosamente il testo (agente 17 della maratona, v0.9.94, 25/08/2026)	corretto	Diciassettesimo agente della maratona, su richiesta esplicita dell'utente ("a valle rifai tutte le verifiche c commit v0.9.93 cercando regressioni introdotte dai fix stessi. Trovato e corretto: sia <code>.Substring()</code> lato server (PowerShell) sia <code>.slice()</code> lato client (JavaScript) caratteri veri - se il taglio cade esattamente a meta' di una coppia surrogata (es. un'emoji, U+10000 e alto orfano. Riprodotto dal vivo lato server: quella stringa passa <code>ConvertTo-Json</code> senza errori, ma q per la chiamata HTTP reale verso l'IA, il surrogato orfano non e' rappresentabile e viene silenziosamen riceverebbe testo di contesto corrotto, senza nessun errore visibile. Lato client lo stesso problema e' sc raggiungibile espandendo con "Mostra tutto"), ma corretto per coerenza. Corretto in entrambi i punti surrogato alto, cosi' la coppia resta sempre intera. Altre aree controllate, nessun problema trovato: nessun altro punto del codice (10 file con riferime dipendeva implicitamente dal vecchio taglio a 3000 caratteri; il tetto <code>\$maxPairs = 8</code> (ultimi 16 scan esatto al confine dei 3000 caratteri gestiti correttamente; il toggle "Mostra tutto" funziona identico sia storico al caricamento pagina. Verificato dal vivo end-to-end su "vnsys-test": risposta di 5270 caratteri "Mostra tutto"/"Mostra meno" testato nel browser reale in entrambe le direzioni dopo il fix.
Analizzatore intestazioni spostato dalle Impostazioni a un pulsante in primo piano nella barra strumenti + reindirizzamento se incollato per errore in chat + conteggio token esteso a TUTTA la chat, non solo l'analizzatore (v0.9.95, 25/08/2026)	corretto	Tre correzioni dal vivo richieste esplicitamente dall'utente dopo aver provato la funzionalita' reale nel p 1) Riposizionamento GUI: la tab "Intestazioni email" viveva dentro il pannello "🔍 Impostazioni tenan Tenant/Documentazione/Motore AI/MCP/Email/Manutenzione) - l'utente ha segnalato che non e' un estemporaneo e reattivo ("ho appena ricevuto un NDR, lo incollo subito"), non per la configurazione u pannello di primo livello proprio (<code>#headers-panel</code>), con un pulsante dedicato "🔍 Intestazioni ema (stesso pattern di apertura/chiusura gia' in uso per il pannello di caricamento file) - nessuna funzionali pulsante combinato copia+apri Microsoft Message Header Analyzer, tutto riverificato dal vivo nel bro 2) Reindirizzamento se incollato nel posto sbagliato: segnalato dal vivo che incollare delle intestazi pannello dedicato) le faceva finire silenziosamente all'IA generica - niente tabella hop, niente verifica S Header Analyzer, niente conteggio token, solo una risposta testuale generica su un blocco di testo che catalogo comandi locali (<code>Gui/CommandCatalog.ps1</code> , <code>PastedEmailHeadersRedirect</code> , controllata 5322 o un blocco NDR/message-trace incollati in chat (stessa euristica di autorilevamento gia' usata d : , o <code>{LED= }</code>) e reindirizza l'utente al pulsante dedicato invece di lasciar rispondere l'IA generica - nes catalogo locale. 3) Conteggio token esteso alla chat generale: il conteggio token (introdotto in v0.9.92) copriva solo chiarito di volerlo anche nella nota "Fonte dati"/"Elaborata da IA" di OGNI risposta della chat principal <code>M365OpsAgentTools.ps1</code> una somma <code>\$totalInputTokens / \$totalOutputTokens / \$totalCac</code> risposta puo' attraversare piu' round di andata/ritorno con l'IA, uno per ogni chiamata a uno strument

		e tre i punti di uscita della funzione (risposta con dati, risposta puramente conversazionale, risposta in per quello Azure OpenAI - stessa formula già collaudata in v0.9.92 ("N token inviati, M ricevuti", di cui Verificato dal vivo: una domanda senza strumenti ("ciao, come stai?") mostra "31254 token inviati, 21 domanda con strumenti ("quanti utenti ci sono nel tenant?") attraverso 3 round reali (30514+30738+3 finale riporta correttamente la SOMMA esatta (93927) e non l'ultimo round soltanto, cache inclusa (83 riposizionato testato end-to-end nel browser (apertura/chiusura, Analizza su un'intestazione reale con nota token). Reindirizzamento testato sia sul caso positivo (blocco intestazioni incollato in chat, nessun domanda normale che nomina "email" e un mittente non scatena mai il redirect).
Autoreview del v0.9.95: falso positivo del redirect intestazioni su una normale frase inglese (agente di regression-review dedicato, v0.9.96, 25/08/2026)	corretto	Rilette SOLO le modifiche del commit v0.9.95 (pannello intestazioni riposizionato, redirect catalogo, cc fix stessi - stesso compito già collaudato dagli agenti precedenti della maratona. Trovato e corretto: il trigger <code>received:\s*from\s</code> della nuova voce <code>PastedEmailHeadersRedir</code> ancorava il match all'inizio riga - una qualunque frase che contenesse quella sottostringa a meta' frase email incollata davvero. Riprodotto dal vivo con un messaggio di lavoro del tutto plausibile: <i>"Your pay to billing@contoso.com"</i> veniva intercettato e redirezionato al pannello intestazioni invece di ricevere l'intestazione <code>Received:</code> e' invece SEMPRE la prima cosa sulla propria riga nel testo grezzo incollato, trigger a inizio riga (<code>(?m)^\s*received:\s*from\s</code> , con spazi di indentazione opzionali per le high chiamate dirette a <code>/api/chat</code> : la stessa frase inglese ora riceve correttamente la risposta AI (nessun un vero blocco NDR (<code>RecipientStatus:</code>) continuano a scatenare il redirect come previsto - conferi l'occorrenza pre-fix (redirect errato) e quella post-fix (risposta corretta) sullo stesso identico messaggi Altre aree controllate, nessun problema trovato: nessun riferimento residuo alla vecchia posizione <code>Gui/index.html</code> ; <code>#headers-panel</code> e <code>#upload-panel</code> possono restare aperti contemporaneamente nessuna sovrapposizione, nessuna interferenza tra le classi <code>active</code> dei due pulsanti) - verificato via l'orizzontale; tutte le funzionalita' del pannello (Analizza, Pulisci, pulsante IA, pulsante combinato copia DOM. Sul conteggio token: <code>\$totalInputTokens += \$response.usage.input_tokens</code> e affini res assente/ <code>\$null</code> (l'accesso a proprieta' su <code>\$null</code> in PowerShell restituisce <code>\$null</code> , sommato come : PowerShell 5.1); l'accumulo su piu' round riverificato dal vivo confrontando i log per-round con il total ("quanti utenti ci sono nel tenant?": 25401+25624+27561=78586 token in ingresso e 0+24832+24832 quanto mostrato in chat - non l'ultimo round soltanto). Formati "Fonte dati: X. Elaborata da IA: Y (...)." tenant." confermati invariati nella struttura, solo con il conteggio token aggiunto in coda come da v0.9 Nota fuori ambito, non corretta in questo giro (pre-esistente, non introdotta da v0.9.95): durante il <code>headers-ai</code> (route introdotta in v0.9.90, invariata da questo commit) rimasta bloccata oltre 3 minuti server e' sincrono a thread singolo - quella singola chiamata bloccata ha reso l'intero server (compresi finche' il processo non e' stato terminato e riavviato manualmente. Non riprodotto in modo sistematico rete/provider IA in questo ambiente sandbox), ma il rischio architetturale e' reale e indipendente dall'a puo' bloccare l'intero server per tutti gli utenti. Segnalato per un giro dedicato separato, non affrontat
Una chiamata IA appesa non blocca piu' l'intero server a tempo indeterminato - timeout esplicito su tutte le chiamate a Claude/Azure OpenAI (v0.9.97, 25/08/2026)	corretto	Seguito diretto del rischio architetturale segnalato (non corretto) dall'agente di regression-review del subito ("correggi certo"). Causa reale: il server GUI (<code>Gui/Server.ps1</code>) usa un <code>System.Net.HttpListener</code> con un unico ciclo <code>\$listener.GetContext(); ... }</code> - sincrono, a thread singolo, una richiesta alla volta. Nessuna de <code>Invoke-M365OpsAgent.ps1</code> , 3 in <code>Invoke-M365OpsAgentTools.ps1</code> , contando anche i tentativi c <code>TimeoutSec</code> - senza, <code>Invoke-RestMethod</code> non applica NESSUN limite proprio (il default e' un'attes (rete lenta, provider IA in difficolta') blocca l'intero server per QUALUNQUE altra richiesta nel frattemp dal vivo dall'agente precedente con una chiamata a <code>/api/analyze-headers-ai</code> rimasta bloccata o del processo. Corretto: aggiunto <code>-TimeoutSec 120</code> a tutte e 6 le chiamate. 120 secondi e' abbondante per una s contesto ampio (i round multipli del ciclo tool-calling in <code>Invoke-M365OpsAgentTools.ps1</code> hanno c altri round della stessa conversazione) mentre limita il tempo massimo di blocco del server a un valore server per essere davvero multi-thread e' stata scartata di proposito per lo stesso motivo già' docume bug di concorrenza su stato condiviso critico per la sicurezza delle scritture (<code>\$pendingWrite</code> , <code>\$scr</code> sarebbe peggiore del problema che risolverebbe - un timeout esplicito e' la correzione mirata e coere Un timeout scaduto viene lanciato come <code>System.Threading.Tasks.TaskCanceledException</code> (ve lento) - la stessa eccezione che i blocchi <code>try/catch</code> già' esistenti attorno a queste chiamate gestisc <code>Invoke-M365OpsAgentTools.ps1</code> ricade nel catch che il chiamante in <code>Gui/Server.ps1</code> intercett strumenti e poi su un messaggio d'errore pulito; dentro la route <code>/api/analyze-headers-ai</code> ricade di errore invece di restare in attesa. Verificato dal vivo: 387 file <code>.ps1</code> sintatticamente puliti dopo la modifica; una domanda normale in prima (nota "Elaborata da IA" col conteggio token invariata); il meccanismo di timeout confermato iso contro un endpoint che risponde dopo 5 secondi - eccezione <code>TaskCanceledException</code> lanciata e c nelle 6 chiamate reali.

16. Roadmap ed estensioni future

- **futuro** Ricerca sul registro di controllo unificato Microsoft Purview ([Search-UnifiedAuditLog](#) — chi ha fatto cosa, quando, su Exchange/SharePoint/Teams). Cmdlet reale e verificata, richiede licenza/ruolo Audit Log dedicato — non ancora costruita, un vero audit log oggi si copre solo parzialmente con i report di sezione 11 (litigation hold, inoltri, regole sospette)
- **futuro** Collegare all'AI i server MCP aggiuntivi oltre a Lokka, oggi solo salvati in configurazione
- **futuro** Ricerca automatica icone app da winget-pkgs per [New-M365OpsWin32App](#)
- **futuro** Valutare Azure SRE Agent come alleato — riguarda però l'affidabilità dell'infrastruttura Azure, non l'amministrazione M365, quindi resta fuori scope diretto
- **futuro** Server multi-threaded (oggi single-threaded per design semplice) — il login Exchange delegato non blocca più (sezione 10.6), ma resta l'unico modello possibile finché il server processa una richiesta alla volta. Finché resta così, un riavvio esterno "brutale" (kill del processo, non il pulsante/endpoint di riavvio) può ancora interrompere una richiesta bloccante in corso (sezione 12.2)
- **futuro** Storico conversazione più lungo/configurabile (oggi fisso a 8 scambi per tenant — sezione 23.2) o riassunto automatico dei turni più vecchi invece di scartarli
- **futuro** Task periodici autonomi (es. report settimanale via email senza tenere aperto M365Ops) tramite le "routine" di Azure AI Foundry Agent Service (trigger cron nativi lato Azure, GA da luglio 2026) - valutato il 17/08/2026: richiede un secondo sistema separato (Agent con propri strumenti verso Graph/Exchange, autenticazione Entra ID dedicata) e andrebbe usato SOLO per letture/report, mai per scritture non supervisionate (il gate di conferma umana vive nel processo locale di M365Ops, non si può delegare a un Agent schedulato). Deliberatamente NON si sposta la chat interattiva su un Agent: il ciclo Thread+Run è più complesso del semplice request/response usato oggi, e il catalogo strumenti/script già si ricostruisce ad ogni messaggio - non c'è nulla da guadagnare spostandolo su una configurazione Agent statica da tenere comunque aggiornata a mano.

Questo documento è pensato per essere aggiornato insieme al codice: quando una riga della tabella in sezione 15 cambia stato, aggiorna anche questa pagina e rigenera il PDF (vedi il commento in testa al file HTML sorgente).

17. Script personalizzati (Scripts\Custom)

Oltre alle cmdlet del modulo core (Public\) e ai 50+ strumenti Exchange (sezione 11), è possibile aggiungere script "home made" per casi d'uso specifici di un tenant/cliente — es. estrazione permessi OneDrive/SharePoint, un report ad hoc — e renderli usabili dall'AI in chat senza toccare nessun file del modulo.

17.1 Come diventano usabili

- 1. Copia Scripts\Custom_TEMPLATE.ps1 , rinominalo come la funzione che definisce (es. Get-M3650psOneDriveSharingReport.ps1 per una funzione con lo stesso nome).
- 2. Scrivi la funzione rispettando la convenzione sotto.
- 3. Riavvia il server (tab Manutenzione) — i file si caricano all'avvio del modulo.

Da quel momento lo script compare nel tab Manutenzione (elenco script rilevati, validi o no con motivo) e l'AI lo vede tra gli strumenti disponibili al messaggio successivo — nessuna registrazione manuale, nessuna modifica al manifest del modulo.

17.2 Convenzione obbligatoria

Requisito	Perché
Un file = una funzione, nome file = nome funzione	Stessa convenzione di Public\ — scoperta automatica senza registrazione manuale.
Blocco di help PowerShell (<# .SYNOPSIS ... #>)	È l'UNICA descrizione che l'AI riceve per decidere se/come usare lo script.
.NOTES con Mode: ReadOnly oppure Mode: Write	Obbligatorio. ReadOnly = eseguibile subito dall'AI; Write = solo proponibile, mai eseguito senza conferma umana esplicita — stesso principio non negoziabile di ogni altra scrittura. Senza questo tag lo script viene ignorato, mai esposto all'AI.
Nessun input interattivo (Read-Host , popup)	Deve essere una funzione pura: parametri in ingresso, oggetti in uscita.
Usa le funzioni di accesso dati già esistenti del modulo	Mai gestire token/credenziali proprie — usa Invoke-M3650psGraphRequest per Graph, Connect-M3650psExchange + cmdlet native per Exchange. Così lo script funziona automaticamente sia sui tenant AppOnly sia Delegated (sezione 10), senza mai assumere quale modalità è attiva.

17.3 Checklist per un buon .SYNOPSIS

L'AI vede *solo* questa riga per decidere se e come usare lo script — deve rispondere a:

- 1. **Quale dato specifico restituisce o azione specifica compie** — non "esegue una query su SharePoint", ma "elenca i link di condivisione pubblici attivi su OneDrive, con proprietario e data".
- 2. **Permessi/scope non standard richiesti**, se presenti — es. Sites.Read.All non è tra i permessi Graph di default (sezione 4.2): va detto qui, prima che l'operatore scopra un errore di permesso a metà.
- 3. **Per gli script Mode: Write**, quali effetti reali ha — l'AI usa questo testo, quasi alla lettera, per spiegare la proposta in chat prima della conferma.
- 4. **Forma dei dati restituiti**, se non ovvia.

17.4 Se uno script fallisce — diagnosi e correzione via AI, mai un crash

Un errore in uno script personalizzato non blocca il resto dell'app. Sia in lettura (custom_script_query) sia in scrittura confermata (CustomWrite), il fallimento passa dallo stesso motore di diagnosi usato per le altre scritture del modulo (Invoke-M3650psErrorTriage , sezione 14): l'errore, il contesto e il codice sorgente vengono analizzati, classificati (transitorio / serve una decisione tua / possibile bug) e, se c'è una correzione precisa da proporre, questa segue lo stesso flusso proponi → conferma → applica di ogni altra scrittura — mai una riscrittura automatica senza conferma esplicita.

Le correzioni proposte devono restare compatibili con **entrambe** le modalità di autenticazione: l'AI viene istruita esplicitamente a non introdurre gestione di token/credenziali proprie nel fix, per lo stesso motivo del punto 17.2 sull'accesso ai dati. Testato dal vivo: uno script di esempio (report link di condivisione OneDrive) ha incontrato un 404 reale su un utente senza OneDrive provisionato, ed è stato classificato correttamente come "serve una decisione tua" (verificare licenza/provisioning) — non un bug nello script.

17.5 File non caricati

Qualunque file il cui nome inizia con _ (come _TEMPLATE.ps1) viene ignorato dallo scanner — utile per esempi o bozze non ancora pronte.

17.6 L'AI può proporre la creazione di uno script nuovo

Dal 17/08/2026, quando un compito è chiaramente qualcosa che tornerà utile di nuovo (un report ricorrente, un'estrazione specifica di questo tenant) e nessuno strumento esistente lo copre, l'AI può scrivere da sola il codice completo di un nuovo script e proporlo — strumento propose_new_custom_script . Non è la prima risorsa: il system prompt le impone di provare prima graph_api_call / exo_query (con lookup_ms_docs se serve un parametro nativo non standard), e di non usarlo mai come scorciatoia per una singola domanda occasionale.

Non viene mai salvato senza conferma esplicita, e la conferma mostra il codice per intero — non un riassunto — esattamente come richiesto: creare uno script nuovo aggiunge una capacità permanente all'app, non è "solo" una scrittura su un dato del tenant, quindi la conferma lo dice esplicitamente in cima al messaggio, prima ancora del codice. Per gli script proposti in modalità Write , ogni loro esecuzione futura richiederà comunque una conferma separata — la conferma sulla creazione non "sblocca" scritture non supervisionate, si limita a far esistere lo strumento.

Validazioni automatiche prima ancora di mostrare la proposta (se una fallisce, l'AI riceve il motivo esatto e può correggere nello stesso turno, invece di proporre qualcosa che verrebbe comunque scartato dopo): il codice deve essere sintatticamente valido PowerShell (analizzato con il parser reale, non un controllo superficiale), deve definire esattamente una funzione il cui

nome coincide con il nome file proposto, deve avere un blocco `.SYNOPSIS`, e il tag `Mode`: nel codice deve corrispondere esattamente al parametro dichiarato. Il nome deve seguire la stessa convenzione Verbo-M365OpsNome di ogni cmdlet del modulo, e non viene mai sovrascritto silenziosamente uno script già esistente con lo stesso nome.

Il codice viene inoltre scansionato per una lista di comandi potenzialmente distruttivi (`Remove-Item`, `Invoke-Expression`, `Stop-Computer`, ecc.) — se presenti, non bloccano la proposta ma vengono segnalati con un'ATTENZIONE ben visibile in cima alla conferma, cosa da leggere con più attenzione prima di dire "sì" (in PowerShell puro non esiste un sandboxing reale, quindi questo resta un avviso mirato, non un blocco tecnico).

Alla conferma: il file viene scritto in `Scripts\Custom\` e il server si riavvia da solo (stessa infrastruttura sicura del pulsante "Riavvia"/ `POST /api/restart`, sezione 12.2 — agisce solo dopo aver già inviato la conferma, non può mai interrompere altro) per caricarlo. Da quel momento è uno strumento vero, scoperto automaticamente come ogni altro script in questa cartella (sezione 17.1).

Perché la revisione umana del codice resta comunque indispensabile. Testato dal vivo: la prima proposta reale generata dall'AI chiamava `Invoke-M365OpsGraphRequest -Uri $uri` — il parametro reale si chiama `-Path`, non `-Uri`. Un errore che il parser PowerShell non può rilevare (sintatticamente valido, fallisce solo in esecuzione) ma che un umano che legge il codice nota subito. Proposta rifiutata prima del salvataggio, e la descrizione dello strumento è stata rafforzata con la firma esatta e verificata di `Invoke-M365OpsGraphRequest / Connect-M365OpsExchange` per ridurre (non eliminare) questa classe di errore — motivo in più per cui la conferma mostra sempre il codice completo, mai un riassunto.

17.7 Sicurezza email — antispam, threat policy, allow/block list, quarantena, hunting

Aggiunto il 17/08/2026 su richiesta esplicita: gestione della sicurezza email (Exchange Online Protection / Defender for Office 365), oltre al message trace già esistente.

Cosa	Come
Criteri anti-spam, anti-phishing, Safe Links/Safe Attachments	lettura via <code>exo_query</code> (<code>Get-M365OpsAntiSpamPolicies</code> , <code>Get-M365OpsAntiPhishPolicies</code> , <code>Get-M365OpsThreatPolicies</code>) — nessun permesso Graph nuovo, usano la stessa connessione certificato-based di Exchange Online (sezione 5)
Tenant Allow/Block List (mittenti, URL, hash file, IP)	lettura con <code>Get-M365OpsTenantAllowBlockList</code> , scrittura proposta (conferma sempre richiesta) con <code>New-/Remove-M365OpsTenantAllowBlockListEntry</code>
Quarantena, incluso il rilascio	lettura con <code>Get-M365OpsQuarantineMessages</code> (default ultimi 7 giorni), rilascio proposto con <code>Release-M365OpsQuarantineMessage</code> (a tutti i destinatari originali, opzionalmente con il mittente aggiunto alla Allow List), eliminazione definitiva proposta con <code>Remove-M365OpsQuarantineMessage</code> — entrambe le scritture passano dalla stessa conferma esplicita di ogni altra azione del modulo
Message trace oltre i 10 giorni (es. "ultimi 30 giorni")	<code>Get-M365OpsMessageTrace</code> incatena da sola più query da 10 giorni (limite del servizio) e unisce i risultati — non serve più spezzare la richiesta a mano. Il risultato si ferma comunque a 1000 righe con un avviso esplicito se superate: una query lunga SENZA filtro mittente/destinatario su un tenant con molto traffico può generare un volume enorme (bug reale osservato: 3000 righe, ~330mila token, ha superato da sola il limite di contesto del modello e fatto fallire l'intera risposta) — specificare sempre a chi si riferisce la richiesta riduce il volume di un ordine di grandezza.
"Explorer"/Threat Hunting di security.microsoft.com	non è un'unica area: alert e incidenti già rilevati da Defender si leggono con <code>graph_api_call</code> su <code>/security/alerts_v2</code> e <code>/security/incidents</code> (permessi <code>SecurityAlert.Read.All</code> / <code>SecurityIncident.Read.All</code> , sezione 4.2); la ricerca libera con query KQL nei dati grezzi (l'uso più vicino a "Explorer" in senso stretto) usa il nuovo strumento <code>security_hunting_query</code> — richiede <code>ThreatHunting.Read.All</code> E la licenza Microsoft Defender for Endpoint Plan 2: senza quella licenza Graph risponde 403 anche a permesso concesso, e l'AI è istruita a spiegarlo come limite di licenza invece di ritentare query diverse sperando che funzionino.

Tutte le nuove cmdlet di lettura/scrittura Exchange qui sopra usano la stessa `Connect-M365OpsExchange` già esistente (certificato per AppOnly, sezione 5) — verificato dal vivo il 17/08/2026 che il modulo Exchange Online v3 espone anche i cmdlet un tempo esclusivi della Security & Compliance PowerShell (quarantena, Tenant Allow/Block List) sotto la stessa connessione, senza bisogno di una sessione separata.

17.8 Invio report via email dalla conversazione libera

`Send-M365OpsReportEmail` esisteva già (invio via Graph `/sendMail`, permesso `Mail.Send`) ma era raggiungibile solo dal catalogo deterministico con una frase fissa ("invalio per email a..."). Una richiesta composita in linguaggio libero (es. "aggiorna il report e mandalo a X") arrivava alla conversazione con l'IA, che non aveva alcuno strumento per l'invio e lo segnalava correttamente invece di inventarsi una capacità inesistente — ma il gap era reale. Aggiunto il 17/08/2026 lo strumento `propose_send_report_email`: propone l'invio dell'ultimo report generato in sessione come allegato, con conferma esplicita sempre richiesta (stesso meccanismo di ogni altra scrittura — inviare un'email è un'azione visibile al destinatario). L'IA scrive anche un testo su misura nel corpo del messaggio (breve overview del contenuto reale del report, non un messaggio generico), con la stessa regola di `generate_report`: solo dati davvero recuperati in conversazione, mai inventati.

Bug reale corretto lo stesso giorno: il percorso del file allegato veniva riletto da `$script:LastReportPath` dentro `Server.ps1`, ma quella variabile — quando scritta da `Invoke-M365OpsAgentTools` — vive nello scope del modulo, non in quello di `Server.ps1` (che non fa parte del modulo, lo consuma soltanto): il valore letto da `Server.ps1` era quindi sempre `$null`, con un crash immediato ("Cannot bind argument to parameter 'Path'"). Stessa classe di bug di scope già vista altre volte in questo progetto — risolto passando il percorso esplicitamente nell'oggetto restituito invece di affidarsi a una variabile condivisa tra scope diversi. Anche il vecchio percorso deterministico ("invalio per email a...") è stato allineato: prima inviava l'email immediatamente, senza alcuna conferma — unica scrittura dell'app con questo comportamento — ora passa dalla stessa conferma di tutte le altre.

17.9 SharePoint e OneDrive

Aggiunto il 17/08/2026 su richiesta esplicita ("tutto quello che è utile"): elenco siti, condivisione esterna, storage, permessi, utilizzo OneDrive e account orfani. Architettura diversa dal resto del modulo — vale la pena capirla prima di fidarsi ciecamente dei dati.

Cmdlet	Cosa dà
Get-M3650psSharePointSites	ogni sito: storage usato/quota, <code>SharingCapability</code> (stato condivisione esterna), template, ultima modifica — un'unica chiamata copre sia il classico report "condivisione esterna" sia lo storage per sito
Get-M3650psSharePointSitePermissions	amministratori raccolta siti + membri Proprietari/Membri/Visitori di UN sito (richiede l'URL, mai indovinato — va preso dall'elenco siti)
Get-M3650psOneDriveUsageReport	ogni OneDrive personale: proprietario, storage usato/quota, ultima attività
Get-M3650psInactiveOneDriveAccounts	OneDrive il cui proprietario è disabilitato o eliminato da Entra ID — dati orfani, rischio compliance/storage, già filtrato

Connessione e permesso diversi dal resto del modulo. Exchange, Graph e le nuove cmdlet di sicurezza email usano tutte lo stesso certificato con permessi Microsoft Graph o nativi Exchange. SharePoint invece passa da `PnP.PowerShell` (`Connect-M3650psSharePoint`), che riusa lo STESSO certificato ma richiede un consenso SEPARATO sotto l'API "SharePoint" — vedi sezione 4.3. Finché quel permesso non è concesso, ogni cmdlet qui sopra fallisce con un errore "Unauthorized" chiaro (non un bug, non va ritentato).

Verificato dal vivo il 17/08/2026, dopo la concessione del permesso di sezione 4.3: tutte e 4 le cmdlet restituiscono dati reali e corretti — 44 siti con `SharingCapability` popolata e variata (Disabled/ExternalUserSharingOnly/ ExternalUserAndGuestSharing), 13 OneDrive personali, permessi reali su un sito di test, e `Get-M3650psInactiveOneDriveAccounts` ha correttamente individuato 2 OneDrive con proprietario eliminato da Entra ID (non un dato di prova - un caso reale trovato al primo giro). Il campo `Owner` è confermato un UPN pulito (es. `nome@dominio.it`), non un formato claims-encoded - la ricerca Entra ID di `Get-M3650psInactiveOneDriveAccounts` funziona quindi come previsto senza fallback necessari.

Bug reale corretto il 17/08/2026 su un secondo tenant (Fabrikam-Prod, Delegato): l'URL del centro di amministrazione veniva derivato assumendo che il TenantId del profilo fosse sempre nel formato `xxxxx.onmicrosoft.com` (vero per contoso-test) - ma un profilo può avere il TenantId salvato come GUID puro, producendo un hostname senza senso (es. `https://a1b2c3d4-...-admin.sharepoint.com` , non risolvibile via DNS) e quindi un errore ancora prima del permesso/autenticazione. Corretto risolvendo il dominio iniziale verificato reale via Microsoft Graph `/organization` quando il TenantId non è già nel formato dominio - funziona sia in AppOnly sia in Delegato, risultato cache-ato per non ripetere la chiamata ad ogni uso. Verificato con un test mirato (non dal vivo su Fabrikam-Prod, che è Delegato e richiede un login interattivo reale): la derivazione produce l'URL corretto sia nel caso GUID sia nel caso dominio, senza chiamate di rete superflue in quest'ultimo.

17.9.1 Scrittura SharePoint

Aggiunta il 18/08/2026, su richiesta esplicita dopo un batch di test con prompt reali che segnalava l'assenza di scrittura (creazione sito, quote, sharing) come gap più visibile. Stesso meccanismo di proposta/conferma di ogni altra scrittura del modulo (mai eseguita senza conferma esplicita in chat) — nessun percorso separato o meno sorvegliato.

Cmdlet	Cosa fa
New-M3650psSharePointSite	crea un Team site (con gruppo Microsoft 365 associato) o un Communication site
Set-M3650psSharePointSiteMember	aggiunge/rimuove un utente da Owners/Members/Visitors di un sito
Set-M3650psSharePointPermissionInheritance	interrompe (permessi unici) o ripristina (torna a ereditare) l'ereditarietà permessi di un sito
Set-M3650psSharePointSiteSharing	imposta la <code>SharingCapability</code> di UN sito (stesso valore letto da <code>Get-M3650psSharePointSites</code>)
Set-M3650psSharePointSiteQuota	imposta la quota di storage di un sito, in GB

Non ancora disponibile: gestione Site Collection/colonne/template (fuori perimetro di questo giro, era una richiesta a parte del catalogo di test) e federazione cross-tenant. Se richiesto, il tool lo dichiara esplicitamente invece di tentare scorciatoie via Graph.

Verificato dal vivo il 18/08/2026 sul tenant contoso-test tramite il flusso reale di chat (proposta → conferma → esecuzione), non chiamando le funzioni a mano: creato un Communication site di test (`New-M3650psSharePointSite`), confermato e poi ritrovato correttamente in un elenco siti riletto in un secondo turno separato (owner e `SharingCapability` corretti).

17.10 Come è governata la connessione (App-only vs Delegata)

La connessione SharePoint (`Connect-M3650psSharePoint`) ha una casella dedicata nel tab **Tenant** (sezione "Stato connessioni", non MCP/Connettori), con un pulsante "Connetti / Test connessione SharePoint" e uno stato in tempo reale.

Modalità	Comportamento
App-only	Connessione silenziosa col certificato già configurato per Exchange (nessun secret/certificato nuovo) - il pulsante serve solo per un test esplicito o per riconnettersi a un URL diverso, non per un login vero e proprio. Pienamente verificato con dati reali (sezione 17.9).
Delegata	Richiede un login interattivo una tantum, fatto da un terminale normale (non dal pulsante della GUI) - vedi il riquadro sotto per il perché e il comando esatto. Dopo quel login, il pulsante della GUI funziona silenziosamente riusando la cache locale, senza bloccare mai il server.

Storia reale di questa sezione: tre bug diversi in sequenza sullo stesso sintomo, lasciata per intero perché istruttiva su come NON bloccarsi su un problema che sembra sempre "lo stesso" ma non lo è.

1. **Client OAuth indovinato.** Un primo tentativo con un flusso a codice dispositivo *custom* (per non bloccare il server) usava un `client_id` "indovinato" a memoria (il presunto "PnP Management Shell", `31359c7f-bd7e-475c-86db-fdb8c937548e`) - fallito con `AADSTS700016 "Application ... was not found in the directory"` : quel client non è mai stato consentito nel tenant, e il device code flow non può creare un primo consenso da solo.
2. **Sostituito con `-Interactive`** (lascia scegliere a PnP il proprio client corretto) - fallito con `System.NotSupportedException: Specified method is not supported.`, senza dettaglio utile nell'eccezione. Ipotesi ragionevole ma SBAGLIATA: apartment threading (il processo server gira in MTA, i controlli COM per finestre embedded richiedono STA) - aggiunto `-STA` all'avvio del processo server, verificato con `Get-CimInstance Win32_Process` che il flag fosse davvero presente nel processo in esecuzione, e l'errore è rimasto **identico**: ipotesi scartata con prove, non per sospetto.
3. **Confronto con Intune (funzionante):** Intune usa anch'esso `-Interactive` per i tenant Delegati, ma con successo - il modulo `IntuneWin32App` implementa il proprio flusso PKCE con redirect su localhost scritto da zero (nessun controllo browser embedded), mentre l'implementazione interna di `-Interactive` in `PnP.PowerShell` è evidentemente meno adatta a un processo headless. Confermato con `Start-Job` che `Connect-PnPOnline -DeviceLogin` nativo (nessun client indovinato) NON lancia l'eccezione e resta correttamente in attesa - ma un processo server headless non ha una console interattiva a cui è agganciato, quindi il codice stampato da `-DeviceLogin` non è recuperabile in modo affidabile da lì.

Soluzione: `-PersistLogin` . Questo parametro di `Connect-PnPOnline` salva il token in una cache locale su disco, riutilizzabile tra sessioni PowerShell diverse e dopo un riavvio del PC - va fornito una sola volta, dopo di che ogni nuova connessione allo stesso tenant lo riusa automaticamente, **anche da un processo headless** come il server di questa app (perché a quel punto non deve più mostrare nessuna UI, si limita a leggere il token già presente). Il login iniziale va fatto UNA TANTUM da un terminale PowerShell 7 normale (non tramite il pulsante della GUI, dove `-Interactive` continua a fallire per il motivo del punto 2 sopra):

```
Import-Module PnP.PowerShell
Connect-PnPOnline -Url "https://<prefisso-tenant>-admin.sharepoint.com" -Interactive -PersistLogin
```

Dopo aver completato il login nel browser che si apre, il pulsante "Connetti / Test connessione SharePoint" nella GUI dovrebbe funzionare silenziosamente. Se in futuro cambi i permessi dell'App Registration usata per il login, la cache va invalidata esplicitamente con `Disconnect-PnPOnline -ClearPersistedLogin` prima di un nuovo login (il token in cache non si aggiorna da solo).

17.11 Microsoft Teams

Aggiunto il 17/08/2026 subito dopo SharePoint, stessa richiesta ("tutto quello che è utile"). Architettura ibrida: una parte funziona subito col certificato esistente, un'altra parte (le policy) richiede una TERZA API separata, ancora diversa da Graph e da SharePoint.

Cmdlet	Cosa dà	Permesso
Get-M3650psTeamsList	ogni Team: visibilità, archiviato, criteri di collaborazione (chi può creare canali/aggiungere app/menzionare tutti)	nessuno in più - stesso certificato di Exchange
Get-M3650psTeamsChannels	canali di UN team (richiede GroupId, mai indovinato - da Get-M3650psTeamsList)	nessuno in più
Get-M3650psTeamsMembers	membri di UN team con ruolo owner/member/guest	nessuno in più
Get-M3650psTeamsPolicies	criteri di riunione/chiamata/messaggistica in un'unica lista (registrazione consentita, chiamate private, modifica/eliminazione messaggi, ecc.)	Skype and Teams Tenant Admin API - sezione 4.4
Get-M3650psTeamsExternalAccessConfig	federazione con organizzazioni esterne + cosa possono fare gli ospiti (chat/riunioni/chiamate) - report di sicurezza classico	Skype and Teams Tenant Admin API - sezione 4.4

Verificato dal vivo il 17/08/2026: Connect-M3650psTeams (modulo MicrosoftTeams, stesso certificato di Exchange) funziona SUBITO per le prime tre cmdlet - 14 Team reali, canali e membri (incl. un membro guest reale) confermati con dati veri. Le due cmdlet di policy invece falliscono oggi con "Access Denied. Provide different credential or request access." - il permesso di sezione 4.4 non è ancora stato concesso, i loro campi restano quindi non verificati (proprietà documentate e stabili delle cmdlet native, non indovinate, ma da riconfermare al primo test reale come già fatto per SharePoint).

Stessa governance connessione di Exchange/SharePoint: casella dedicata nel tab **Tenant** (sezione "Stato connessioni", non MCP/Connettori - vedi sezione 10.7) con pulsante "Connetti / Test connessione Teams" e stato in tempo reale.

BUG STRUTTURALE trovato dal vivo il 21/08/2026 (non un'ipotesi teorica - successo due volte all'utente): -UseDeviceAuthentication lasciava l'utente bloccato per sempre. Quel parametro stampa il codice dispositivo con Write-Host sulla CONSOLE DEL PROCESSO SERVER, non nella GUI - invisibile e irrecuperabile nell'uso normale dell'app (il launcher avvia il server con finestra nascosta). L'utente vedeva solo "login in corso" bloccato per sempre, nessun popup ne' codice da nessuna parte - comportamento CORRETTO per un device-code flow (che non apre mai un browser da solo), ma il codice stesso non arrivava mai all'utente. A differenza di Exchange/Graph (sezioni 6.5/10, che hanno un vero flusso non bloccante start/poll con codice mostrato in GUI), il modulo MicrosoftTeams non espone cmdlet separati per "inizia" e "completa" un device code - Connect-MicrosoftTeams -UseDeviceAuthentication e' un'unica chiamata atomica bloccante, non costruibile come start/poll senza intercettare la sua console (fragile).

Corretto RIMUOVENDO -UseDeviceAuthentication : senza, il modulo usa il proprio flusso interattivo di default (popup browser reale, come già visto funzionare per SharePoint via Connect-PnPOnline -Interactive). Il motivo per cui -UseDeviceAuthentication era stato scelto in origine (lasciare al modulo la scelta del proprio client_id interno corretto, per evitare un client_id indovinato a memoria - stesso bug reale già visto su SharePoint, AADSTS700016, sezione 17.10) resta valido: e' una proprietà del modulo stesso, non del parametro specifico - il modulo sceglie il client corretto in ENTRAMBI i flussi. Verificato dal vivo dopo il fix: popup browser apparso correttamente, login completato senza bloccare il server.

17.11.1 Scrittura Teams

Aggiunta il 18/08/2026, stesso motivo/richiesta di 17.9.1 (SharePoint). Stesso meccanismo di proposta/conferma di ogni altra scrittura del modulo.

Cmdlet	Cosa fa	Percorso API
New-M3650psTeam	crea un Team (e il gruppo Microsoft 365 associato)	Graph - stesso certificato di Exchange, nessun permesso in più
Set-M3650psTeam	modifica nome/descrizione/visibilità/archiviazione di un Team esistente	Graph
Remove-M3650psTeam	rimuove un Team (irreversibile oltre il cestino Entra ID a 30 giorni) - stessa conferma esplicita di ogni scrittura distruttiva del modulo, nessun percorso separato	Graph
Grant-M3650psTeamsPolicy	assegna (o resetta al default, omettendo il nome) una policy meeting/messaging/app setup/app permission ESISTENTE a un utente - non crea/modifica il contenuto della policy	Cs* legacy - stesso permesso "Skype and Teams Tenant Admin API" di Get-M3650psTeamsPolicies (sezione 4.4)

Verificato dal vivo il 18/08/2026 sul tenant contoso-test tramite il flusso reale di chat: creato un Team di test privato (New-M3650psTeam), confermato e ritrovato correttamente in un elenco riletto in un secondo turno separato.

Bug reale trovato e corretto durante il test: Grant-CsTeamsMeetingPolicy (e gli altri Grant-CsTeams*Policy) trattano -PolicyName come parametro OBBLIGATORIO - ometterlo del tutto per resettare al default falliva con "missing mandatory parameters: PolicyName" invece di funzionare. Va passato esplicitamente -PolicyName \$null (comportamento nativo dei cmdlet Microsoft, non un'assunzione ragionevole) - corretto in Grant-M3650psTeamsPolicy.ps1 .

Limite noto, non ancora risolto: dopo la correzione sopra, Grant-M3650psTeamsPolicy fallisce comunque su contoso-test con un errore di serializzazione interno del modulo MicrosoftTeams ("Dictionary...Keys must be strings", dentro Microsoft.TeamsCmdlets.PowerShell.Connect) - stesso percorso legacy Cs* di Get-M3650psTeamsPolicies , che oggi fallisce con "Access Denied" per lo stesso permesso mancante (sezione 4.4). Il sintomo è diverso (un bug di serializzazione invece di un messaggio di permesso pulito) ma la causa più probabile è la stessa: finché il permesso "Skype and Teams Tenant Admin API" non è concesso, non è possibile confermare se il codice del wrapper è corretto - da riverificare non appena il permesso è disponibile, prima di fidarsi ciecamente di questa cmdlet in produzione.

17.13 Knowledge Base per tenant (tab "Documentazione")

Aggiunta il 18/08/2026: documentazione specifica di UN cliente (peculiarità di infrastruttura, procedure di troubleshooting, note operative) caricata dall'operatore e consultata dall'AI senza doverla ripetere ogni volta in chat. Formati accettati: `.txt`, `.md`, `.docx`, `.xlsx` / `.xls`, `.pdf` (tramite il modulo `PdfLexer`, installato automaticamente al primo uso).

Come funziona

All'upload il testo viene estratto e l'AI lo cataloga con **una sola chiamata** (titolo, argomenti, riassunto) - il catalogo (leggero) resta sempre nel prompt di sistema, nessun costo aggiuntivo per le domande successive. Il testo **completo** di un documento specifico viene recuperato solo quando l'AI lo ritiene rilevante, tramite lo strumento dedicato `kb_query` - mai tutto il contenuto di tutti i documenti ad ogni domanda.

Isolamento tra tenant, verificato dal vivo (non solo per progetto): caricato un documento di prova su un tenant con un "codice segreto" fittizio, l'AI lo ha recuperato correttamente SOLO su quel tenant; passando a un secondo tenant il catalogo risultava vuoto e l'AI ha rifiutato esplicitamente di inventare una risposta invece di confondere i due clienti. `kb_query` non accetta né espone un parametro "tenant": legge sempre e solo il tenant realmente attivo in quel momento - una garanzia strutturale, non solo una convenzione rispettata dal codice.

Consigli pratici

- **Un documento per argomento** funziona meglio di un unico file enorme - il riassunto nel catalogo aiuta l'AI a capire QUANDO consultare il testo completo, e riassunti più mirati sono più utili di uno generico su tutto.
- **.doc legacy (binario, non .docx) non è supportato** - salva da Word come `.docx` prima di caricarlo, un tentativo con `.doc` viene rifiutato con un messaggio chiaro invece di produrre testo corrotto.
- **PDF scansionati come immagine** (senza testo selezionabile, es. un documento cartaceo fotografato) non vengono estratti - il tool lo segnala esplicitamente invece di catalogare un documento vuoto in silenzio.
- **Ricaricare un file con lo stesso nome sostituisce** la voce precedente - comodo per aggiornare una procedura senza dover prima rimuovere la versione vecchia.
- **Meglio per contenuti procedurali/operativi** (procedure, contatti, note tecniche specifiche del cliente) che per dati che cambiano spesso - quelli è meglio chiederli in tempo reale al tenant (es. "quanti utenti ha"), la Knowledge Base è pensata per "cose che un admin dovrebbe sapere a memoria su questo cliente", non per stato live.
- Cambiando tenant nella GUI, l'elenco documenti visibile nel tab cambia da solo - non serve alcuna azione manuale, ogni cliente ha la propria Knowledge Base, mai mescolate.

Knowledge Base GLOBALE (aggiunta il 20/08/2026)

Estensione richiesta esplicitamente dall'utente: oltre alla Knowledge Base per-tenant sopra, esiste un secondo bucket **riservato** (nome tenant interno `_global`, prefisso `_` scelto apposta - i profili tenant reali in `Config\tenants.json` non iniziano mai per convenzione con underscore, quindi nessuna collisione possibile) per documentazione che riguarda **l'app stessa**, non un cliente specifico - a partire dalla guida di configurazione che stai leggendo ora, caricata come `Guida-Configurazione.pdf`. Questo bucket è visibile **sempre**, indipendentemente da quale tenant è attivo - a differenza della Knowledge Base per-tenant sopra, che resta isolata per cliente.

Si aggiorna da sola (aggiunto il 21/08/2026): ad ogni avvio del server, `Gui\Server.ps1` confronta l'hash SHA256 di `docs\Guida-Configurazione.pdf` con l'ultimo caricato (`Config\global-kb-guide-hash.txt`) e, se è cambiato, lo ricarica automaticamente nella Knowledge Base globale - nessun passo manuale da ricordare dopo aver modificato la guida. Se il PDF non è cambiato, il riavvio non fa nulla (nessuna chiamata AI inutile). Verificato dal vivo: un primo riavvio dopo una modifica della guida ha risincronizzato correttamente (log: "Guida di configurazione cambiata..."), un secondo riavvio immediato, a guida invariata, non ha risincronizzato.

Con questo, si può chiedere in chat qualcosa come "come configuro l'app per Exchange Online?" o "quali permessi Graph servono per Intune?" a prescindere dal tenant attivo, e l'AI consulta la guida vera tramite `kb_query` invece di rispondere a memoria - verificato dal vivo il 20/08/2026 (log: chiamata reale a `kb_query`, risposta con dettagli specifici della guida come il permesso `Exchange.ManageAsApp` e l'elenco completo dei permessi Graph richiesti).

L'isolamento tra tenant resta intatto: il bucket globale è un pool **aggiuntivo e separato**, mai un modo di raggiungere il KB di un altro cliente reale - `kb_query` prova prima il tenant attivo, poi il bucket globale come fallback, mai il contrario.

Per caricare o aggiornare la guida nel bucket globale (operazione manuale, non da GUI per ora):

```
Add-M365OpsKnowledgeDocument -TenantName '_global' -FilePath 'docs\Guida-Configurazione.pdf' `
-OriginalFileName 'Guida-Configurazione.pdf' -Provider AzureOpenAI
```

Bug reale trovato caricando la guida per la prima volta (52 pagine): il modulo `PdfLexer` restituisce un array di stringhe, una per pagina, non un'unica stringa - il codice originale lo passava così com'era, quindi `.Length` contava le PAGINE (52) invece dei caratteri, scartando di fatto tutto il testo vero e mandando in errore la catalogazione AI (un array non converte a un parametro tipizzato stringa). Corretto unendo esplicitamente le pagine in `Get-M365OpsExtractedFileText.ps1` prima di restituire il testo - stessa famiglia di bug "array trattato come scalare" già vista più volte in questo progetto (sezione 20.5).

Diagnosi errori che consulta anche la Knowledge Base (aggiunta il 20/08/2026)

`Invoke-M365OpsErrorTriage` (diagnosi di un'azione fallita) e `Invoke-M365OpsWriteRecovery` (proposta di correzione immediata per una scrittura Graph fallita) ora consultano, oltre a Microsoft Learn, anche la Knowledge Base - sia quella globale sia quella del tenant attivo, combinate. Match per parola chiave semplice tra il testo dell'errore/contesto e gli argomenti/titolo dei documenti in catalogo (nessuna chiamata AI aggiuntiva per la ricerca: i cataloghi restano piccoli). Se un problema è specifico di configurazione/setup di QUESTA app (es. il limite RBAC app-only della sezione 6.4 qui sopra), la documentazione interna del progetto è una fonte più precisa e mirata di una ricerca generica su Microsoft Learn.

Verificato dal vivo il 20/08/2026: passato a `Invoke-M365OpsErrorTriage` lo stesso errore reale "term not recognized" di `New-AcceptedDomain` della sezione 6.4 - la diagnosi ha citato correttamente la finestra di propagazione RBAC documentata nella guida invece di rispondere alla cieca, confermando (anche con un controllo indipendente del solo meccanismo di match per parola chiave, senza passare dall'AI) che il documento giusto viene trovato e usato.

17.14 Aggiornamenti (tab "Manutenzione")

Aggiunta il 18/08/2026, dopo la pubblicazione del repository su GitHub: la webapp puo' controllare, scaricare e applicare da sola la versione piu' recente del codice, senza bisogno di `git` installato ne' di comandi manuali - tutto da un pulsante nel tab Manutenzione.

Come si decide quando un aggiornamento diventa disponibile

Nessun commit pubblicato su GitHub diventa mai un aggiornamento da solo. Un aggiornamento esiste solo quando viene pubblicata esplicitamente una **Release** GitHub (un tag di versione, es. `v0.2.0`, con note di rilascio) - un atto deliberato, mai automatico. Finche' non si pubblica una Release, ogni installazione resta ferma alla propria versione, indipendentemente da quanti commit nel frattempo finiscono su `main`.

Canale Stable vs Testing

La distinzione usa un meccanismo **nativo di GitHub**, non qualcosa di inventato per questo progetto: al momento di pubblicare una Release, GitHub offre la casella "Set as a pre-release".

- **Stable** (default): propone solo Release NON marcate pre-release. La webapp chiama `GET /repos/<owner>/<repo>/releases/latest` - un endpoint che GitHub stesso filtra gia' per restituire solo l'ultima Release definitiva, nessuna logica di selezione scritta qui.
- **Testing**: propone SEMPRE la Release piu' recente in assoluto, marcata pre-release o no - utile solo per chi vuole provare novita' in anteprima, sconsigliato su un tenant di produzione.

Il canale e' una preferenza **locale a questa installazione** (`Config\update-channel.txt`, escluso da git) - cambiarlo non ha alcun effetto sul repository, solo su quali Release questa copia dell'app propone.

Sicurezza dei dati del tenant, per struttura non per promessa: quando si applica un aggiornamento, il codice scarica l'archivio della Release da GitHub e sovrascrive i file dell'installazione con quelli scaricati. Quell'archivio e' **esattamente l'albero tracciato da git** a quel tag - `Config\`, `Logs\`, `Uploads\`, `Reports\`, `Testing\` non ci sono MAI dentro (esclusi da `.gitignore`, mai committati), quindi l'aggiornamento non puo' fisicamente toccare tenant, chat, Knowledge Base, secret o report esistenti - non perche' vengono esclusi "a mano", ma perche' semplicemente non esistono nel materiale scaricato. Stessa garanzia strutturale gia' usata per l'isolamento della Knowledge Base (sezione 17.13).

Limite noto

Un file rimosso in una versione piu' recente non viene ripulito automaticamente da un aggiornamento - vengono applicate solo aggiunte e modifiche. Per una pulizia completa (raro che serva) resta disponibile il percorso garantito al 100%: cancellare la cartella e riclonare il repository.

L'azione di aggiornamento e' **SOLO da GUI** (pulsante "Aggiorna ora"), mai esposta come strumento all'AI - modifica l'app stessa e richiede un riavvio, non qualcosa che una richiesta in chat deve poter innescare, stessa logica gia' applicata alla cancellazione di un documento dalla Knowledge Base.

17.15 Copertura Exchange Online: cosa c'e' e cosa manca deliberatamente

Il 18/08/2026 e' stato fatto un censimento reale (non a memoria) dei cmdlet Exchange Online ufficiali - scaricati dal repository sorgente della documentazione Microsoft su GitHub ([MicrosoftDocs/office-docs-powershell](#)), **1.438 cmdlet totali**. M365Ops non punta alla parita' 1:1 con l'intero prodotto Exchange (che include amministrazione on-premise/ ibrida, RBAC, eDiscovery, Unified Messaging - aree deliberatamente fuori scope) - punta a coprire bene le aree di amministrazione M365 cloud-only quotidiane.

Aree estese il 18/08/2026 (36 nuovi cmdlet)

- **Gruppi di distribuzione:** `Set-/Enable-/Disable-M365OpsDistributionGroup` , `Set-/Remove-M365OpsDynamicDistributionGroup` , `Get-M365OpsDynamicDistributionGroupMember` , `Update-M365OpsDistributionGroupMember` (sostituisce l'INTERA membership, non incrementale)
- **Tenant Allow/Block List - spoof:** `Get-/New-/Remove-M365OpsTenantAllowBlockListSpoofItem(s)` , `Set-M365OpsTenantAllowBlockListItem` - sotto-lista separata dalle voci normali per mittente/URL/hash/IP
- **Mail flow / connettori:** `Get-/Set-M365OpsTransportConfig` (impostazioni globali), `Get-/New-/Set-/Remove-M365OpsInboundConnector` e `...OutboundConnector` (rinominati il 21/08/2026 da `...ReceiveConnector/...SendConnector` - vedi riquadro sotto), `Get-/New-/Set-/Remove-M365OpsRemoteDomain` , `New-/Set-/Remove-M365OpsAcceptedDomain` , `Get-M365OpsMailDetailTransportRuleReport`
- **Quarantena (policy):** `Get-/New-/Set-/Remove-M365OpsQuarantinePolicy` , `Get-M365OpsQuarantineMessageHeader`

Testato dal vivo con scritture reali (creato, verificato, ripulito su vnsys-test): `Set-M365OpsDistributionGroup` (nascondi/mostra in rubrica), `New-/Remove-M365OpsQuarantinePolicy` (ciclo completo). Le letture `Get-M365OpsTransportConfig` / `RemoteDomain` / `QuarantinePolicy` sono verificate dal vivo con dati reali del tenant. Aggiunto il 21/08/2026: `Set-M365OpsRemoteDomain` (toggle di `AutoForwardEnabled` sulla voce "Default", verificato con lettura indipendente e ripristinato al valore originale) e il ciclo completo `New-/Set-/Remove-M365OpsDynamicDistributionGroup` (gruppo dinamico di test creato con un filtro OPATH reale, rinominato, poi rimosso - ogni passaggio verificato con una lettura indipendente, ambiente di test tornato pulito). **Aggiunto il 21/08/2026:** ciclo completo create/verifica/modifica/verifica/rimuovi/ verifica per `New-/Set-/Remove-M365OpsInboundConnector` e `...OutboundConnector` , IN APP-ONLY - funzionano perfettamente, nessun limite RBAC reale (vedi il riquadro sotto per la storia completa).

BUG STRUTTURALE trovato e corretto il 21/08/2026 (non un limite RBAC come ipotizzato inizialmente): `Get-/New-/Set-/Remove-ReceiveConnector` e `...SendConnector` fallivano con "term not recognized" sul tenant di test - scambiato in un primo momento per lo stesso schema RBAC gia' documentato per Teams Policies/Purview (sezione 6.4). Approfondendo su richiesta esplicita dell'utente ("puoi fare test in modalita' delegata?"), un login Delegated reale con un utente membro completo di "Organization Management" ha dato lo STESSO errore - impossibile se fosse stato RBAC. Causa vera, confermata su documentazione ufficiale Microsoft: questi cmdlet sono **esclusivi di Exchange on-premises**, mai esistiti in Exchange Online. Sostituiti gli 8 wrapper con i veri equivalenti Exchange Online, `Get-/New-/Set-/Remove-InboundConnector` e `...OutboundConnector` (parametri diversi: `SenderDomains` invece di `Bindings` / `RemoteIPRanges` , concetti legati a un server fisico che non esiste in cloud) - testato dal vivo con un ciclo completo per entrambi, funzionano subito in App-only, zero problemi di permessi. Vedi sezione 6.4 per la cronologia completa dell'indagine.

`New-/Set-/Remove-M365OpsAcceptedDomain` ha la sintassi verificata contro la documentazione ufficiale Microsoft. Resta NON testato con una scrittura reale, deliberatamente (azione ad alto impatto, puo' interrompere la posta su un intero dominio) - ma il limite RBAC che lo bloccava e' ora CONFERMATO e circoscritto (sezione 6.4, 21/08/2026): funziona in modalita' Delegated (`Get-AcceptedDomain` verificato dal vivo con 5 domini reali), resta bloccato in App-only indipendentemente da quanti ruoli RBAC si assegnano al service principal - un limite della piattaforma per questo specifico cmdlet, non un bug di questo progetto. Chi ha bisogno di gestirlo da M365Ops deve usare un profilo tenant Delegated.

Cosa resta fuori scope (per scelta, non dimenticanza)

Il censimento ha anche quantificato aree grandi lasciate deliberatamente fuori per ora: Retention/Compliance Purview incluso DLP ed eDiscovery (~65 cmdlet - oggi solo lettura, comunque limitata dal ruolo RBAC non assegnato), migrazione Public Folder (~40 cmdlet, sotto-mondo separato mai iniziato). Candidate per un'estensione futura, su richiesta esplicita. Anti-spam/anti-phish/ malware in scrittura (~98 cmdlet), che era in questo elenco, e' stato implementato il 21/08/2026 - vedi sezione 17.19.

17.16 Copertura Intune: perche' il conteggio cmdlet non e' la metrica giusta

Censimento reale (19/08/2026, non a memoria) del repository sorgente della documentazione Microsoft Graph PowerShell SDK ([MicrosoftDocs/microsoftgraph-docs-powershell](#) , via Git Trees API): i moduli genuinamente Intune ([Microsoft.Graph.DeviceManagement](#) , [.Administration](#) , [.Enrollment](#) , [.Functions](#) , [Devices.CorporateManagement](#) - quest'ultimo copre [deviceAppManagement](#) , quindi le app Win32/mobile e le policy di protezione app) totalizzano **3.693 cmdlet** (1.033 in v1.0, 2.660 in beta) tra v1.0 e beta.

Un confronto diretto "N di 3.693 coperti" sarebbe fuorviante, a differenza del censimento Exchange sopra: quei cmdlet sono generati automaticamente, uno per ogni combinazione risorsa/verbo REST del namespace `deviceManagement / deviceAppManagement` (es. `Get-MgDeviceManagementDeviceCompliancePolicySettingStateSummary`) - la stragrande maggioranza sono letture/scritture strette su sotto-risorse molto specifiche, non funzionalita' distinte per un amministratore. M365Ops non li wrappa 1:1 come fa per Exchange: usa `graph_api_call / propose_graph_write` generici, che possono gia' raggiungere QUALUNQUE endpoint sotto `deviceManagement / deviceAppManagement` - la vera limitazione non e' "quale cmdlet manca", ma quali permessi Graph sono concessi all'App Registration e quanto lo strato AI conosce lo schema esatto di un payload prima di scriverlo (vedi i bug reali del 19/08/2026 sopra, es. `omaSettings`).

Wrapper dedicati esistono solo dove la generalizzazione non basta: `Get-M365OpsManagedDevices` , `Get-M365OpsDeviceComplianceReasons` , `Get-M365OpsCompliancePatterns` (lettura/analisi dispositivi e compliance), `New-M365OpsWin32App / Set-M365OpsAppAssignment` (packaging e assegnazione Win32, dove passare dal modulo community `IntuneWin32App` - non solo Graph REST - serve davvero, per la codifica/upload del pacchetto `.intunewin`). Solo 5 radici Graph risultano referenziate da codice reale nell'intero modulo (verificato per grep, non a memoria): `deviceAppManagement/mobileApps` , `deviceManagement/assignmentFilters` , `deviceManagement/deviceCompliancePolicies` , `deviceManagement/deviceConfigurations` , `deviceManagement/managedDevices` - tutto il resto, quando funziona, funziona solo perche' l'AI costruisce la chiamata Graph al volo via `graph_api_call / propose_graph_write` , senza alcun codice dedicato che l'abbia mai verificata dal vivo.

Censimento reale per area funzionale (19/08/2026, aggiornato dopo l'implementazione)

Conteggio cmdlet SDK per area (non conteggio di funzionalita' distinte - vedi sopra il perche' - ma utile per capire dove si concentra la superficie Intune) e stato reale in M365Ops. Tutte le aree elencate come "non implementato" nel censimento iniziale sono state costruite lo stesso giorno (19/08/2026, ~60 nuovi cmdlet in `Public\`) su richiesta esplicita dell'utente, con schema Graph verificato dal vivo su Microsoft Learn per ognuna (mai a memoria) e wiring completo nello strato AI (`intune_query / propose_intune_write` , stesso meccanismo generico query/proposta-scrittura di `exo_query / propose_exo_write`). Verificate dal vivo su `vnsys-test` il giorno stesso via chat reale (non solo import del modulo):

Area	Cmdlet SDK (v1.0+beta)	Stato in M365Ops
Dispositivi gestiti (elenco, dettaglio, azioni remote)	referenziato in codice	funzionante - lettura/compliance testate dal vivo ripetutamente. Azioni remote (retire/wipe/sync/restart/fresh start/autopilot reset) PROPOSTE dal vivo con successo via AI generica (mai confermate per non toccare dispositivi reali) - nessun wrapper dedicato, nessun guardrail oltre alla conferma umana standard
Compliance Policy (criteri di conformita')	66 (v1.0) + 77 (beta) = 143	solo lettura generica - elenco/dettaglio letti dal vivo via <code>graph_api_call</code> , MAI creata/modificata una policy (nessun wrapper, nessun test)
Configuration Profile (profili di configurazione "classici")	46 (v1.0) + 85 (beta) = 131	parziale, non affidabile - <code>New-M365OpsDeviceConfigurationProfile</code> esiste ma e' orfano (nessun tool AI/GUI la richiama); il tentativo di creazione via AI generica in questo stesso giro di test e' FALLITO due volte (<code>omaSettings</code>) prima di essere abbandonato - oggi non affidabile. Superato in pratica dal Settings Catalog sotto, la via moderna consigliata da Microsoft stessa
Settings Catalog (<code>configurationPolicies</code> , il rimpiazzo moderno dei profili classici)	39 (solo beta)	funzionante - <code>Get-/New-/Set-/Remove-M365OpsConfigurationPolicy</code> , <code>Set-M365OpsConfigurationPolicyAssignment</code> , <code>Get-M365OpsConfigurationSettingDefinitions</code> (ricerca impostazioni). Lettura verificata dal vivo il 19/08/2026 su <code>vnsys-test</code> (2 criteri reali restituiti correttamente via chat)
Endpoint Security (Antivirus, Disk Encryption, Firewall, EDR, Attack Surface Reduction - <code>intents</code>)	60 (solo beta)	funzionante - stesso motore/cmdlet del Settings Catalog sopra (distinto da <code>templateReference.templateFamily</code>), piu' <code>Get-M365OpsConfigurationPolicyTemplates</code> per i modelli disponibili
Security Baseline	30 (solo beta)	funzionante - stesso motore Settings Catalog (<code>templateFamily=baseline</code>), nessun cmdlet separato necessario
Autopilot (identita' dispositivo + profili di deployment + ESP)	13 (v1.0) + 52 (beta) = 65	funzionante (codice) , non verificabile end-to-end - 9 cmdlet (import/lettura/modifica/rimozione dispositivo, profili di deployment + assegnazione). L'import di un dispositivo reale richiede un hash hardware autentico (4K HH) che non e' fabbricabile per un test - mai verificato dal vivo con un dispositivo reale, solo sintassi e schema Graph
Script PowerShell Windows (<code>deviceManagementScripts</code> - DIVERSO dal packaging Win32: esecuzione diretta di uno script, non un'app)	33 (solo beta)	funzionante - <code>New-/Get-/Remove-M365OpsDeviceScript</code> , <code>Set-M365OpsDeviceScriptAssignment</code> (Windows e macOS). Lettura testata dal vivo il 19/08/2026: 403 chiaro per permesso <code>DeviceManagementScripts.ReadWrite.All</code> mancante su <code>vnsys-test</code> (vedi sezione 4.2) - codice corretto, permesso da concedere
Script shell macOS	32 (solo beta)	funzionante - stesso cmdlet di sopra con <code>-Platform macOS</code> , stesso gap di permesso
Proactive Remediation (<code>deviceHealthScripts</code> - rileva+correggi)	41 (solo beta)	funzionante - <code>New-/Get-/Remove-M365OpsProactiveRemediation</code> , <code>Set-M365OpsProactiveRemediationAssignment</code> (pianificazione giornaliera/oraria). Stesso gap di permesso <code>DeviceManagementScripts.ReadWrite.All</code> dello script Windows sopra (verificato dal vivo)
Admin Template / Group Policy Analytics	20 (solo beta)	funzionante - <code>New-/Get-/Remove-M365OpsAdminTemplate</code> , <code>Set-M365OpsAdminTemplateAssignment</code> , <code>Find-M365OpsAdminTemplateSetting</code> (ricerca impostazioni GP) + <code>Set-M365OpsAdminTemplateSetting</code> (azione <code>updateDefinitionValues</code>). Lettura verificata dal vivo il 19/08/2026 (nessun profilo configurato su <code>vnsys-test</code> , risposta corretta senza errori)
Windows Update rings (Feature/Quality update profiles)	22 (solo beta)	funzionante - <code>New-/Get-/Remove-M365OpsUpdateRing</code> , <code>Set-M365OpsUpdateRingAssignment</code> . Lettura verificata dal vivo il 19/08/2026 (nessun anello configurato, risposta corretta)
App Protection / MAM (Managed App Protection Android/iOS)	50+49 = 99 (v1.0, Android+iOS)	funzionante - <code>New-/Get-/Remove-M365OpsAppProtectionPolicy</code> , <code>Set-/Remove-M365OpsAppProtectionAssignment</code> , <code>Set-M365OpsAppProtectionTargetApps</code> . Lettura verificata dal vivo il

		19/08/2026 (nessun criterio configurato, risposta corretta). Windows Information Protection resta fuori scope (funzionalità in via di deprecazione da parte di Microsoft)
Scope Tag (RBAC per oggetto Intune)	14 (solo beta)	funzionante (codice) , permesso da concedere - <code>New-/Get-/Remove-M365OpsScopeTag</code> per i tag stessi (oltre alla sola assegnazione già esistente su <code>New-M365OpsWin32App -ScopeTagNames</code>). Creazione testata dal vivo il 19/08/2026: 403 chiaro per <code>DeviceManagementRBAC.ReadWrite.All</code> mancante (vedi sezione 4.2) - pipeline proposta→conferma→esecuzione→errore Graph confermata funzionante end-to-end, manca solo il permesso
Enrollment Configuration (restrizioni/profili di iscrizione)	12 (v1.0)	funzionante (codice) , permesso da concedere - <code>Get-M365OpsEnrollmentConfigurations</code> , <code>New-M365OpsEnrollmentLimitConfiguration</code> , <code>New-M365OpsEnrollmentPlatformRestriction</code> , <code>Set-M365OpsEnrollmentConfigurationPriority / Assignment</code> , <code>Remove-M365OpsEnrollmentConfiguration</code> . Lettura testata dal vivo il 19/08/2026: 403 per <code>DeviceManagementServiceConfig.ReadWrite.All</code> mancante
Device Category	17 (v1.0)	non implementato - area di nicchia, non richiesta esplicitamente
Notification Message Template	11 (v1.0)	funzionante - <code>New-/Get-/Remove-M365OpsNotificationTemplate</code> , <code>Set-M365OpsNotificationTemplateMessage</code> (multi-lingua) , <code>Send-M365OpsNotificationTemplateTest</code> . Lettura verificata dal vivo il 19/08/2026 (nessun modello configurato, risposta corretta)
Role Definition (ruoli RBAC custom Intune)	13 (v1.0)	funzionante (codice) , permesso da concedere - <code>New-/Get-/Remove-M365OpsCustomRole</code> , <code>Get-M365OpsRoleDefinitionActions</code> (catalogo azioni di riferimento) , <code>Set-/Remove-M365OpsRoleAssignment</code> . Lettura testata dal vivo il 19/08/2026: stesso 403 <code>DeviceManagementRBAC.ReadWrite.All</code> dello Scope Tag sopra
Certificati/Wifi/VPN/Email (profili di connettività)	incluso in Configuration Profile sopra	solo generico, mai testato - sono sotto-tipi di <code>deviceConfiguration</code> , quindi teoricamente raggiungibili con la stessa via generica, ma zero verifica dal vivo su nessuno di questi
Connettori partner (Mobile Threat Defense, Exchange, Compliance Management Partner, Remote Assistance, DEP/ABM, VPP token)	7+6+5+5+5+6 = 34 (v1.0)	funzionante (sola lettura) - <code>Get-M365OpsPartnerConnectorsStatus</code> , sola lettura per design: l'attivazione iniziale richiede un consenso OAuth interattivo con il servizio partner, non realizzabile da chat/REST. Verificato dal vivo il 19/08/2026 (query riuscita, nessun connettore configurato)
Restano deliberatamente fuori scope: Device Category (area di nicchia), Windows Information Protection (in deprecazione), profili di connettività 'Certificati/Wifi/VPN/Email' (raggiungibili solo via <code>propose_graph_write</code> generico, mai verificati). Tre aree (Scope Tag, Enrollment Configuration, Role Definition/RBAC) hanno il codice completo e verificato ma restano bloccate da permessi Graph Application non ancora concessi su <code>vnsys-test</code> (<code>DeviceManagementRBAC.ReadWrite.All</code> , <code>DeviceManagementServiceConfig.ReadWrite.All</code>) - vedi sezione 4.2 per i nomi esatti da aggiungere nell'App Registration.		

17.17 Check permessi app (pulsante "🔒 Stato permessi")

Aggiunto il 21/08/2026, richiesto esplicitamente dall'utente: un pulsante in alto nella chat (o scrivendo "verifica permessi app"/"che permessi ha l'app") interroga Microsoft Graph in tempo reale per vedere quali permessi sono DAVVERO concessi (con "Grant admin consent") all'App Registration del tenant attivo, e li traduce per area funzionale (Intune, Entra ID, Report, MFA, Sicurezza, Teams, SharePoint, Exchange) con stato "nessun accesso" / "lettura" / "lettura+scrittura" - invece di scoprirlo un errore alla volta durante l'uso normale.

Nessun elenco statico: legge le assegnazioni REALI del service principal (`GET /servicePrincipals/{id}/appRoleAssignments`) - un permesso aggiunto in Entra ID ma senza "Grant admin consent" non compare, esattamente come nella realtà non funziona. Costo zero: nessuna chiamata AI, la funzione (`Get-M365OpsAppPermissionsCheck`) e' puro codice deterministico, catalogata come comando locale (sezione 17, `CommandCatalog.ps1`).

Bug reale trovato durante lo sviluppo, corretto prima di consegnare: il nome della risorsa SharePoint restituito da Graph e' "Office 365 SharePoint Online", NON "SharePoint Online" come si potrebbe supporre dal nome "SharePoint" mostrato nel selettore del portale Entra ID (sezione 4.3) - un primo tentativo con quel nome produceva sempre "nessun accesso" anche a permesso presente (un mismatch di stringa nel codice, non un vero permesso mancante), scoperto confrontando l'output con le assegnazioni grezze reali di vnsys-test prima di fidarsi del risultato.

Secondo bug reale, trovato dal vivo dall'UTENTE il 21/08/2026 (non durante lo sviluppo, dopo la consegna): "l'app ha Sites.FullControl.All, che dovrebbe essere oltre Read.All, ma il check non se ne rende conto" - confermato: il confronto era per stringa ESATTA, quindi `Sites.FullControl.All` concesso non soddisfaceva un requisito di `Sites.Read.All`, anche se chi ha accesso completo puo' ovviamente anche solo leggere. Corretto aggiungendo il riconoscimento della gerarchia standard Microsoft Graph `Read.All < ReadWrite.All < Manage.All < FullControl.All` : un permesso piu' ampio con lo stesso prefisso ora soddisfa automaticamente un requisito piu' stretto. Verificato dal vivo dopo il fix: l'area "SharePoint (lettura diretta via Graph)" e' passata da "nessun accesso" a "lettura", coerente con il permesso realmente posseduto.

Quattro "aree di concessione" Microsoft distinte vengono controllate, ognuna con il proprio consenso separato nel portale Entra ID: Microsoft Graph, "Office 365 Exchange Online" (`Exchange.ManageAsApp`), "Office 365 SharePoint Online" (`Sites.FullControl.All`, sezione 4.3) e "Skype and Teams Tenant Admin API" (solo per i cmdlet di POLICY Teams, sezione 4.4 - nome risorsa non ancora verificato dal vivo su un tenant che abbia quel permesso concesso, l'unico effetto possibile di un nome sbagliato sarebbe un falso "nessun accesso", mai un falso positivo). Teams e SharePoint "di base" (moduli MicrosoftTeams/PnP) sono segnalati separatamente come sempre disponibili con lo stesso certificato/secret gia' usato per l'autenticazione - non dipendono da nessuno di questi permessi (verificato dal vivo il 17/08/2026, sezione 17.11).

Il livello RBAC REALE di Exchange Online (quali cmdlet funzionano davvero, non solo se la connessione riesce) dipende anche dal ruolo assegnato al service principal DENTRO Exchange (sezione 6) - non verificabile da questo check senza una connessione Exchange live, segnalato come nota esplicita nel risultato invece di dare un falso senso di completezza.

Estensione per tenant Delegati (21/08/2026)

Aggiunta richiesta esplicitamente dall'utente dopo aver notato che, su un tenant Delegato, il pulsante si limitava a un errore ("il check si applica solo ai tenant App-only"): "non sarebbe meglio listare i ruoli attivi dell'utente e dirgli cosa puo'/non puo' fare con quei ruoli?". Su un tenant Delegato non esiste un'App Registration da verificare - l'accesso dipende dai RUOLI RBAC EXCHANGE REALI dell'utente con cui si e' fatto login. `Get-M365OpsDelegatedPermissionsCheck` (nuova, Private) interroga `Get-ManagementRoleAssignment -RoleAssignee` per l'utente delegato e confronta i ruoli posseduti con una tabella area=ruolo.

Onestà sulla copertura: solo 2 righe della tabella sono VERIFICATE dal vivo con una scrittura reale in modalità Delegated ("Domini accettati" via "Remote and Accepted Domains", confermato lo stesso giorno con un login reale; "Anti-spam/ anti-phishing/malware" via "Transport Hygiene", confermato in App-only con lo stesso ruolo). Le altre righe (Connettori, Mailbox/gruppi, Regole di trasporto, Role Management) sono dedotte dal NOME del ruolo Exchange, non ancora confermate con una scrittura reale in Delegated - marcate esplicitamente come tali nel risultato (nota per riga), per non dare un falso senso di certezza. L'accesso Graph (Intune/Entra ID/Teams/SharePoint) in Delegated dipende dal ruolo DIRECTORY Entra ID dell'utente - un controllo diverso, non ancora implementato, segnalato come tale nel risultato invece di ometterlo in silenzio.

L'elenco COMPLETO e grezzo dei ruoli RBAC Exchange posseduti dall'utente e' sempre incluso nel risultato (riga "informativo" dedicata) - anche per le aree non ancora mappate, così non si perde informazione reale in nome di una tabella incompleta.

Verificato dal vivo il 21/08/2026 con un login Delegato reale (`diego@vnsys.it`, membro di "Organization Management"): il pulsante "🔒 Stato permessi" mostra correttamente tutte le aree mappate come disponibili, l'elenco completo dei ~90 ruoli posseduti, e la nota sul gap Graph/directory role - nessun errore, nessuna informazione fuorviante.

Due bug reali trovati dall'utente subito dopo, corretti lo stesso giorno

- Intestazione sbagliata:** il risultato apriva sempre con "Permessi reali dell'app registrata su questo tenant" - testo copiato dal ramo App-only, senza senso su un tenant Delegato (non esiste nessuna App Registration). Corretto rendendo l'intestazione condizionale in `CommandCatalog.ps1`.
- Mancavano i ruoli DIRECTORY Entra ID:** "mi aspettavo di vedere i miei ruoli, tipo Global Admin nel mio caso" - i ruoli RBAC Exchange sopra sono un sistema COMPLETAMENTE SEPARATO dai ruoli directory Entra ID che governano Intune/Entra ID/Teams/SharePoint via Graph (gia' segnalato come gap noto nella prima versione, ora colmato). Aggiunta una chiamata a `GET /me/transitiveMemberOf/microsoft.graph.directoryRole` usando lo stesso token Graph delegato gia' ottenuto con "Accedi a Microsoft Graph col mio utente" - `Invoke-M365OpsGraphRequest` sceglie gia' da solo il token giusto in base all'AuthMode, nessun codice di autenticazione nuovo necessario. Mappa i ruoli directory standard Microsoft (Global Administrator, Intune Administrator, User Administrator, Groups Administrator, Teams Administrator, SharePoint Administrator) su Intune/Entra ID/Teams/SharePoint, oltre a mostrare sempre l'elenco completo grezzo dei ruoli directory posseduti.

17.18 Tema chiaro/scuro e pannello Upload a scomparsa

Aggiunto il 21/08/2026, richiesto esplicitamente dall'utente. Due modifiche indipendenti alla GUI:

- **Tema scuro** - pulsante 🌙/☀️ nell'header. Scelta SOLO dell'utente (mai dedotta dal tema del sistema operativo), persistita in `localStorage` così resta la stessa tra un riavvio del browser e l'altro. Tutta la GUI usa già variabili CSS per i colori (nessun colore scritto a fisso in nessun punto della pagina) - il tema scuro è quindi un solo blocco di variabili alternative, non una riscrittura dei singoli stili.
- **Pannello Upload a scomparsa** - i 3 pulsanti di caricamento (file da pacchettizzare, icona, CSV migrazione), prima sempre visibili sopra la casella di testo, ora sono dentro un pannello che si apre/chiude col pulsante "📁 Upload" nell'header - stesso pattern toggle già usato per il pannello Impostazioni.

Contestualmente, i pulsanti dell'header ("⚙️ Impostazioni tenant", "🔒 Stato permessi", "📁 Upload", tema) sono stati unificati sotto un solo stile condiviso (`.header-btn`) - prima "Impostazioni tenant" aveva una regola CSS dedicata mentre "Stato permessi" (aggiunto in v0.9.18) non ne aveva nessuna, risultando visivamente diverso (osservato dall'utente).

Un file HTML/JS come `Gui\index.html` viene tenuto in memoria dal server all'avvio (`Get-Content -Raw in Gui\Server.ps1` , mai riletto ad ogni richiesta) - a differenza di un cambio a un file `.ps1` del modulo (che serve comunque un riavvio per lo stesso motivo), un cambio SOLO a `index.html` può sembrare non applicarsi se si controlla senza riavviare: verificato dal vivo durante lo sviluppo di questa stessa funzione (un primo controllo subito dopo l'edit mostrava ancora la pagina vecchia).

17.19 Anti-spam, anti-phishing e filtro malware in scrittura

Aggiunto il 21/08/2026, richiesto esplicitamente dall'utente ("passiamo all'implementazione di Anti-spam/anti-phishing/malware in scrittura — ~98 cmdlet, oggi solo lettura"). Copre le tre aree con lo stesso schema "criterio + regola" già visto per Accepted Domain/Connettori: un CRITERIO (policy) definisce COSA succede (azioni, soglie), una REGOLA (rule) definisce A CHI si applica - un criterio senza una regola collegata non si applica a nessuno (tranne quelli predefiniti del sistema).

Area	Cmdlet nativo	Wrapper M365Ops
Anti-spam	HostedContentFilterPolicy / HostedContentFilterRule	Get-/New-/Set-/Remove-M365OpsAntiSpamPolicy , Get-/New-/Set-/Remove-M365OpsAntiSpamRule , Enable-/Disable-M365OpsAntiSpamRule
Anti-phishing	AntiPhishPolicy / AntiPhishRule	Get-/New-/Set-/Remove-M365OpsAntiPhishPolicy , Get-/New-/Set-/Remove-M365OpsAntiPhishRule , Enable-/Disable-M365OpsAntiPhishRule
Filtro malware	MalwareFilterPolicy / MalwareFilterRule	Get-/New-/Set-/Remove-M365OpsMalwareFilterPolicy , Get-/New-/Set-/Remove-M365OpsMalwareFilterRule , Enable-/Disable-M365OpsMalwareFilterRule

Nessun nuovo ruolo RBAC richiesto: il ruolo Exchange "Transport Hygiene", già assegnato al service principal di questo progetto per altre funzionalità, copre già tutti e sei i cmdlet New- . Verificato dal vivo (non assunto dalla documentazione): tutti e 24 i wrapper testati in App-only con un ciclo completo create/verifica/modifica/verifica/rimuovi/verifica per ognuna delle tre aree - nessun errore di permesso, ambiente di test tornato pulito ad ogni ciclo.

Bug reale trovato dal vivo durante il test, non dalla documentazione: NESSUNO dei tre Set-*Rule (Set-HostedContentFilterRule , Set-AntiPhishRule , Set-MalwareFilterRule) accetta un parametro -Enabled , nonostante alcune fonti (inclusi risultati di ricerca web) suggerissero il contrario - verificato leggendo la lista PARAMETRI REALE del cmdlet live con Get-Command , non la sola documentazione. Il tentativo iniziale (Set-M365OpsAntiSpamRule -ExtraParams @{Enabled=\$false}) falliva con "A parameter cannot be found that matches parameter name 'Enabled'". Corretto aggiungendo 6 cmdlet dedicati Enable-/Disable-M365OpsXRule (che chiamano i veri Enable-/Disable-HostedContentFilterRule ecc., cmdlet nativi separati che ESISTONO e funzionano) - verificato dal vivo per tutti e tre i tipi.

Sintassi di tutti i cmdlet New- verificata contro la documentazione ufficiale Microsoft prima di scrivere i wrapper (non a memoria): unico parametro davvero obbligatorio per un criterio e' Name ; per una regola sono Name + il riferimento al criterio (HostedContentFilterPolicy / AntiPhishPolicy / MalwareFilterPolicy a seconda del tipo) - un criterio può essere collegato a UNA SOLA regola, Microsoft lo impedisce direttamente al momento della creazione se si tenta di riusarlo.

17.20 La guida resta raggiungibile anche a zero configurazione (`/guida`)

Aggiunto il 21/08/2026, richiesto esplicitamente dall'utente dopo un test end-to-end su un PC completamente pulito: "quando l'app parte e non e' configurata la sua guida deve essere raggiungibile per essere consultata, o non fa perche' manca l'IA?".

Problema reale, non solo di messaggistica: prima di questo fix, la guida era leggibile SOLO tramite lo strumento `kb_query` dentro il motore AI (`Invoke-M365OpsAgentTools`) - senza nessuna chiave AI configurata, nemmeno la domanda piu' semplice sulla guida stessa ("come genero il certificato Exchange?") trovava risposta: un problema dell'uovo e della gallina, serviva la guida per sapere come configurare l'AI, ma la guida stessa richiedeva l'AI per essere consultata.

Corretto con una route statica `GET /guida` sul server web dell'app, che serve `docs\Guida-Configurazione.pdf` direttamente come file - nessun passaggio dal motore AI, nessuna chiave richiesta, nessun tenant necessario. Raggiungibile aprendo `http://localhost:<porta>/guida` in qualunque browser, anche al primissimo avvio dell'app senza nulla configurato. I messaggi di errore per chiave AI mancante (`Invoke-M365OpsAgentTools.ps1`) e il messaggio di onboarding al primo avvio (sezione 7) ora indicano esplicitamente questo percorso. Verificato dal vivo: PDF di 8 pagine servito correttamente via una richiesta GET reale.

5. Se un tenant è in modalità Delegated, fai il login la prima volta che ti serve (tab Tenant)

19. Log delle azioni — storico e debug

Ogni messaggio ricevuto in chat, ogni strumento AI chiamato (con esito), e ogni azione confermata eseguita vengono registrati in `Logs\m365ops-YYYYMMDD.log` (un file al giorno, testo semplice, append-only) — indipendentemente dai messaggi `Write-Host` già visibili nel terminale se stai guardando la finestra del server.

19.1 Consultarli

Tab Manutenzione → sezione "Log delle azioni (oggi)" mostra le ultime 200 righe, con un pulsante "Aggiorna". Oppure apri direttamente il file di testo in `Logs\`.

```
# formato di ogni riga: data/ora TAB [livello] TAB tenant[modalità] TAB messaggio
2026-08-15 10:20:46 [Info] Fabrikam-Prod[Delegated] Messaggio ricevuto: elenca le mailbox condivise
2026-08-15 10:20:46 [Info] Fabrikam-Prod[Delegated] Avvio Invoke-M365OpsAgentTools (fallback AI)...
2026-08-15 10:20:48 [Info] Fabrikam-Prod[Delegated] Tool AI chiamato: exo_query Get-M365OpsSharedMailboxes
2026-08-15 10:20:48 [Error] Fabrikam-Prod[Delegated] Tool AI fallito: ... Sessione Exchange non attiva ...
2026-08-15 10:20:48 [Info] Fabrikam-Prod[Delegated] Tool AI completato: exo_query Get-M365OpsSharedMailboxes
```

19.2 Cosa viene registrato

- Avvio del server e tenant iniziale caricato
 - Ogni messaggio ricevuto in chat (testo completo)
 - Inizio/fine di ogni chiamata al motore AI con tool-calling, con timestamp separati — se il server si blocca, l'ultima riga "chiamato" senza il corrispondente "completato" indica esattamente quale strumento è rimasto appeso (è così che è stato diagnosticato l'incidente di sezione 10.6)
 - Ogni tool AI chiamato/completato/fallito, con l'errore esatto in caso di fallimento
 - Ogni azione confermata eseguita (creazione gruppo, scrittura Exchange, script personalizzato...)
- Uso da Claude Code (o da chiunque debugga il progetto): quando qualcosa si comporta in modo strano, il primo posto da controllare è l'ultima riga di `Logs\m365ops-*.log` di oggi — spesso basta quella a capire dove si sia fermato.

20. Stato e utilizzo del motore AI

Il tab **Motore AI** mostra, oltre alle impostazioni, un pannello "Stato e utilizzo" con: provider attivo, quale delle due chiavi (Claude/Azure OpenAI) risulta configurata, e un **contatore delle chiamate** fatte a ciascun provider in questa sessione del server (azzerato ad ogni riavvio — non persistito, è un contatore di sessione; il provider *selezionato*, invece, è persistito — sezione 8).

20.1 Chi usa davvero quale provider

Dal 15/08/2026 il provider selezionato nel tab Motore AI vale per **tutto**: sia la chat libera con strumenti (`Invoke-M365OpsAgentTools` — la stragrande maggioranza delle domande) sia le chiamate singole senza strumenti (analisi pattern, diagnosi/correzione errori). Non esiste più una parte dell'app che ignora silenziosamente la scelta fatta in GUI — vedi sezione 8.1/8.2 per come funziona il tool-calling su entrambi i provider e il bug di scoping reale trovato e corretto durante questo lavoro.

20.2 Verificare la connessione dal vivo

Il pulsante "Testa connessione (provider attivo)" fa una vera chiamata reale (pochi token, non gratuita) al provider selezionato e conferma se risponde correttamente — non si limita a controllare che la chiave sia presente. Usalo su richiesta esplicita, non in automatico: costa una chiamata reale ogni volta.

20.3 Indicatore di avanzamento onesto

Il server è a thread singolo (sezione 10.6): mentre elabora una domanda non può rispondere a nessun'altra richiesta, nemmeno a un polling di stato. Un messaggio statico "sto elaborando..." sarebbe quindi indistinguibile da un blocco reale - bug reale segnalato dal vivo: "dice che sta facendo roba quando dentro non sta facendo un cazzo". La GUI ora mostra invece un contatore dei secondi realmente trascorsi (misurato lato browser, non simulato), con messaggi che cambiano dopo le prime decine di secondi per suggerire cosa potrebbe star succedendo (molte chiamate Graph in sequenza, un report con molti dati, o - oltre i 60-90s - un login interattivo Exchange/Intune avviato e mai completato in un'altra finestra, l'unico caso di blocco genuino di lunga durata). Non è un vero progresso passo-passo (il server non può emettere eventi mentre è occupato), ma è un segnale onesto invece che uno finto.

20.4 Budget di token e risposte vuote sui modelli reasoning

Il limite di token per ogni singolo giro del ciclo di tool-calling (`max_tokens` per Claude, `max_completion_tokens` per Azure OpenAI) è 4000, non più 2000. Bug reale scoperto il 17/08/2026 testando la creazione di script da parte dell'AI (sezione 17.6): su un modello reasoning come `gpt-5-mini`, i token di ragionamento interno condividono lo stesso budget dei token restituiti come risposta — con un compito che richiede scrivere codice vero (non solo un breve JSON), il modello poteva esaurire l'intero budget nel ragionamento e restituire un contenuto **vuoto**, senza alcun errore lanciato. L'utente vedeva semplicemente un messaggio bianco in chat.

Corretto in due modi complementari: il budget è stato alzato a 4000 (riduce la probabilità che succeda), e — indipendentemente dalla causa — un contenuto vuoto non arriva più silenzioso all'utente: viene rilevato, registrato nel log con `finish_reason` e il conteggio dei token di ragionamento usati (utile per capire se serve alzare ancora il budget in futuro), e sostituito con un messaggio che spiega cosa è successo invece di una risposta bianca.

20.5 Rete di sicurezza contro un messaggio con 'role' nullo

Bug osservato due volte in questo progetto con la stessa identica firma: Azure OpenAI rifiuta l'INTERA richiesta con 400 "Invalid type for messages[N].role... got null instead" se anche un solo messaggio nell'array ha 'role' mancante o nullo. La prima occorrenza (storico conversazione persistito corrotto) era stata mitigata filtrando le voci non valide in `Get-M365OpsChatHistory`. Riapparso il 17/08/2026 su un tenant con storico pulito (verificato leggendo direttamente il file) - l'origine esatta di QUESTA seconda occorrenza non è stata individuata con certezza (probabile causa: una voce costruita durante il ciclo di tool-calling stesso, non nello storico persistito).

Invece di continuare a inseguire ogni possibile origine, la mitigazione è stata spostata al punto più a valle possibile: l'array `messages` viene filtrato appena prima di ogni chiamata reale (sia il ramo Azure sia il ramo Claude), scartando qualunque voce senza un 'role' valido. Una voce malformata da QUALSIASI causa, presente o futura, degrada quindi a "quel turno manca dal contesto" invece di far fallire l'intera risposta.

Causa radice trovata il 19/08/2026 (richiesto esplicitamente dall'utente dopo averlo rivisto ricomparire durante un bug-hunt sul catalogo comandi): il filtro sopra continuava a scartare 1 voce ad OGNI chiamata AI che parte da uno storico vuoto (non un caso raro - praticamente ogni "nuova conversazione"). Instrumentato temporaneamente il codice per vedere `$messages` nel momento esatto del filtro: la voce fantasma era sempre `{"role":null,"content":null}` come primo elemento. Causa, verificata empiricamente in isolamento passo per passo: `Get-M365OpsChatHistory` fa `return @()` a storico vuoto, ma `Gui\Server.ps1` assegnava il risultato SENZA avvolgerlo in `@(...)` - un array vuoto restituito così da una funzione PowerShell **collassa a \$null** in un'assegnazione scalare (verificato: `$a = FunzioneCheRestituisceArrayVuoto` rende `$a` letteralmente `$null`, non un array di lunghezza 0). `-History $null` arriva così in `Invoke-M365OpsAgentTools` (un `$null` passato esplicitamente NON attiva il default del parametro, quello scatta solo se il parametro è del tutto omesso), e `$null | ForEach-Object {...}` NON esegue zero iterazioni come ci si aspetterebbe da una collezione vuota - ne esegue UNA, con `$_ = $null`, producendo esattamente la voce fantasma osservata. Corretto in due punti: `@(...)` aggiunto attorno alla chiamata mancante in `Server.ps1` (l'altro punto che già la chiamava, `GET /api/chat/history`, ce l'aveva già), più `Where-Object { $_ }` prima del `ForEach-Object` che costruisce `$messages` da `$History` in `Invoke-M365OpsAgentTools.ps1` - rete di sicurezza simmetrica a quella già esistente sull'output, così un `$History` nullo non può più produrre la voce fantasma indipendentemente da futuri errori di chiamata. Verificato dal vivo: reset storico + messaggio → nessun "Filtrate...role mancante" nei log (prima compariva sempre in questo esatto scenario).

21. Stato e reset MFA — dettagli tecnici

Vedi sezione 13.5 per l'uso quotidiano in chat. Questa sezione documenta i dettagli verificati sulla documentazione Microsoft reale (non a memoria), utili se qualcosa non torna.

21.1 Perché non c'è un singolo flag "MFA abilitata"

La pagina "Authentication states" di Microsoft Graph (vedere/impostare lo stato MFA per-utente con un valore unico) è esplicitamente **non ancora supportata in v1.0** - lo stato reale è governato da Conditional Access o dalle vecchie impostazioni per-utente dell'admin center, non da un campo Graph interrogabile. Il modo corretto, verificato su learn.microsoft.com/graph/api/authentication-list-methods e [.../authenticationmethods-overview](https://learn.microsoft.com/graph/api/authentication-methods-overview), è elencare `GET /users/{id}/authentication/methods` e guardare quali tipi risultano registrati oltre alla password: se c'è solo la password, l'utente non ha configurato nessun metodo di verifica aggiuntivo. `Get-M365OpsUserMfaStatus` esclude dal conteggio "MFA configurata" sia la password (fattore primario, non un secondo fattore) sia il metodo email (serve solo per il reset password self-service, non per l'MFA).

21.2 Cosa significa davvero "resettare l'MFA"

Non esiste un'azione atomica equivalente in Graph: la documentazione ufficiale stessa indica di eliminare ogni metodo uno per uno, tramite l'endpoint REST specifico del suo tipo (es. `DELETE /users/{id}/authentication/microsoftAuthenticatorMethods/{methodId}`, `.../phoneMethods/{methodId}`, `.../fido2Methods/{methodId}`, e così via per app OATH, Windows Hello for Business, Temporary Access Pass). `Reset-M365OpsUserMfa` fa esattamente questo, uno per uno, senza mai toccare password o email, e restituisce sia i metodi rimossi con successo sia quelli falliti (invece di fermarsi al primo errore).

21.3 Permessi e ruolo — due requisiti separati

Serve **sia** lo scope Graph `UserAuthenticationMethod.Read.All` (lettura) o `.ReadWrite.All` (reset) **sia** un ruolo Entra assegnato a chi ha fatto login: Global Reader per la sola lettura; Authentication Administrator o Privileged Authentication Administrator per il reset. Senza il ruolo, Graph risponde 403 "Request Authorization failed" anche con lo scope corretto nel token - un errore facile da scambiare per uno scope mancante, ma è invece un requisito di ruolo separato. Incontrato dal vivo il 15/08/2026 e risolto assegnando il ruolo giusto all'utente che fa login.

22. Report AI-orchestrati con grafici — dettagli tecnici

Vedi sezione 13.4 per l'uso quotidiano in chat. Architettura: l'AI raccoglie SEMPRE i dati grezzi completi (mai un riassunto pre-aggregato) tramite gli strumenti di lettura già esistenti (`graph_api_call` , `exo_query`), poi passa l'intero dataset più un elenco opzionale di campi da graficare (`chartFields`) allo strumento `generate_report` . Da lì in poi non c'è più ragionamento AI coinvolto:

- **Excel:** tutte le righe raccolte, via `Export-M365opsReport -Format xlsx` .
- **Aggregazione grafici:** `Group-Object` sul campo richiesto, conteggio per valore distinto - fatto dal codice, MAI chiesto all'AI. Un conteggio sbagliato su centinaia di righe sarebbe un errore silenzioso difficile da notare guardando solo il grafico finale.
- **Grafici:** SVG generati internamente (`New-M365OpsSvgBarChart` / `New-M365OpsSvgPieChart`), nessuna libreria JavaScript esterna, nessuna dipendenza da internet in fase di stampa - stessa filosofia di Edge headless locale già usata per ogni altro export PDF.
- **PDF:** HTML con i grafici SVG incorporati + tabella dati completa, stampato via Microsoft Edge headless, stesso meccanismo di `Export-M365opsReport` .

Entrambi i file compaiono in chat come pulsanti di download reali (`GET /api/reports/download?file=...` , protetto da path traversal: qualunque nome file con `/ o \` viene rifiutato) - il testo di risposta dell'AI non indica mai un percorso assoluto su disco, corretto dopo un bug reale in cui l'AI annunciava "genero il report" senza aver davvero chiamato lo strumento (vedi il blocco "REGOLA CRITICA" nel system prompt di `Invoke-M365opsAgentTools` , più un controllo di sicurezza lato server che segnala il sospetto se il testo promette un report senza allegati).

23. Memoria della conversazione — dettagli tecnici

Vedi sezione 13.6 per l'uso quotidiano. Prima di questa funzione (aggiunta il 15/08/2026), ogni domanda all'AI ripartiva da zero: `Invoke-M365OpsAgentTools` costruiva la conversazione con il solo messaggio corrente, mai i turni precedenti - una domanda di follow-up del tipo "e quello di prima?" non aveva alcun contesto a cui agganciarsi.

23.1 Cosa viene salvato e dove

`Config\ChatHistory-<nometenant>.json`, uno storico per tenant (mai condiviso tra profili diversi, stesso principio di isolamento multi-tenant della sezione 14). Ogni scambio (domanda utente + risposta finale, qualunque sia stato il percorso - catalogo locale o AI) viene accodato in un unico punto in `Server.ps1`, così lo storico resta coerente con ciò che l'utente ha davvero visto in chat, non solo con le risposte generate dall'AI.

Bug reale corretto il 16/08/2026. L'endpoint che legge lo storico (`GET /api/chat/history`) pipelava l'array in `ConvertTo-Json` - per un array VUOTO (tenant senza nessuno scambio ancora, es. subito dopo un riavvio), questo restituisce `$null` vero, non la stringa "[]": `GetBytes($null)` lanciava "Value cannot be null" ad ogni caricamento pagina. Corretto passando a `ConvertTo-Json -InputObject $array` (mai pipelato), che serializza correttamente 0, 1 o più elementi senza bisogno di controlli separati - stesso fix applicato a tutti gli altri punti a rischio del progetto (`/api/tenants` , `/api/setup-status` , `/api/logs` , il salvataggio dello storico stesso) dopo averli controllati uno per uno.

23.2 Limiti deliberati

- **Ultimi 8 scambi** (16 voci) per tenant: oltre, i più vecchi vengono scartati. Senza un tetto, una conversazione lunga rischierebbe di gonfiare ogni chiamata successiva in costo/tempo, fino a rischiare di sfiorare il limite di contesto del modello.
- **Ogni messaggio troncato a 3000 caratteri** prima del salvataggio: un elenco dispositivi o un dump JSON molto lungo non deve consumare da solo tutto il budget di contesto delle domande successive.
- **Password redatte prima del salvataggio** - qualunque campo `"password": "..."` nel testo (es. il corpo di una proposta di creazione utente) viene sostituito con `[REDACTED]` prima di scrivere su disco, stesso principio "zero segreti su disco" già applicato a `tenants.json` (sezione 14), esteso qui ai segreti che possono comparire dentro una conversazione stessa.

23.3 Azzerare lo storico

Pulsante "Nuova conversazione" in GUI, oppure scrivere "nuova conversazione" / "dimentica tutto" / "cancella cronologia" / "resetta la chat" in chat (riconosciuto dal catalogo comandi locale, nessun costo AI) - elimina il file per il tenant attivo e annulla anche un'eventuale proposta di scrittura ancora in sospeso, per evitare di ripartire "puliti" in chat ma con un'azione dimenticata ancora pronta a essere eseguita da un vecchio "sì".

24. Report di utilizzo Microsoft 365 Copilot (22/08/2026)

Primo passo dell'integrazione con l'amministrazione di Microsoft 365 Copilot, richiesta esplicitamente dall'utente: "dobbiamo aggiungere tutto quello che riguarda il supporto copilot M365... quello che si vede da admin center". Ambito volutamente ristretto alla parte amministrativa/di reporting - **M365Ops non chiama mai Copilot come motore AI** (usa sempre Claude/Azure OpenAI per quello, sezione 8), qui si tratta solo di leggere DATI su come Copilot viene usato nel tenant, non di usarlo.

24.1 Cosa e' disponibile, cosa no

Ricerca fatta contro la documentazione Microsoft Learn reale (non a memoria) prima di implementare, per evitare di costruire su un'API instabile o su un percorso che poi risulta Delegato-only mentre questo progetto lavora principalmente in App-only:

Area	Stato	Motivo
Report di utilizzo (per-utente e aggregato)	implementato	Endpoint Graph v1.0 (GA, stabile), permesso <code>Reports.Read.All</code> - funziona identico sia App-only sia Delegato, nessuna limitazione. Vedi 24.2.
Impostazioni Copilot (User access/Data access/Copilot actions)	non implementato	API <code>/copilot/admin/policySettings</code> - permesso <code>CopilotPolicySettings.Read/.ReadWrite</code> risulta letteralmente "Application: Not supported" nella documentazione ufficiale - SOLO Delegato, mai utilizzabile su un tenant App-only.
Catalogo agenti/app Copilot	non implementato	API <code>/copilot/admin/catalog/packages</code> - ancora in preview, rollout tenant-per-tenant con errori 403 documentati anche a permessi corretti quando il feature flag non e' ancora attivo - troppo instabile per costruirci sopra ora.
Connettori federati Copilot	non implementato	<code>Set-FederatedConnectorToggle</code> (modulo <code>Connector.cmd</code>) risulta in dismissione dal 25/08/2026, sostituito da "Allowed agent types" in Agent 365 dal 5/10/2026 - costruirci sopra ora avrebbe significato shippare qualcosa gia' morto in pochi giorni.
Cronologia interazioni Copilot (contenuto reale dei prompt)	non implementato	Permesso <code>AiEnterpriseInteraction.Read.All</code> (solo Application) esisterebbe, ma espone il CONTENUTO REALE di cosa gli utenti hanno scritto a Copilot - dato di livello compliance/eDiscovery, non semplice telemetria di utilizzo. Non aggiunto senza una richiesta esplicita e consapevole dell'utente.

24.2 Get-M365OpsCopilotUsageReport / Get-M365OpsCopilotUsageSummary

`Get-M365OpsCopilotUsageReport -Period D30` (report per-utente: ultima attivita' complessiva e per singola app - Teams/Word/Excel/PowerPoint/Outlook/OneNote/Loop/Copilot Chat) e `Get-M365OpsCopilotUsageSummary -Period D30` (numero aggregato di utenti abilitati/ attivi per app) - gli stessi identici dati mostrati in "Report > Utilizzo > Microsoft 365 Copilot" nell'admin center. `-Period` accetta `D7` / `D30` / `D90` / `D180` / `ALL`.

Endpoint Graph v1.0 (`/copilot/reports/getMicrosoft365CopilotUsageUserDetail` e `getMicrosoft365CopilotUserCountSummary`) - a differenza della maggior parte delle altre API di questo progetto, la risposta v1.0 e' SEMPRE un file CSV (comportamento documentato su Microsoft Learn, non un bug), mai JSON: entrambe le cmdlet lo convertono con `ConvertFrom-Csv` e rinominano le colonne in PascalCase per coerenza con lo stile del resto del modulo.

Permesso mancante su vnsys-test, verificato dal vivo il 22/08/2026: `Reports.Read.All` (Application) non e' ancora concesso - Graph risponde 403 "S2SUnauthorized: Invalid permission." Va aggiunto in Entra ID > App Registration > API permissions > Add a permission > Microsoft Graph > Application permissions > `Reports.Read.All` > Grant admin consent (sezione 4.2) - lo stesso permesso serve gia' per altri report generali di Microsoft 365, non e' specifico di Copilot. Il parsing CSV di queste due cmdlet non e' quindi ancora stato verificato dal vivo con dati reali - solo il percorso di errore (403) e' stato confermato.

24.3 Knowledge Base su Copilot

Per le aree NON implementate (24.1), invece di costruire integrazioni fragili o premature, l'AI puo' comunque rispondere a domande concettuali su Microsoft 365 Copilot (cos'e' il Copilot Control System, perche' le impostazioni Policy sono solo Delegate, cosa mostra l'admin center) attingendo alla Knowledge Base globale (sezione 17.13) - stesso meccanismo gia' usato per la guida stessa, riusabile per qualunque documentazione aggiuntiva caricata con `Add-M365OpsKnowledgeDocument -TenantName '_global'`.

Appendice A — Struttura dei file

```

M365Ops\
├─ M365Ops.psd1 / M365Ops.psm1    manifest e loader del modulo (export dinamico, sezione 17)
├─ Public\
│   ├── Get-M365OpsUserMfaStatus.ps1 / Reset-M365OpsUserMfa.ps1    stato/reset MFA (sezione 21)
│   ├── Export-M365OpsDataReport.ps1                                report AI-orchestrati con grafici (sezione 22)
│   ├── Connect-M365OpsIntune.ps1                                  connessione Intune, entrambe le modalità (sezione 10.9)
│   ├── Invoke-M365OpsActionRecovery.ps1                          retry AI-assistito su scritture fallite (sezione 13.7)
│   └─ Get-M365OpsChatHistory.ps1 / Add-M365OpsChatHistoryTurn.ps1 / Clear-M365OpsChatHistory.ps1    memoria conversazione (sezione 23)
├─ Private\
│   ├── funzioni interne (token, chiamate Graph/MCP, device code flow)
│   ├── Test-M365OpsIntuneAuthValid.ps1                          verifica dal vivo del token Intune (sezione 10.9)
│   ├── New-M365OpsSvgChart.ps1                                    grafici SVG per i report (sezione 22)
│   └─ ConvertTo-M365OpsHashtable.ps1                             normalizza i parametri AI (PSCustomObject → Hashtable) prima di ogni scrittura
├─ Scripts\Custom\
│   └─ script personalizzati (sezione 17) - README.md e _TEMPLATE.ps1
├─ Config\
│   ├── tenants.json                                              profili tenant (mai segreti in chiaro)
│   ├── M365Ops-Exchange.cer                                       chiave pubblica del certificato Exchange
│   ├── ai-provider.json                                           provider AI selezionato in GUI, persistito (sezione 8)
│   ├── last-active-tenant.txt                                     ultimo tenant attivo, letto da Launch-M365Ops.ps1 (sezione 12.1)
│   └─ ChatHistory-<tenant>.json                                    storico conversazione per tenant, password redatte (sezione 23)
├─ Gui\
│   ├── Server.ps1                                                server HTTP + logica chat/conferme
│   ├── index.html                                                interfaccia web di chat
│   └─ CommandCatalog.ps1                                         comandi deterministici a costo zero
├─ Reports\
│   └─ output export CSV/XLSX/PDF (anche i grafici SVG generati, sezione 22)
├─ Uploads\
│   └─ file caricati da GUI (installer, icone, CSV migrazione)
├─ Logs\
│   └─ storico azioni, un file al giorno (sezione 19)
├─ Tools\
│   └─ IntuneWinAppUtil.exe, New-M365OpsIcon.ps1 / New-M365OpsShortcut.ps1 (sezione 3.1)
├─ Assets\
│   └─ M365Ops.ico                                                icona dell'app, usata dal collegamento sul Desktop (sezione 3.1)
├─ docs\
│   ├── Guida-Configurazione.html                                  sorgente di questa guida
│   └─ Guida-Configurazione.pdf                                    versione stampabile
├─ Launch-M365Ops.ps1 / M365Ops.bat                                avvio con un click (12.1) o da terminale (12.2), resume-session, auto-install prerequisiti
├─ Show-M365OpsPrereqInstaller.ps1                                GUI di installazione prerequisiti, richiamata da M365Ops.bat solo se manca qualcosa (sezione 3.1)
├─ Bootstrap-WinGet.ps1                                           installa winget stesso se assente, richiamato da Show-M365OpsPrereqInstaller.ps1 (sezione 3.1)
├─ Kill-And-Restart-M365Ops.ps1 / .bat                             termina il server (anche se bloccato) e lo riavvia - ultima risorsa se il riavvio normale non risponde (sezione 12.3)
├─ Start-M365OpsWebApp.ps1 / Start-M365Ops.ps1                   superati da Launch-M365Ops.ps1 (mancano rilevamento porta e resume-session) - lasciati solo per compatibilità con script
└─ README.md                                                       guida rapida sviluppatore

```

Appendice B — Tutto avviene dalla GUI, non serve PowerShell

Per scelta esplicita, l'uso quotidiano di M365Ops non richiede mai di scrivere un comando PowerShell: setup del tenant, motore AI, email, connettori MCP, report, reset MFA, tracking messaggi, migrazioni, sicurezza email, SharePoint/OneDrive, Teams — ogni operazione descritta in questa guida ha un campo, un pulsante o una richiesta in chat come suo equivalente diretto in GUI (vedi in particolare le sezioni 7, 9, 11.1, 13, 17). Le cmdlet PowerShell (`Get-Command -Module M365Ops` per l'elenco completo) restano lo strato che la GUI stessa chiama dietro le quinte, e sono comunque disponibili per chi vuole scriptare qualcosa di più avanzato da un terminale — ma non sono la via prevista per l'uso normale, e questa guida non le presenta più come tale.

Le uniche eccezioni reali, dove un passaggio manuale fuori dalla GUI resta inevitabile perché riguarda strumenti esterni (Azure, Exchange, il certificate store di Windows) e non l'app in sé, sono documentate nel punto in cui servono: creazione dell'App Registration e del certificato (sezioni 4-5), assegnazione del ruolo RBAC su Exchange Online (sezione 6), e trasferimento del certificato su un PC nuovo (sezione 18.3).

M365Ops — Guida alla Configurazione — documento vivo, aggiornato ad ogni modifica rilevante